

Trabajo Fin de Máster
Máster en Seguridad en Tecnologías de la
Información y las Comunicaciones

Evaluación de Velociraptor como HIDS para la
detección de técnicas MITRE ATT&CK Enterprise

Autor: Julio García-Amorena Reina

Tutor: Francisco Javier Muñoz Calle

Dpto. de Ingeniería Telemática
Escuela Técnica Superior de Ingeniería
Universidad de Sevilla

Sevilla, 2026



Trabajo Fin de Máster

Máster en Seguridad en Tecnologías de la Información y las Comunicaciones

Evaluación de Velociraptor como HIDS para la detección de técnicas MITRE ATT&CK Enterprise

Autor:

Julio García-Amorena Reina

Tutor:

Francisco Javier Muñoz Calle

Profesor Colaborador

Dpto. de Ingeniería Telemática

Escuela Técnica Superior de Ingeniería

Universidad de Sevilla

Sevilla, 2026

Trabajo Fin de Máster: Evaluación de Velociraptor como HIDS para la detección de técnicas MITRE ATT&CK Enterprise

Autor: Julio García-Amorena Reina

Tutor: Francisco Javier Muñoz Calle

El tribunal nombrado para juzgar el proyecto arriba indicado, compuesto por los siguientes miembros:

Presidente:

Vocales:

Secretario:

Acuerdan otorgarle la calificación de:

Sevilla, 2026

Secretario del Tribunal

Dedico este Trabajo de Fin de Máster a mi familia. A mi padre, Agustín García-Amorena, por haberme apoyado siempre, por ser una inspiración en mi vida y un ejemplo constante y a mi madre, Sara Reina, por su protección, cariño y presencia incondicional. También quiero dedicar este trabajo a mis primos, Guillermo Reina y Jesús Gómez de la Mata, mis mejores amigos y compañeros de vida; y a mi pareja, Irene Reyes, quien no solo me animó a tomar este camino, sino que me ha acompañado en cada paso.

Finalmente, quiero recordar de forma especial a mi padrino, Juan García-Amorena, y nuevamente a mi padre, Agustín García-Amorena, quienes me inculcaron el amor por el conocimiento técnico y han sido dos de las personas que más he admirado.

Agradecimientos

Quiero expresar mi más sincero agradecimiento a todas las personas que me han apoyado, aconsejado y escuchado a lo largo del desarrollo de este proyecto.

En primer lugar, a mi familia: a mi padre, Agustín, pieza fundamental en mi vida y ejemplo a seguir y a mi madre, Sara, por estar siempre a mi lado y ayudarme a forjar mi carácter. A mi pareja, Irene, quiero agradecerle especialmente que me diera el valor para seguir el camino que realmente deseaba, ayudándome a superar prejuicios, dudas y complejos. Su apoyo incondicional, su confianza en mí y su constante motivación han sido esenciales para alcanzar mis objetivos. A mis amigos y compañeros, Jorge, Miguel, Alimar y Reisaac, gracias por acompañarme durante este año, por vuestro apoyo y por haber hecho más llevadero el curso. A mis primos, Guillermo y Jesús, gracias por estar siempre disponibles cuando he necesitado una corrección, un consejo o un punto de vista sincero. Les estoy profundamente agradecido.

Finalmente, quiero expresar mi más sincero agradecimiento a mi tutor, Francisco Javier Muñoz Calle, por su orientación, disponibilidad y apoyo durante el desarrollo de este Trabajo de Fin de Máster. Su implicación, cercanía y criterio han sido fundamentales para enfocar y completar este proyecto, así como para convertirlo en una propuesta plenamente alineada con mis intereses académicos y profesionales.

A todos, mi más sincero agradecimiento.

Por último, al igual que cerré mi Trabajo de Fin de Grado con los versos de “Caminante, no hay camino”, pertenecientes a *Proverbios y cantares XXIX*, del ilustre poeta Antonio Machado, me gustaría cerrar este Trabajo de Fin de Máster con los siguientes versos del mismo autor:

*Todo pasa y todo queda,
pero lo nuestro es pasar;
pasar haciendo caminos,
caminos sobre la mar.*

Antonio Machado, *Proverbios y cantares XXIX*

Resumen

El incremento de los ciberataques dirigidos a infraestructuras corporativas ha puesto de manifiesto las limitaciones de los enfoques de seguridad basados exclusivamente en la prevención. En los entornos actuales, donde los atacantes emplean técnicas cada vez más sofisticadas y adaptativas, se asume que los mecanismos defensivos perimetrales pueden ser eventualmente superados. En consecuencia, resulta esencial disponer de capacidades avanzadas de detección y respuesta a nivel de endpoint. En este contexto, las tecnologías de detección de intrusiones basadas en host (HIDS), así como soluciones más avanzadas como EDR (Endpoint Detection and Response) y XDR (Extended Detection and Response), desempeñan un papel fundamental al proporcionar visibilidad sobre la actividad del sistema operativo, los procesos en ejecución y otros indicadores de compromiso.

El presente Trabajo Fin de Máster, titulado “Evaluación de Velociraptor como HIDS para la detección de técnicas MITRE ATT&CK Enterprise”, tiene como objetivo analizar la capacidad de la herramienta Velociraptor para detectar técnicas de ataque recogidas en la matriz MITRE ATT&CK Enterprise. El estudio evalúa en qué medida Velociraptor puede emplearse como sistema HIDS, considerando tanto sus capacidades de monitorización del sistema como su flexibilidad para implementar mecanismos de detección mediante artifacts nativos y personalizados. Para ello, se ha diseñado un entorno de laboratorio controlado mediante VM con arquitectura cliente-servidor y un SO Windows instrumentado con el agente de Velociraptor, sobre el que se ejecutan de forma controlada distintas técnicas de ataque.

Los resultados obtenidos muestran que los artifacts personalizados desarrollados para Velociraptor generaron detecciones específicas para las nueve técnicas evaluadas en el entorno experimental. En la evaluación de artifacts públicos se obtuvieron seis resultados positivos de visibilidad, mientras que únicamente Hayabusa alcanzó detección específica en 3 de las 9 técnicas. Como referencia comparativa, Wazuh obtuvo cobertura específica en 3/9 técnicas mediante su configuración base y en 4/9 mediante reglas personalizadas, alcanzando conjuntamente 7/9 técnicas.

Adicionalmente, el trabajo analiza la viabilidad de los artifacts públicos y de la comunidad, así como el impacto en el rendimiento derivado de la monitorización continua. El benchmark se realizó mediante nueve ejecuciones distribuidas en tres escenarios, observándose durante la ejecución del runner una utilización media aproximada del 0,95 % de CPU del cliente y una huella de memoria cercana a 57 MB. Los resultados deben interpretarse dentro del alcance del laboratorio empleado, basado en una única máquina virtual Windows.

Palabras clave: ciberseguridad, HIDS, Velociraptor, MITRE ATT&CK, ransomware, DFIR, detección de intrusiones, monitorización de endpoints.

Abstract

The increasing number of cyberattacks targeting corporate infrastructures has highlighted the limitations of security approaches based solely on prevention. In modern threat environments, where attackers employ increasingly sophisticated and adaptive techniques, it is assumed that perimeter defence mechanisms may eventually be bypassed. Consequently, it is essential to develop advanced detection and response capabilities at the endpoint level. In this context, host-based intrusion detection technologies (HIDS), together with more advanced solutions such as EDR (Endpoint Detection and Response) and XDR (Extended Detection and Response), play a fundamental role by providing visibility into operating system activity, running processes, and other indicators of compromise.

This Master's Thesis, entitled “Evaluation of Velociraptor as a HIDS for Detecting MITRE ATT&CK Enterprise Techniques”, aims to analyse the ability of Velociraptor to detect the attack techniques evaluated in the MITRE ATT&CK Enterprise framework. The study evaluates the extent to which Velociraptor can operate as a HIDS, considering both its system monitoring capabilities and its flexibility to implement detection mechanisms through native and custom artifacts. To achieve this objective, a controlled laboratory environment has been designed using a virtualized client-server architecture and a Windows operating system instrumented with the Velociraptor agent, on which several attack techniques are executed in a controlled manner.

The results show that the custom artifacts developed for Velociraptor generated specific detections for all nine techniques evaluated in the experimental environment. In the evaluation of public artifacts, six positive visibility results were obtained, while only Hayabusa achieved specific detection for 3 of the 9 techniques. For comparison, Wazuh achieved specific coverage for 3 out of 9 techniques using its base configuration and for 4 out of 9 using custom rules, covering a total of 7 out of 9 techniques.

Furthermore, the study analyses the viability of public and community artifacts, as well as the impact on performance resulting from continuous monitoring. The benchmark was carried out using nine runs distributed across three scenarios; during the runner's execution, an average CPU utilisation of approximately 0.95 per cent on the client machine and a memory footprint of around 57 MB were observed. The results are limited to a controlled laboratory based on a single Windows virtual machine.

Keywords: cybersecurity, HIDS, Velociraptor, MITRE ATT&CK, ransomware, DFIR, intrusion detection, endpoint monitoring.

Índice

Capítulo I: Introducción.....	1
1.1. Motivación y descripción del problema.....	1
1.2. Objetivos del proyecto	2
1.3. Estimación y plazos	2
Capítulo II: Fundamentos teóricos.....	4
2.1. Detección en endpoint: HIDS, EDR y XDR.....	4
2.1.1. Detección en HIDS tradicionales	4
2.1.2. Detección en DFIR	5
2.1.3. Detección en EDR.....	5
2.1.4. MITRE ATT&CK como marco de referencia.....	6
2.1.5. Criterios de evaluación de herramientas de detección	6
2.2. Velociraptor.....	7
2.2.1. Naturaleza, capacidades y uso habitual.....	7
2.2.2. Posición de Velociraptor y comparación funcional con otras soluciones	8
2.2.3. Arquitectura interna Velociraptor	10
2.2.4. Artifacts y modelo de ejecución.....	13
2.2.5. Velociraptor Query Language (VQL)	16
2.2.6. Telemetría disponible en el endpoint	18
2.2.7. Limitaciones de visibilidad en HIDS/DFIR.....	19
Capítulo III: Desarrollo experimental y ejecución de pruebas.....	21
3.1. Metodología experimental	21
3.1.1. Principios metodológicos.....	22
3.1.2. Flujo general de trabajo.....	23
3.2. Selección de técnicas MITRE ATT&CK	23
3.2.1. Criterios de selección	23
3.2.2. Técnicas seleccionadas	24
3.2.3. Tácticas ATT&CK representadas por el subconjunto evaluado.....	25

3.3. Diseño del entorno	25
3.3.1. Configuración base de la máquina virtual.....	25
3.3.2. Componentes técnicos desplegados	26
3.3.3. Identificación del cliente y alcance del laboratorio.....	27
3.4. Instalación, instrumentación y configuración base	27
3.4.1. Servidor Velociraptor	27
3.4.2. Agente Velociraptor.....	28
3.4.3. Instrumentación con Sysmon, PowerShell 4104 y eventos Windows	30
3.4.4. Organización del entorno de trabajo	30
3.4.5. Control de versiones y trazabilidad del desarrollo	31
3.4.6. Matriz principal de control experimental.....	32
3.4.7. Registro de evidencias	33
3.4.8. Diseño de pruebas controladas.....	34
3.4.9. Scripts de laboratorio validados	34
3.5. Estrategia de artifacts y monitorización.....	35
3.5.1. Artifacts públicos considerados	35
3.5.2. Brechas detectadas y necesidad de artifacts personalizados	36
3.6. Benchmark y alertabilidad	37
3.6.1. Validación operativa mediante alertas.....	39
3.7. Ejecución de ataques.....	40
3.7.1. TEC-001 - Ejecución mediante PowerShell	40
3.7.2. TEC-002 - Ejecución mediante Windows Command Shell	41
3.7.3. TEC-003 - Creación de tarea programada	42
3.7.4. TEC-004 - Persistencia mediante claves Run del registro	44
3.7.5. TEC-005 - Ejecución mediante servicio Windows	45
3.7.6. TEC-006 - Descubrimiento de software de seguridad	46
3.7.7. TEC-007 - Cifrado de datos	47
3.7.8. TEC-008 – Destrucción de datos	48
3.7.9. TEC-009 - Staging, archivado ZIP y exfiltración HTTP controlada.....	49

3.8. Ejecución de pruebas legítimas.....	51
3.9. Recopilación de telemetría.....	54
3.9.1. Telemetría captada en el sistema.....	54
3.9.2. Telemetría captada en Velociraptor	56
3.9.3. Salida externa de alertado (JSONL/Discord).....	57
3.10. Configuración y uso de artifacts	58
3.10.1. Artifacts personalizados	58
3.10.2. Limitaciones encontradas durante la configuración.....	59
Capítulo IV: Resultados de detección.....	61
4.1. Detecciones ante actividad legítima.....	62
4.1.1. Resultados de la campaña de técnicas benignas con artifacts personalizados	62
4.2. Detecciones ante ataques	63
4.2.1. Resultados por ataque en campaña personalizada	63
4.2.2. Campaña de ataque con Artifacts públicos	80
4.3. Análisis comparativo integrado de los resultados	82
4.3.1. Artifacts públicos frente a personalizados.....	82
4.3.2. Intensidad de señal por técnica	84
4.3.3. Cobertura y selectividad de los perfiles P1-P4	85
4.3.4. Relación entre detección y falsos positivos	87
4.3.5. Síntesis de resultados	88
Capítulo V: Evaluación de Velociraptor y benchmark de rendimiento	91
5.1. Metodología de benchmark.....	92
5.2. Métricas, unidad de análisis y exclusiones	93
5.3. Impacto en CPU.....	95
5.4. Impacto en memoria	97
5.5. Impacto en disco	98
5.6. Relación detección / coste.....	100
5.7. Conclusiones	101
Capítulo VI: Validación comparativa con Wazuh como HIDS.....	102

6.1. Objetivo de la comparación	102
6.2. Arquitectura del entorno Wazuh.....	103
6.3. Fuentes de telemetría configuradas.....	104
6.4. Reglas personalizadas TFM en Wazuh	106
6.5. Metodología de campañas separadas	107
6.6. Campaña B: Wazuh base.....	108
6.7. Campaña C: Wazuh custom	110
6.8. Evidencias gráficas en el dashboard de Wazuh.....	114
6.9. Comparación Wazuh – Velociraptor	118
6.10. Limitaciones observadas.....	120
Capítulo VII: Conclusiones	122
7.1. Conclusiones principales.....	122
7.2. Respuesta a objetivos	123
7.3. Limitaciones identificadas	123
7.4. Valor del enfoque	124
Capítulo VIII: Discusión y trabajo futuro	126
8.1. Discusión integrada de resultados.....	126
8.2. Ampliación de técnicas MITRE	126
8.3. Desarrollo de nuevos artifacts.....	127
8.4. Automatización de detecciones.....	128
8.5. Integración SIEM/SOC y despliegue en entornos reales	128
Anexo A: Instalación del entorno de laboratorio	129
Anexo B: Repositorio de GitHub.....	136
Anexo C. Técnicas MITRE seleccionadas.....	138
Anexo D. Artifacts Velociraptor validados.....	144
Anexo E. Benchmark y rendimiento	152
Anexo F. Runners, scripts y ground truth	161
Anexo G. Configuración y resultados de Wazuh.....	165
Anexo H. Planificación del proyecto ampliada.....	171

Anexo I. Declaración responsable sobre el uso de inteligencia artificial.....	181
Referencias.....	182

Índice de tablas

Tabla 1: Funciones, limitaciones y riesgos del enfoque HIDS	5
Tabla 2: Funciones, limitaciones y riesgos del enfoque DFIR.....	5
Tabla 3: Funciones, limitaciones y riesgos del enfoque EDR.....	6
Tabla 4: Tipos de artifacts según su modo de ejecución y objetivo	8
Tabla 5: Comparación funcional de Velociraptor con otras soluciones de seguridad orientadas al endpoint	9
Tabla 6: Formatos de datos en el flujo cliente-servidor de Velociraptor.....	12
Tabla 7: Campos principales de definición de artifacts.....	14
Tabla 8: Tipos de artifacts según el contexto de ejecución	14
Tabla 9: Diferencias entre collections, hunts, monitoring y SERVER_EVENT.....	15
Tabla 10: Fuentes principales de telemetría y utilidad en detección.....	19
Tabla 11: Diferencia entre visibilidad, detección, alerta y salida externa	19
Tabla 12: Criterios metodológicos del capítulo III	21
Tabla 13: Principios metodológicos aplicados en la fase experimental	23
Tabla 14: Flujo general de trabajo por técnica	23
Tabla 15: Criterios usados para seleccionar técnicas MITRE ATT&CK	24
Tabla 16: Técnicas MITRE ATT&CK seleccionadas para la fase experimental - Fuente: elaboración propia a partir de las técnicas ATT&CK [2], [20]-[30].....	24
Tabla 17: Tácticas ATT&CK representadas por el subconjunto evaluado empleando denominador	14 25
Tabla 18: Parámetros iniciales de la máquina virtual.....	26
Tabla 19: Componentes técnicos principales	27
Tabla 20: Identificación del endpoint usado en el laboratorio	27
Tabla 21: Parámetros funcionales del servidor Velociraptor.....	28
Tabla 22: Fuentes de telemetría habilitadas y técnicas relacionadas	30
Tabla 23: Snapshots utilizados para control del laboratorio	32
Tabla 24: Hojas principales de la matriz de control experimental	33
Tabla 25: Tipos de evidencias documentadas por técnica.....	34
Tabla 26: Controles de seguridad aplicados a las pruebas	34
Tabla 27: Scripts de laboratorio y papel metodológico.....	34
Tabla 28: Artifacts públicos considerados en la evaluación metodológica	36
Tabla 29: Brechas y limitaciones de los artifacts públicos en la evaluación.....	37
Tabla 30: Planteamiento inicial del benchmark de rendimiento	38
Tabla 31: Escenarios definitivos del benchmark centrado en el cliente.....	39

Tabla 32: Casos benignos incluidos en el runner de falsos positivos.....	53
Tabla 33: Telemetría esperada por técnica experimental	56
Tabla 34: Artifacts custom por perfil y función operativa	59
Tabla 35: Resultados definitivos del runner de técnicas benignas	63
Tabla 36: Resultado de ejecución y telemetría comprobada por el runner	64
Tabla 37: Distribución de filas CLIENT_EVENT emitidas/exportadas por perfil custom.....	67
Tabla 38: Distribución de filas CLIENT_EVENT por técnica en la campaña real	68
Tabla 39: Resultados definitivos de las campañas con artifacts públicos	81
Tabla 40: Cobertura y densidad de los perfiles personalizados P1-P4.....	86
Tabla 41: Síntesis cuantitativa de los resultados de detección	89
Tabla 42: Síntesis del impacto observado	92
Tabla 43: Escenarios y ventanas de medición.....	92
Tabla 44: Estructura del Excel definitivo de benchmark	94
Tabla 45: CPU del cliente en VR_TEC_RUNNER por repetición	96
Tabla 46: Memoria del cliente por escenario y repetición	97
Tabla 47: Actividad global de disco por escenario.....	99
Tabla 48: Disco global en VR_TEC_RUNNER por repetición	100
Tabla 49: Síntesis de evidencias y conclusiones	100
Tabla 50: Controles de calidad del benchmark final	101
Tabla 51: Criterios empleados para evaluar Wazuh como solución de comparación	103
Tabla 52: Parámetros principales del entorno Wazuh desplegado	104
Tabla 53: Fuentes de telemetría utilizadas por Wazuh en la campaña	106
Tabla 54: Reglas personalizadas Wazuh utilizadas para aproximar los perfiles P1-P4 de Velociraptor	106
Tabla 55: Campañas definidas para la validación comparativa	108
Tabla 56: Resultados de visibilidad y alerta de la campaña Wazuh base	110
Tabla 57: Resultados agregados de la campaña Wazuh custom.....	112
Tabla 58: Distribución de reglas TFM activadas durante la campaña custom	113
Tabla 59: Comparación final entre Wazuh base, Wazuh custom y Velociraptor custom	120
Tabla 60: Mapeo de severidad usado entre perfiles Velociraptor y niveles Wazuh	120
Tabla 61: Correspondencia entre preguntas de investigación, resultados y evidencias	123
Tabla 62: Técnicas MITRE ATT&CK propuestas para trabajo futuro.....	127
Tabla 63: Repositorio de GitHub	137
Tabla 64: Técnicas MITRE evaluadas, procedimientos y telemetría esperada (I)	142
Tabla 65: Evidencia observada y conclusión por técnica MITRE evaluada (II).....	143
Tabla 66: Definición de artifacts personalizados: perfil, tipo, sources y técnicas declaradas (I).....	147
Tabla 67: Resultados y función metodológica de los artifacts personalizados (II)	148

Tabla 68: Capacidad de detección, alerta y salida externa de los artifacts públicos (I)	150
Tabla 69: Resultados y fuentes de evidencia de las campañas con artifacts públicos (II)	151
Tabla 70: Resumen de rendimiento por escenario de benchmark	153
Tabla 71: Indicadores globales y controles de calidad del benchmark	154
Tabla 72: Reglas Wazuh observadas y veredicto por técnica.....	166
Tabla 73: Comparación agregada de cobertura entre Velociraptor y Wazuh	168
Tabla 74: Resultados de detección por técnica en Velociraptor y Wazuh	170
Tabla 75: Planificación general.....	172
Tabla 76: Planificación del capítulo I	173
Tabla 77: Planificación del capítulo II	174
Tabla 78: Planificación del capítulo III.....	175
Tabla 79: Planificación del capítulo IV.....	175
Tabla 80: Planificación del capítulo V	176
Tabla 81: Planificación del capítulo VI.....	177
Tabla 82: Planificación del capítulo VII	177
Tabla 83: Planificación del capítulo VIII	178
Tabla 84: Planificación de referencias bibliográficas y anexos	178

Índice de figuras

Figura 1: Diagrama de Gantt general.....	3
Figura 2: Arquitectura interna básica VR.....	11
Figura 3: Flujo de datos de Velociraptor.....	13
Figura 4: Instalación del agente Velociraptor.....	29
Figura 5: Datos recibidos desde los agentes Velociraptor.....	29
Figura 6: Estructura de directorios de la máquina física.....	31
Figura 7: Directorio de la máquina virtual.....	31
Figura 8: Arquitectura del flujo de alertas	40
Figura 9: Evidencia de ejecución de TEC-001	41
Figura 10: Evidencia de ejecución de TEC-002	42
Figura 11: Evidencia de la tarea programada TEC-003	43
Figura 12: Evidencia de persistencia mediante clave Run en TEC-004	44
Figura 13: Evidencia de ejecución del servicio TEC-005.....	45
Figura 14: Evidencia de descubrimiento de software de seguridad en TEC-006	46
Figura 15: Resultado del cifrado controlado: directorio con ficheros cifrados.....	48
Figura 16: Copia de respaldo de los ficheros tras el cifrado controlado	48
Figura 17: Receptor HTTP controlado activo para la prueba TEC-009.....	50
Figura 18: Resumen de la campaña de pruebas benignas	51
Figura 19: Resultado detallado de la campaña de pruebas benignas	52
Figura 20: Estado de Sysmon y del canal de telemetría del endpoint.....	54
Figura 21: Verificación de las fuentes de telemetría Sysmon habilitadas y registradas en el endpoint	55
Figura 22: Verificación de PowerShell Script Block Logging y Event ID 4104	55
Figura 23: Evidencia de telemetría registrada en Windows durante las pruebas	56
Figura 24: Evidencia de telemetría observada en Velociraptor durante las pruebas	57
Figura 25: Configuración Discord-Webhook.....	57
Figura 26: Resultado general de cobertura por campaña de ataques	61
Figura 27: Campaña definitiva de falsos positivos	62
Figura 28: Resumen del runner global de técnicas	63
Figura 29: Filas CLIENT_EVENT por caso TEC y perfil.....	64
Figura 30: Filas CLIENT_EVENT por categoría táctica.....	65
Figura 31: Media de filas CLIENT_EVENT por caso TEC según categoría táctica.....	65
Figura 32: Distribución de alertas por técnica cubierta	66
Figura 33: Arquitectura de artifacts custom y router	67
Figura 34: Captura textual de la evidencia de upload HTTP controlado en TEC-009.....	69

Figura 35: Notificación recibida en Discord para TEC-001 (1 de 3).....	70
Figura 36: Notificación recibida en Discord para TEC-001 (2 de 3).....	70
Figura 37: Notificación recibida en Discord para TEC-001 (3 de 3).....	71
Figura 38: Notificación recibida en Discord para TEC-002	71
Figura 39: Notificación recibida en Discord para TEC-003	72
Figura 40: Notificación recibida en Discord para TEC-004	73
Figura 41: Notificación recibida en Discord para TEC-005	73
Figura 42: Notificación recibida en Discord para TEC-006 (1 de 2).....	74
Figura 43: Notificación recibida en Discord para TEC-006 (2 de 2).....	74
Figura 44: Notificación recibida en Discord para TEC-007 (1 de 2).....	75
Figura 45: Notificación recibida en Discord para TEC-007 (2 de 2).....	76
Figura 46: Notificación recibida en Discord para TEC-008 (1 de 3).....	77
Figura 47: Notificación recibida en Discord para TEC-008 (2 de 3).....	77
Figura 48: Notificación recibida en Discord para TEC-008 (3 de 3).....	78
Figura 49: Notificación recibida en Discord para TEC-009 (1 de 3).....	79
Figura 50: Notificación recibida en Discord para TEC-009 (2 de 3).....	79
Figura 51: Notificación recibida en Discord para TEC-009 (3 de 3).....	80
Figura 52: Artifacts públicos: filas en la campaña TEC y en la campaña benigna	81
Figura 53: Cobertura específica agregada de artifacts públicos y personalizados	83
Figura 54: Comparación de detección por técnica entre artifacts públicos y custom	83
Figura 55: Concentración de señales CLIENT_EVENT por técnica.....	84
Figura 56: Fuentes de telemetría que sostienen la visibilidad host.....	85
Figura 57: Distribución de las 379 filas CLIENT_EVENT por perfil.....	86
Figura 58: Relación entre cobertura y densidad de señales de los perfiles P1-P4	87
Figura 59: Matriz de clasificación a nivel de escenario experimental	88
Figura 60: Duración por repetición y escenario.....	93
Figura 61: Coste operativo observado del cliente Velociraptor por escenario	95
Figura 62: CPU media, percentil 95 y máximo por repetición en VR_TEC_RUNNER	96
Figura 63: Memoria media, percentil 95 y máxima por repetición.....	98
Figura 64: Actividad de disco del sistema por escenario	99
Figura 65: Validación del agente LT29 como agente activo desde agent_control	104
Figura 66: Reactivación de reglas personalizadas TFM y servicio wazuh-manager activo.....	107
Figura 67: Resumen de la campaña Wazuh base, con ausencia de reglas TFM 110xxx.....	110
Figura 68: Cobertura específica comparada sobre nueve técnicas experimentales	113
Figura 69: Resumen de campaña Wazuh custom con reglas TFM activadas.....	114
Figura 70: Resumen de severidades en el dashboard tras la campaña	115
Figura 71: Dashboard de Wazuh filtrado por agent.name LT29	115

Figura 72: Visibilidad de PowerShell 4104 en Wazuh.....	116
Figura 73: Visibilidad de Sysmon Event ID 1, asociado a creación de procesos.....	116
Figura 74: Visibilidad de Sysmon Event ID 11, asociado a creación de ficheros.....	117
Figura 75: Ejemplo de detección custom mediante la regla 110202.....	117
Figura 76: Ausencia de resultados para rule.id 110201 en la campaña custom	117
Figura 77: Vista de alertas de severidad alta/crítica generadas en Wazuh	118
Figura 78: Detección por técnica Velociraptor vs Wazuh	119
Figura 79: Cobertura por técnica wazuh vs VR.....	119
Figura 80: Resultado del análisis de un componente Oracle en VirusTotal.....	129
Figura 81: Máquina virtual del laboratorio desplegada y en ejecución	129
Figura 82: Panel de acceso al dashboard de Wazuh.....	133
Figura 83: Validación de conectividad e instalación del agente Wazuh desde PowerShell	135
Figura 84: Comprobación del servicio WazuhSvc en ejecución en el endpoint Windows	135
Figura 85: CPU media del cliente Velociraptor por escenario de benchmark.....	155
Figura 86: CPU máxima del cliente Velociraptor por escenario de benchmark	156
Figura 87: Memoria RAM media del cliente Velociraptor por escenario de benchmark.....	157
Figura 88: Memoria RAM máxima del cliente Velociraptor por escenario de benchmark.....	158
Figura 89: Duración media de las ejecuciones por escenario de benchmark.....	159
Figura 90: Duración de cada repetición por escenario de benchmark	160
Figura 91: Evidencia de finalización de la campaña global (1 de 2)	162
Figura 92: Evidencia de finalización de la campaña global (2 de 2)	162
Figura 93: Monitorización con Toggl Track	171
Figura 94: Planificación Gantt de los capítulos I y II	179
Figura 95: Planificación Gantt del capítulo III	179
Figura 96: Planificación Gantt del capítulo IV	179
Figura 97: Planificación Gantt del capítulo V.....	180
Figura 98: Planificación Gantt del capítulo VI	180
Figura 99: Planificación Gantt del capítulo VII.....	180
Figura 100: Planificación inicial del capítulo VIII y del bloque de referencias y anexos.....	180

Índice de códigos

Código 1: Ejemplo de consulta VQL	16
Código 2: Ejemplo de obtención de información mediante VQL.....	16
Código 3: Ejemplo de consulta de claves del registro mediante VQL.....	17
Código 4: Ejemplo de evaluación diferida en VQL.....	17
Código 5: Procedimiento de ejecución de TEC-001	41
Código 6: Procedimiento de ejecución de TEC-002.....	42
Código 7: Procedimiento de ejecución de TEC-003.....	43
Código 8: Procedimiento de ejecución de TEC-004.....	44
Código 9: Procedimiento de ejecución de TEC-005.....	45
Código 10: Procedimiento de ejecución de TEC-006 mediante PowerShell.....	46
Código 11: Procedimiento de ejecución de TEC-006 mediante WMIC	46
Código 12: Procedimiento de ejecución de TEC-007.....	47
Código 13: Procedimiento de ejecución de TEC-008.....	48
Código 14: Ejecución del receptor HTTP controlado	50
Código 15: Ejecución del runner de falsos positivos.....	51
Código 16: Fuentes eventchannel configuradas en el agente Windows.....	105
Código 17: Reinicio y comprobación del agente Wazuh en Windows	105
Código 18: Deshabilitación de la primera versión de reglas tras problemas de estabilidad	106
Código 19: Validación sintáctica y reinicio tras cargar reglas custom TFM.....	107
Código 20: Desactivación de reglas custom para la campaña Wazuh base	108
Código 21: Ejecución de runners TEC y falsos positivos desde Windows.....	108
Código 22: Marcado de inicio y líneas iniciales de logs en campaña base.....	109
Código 23: Extracción de delta de archives.json y alerts.json en campaña base.....	109
Código 24: Reactivación de reglas custom TFM para la campaña Wazuh custom.....	111
Código 25: Inicio de campaña Wazuh custom	111
Código 26: Extracción de delta en campaña Wazuh custom.....	112
Código 27: Nombre VR	130
Código 28: Generación interactiva de la configuración de Velociraptor	130
Código 29: Documento_config_VR	130
Código 30: Agente instalación II	131
Código 31: Sysmon.....	131
Código 32: Instalación de Sysmon.....	131
Código 33: Sysmon comprobación.....	131
Código 34: habilitar PowerShell Script Block Logging	132

Código 35: Comprobar_Script_block	132
Código 36: Instalación base de dependencias y despliegue all-in-one de Wazuh.....	133
Código 37: Validación de conectividad desde Windows hacia Wazuh	134
Código 38: Instalación del agente Wazuh en Windows	135
Código 39: Ejecución del runner global de técnicas TEC-001 a TEC-009.....	161

Acrónimos y abreviaturas

ATT&CK	Adversarial Tactics, Techniques and Common Knowledge. Framework de MITRE para clasificar tácticas, técnicas y procedimientos de atacantes.
CPU	Central Processing Unit.
CSV	Comma-Separated Values. Formato tabular basado en texto donde los campos se separan normalmente por comas o punto y coma. Se usa para exportar datos a hojas de cálculo o herramientas de análisis.
DFIR	Digital Forensics and Incident Response.
EDR	Endpoint Detection and Response.
EID	Event Identifier.
FN	Falso Negativo.
FP	Falso Positivo.
GUI	Graphical User Interface. Interfaz gráfica de usuario.
HIDS	Host-based Intrusion Detection System. Sistema de detección de intrusiones basado en host.
HTTPS	HyperText Transfer Protocol Secure. Versión segura de HTTP que utiliza TLS para cifrar la comunicación entre cliente y servidor.
I/O	Input/Output.
IOA	Indicator of Attack.
IOC	Indicator of Compromise.
JSON	JavaScript Object Notation. Formato de intercambio de datos basado en texto, legible por humanos y ampliamente usado en APIs, configuraciones y exportaciones.
JSONL	JSON Lines. Formato en el que cada línea contiene un objeto JSON independiente. Es útil para almacenar y procesar grandes volúmenes de eventos o registros.
CH	Critical/High
CHM	Critical/High/Medium
P1-P4	Perfiles personalizados de detección de Velociraptor empleados en el TFM, ordenados según prioridad y función operativa.
ETW	Event Tracing for Windows

HTTP	Hypertext Transfer Protocol
NIDS	Network-based Intrusion Detection System
TTP	Tactics, Techniques and Procedures
VM	Virtual Machine
MSI	Microsoft Installer. Formato de paquete de instalación utilizado en Windows para desplegar software, servicios o agentes de forma estructurada.
OS / SO	Operating System / Sistema Operativo. Software base del sistema.
PID	Process Identifier. Identificador único de un proceso en ejecución.
PowerShell 4104	Event ID 4104 de PowerShell, asociado al Script Block Logging. Registra bloques de código PowerShell ejecutados, siendo útil para detectar scripts sospechosos, comandos ofuscados o actividad fileless.
Protobuf	Protocol Buffers. Formato binario de serialización de datos desarrollado por Google, usado para estructurar mensajes de forma compacta y eficiente entre sistemas.
RAM	Random Access Memory.
SIEM	Security Information and Event Management.
Sigma	Lenguaje abierto y genérico para describir reglas de detección de amenazas en logs. Permite expresar patrones de detección reutilizables y adaptables a distintas plataformas SIEM o herramientas de análisis.
SOC	Security Operations Center.
Sysmon	System Monitor. Herramienta de Microsoft Sysinternals para generar telemetría avanzada de Windows.
Sysmon Event ID	Identificador numérico de evento generado por Sysmon. Cada Event ID representa un tipo concreto de actividad del sistema, como creación de procesos, conexiones de red, creación de ficheros o modificaciones del registro.
TEC-00X	Identificador interno usado en el TFM para cada técnica evaluada.
TLS	Transport Layer Security. Protocolo criptográfico utilizado para proteger comunicaciones en red mediante cifrado, autenticación e integridad de los datos transmitidos.
Txxxx	Identificador de técnica MITRE ATT&CK.
VN	Verdadero Negativo.
VP	Verdadero Positivo.

VQL	Velociraptor Query Language.
VR	Velociraptor.
XDR	Extended Detection and Response.
YAML	YAML Ain't Markup Language. Formato de serialización legible empleado para definir artifacts y configuraciones.

Capítulo I: Introducción

Este capítulo presenta el marco general en el que se desarrolla el trabajo y establece las bases necesarias para comprender el problema. En primer lugar, se expone la motivación del estudio y la descripción del problema, analizando la relevancia de las capacidades de detección en ciberseguridad y el papel de los sistemas HIDS en entornos actuales. A continuación, se definen los objetivos del proyecto, especificando el alcance del análisis y las metas que se persiguen con la evaluación de Velociraptor. Finalmente, se presenta la planificación, estimación temporal y organización general del proyecto.

1.1. Motivación y descripción del problema

Tradicionalmente, las organizaciones han abordado la seguridad desde una perspectiva principalmente preventiva, asumiendo que la inversión en soluciones de protección o personal especializado permitiría evitar estos ciberataques. Sin embargo, la evolución del panorama ha demostrado que incluso las organizaciones con más inversión en ciberseguridad pueden ser comprometidas. Informes recientes muestran que las intrusiones en infraestructuras corporativas continúan aumentando y que los atacantes reducen cada vez más el tiempo necesario para moverse dentro de estas. Según el Data Breach Investigations Report 2025, el 60% [1] de las brechas analizadas involucraron el factor humano o el abuso de credenciales, lo que pone de manifiesto que los mecanismos de prevención tradicionales no siempre son suficientes para evitar el acceso a los sistemas.

Ante esta situación, el modelo defensivo ha evolucionado hacia enfoques en los que se asume que el compromiso puede llegar a producirse, por lo que la prioridad pasa a ser la detección temprana y la respuesta. En este contexto, las tecnologías de monitorización del endpoint, como los HIDS y las soluciones EDR, permiten observar eventos críticos del sistema, como la creación de procesos, la ejecución de scripts o la modificación de configuraciones críticas.

En este marco, resulta de interés analizar el papel de Velociraptor como herramienta de detección basada en host. Aunque su uso está consolidado en entornos de DFIR, su arquitectura basada en artifacts y sobre todo su modo de interactuar con los agentes permite también implementar mecanismos de monitorización continua. Por ello, este trabajo se centra en evaluar su capacidad para detectar técnicas MITRE ATT&CK a nivel de host, mediante artifacts nativos y personalizados, y en analizar el impacto de dicha monitorización sobre el rendimiento del sistema.

1.2. Objetivos del proyecto

El objetivo principal de este Trabajo Fin de Máster es evaluar la capacidad de Velociraptor para detectar un subconjunto acotado de técnicas MITRE ATT&CK a nivel de host, analizando su posible uso como sistema HIDS en un entorno de laboratorio controlado. Para ello, el estudio se centra en comprobar qué capacidad de detección ofrece la herramienta mediante el uso de artifacts nativos y cuando resulte necesario, mediante la personalización de artifacts.

A partir de este objetivo general, el trabajo plantea las siguientes cuestiones:

- RQ1: ¿Qué cobertura de detección ofrece Velociraptor, mediante artifacts CLIENT_EVENT personalizados, sobre el subconjunto TEC-001 a TEC-009 evaluado en Windows?
- RQ2: ¿Qué capacidades de visibilidad y detección específica aportan los artifacts públicos evaluados, y qué limitaciones presentan en el subconjunto experimental?
- RQ3: ¿Qué impacto en CPU, memoria y duración introduce el cliente Velociraptor en el equipo?
- RQ4: ¿Cómo responde la arquitectura personalizada P1–P4 a las brechas de detección observadas en los artifacts públicos sin confundir detección, contexto y evidencia forense?
- RQ5: ¿Qué señales definitivas generan los artifacts personalizados ante los diez escenarios benignos controlados y qué limitaciones tiene esta comprobación?
- RQ6: ¿Qué aporta Velociraptor frente a soluciones HIDS ampliamente extendidas como Wazuh?

La comparación prevista inicialmente con FortiEDR se sustituyó por Wazuh debido a problemas con la licencia de FortiEDR. Este cambio no altera el objetivo comparativo y no implica equivalencia directa entre las unidades de ambos sistemas.

1.3. Estimación y plazos

El desarrollo del proyecto se estructuró en distintas fases, incluyendo el estudio previo, la preparación del entorno experimental, el desarrollo y ejecución de las pruebas, el análisis de resultados y la elaboración de la memoria.

La planificación inicial contemplaba una dedicación de 150 horas, aunque la ejecución real alcanzó las 290 horas debido principalmente a la ampliación del alcance experimental. La planificación temporal detallada, junto con las estimaciones iniciales, horas reales y diagramas de Gantt, puede consultarse en el Anexo H: Planificación del proyecto ampliada. De forma general, la Figura 1 resume la planificación inicial.

Evaluación de Velociraptor como HIDS para la detección de técnicas MITRE ATT&CK Enterprise

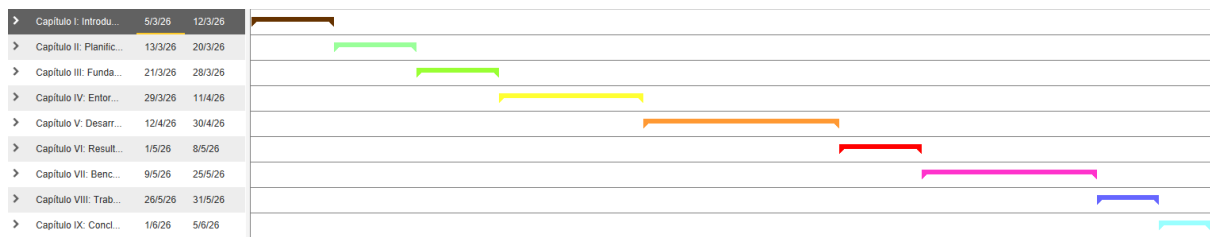


Figura 1: Diagrama de Gantt general

Capítulo II: Fundamentos teóricos

En este capítulo se establecen los fundamentos técnicos necesarios para evaluar la viabilidad de Velociraptor como plataforma HIDS/EDR en endpoints Windows. En primer lugar, se revisan los enfoques de detección y análisis en endpoint (HIDS tradicionales, DFIR y EDR) para situar correctamente el papel de Velociraptor. A continuación, se describe su arquitectura cliente-servidor, el modelo de ejecución basado en artifacts, el lenguaje Velociraptor Query Language (VQL), las fuentes de telemetría host-based disponibles y las limitaciones inherentes a la visibilidad en sistemas basados en host.

2.1. Detección en endpoint: HIDS, EDR y XDR

Para empezar introduciendo las herramientas de visibilidad la primera que se va a analizar es el HIDS (Host Intrusion Detection System). Este es un sistema de seguridad diseñado para monitorizar la actividad de un host con el objetivo de detectar comportamientos potencialmente maliciosos. Para ello analiza eventos generados por el sistema operativo, como la creación de procesos, modificaciones en archivos del sistema, cambios en configuraciones críticas o registros de actividad. Su función principal es identificar indicadores de compromiso o comportamientos anómalos dentro del propio endpoint, sin necesitar acceso a internet. En el caso de basarse en comportamientos de red, estaríamos hablando entonces de un NIDS (Network Intrusion Detection System), aunque hay soluciones que mezclan ambos conceptos.

Las soluciones Endpoint Detection and Response (EDR) amplían el enfoque de los HIDS tradicionales incorporando capacidades adicionales de recopilación continua de telemetría, análisis de comportamiento y sobre todo respuesta ante incidentes. Mientras que un HIDS se centra en la detección de eventos sospechosos en un host, las plataformas EDR permiten correlacionar eventos, investigar incidentes y aplicar acciones de respuesta sobre los endpoints comprometidos.

Por su parte, las plataformas Extended Detection and Response (XDR) extienden este modelo integrando información procedente de múltiples fuentes de seguridad, como otros endpoints, red, correo electrónico o servicios en la nube. Su objetivo es correlacionar eventos entre distintos sistemas para mejorar la visibilidad global del entorno y facilitar la detección de ataques que se desarrollan a través de múltiples vectores.

2.1.1. Detección en HIDS tradicionales

Un HIDS[7] o sistema de detección de intrusiones basado en host monitoriza las características de un único equipo y los eventos que se producen dentro de ese host para identificar actividad

sospechosa o potencialmente maliciosa. Su ámbito natural es local: procesos, eventos del sistema, integridad de ficheros, registros, autenticaciones, servicios y cambios de configuración.

En términos generales, los HIDS tradicionales se basan en reglas, firmas o patrones de eventos. Su principal ventaja es que observan el resultado de la actividad directamente en el host, incluso cuando esta no es visible desde una sonda de red. Sin embargo, su eficacia, al igual que la del DFIR, como veremos más adelante, depende de la calidad de las reglas y de la configuración de logs y eventos del sistema operativo.

Fortalezas	visibilidad local, reglas explícitas, bajo coste relativo y utilidad para auditoría
Limitaciones	dependencia de logs habilitados, reglas rígidas, dificultad para contextualizar comportamientos legítimos (riesgo de ruido operativo)
Relación con este TFM	proporcionan el marco conceptual de detección host-based

Tabla 1: Funciones, limitaciones y riesgos del enfoque HIDS

2.1.2. Detección en DFIR

El enfoque DFIR [8] se centra en la adquisición, preservación, análisis e interpretación de evidencias sobre todo después del incidente, a diferencia de un HIDS, cuyo objetivo principal es detectar actividad durante su ejecución.

Velociraptor encaja de forma natural en este ámbito porque permite ejecutar consultas distribuidas sobre endpoints, recoger evidencias concretas y realizar análisis remoto de forma selectiva. No obstante, la capacidad DFIR no implica automáticamente capacidad HIDS. Para aproximarse a un funcionamiento HIDS es necesario que existan artifacts de monitorización continua, filtros de detección, normalización de resultados y sobre todo, mecanismos de alertado.

Fortalezas	profundidad analítica, adquisición selectiva, reconstrucción de forma distribuida
Limitaciones	dependencia de evidencias que permanezcan y de configuración de eventos del host
Relación con este TFM	Velociraptor es un DFIR con capacidad potencial de operar como HIDS

Tabla 2: Funciones, limitaciones y riesgos del enfoque DFIR

2.1.3. Detección en EDR

Las soluciones EDR [9], Endpoint Detection and Response, monitorizan endpoints de forma continua para detectar indicios de amenaza y facilitar acciones de investigación, contención o respuesta. Suelen incorporar telemetría persistente, reglas mantenidas por el proveedor, analítica de

comportamiento, scoring y lo más relevante, acciones de respuesta. A diferencia de un EDR, Velociraptor no tiene por defecto priorización de alertas, y todavía menos, capacidad de respuesta.

Fortalezas	monitorización continua del endpoint, correlación de eventos, analítica de comportamiento, priorización de alertas y capacidades de respuesta
Limitaciones	dependencia del proveedor, menor transparencia interna de las reglas, coste económico superior, posible efecto “caja negra”
Relación con este TFM	proporcionan el marco conceptual de referencia para comparar qué capacidades se esperan de una plataforma moderna de detección en endpoint

Tabla 3: Funciones, limitaciones y riesgos del enfoque EDR

2.1.4. MITRE ATT&CK como marco de referencia

MITRE ATT&CK (Adversarial Tactics, Techniques and Common Knowledge) es un framework que recopila y organiza tácticas y técnicas utilizadas por adversarios. La matriz ATT&CK Enterprise se centra en entornos corporativos y describe comportamientos asociados a distintas fases de un ataque, como ejecución, persistencia, discovery, exfiltración, etc. [2]

En el proyecto se utiliza ATT&CK como marco de referencia para seleccionar y describir técnicas debido a su uso ampliamente distribuido en el mercado y al tratarse de una de las bases del mismo.

2.1.5. Criterios de evaluación de herramientas de detección

Para la evaluación de las dos herramientas de detección utilizadas durante el proyecto se consideraron los siguientes criterios:

1. Cobertura de detección: capacidad de la herramienta para identificar las técnicas MITRE ATT&CK ejecutadas durante las pruebas. Se medirá como tasa de detección por técnica, indicando si la actividad generada produce o no evidencia observable mediante los artifacts utilizados.
2. Visibilidad del sistema / falsos negativos: análisis de los casos en los que una técnica se ejecuta correctamente pero no genera información detectable a través de los artifacts evaluados. Este criterio permite identificar limitaciones de visibilidad o dependencias de fuentes de datos concretas.
3. Impacto en rendimiento: evaluación del coste que introduce la monitorización mediante Velociraptor sobre el endpoint. Esta parte se expandirá en el capítulo de benchmark, pero a rasgos generales se analizarán métricas como CPU (%), uso de memoria RAM y actividad de disco (I/O).

4. Complejidad operativa: análisis del esfuerzo necesario para desplegar y utilizar las capacidades de detección. Se evaluará principalmente:

- Si es necesario o no modificar artifacts existentes
- Desarrollo de artifacts/YAMLs personalizados
- Ajustes necesarios en la configuración del agente o del servidor.

Estos criterios permiten justificar las métricas utilizadas para evaluar el comportamiento de Velociraptor como posible sistema de detección basado en host.

2.2. Velociraptor

La protección y análisis de endpoints puede abordarse desde varios enfoques complementarios. Aunque todos trabajan sobre sistemas finales, no tienen el mismo objetivo ni operan igual en profundidad o capacidad de respuesta. Como se ha comentado en la introducción, se diferencian cuatro aproximaciones: HIDS tradicionales, DFIR, EDR y Velociraptor.

Las Tablas 1 a 3 y 5 comparan las funciones, fortalezas y limitaciones de HIDS, DFIR y EDR, y sitúan a Velociraptor frente a estos enfoques.

2.2.1. Naturaleza, capacidades y uso habitual

Velociraptor es una plataforma de código abierto orientada a Digital Forensics and Incident Response (DFIR) que permite la recopilación remota y el análisis de información procedente de múltiples endpoints mediante una arquitectura cliente-servidor. Su diseño permite ejecutar consultas sobre sistemas monitorizados para obtener evidencias, analizar actividad sospechosa o investigar incidentes de seguridad. [3]

El funcionamiento de la herramienta se basa en artifacts, que empaquetan consultas VQL y configuración en ficheros YAML. VQL permite interactuar con fuentes del endpoint como procesos, archivos, registro, eventos del sistema operativo, Sysmon o PowerShell. Los artifacts pueden emplearse para consultas puntuales, análisis histórico o monitorización continua cuando se diseñan como event artifacts. [6]

Aunque Velociraptor se ha diseñado principalmente como herramienta DFIR, su arquitectura distribuida y su diseño permiten implementar monitorización continua mediante colas de eventos y server monitoring. Esa capacidad abre la posibilidad de aproximarlos a una herramienta de detección basada en host, siempre que exista una lógica de detección explícita encima de la telemetría.

En función de su comportamiento y de la fuente de información utilizada, para este proyecto, los artifacts pueden clasificarse en tres tipos principales:

La Tabla 4 resume los tipos de artifacts según su modo de ejecución y su objetivo operativo [3], [5], [6].

Tipo	Contexto	Funcionamiento
CLIENT	Endpoint	Recogida puntual de logs
CLIENT_EVENT	Endpoint	Monitorización continua y envío de eventos en tiempo real
SERVER_EVENT	Servidor	Procesamiento continuo de colas de eventos y correlación para salida externa

Tabla 4: Tipos de artifacts según su modo de ejecución y objetivo

En esta memoria, los artifacts de tipo CLIENT_EVENT son los más relevantes para aproximar monitorización HIDS, porque ejecutan la lógica de observación en el endpoint y permiten clasificar eventos host-based de forma continua. Esta clasificación no convierte automáticamente cualquier evento visible en alerta; exige reglas, contexto y evidencia trazable que se estipulan mediante artifacts.

2.2.2. Posición de Velociraptor y comparación funcional con otras soluciones

Las Tablas 1 a 3 permiten diferenciar conceptualmente los enfoques HIDS, DFIR y EDR. Sin embargo, para situar Velociraptor frente a soluciones reales resulta necesario comparar también plataformas concretas con capacidades de monitorización, detección o respuesta sobre endpoints.

Velociraptor se sitúa en una posición intermedia entre estos enfoques. Su documentación lo define como una plataforma open-source para monitorización de endpoints, análisis forense digital y respuesta ante incidentes. Su principal elemento diferenciador es VQL (Velociraptor Query Language), que permite desarrollar y modificar artifacts para recoger información, realizar hunts, ejecutar consultas y mantener monitorización continua sobre los endpoints.

Para contextualizar funcionalmente la herramienta utilizada en este trabajo, se compara con tres soluciones representativas de enfoques diferentes: Wazuh, CrowdStrike Falcon y GRR Rapid Response. La comparación es exclusivamente funcional y se basa en las capacidades documentadas de cada plataforma; no constituye un benchmark experimental entre las cuatro herramientas. La comparación experimental desarrollada en este TFM se limita posteriormente a Velociraptor y Wazuh.

Herramienta	Enfoque principal	Monitorización y detección	DFIR / Hunting	Respuesta
Velociraptor	DFIR y monitorización de endpoints	Monitorización continua mediante event artifacts y VQL	Alta: adquisición, consultas y hunts distribuidos	Automatización y acciones configurables
Wazuh	HIDS / XDR / SIEM	Agente, reglas, decoders, análisis de logs y generación de alertas	Investigación basada en telemetría y eventos centralizados	Active Response mediante scripts asociados a reglas
CrowdStrike Falcon Insight XDR	EDR / XDR comercial	Detección continua basada en telemetría de endpoint, analítica e inteligencia de amenazas	Investigación de incidentes y threat hunting	Alta: Real Time Response y automatización mediante SOAR
GRR Rapid Response	DFIR / respuesta a incidentes	No orientado principalmente a detección HIDS continua	Alta: adquisición remota, triage y hunts a gran escala	Acciones y flows remotos sobre endpoints

Tabla 5: Comparación funcional de Velociraptor con otras soluciones de seguridad orientadas al endpoint

Wazuh representa el enfoque más próximo a un HIDS/XDR tradicional dentro de las herramientas consideradas. Su agente recoge información de los endpoints que puede ser procesada mediante reglas y decoders, mientras que el módulo Active Response permite ejecutar acciones automáticas cuando se producen determinados eventos o alertas.

CrowdStrike Falcon representa, por su parte, el enfoque de un EDR/XDR comercial. Falcon Insight XDR integra capacidades de detección y respuesta sobre endpoints con información procedente de otros dominios, incorpora investigación de amenazas y proporciona mecanismos de respuesta directa y automatizada. A diferencia de Velociraptor, su funcionamiento se basa en una plataforma propietaria orientada a proporcionar detecciones y capacidades de respuesta integradas como producto.

GRR Rapid Response constituye una referencia más próxima al ámbito DFIR. Se trata de un framework open-source orientado a remote live forensics, diseñado para realizar adquisición, triage e investigación de sistemas de forma remota y escalable. Aunque permite efectuar hunts sobre grandes conjuntos de endpoints, su propósito principal no es proporcionar un sistema HIDS de detección continua.

La comparación muestra que Velociraptor comparte con GRR una fuerte orientación hacia DFIR y la investigación distribuida, mientras que incorpora además mecanismos de monitorización continua que permiten aproximarlos al ámbito HIDS. Frente a Wazuh, ofrece una mayor libertad para definir mediante VQL exactamente qué información se recoge y cómo se procesa, mientras que Wazuh dispone de un modelo más directamente orientado a reglas, alertas centralizadas y respuestas asociadas. CrowdStrike Falcon representa un enfoque distinto, basado en una solución EDR/XDR comercial que integra de forma nativa detección, investigación y respuesta.

Por tanto, el interés de Velociraptor en este TFM no reside en considerarlo un sustituto directo de un EDR comercial ni en asumir que sea funcionalmente superior a las demás soluciones, sino en evaluar hasta qué punto una plataforma originalmente orientada a DFIR y altamente personalizable puede incorporar mediante artifacts y VQL capacidades de detección continua propias de un HIDS. Esta cuestión constituye precisamente uno de los objetivos experimentales desarrollados en los capítulos posteriores.

2.2.3. Arquitectura interna Velociraptor

Velociraptor utiliza una arquitectura cliente-servidor. El analista interactúa con la interfaz gráfica o mediante la API del servidor por consola, selecciona artifacts, define parámetros y lanza colecciones, hunts o monitorización continua. El servidor coordina la ejecución, pero la lógica de consulta se ejecuta en el endpoint, donde el cliente accede a las fuentes locales de telemetría. El modo de ejecución de Velociraptor se va a desarrollar más adelante, pues es una de las principales fortalezas de VR sobre otras soluciones. [3], [10], [11]

En la Figura 2 se ha resumido el flujo funcional desde la intervención del analista hasta la devolución de resultados desde los endpoints. Para ello se ha realizado un esquema usando la aplicación web draw.io¹.

¹ draw.io (diagrams.net): herramienta gratuita para crear diagramas y esquemas visuales.

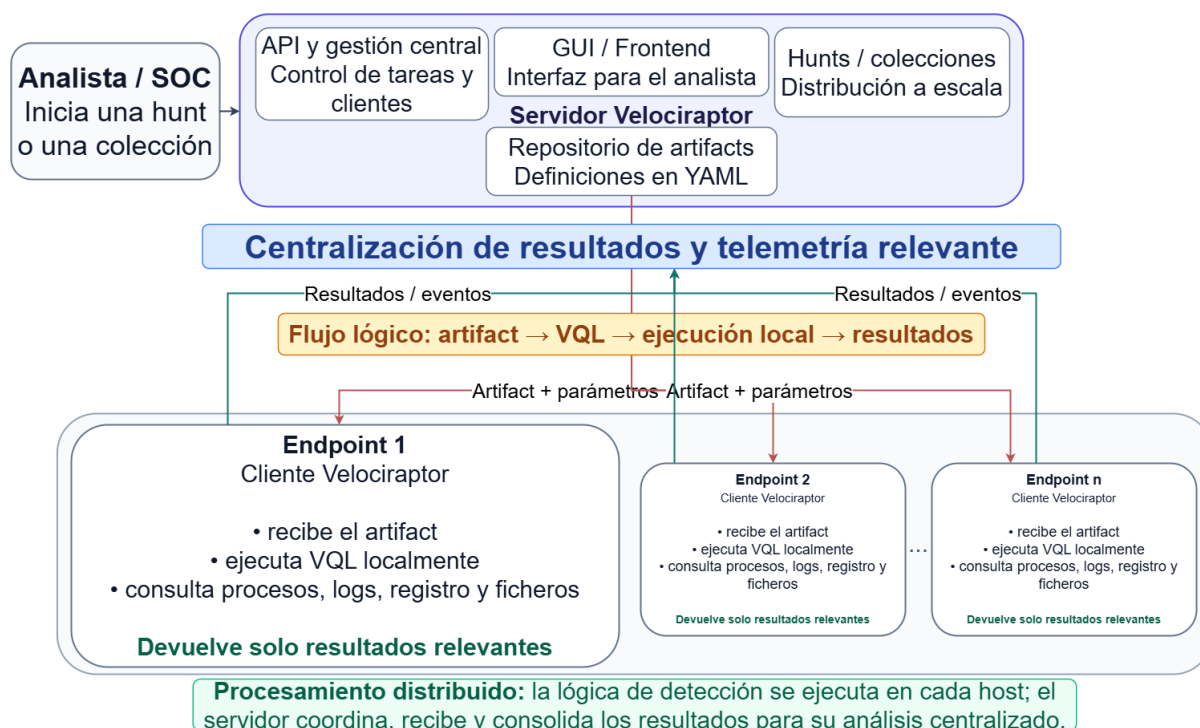


Figura 2: Arquitectura interna básica VR

2.2.3.1. Diferencias entre capas de Velociraptor

Para comprender el funcionamiento de Velociraptor como herramienta HIDS/DFIR, es necesario distinguir sus principales capas funcionales.

- Client: agente instalado en el endpoint. Proporciona visibilidad directa sobre el host y accede a procesos, ficheros, registro, eventos de Windows, Sysmon, etc.
- Server: consola y backend central. Gestiona clientes, colecciones, hunts, mantiene la configuración de monitoring, recibe resultados y almacena evidencias.
- Artifacts: Definidos en YAML y VQL, determinan qué se recoge y cómo se filtra.
- Modos de ejecución: collections, hunts y monitoring. Definen el rango de la ejecución.

Esta separación es relevante para el TFM porque permite diferenciar tres conceptos: visibilidad, detección/alertabilidad y salida externa. Que Velociraptor observe un evento no implica necesariamente que lo clasifique como una técnica MITRE, que genere una alerta operativa, ni que esta sea enviada a un canal externo. Para este proyecto necesitaremos que VR sepa diferenciar entre monitorización y alerta y entre evento maligno y benigno lo máximo posible.

2.2.3.2. Flujo cliente-servidor y formatos de datos

Esta es una de las fortalezas más relevantes de Velociraptor. Este no envía los resultados como JSON plano entre cliente y servidor. Internamente, la comunicación cliente-servidor se realiza mediante

mensajes Protobuf² cifrados, transportados sobre HTTPS POST. Esto reduce el volumen de datos transmitidos, mejora ampliamente el rendimiento y dificulta la lectura directa del tráfico. Posteriormente, el servidor representa los resultados como tablas o permite exportarlos en formatos como JSON, JSONL, CSV o ficheros descargables.

Conviene distinguir entre formato lógico, formato de transporte y formato de explotación analítica. La Tabla 6 resume los formatos empleados en cada fase y la Figura 3 representa el flujo de transporte de datos entre cliente y servidor.

Fase	Formato	Función
Definición del artifact	YAML	Describe el artifact, parámetros, sources y metadatos
Lógica de consulta	VQL	Define qué fuente se consulta, qué campos se devuelven y qué filtros se aplican
Transporte interno	Protobuf	Agrupar mensajes cliente-servidor
Canal	HTTPS POST	Transporta los MessageList entre cliente y servidor
Representación	tablas	Permite visualizar en filas, columnas desde la GUI
Exportación	JSON, CSV, ZIP	Facilita análisis externo, anexos o tratamiento
Salida externa	JSON sobre HTTP POST	Permite enviar alertas a herramientas como webhooks o SIEMs desde SERVER_EVENT

Tabla 6: Formatos de datos en el flujo cliente-servidor de Velociraptor

Para representar el flujo de transporte de datos se ha creado un diagrama:

² Protobuf (Protocol Buffers): Es un formato binario de serialización desarrollado por Google que permite estructurar y transmitir datos de forma compacta y eficiente.

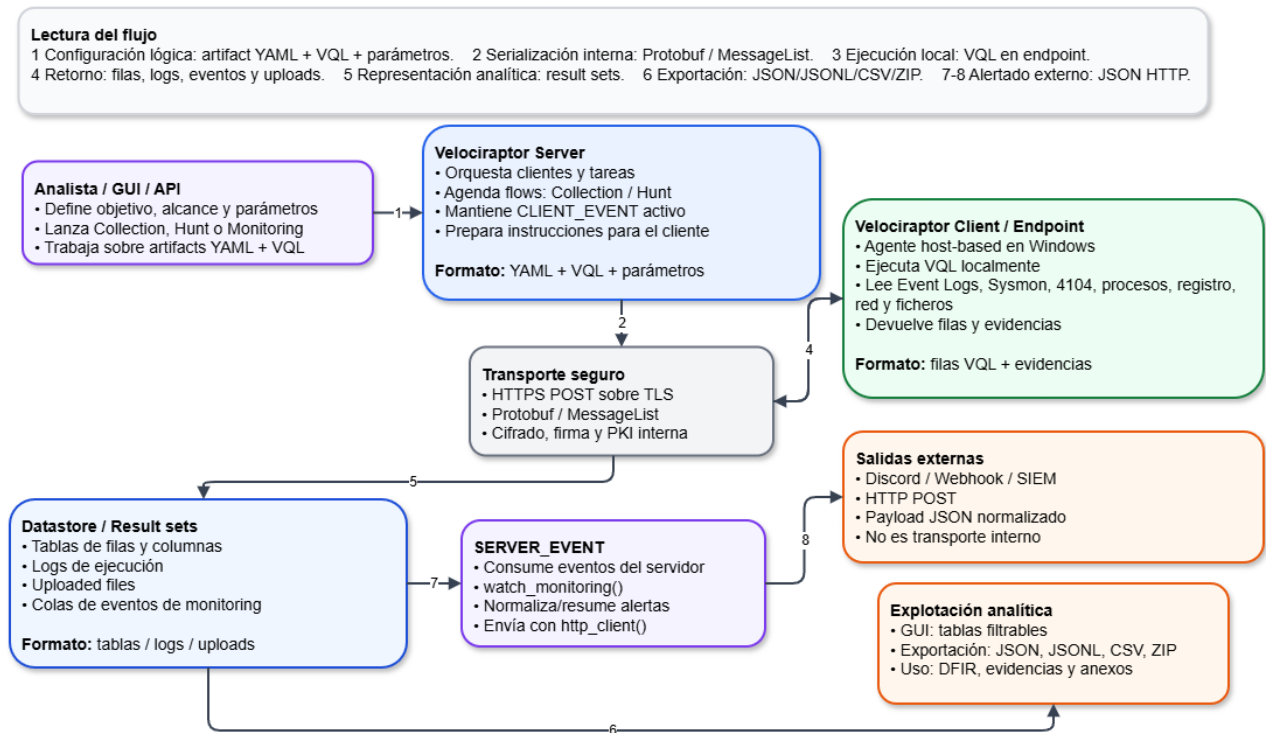


Figura 3: Flujo de datos de Velociraptor

2.2.4. Artifacts y modelo de ejecución

Como se ha explicado anteriormente, Velociraptor utiliza un modelo de ejecución distribuido. En lugar de enviar todos los datos del endpoint al servidor para analizarlos allí, la herramienta envía al propio equipo las consultas VQL que debe ejecutar. De esta forma, el cliente procesa la información localmente y devuelve únicamente los resultados relevantes.

Las Tablas 7 a 9 describen los campos de un artifact, sus contextos de ejecución y las diferencias entre collections, hunts, monitoring y SERVER_EVENT.

Este funcionamiento se apoya en la evaluación diferida o lazy evaluation, que evita procesar datos hasta que realmente son necesarios. Junto con el uso de Protobuf para transmitir la información de forma compacta, representan la mayor ventaja de esta herramienta sobre otras, ya que este enfoque reduce el consumo de recursos y de ancho de banda, facilitando el despliegue de Velociraptor en entornos con un gran número de equipos.

2.2.4.1. Qué es un artifact

Ya se ha introducido el término artifact, pero se debe ampliar. Un artifact, en resumen, es la unidad lógica principal de Velociraptor. Permite empaquetar una o varias consultas VQL junto con su configuración en un fichero YAML estructurado. Gracias a este modelo, el analista no necesita ejecutar manualmente consultas VQL aisladas, sino que puede seleccionar artifacts reutilizables, parametrizables y gestionados desde el servidor.

La documentación oficial [12] indica además que cada source genera una tabla de resultados y que, si hay varias sources, deben tener nombres distintos para evitar ambigüedad en la salida.

Los elementos principales de un artifact son:

Campo	Función
Name	Identificador único del artifact
description	Descripción funcional y buscable desde la interfaz
Type	Contexto de ejecución: CLIENT_EVENT, SERVER, SERVER_EVENT...
parameters	Variables configurables por el analista
sources	Bloques de ejecución VQL que generan resultados
precondition	Condición opcional para decidir si debe ejecutarse
Query	Consulta VQL que realiza la recogida, filtrado o detección

Tabla 7: Campos principales de definición de artifacts

En el contexto del TFM, los artifacts públicos se consideran una línea base de visibilidad y detección parcial. Los artifacts personalizados permiten adaptar la lógica al entorno, normalizar campos, reducir falsos positivos y clasificar señales según técnicas MITRE seleccionadas.

2.2.4.2. Tipos de artifacts según el contexto de ejecución

Velociraptor diferencia los artifacts según el lugar y el modo en que se ejecutan. Esta clasificación es importante porque no todos tienen valor HIDS directo.

Tipo	Dónde actúa	Uso principal
CLIENT	Endpoint	Recogida puntual de evidencias
CLIENT_EVENT	Endpoint	Monitorización continua del host
SERVER	Servidor	Tareas administrativas o consultas del servidor
SERVER_EVENT	Servidor	Procesamiento de eventos, alertado y automatización
INTERNAL	Servidor	Colas internas del sistema

Tabla 8: Tipos de artifacts según el contexto de ejecución

Solamente nos interesa el tipo CLIENT_EVENT para el uso como HIDS. También es relevante SERVER_EVENT para la visibilidad de alertas fuera del sistema usando webhooks.

Según la documentación oficial [13], tanto los artifacts CLIENT_EVENT como los SERVER_EVENT establecen colas de eventos en el servidor, identificadas mediante el nombre del artifact. En los CLIENT_EVENT, el VQL definido en las sources se envía y ejecuta en los endpoints

monitorizados, y las filas generadas se transmiten casi en tiempo real a la cola correspondiente del servidor. Por su parte, los SERVER_EVENT ejecutan su lógica en el servidor y se utilizan normalmente para consumir y procesar eventos recibidos en las colas, permitiendo aplicar tareas de filtrado, normalización, correlación, persistencia o salida externa.

2.2.4.3. Collections, Hunts y Monitoring

Los artifacts pueden ejecutarse mediante distintos modos: [3], [5], [6]

Modo	Alcance	Temporalidad	Uso adecuado
Collection	Uno o varios endpoints seleccionados	Puntual; finaliza al devolver resultados.	Investigación concreta, adquisición de evidencias, validación de hipótesis.
Hunt	Múltiples endpoints	Puntual o programado; orientado a escala.	Threat hunting, búsqueda de hashes, procesos, ficheros o indicadores.
CLIENT_EVENT	Endpoints con la configuración de monitoring aplicada	Continuo; permanece activo.	Detección HIDS en tiempo casi real y generación de eventos.
SERVER_EVENT	Servidor	Continuo sobre colas de eventos.	Normalización, correlación ligera, alertado y salida externa.

Tabla 9: Diferencias entre collections, hunts, monitoring y SERVER_EVENT

2.2.5. Velociraptor Query Language (VQL)

Como se ha podido deducir por las explicaciones previas, el Velociraptor Query Language (VQL) [14] es el lenguaje utilizado por Velociraptor para consultar y procesar información procedente de los endpoints. Su sintaxis se inspira en SQL, pero su funcionamiento se diferencia principalmente en que en lugar de consultar tablas persistentes, VQL opera sobre plugins, que generan dinámicamente información del sistema en forma de filas. [4]

2.2.5.1. Estructura de una consulta VQL

Como se acaba de comentar en el punto previo, las consultas VQL siguen una estructura similar a las consultas SQL, basada principalmente en las cláusulas SELECT y FROM. La cláusula FROM especifica el plugin que actuará como fuente de datos, mientras que SELECT define qué campos o expresiones se devolverán como resultado.

Una estructura de ejemplo sería la siguiente:

```
$ SELECT columna1, columna2
$ FROM plugin(parámetros)
$ WHERE condicion
```

Código 1: Ejemplo de consulta VQL

En este modelo, el plugin indicado en la cláusula FROM genera filas de datos que posteriormente pueden filtrarse, transformarse o procesarse mediante condiciones y expresiones dentro de la consulta.

Un ejemplo más realista, para obtener la información de un endpoint sería algo así:

```
$ SELECT Name, Pid
$ FROM pslist()
```

Código 2: Ejemplo de obtención de información mediante VQL

Esta consulta utiliza el plugin pslist() para obtener los procesos activos del sistema y devuelve el nombre del proceso junto con su identificador.

Otro ejemplo para consulta de claves de persistencia en el registro:

```
$ SELECT Key, Value
$ FROM registry(key=
"HKEY_LOCAL_MACHINE\\Software\\Microsoft\\Windows\\CurrentVersion\\Run")
```

Código 3: Ejemplo de consulta de claves del registro mediante VQL

2.2.5.2. Query Engine, plugins y lazy evaluation

El Query Engine es el motor que procesa las consultas VQL. Su funcionamiento básico consiste en llamar al plugin indicado en la cláusula FROM, recibir filas generadas por dicho plugin, aplicar filtros y emitir únicamente las filas que cumplen los criterios definidos. La documentación describe que los plugins VQL no son tablas estáticas en disco, sino componentes que producen filas dinámicamente.

Internamente, el funcionamiento básico es:

- El plugin indicado en FROM genera una o varias filas.
- Cada fila se evalúa frente a filtros y condiciones.
- Las columnas del SELECT se calculan cuando son necesarias.
- El resultado final se devuelve como filas estructuradas.

Como se ha comentado antes, la evaluación diferida (lazy evaluation) es una característica fundamental de VR, y esta viene dada por el motor de consultas (Query Engine). Durante la ejecución, el motor recibe las filas generadas por los plugins y retrasa el cálculo de las expresiones hasta que su resultado es realmente necesario, por ejemplo, para evaluar un filtro o mostrar una columna. De esta forma, evita ejecutar operaciones costosas sobre filas que posteriormente serán descartadas. En la documentación oficial [14] se deja claro que este comportamiento es esencial para la eficiencia de VQL.

Un ejemplo del funcionamiento del lazy evaluation:

```
$ SELECT Name, hash(path=Exe)
$ FROM pslist()
$ WHERE Name = "powershell.exe"
```

Código 4: Ejemplo de evaluación diferida en VQL

En el ejemplo anterior, pslist() genera filas de procesos. El filtro WHERE limita el resultado a powershell.exe y solo para esas filas se calcula el hash del ejecutable. Sin lazy evaluation, el motor tendría que calcular el hash de todos los procesos antes de descartar los que no coinciden, lo que incrementaría operaciones de disco y consumo de CPU.

2.2.5.3. Event queries y monitorización continua

Las consultas VQL convencionales devuelven un conjunto de resultados y finalizan. Las event queries [15], en cambio, se apoyan en event plugins que no terminan por sí mismos: permanecen esperando sucesos y emiten filas cuando el evento ocurre. La documentación oficial indica que estas consultas devuelven resultados parciales hasta que son canceladas.

En la arquitectura de client monitoring [16], el cliente mantiene un conjunto de event queries en paralelo dentro de su tabla de monitorización. Estas consultas pueden ejecutarse de forma autónoma en el endpoint, incluso aunque el servidor solo sea necesario para sincronizar cambios de configuración. Cuando producen filas, el cliente las envía al servidor, que las escribe en el datastore.

Este modelo es el más relevante para una aproximación HIDS, ya que permite capturar actividad efímera que podría perderse con una collection puntual. No obstante, los eventos no se remiten necesariamente de forma instantánea: Velociraptor puede agrupar filas en memoria y enviarlas por lotes para evitar comunicaciones demasiado frecuentes. Por ello, en el TFM, cuando se hable de tiempo real, realmente será tiempo casi real, no de tiempo real estricto, al igual que podría pasar en un SIEM cuando correlaciona los datos cada x tiempo.

2.2.6. Telemetría disponible en el endpoint

Como se ha comentado con anterioridad, la capacidad de detección de Velociraptor depende de la telemetría disponible en el endpoint. Esta información procede de distintas fuentes, lo cual es clave para saber dónde consultar con los artifacts. La instrumentación utilizada se apoyó en Sysmon y en PowerShell Script Block Logging [17], [18].

Las Tablas 10 y 11 relacionan las fuentes de telemetría con su utilidad y separan visibilidad, detección, alerta, evidencia y salida externa.

Fuente de telemetría	Qué puede observar Velociraptor	Qué puede ayudar a detectar
Procesos	procesos activos, PID, usuario, ruta, línea de comandos, relación padre-hijo	ejecución sospechosa, LOLBins, procesos anómalos, post-explotación
Windows Event Logs	autenticaciones, creación de procesos, errores, eventos de seguridad, PowerShell, Sysmon si existe	accesos anómalos, ejecución de comandos, persistencia, movimiento lateral
Sistema de ficheros	archivos, rutas, fechas, tamaños, hashes	malware en disco, ficheros sospechosos, cambios en ubicaciones sensibles
Registro de Windows	claves, valores y rutas de configuración	persistencia, modificaciones del sistema, claves Run/RunOnce
PowerShell	comandos ejecutados, scripts, Script Block Logging, actividad registrada	ataques fileless, uso malicioso de scripts, automatización ofensiva

Tabla 10: Fuentes principales de telemetría y utilidad en detección

2.2.7. Limitaciones de visibilidad en HIDS/DFIR

Para no sobredimensionar las capacidades de la herramienta y en base a lo comentado en la introducción del capítulo, se estableció una separación entre visibilidad, detección, alertabilidad, evidencia, salida externa y ruido, distinguiendo seis niveles diferentes. La observación de un evento no equivale a detectar una técnica MITRE ATT&CK; del mismo modo, generar una detección no implica necesariamente que exista una alerta operativa. Asimismo, el envío de una notificación externa no demuestra por sí solo que la lógica de detección sea correcta.

Nivel	Significado
Visibilidad	El evento existe y puede observarse
Detección	Una lógica lo clasifica como relevante
Alerta RT	Se genera desde monitorización continua
Salida externa	Se envía a JSONL, Discord, webhook o SIEM
Evidencia	Queda rastro verificable: log, captura, hash, CSV, JSON
Ruido / FP	Actividad legítima clasificada como sospechosa

Tabla 11: Diferencia entre visibilidad, detección, alerta y salida externa

2.2.7.1. Visibilidad parcial del comportamiento del atacante

La visibilidad de un HIDS es parcial por naturaleza, aunque el agente pueda acceder a múltiples fuentes del endpoint, no siempre es posible reconstruir completamente el comportamiento del atacante. Una misma técnica MITRE ATT&CK puede ejecutarse de formas distintas y no todas generan las mismas evidencias.

Esto puede provocar falsos negativos en varios casos:

- La técnica no genera eventos observables.
- El evento existe, pero no está habilitado el log correspondiente.
- El artifact no monitoriza la fuente adecuada.
- La regla de detección no contempla esa variante concreta.
- El comportamiento se confunde con actividad legítima del sistema.

Por este motivo, la detección no se valora únicamente por la existencia de eventos, sino por la capacidad de transformar esa telemetría en señales útiles, contextualizadas y con bajo nivel de falsos positivos.

Capítulo III: Desarrollo experimental y ejecución de pruebas

En este capítulo se define el entorno experimental utilizado para evaluar Velociraptor como plataforma HIDS/DFIR, la metodología de trabajo aplicada y el criterio de selección de técnicas MITRE ATT&CK. El objetivo no es presentar todavía los resultados, sino fijar una base metodológica reproducible que permita interpretar posteriormente la visibilidad obtenida, la capacidad de detección, la generación de alertas en tiempo casi real y el impacto de la monitorización sobre el endpoint.

La metodología se apoya en tres principios: ejecución controlada de técnicas, trazabilidad de evidencias y separación estricta entre visibilidad, detección, alerta operativa y salida externa. Esta separación evita considerar como detección aquello que únicamente corresponde a telemetría bruta, transporte de eventos o notificación externa.

La Tabla 12 resume los criterios metodológicos aplicados durante el desarrollo experimental.

Criterio	Aplicación en el TFM
Entorno	Laboratorio controlado sobre Windows 10.
Alcance	Evaluación de detección host-based de los casos TEC-001 a TEC-009.
Instrumentación	Uso de las fuentes de telemetría y herramientas descritas en este capítulo.
Evaluación	Separación entre ejecución, evidencia observada y señal de detección.

Tabla 12: Criterios metodológicos del capítulo III

Este capítulo establece el diseño experimental. Los resultados concretos por técnica, la tasa de detección, los falsos positivos, las evidencias observadas y el impacto cuantitativo de rendimiento se analizarán en los capítulos posteriores.

3.1. Metodología experimental

La metodología seguida fue iterativa e incremental. Aunque la memoria se presenta de forma secuencial, el desarrollo técnico combinó revisión documental, preparación del laboratorio, ejecución de pruebas, ajuste de artifacts, captura de evidencias y registro en la matriz experimental.

El antecedente [19] sitúa la evaluación de capacidades de detección mediante ATT&CK en un entorno ICS y en las herramientas identificadas en su título. Este TFM centra el experimento en telemetría host de Windows, artifacts de Velociraptor y separación entre detección, evidencia y salida externa. La comparación detallada de metodologías y resultados con ese antecedente debe limitarse a aspectos efectivamente revisados.

Además de las ejecuciones individuales, se empleó un runner global para repetir la campaña completa TEC-001 a TEC-009 bajo un patrón homogéneo. También se diseñó un runner de falsos positivos para generar actividad benigna controlada y evaluar si la lógica de detección producía ruido operativo o sobreclasificación.

Las Tablas 13 a 17 sistematizan los principios metodológicos, el flujo de trabajo, los criterios de selección y el alcance MITRE ATT&CK del experimento.

3.1.1. Principios metodológicos

El proceso por el que se ha ido iterando el trabajo ha sido el siguiente:

Principio	Aplicación en el TFM
Reproducibilidad	Cada técnica se ejecuta mediante un runner que permite ejecutar todas las pruebas de forma consistente y sencilla. Todo procedimiento está definido y permite ser reproducible en cualquier entorno para comparar resultados. Luego de cada ejecución, el mismo runner guarda las evidencias de forma automática en forma de log, permitiendo comparar todas las campañas históricas de forma rápida y sencilla, aunque, con el volumen de datos generados, ha sido necesario un sistema de tratamiento de datos a una escala de GB
Seguridad	No se ejecuta malware con payload ni se trabaja sobre datos reales. Las acciones se limitan a directorios de prueba DUMP, creados previamente y restaurados en cada prueba mediante el runner
Trazabilidad	Cada prueba queda vinculada a una técnica MITRE, artifact, evidencia, resultado y benchmark en el Excel matriz. Además, como se ha comentado en el punto de reproducibilidad, se almacenan todos los logs históricos de las pruebas realizadas
Separación conceptual	Se distingue entre visibilidad, detección, alerta y salida externa
Cobertura	La cobertura MITRE se calcula sobre las 14 tácticas de ATT&CK Enterprise y se separa en cobertura principal, ampliada y contextual. En el trabajo solo se abordan 9 técnicas concretas, distribuidas entre algunas de estas tácticas

Tabla 13: Principios metodológicos aplicados en la fase experimental

3.1.2. Flujo general de trabajo

El trabajo experimental se desarrolló siguiendo un ciclo común para cada técnica analizada. Aunque cada caso requirió ajustes concretos, el flujo general fue el siguiente:

Paso	Actividad
1	Preparar o restaurar snapshot de laboratorio
2	Seleccionar técnica MITRE ATT&CK y objetivo de prueba
3	Diseñar ejecución controlada y segura
4	Ejecutar script o comando en el endpoint (Runner)
4.1	Observar telemetría en Velociraptor, Sysmon y logs Windows.
4.2	Evaluar artifacts públicos disponibles
4.3	Ajustar consultas VQL o artifacts personalizados si procede
4.4	Registrar evidencias, capturas y referencias de logs
4.5	Clasificar resultado: visibilidad, detección, alerta RT y salida externa.
4.6	Medir impacto de rendimiento asociado
4.7	Documentar limitaciones, ruido y falsos positivos
5	Pasar a la siguiente técnica ejecutando desde el paso 4

Tabla 14: Flujo general de trabajo por técnica

3.2. Selección de técnicas MITRE ATT&CK

La selección de técnicas no busca cubrir toda la matriz MITRE ATT&CK Enterprise, sino construir un conjunto acotado y representativo para evaluar detección host-based en Windows. La selección final incorpora nueve técnicas internas TEC-001 a TEC-009, mapeadas contra ATT&CK Enterprise y documentadas en la matriz de evidencias del TFM.

3.2.1. Criterios de selección

Criterio	Descripción
Observabilidad en host	La técnica debe generar señales consultables mediante procesos, logs, Sysmon, PowerShell, registro, ficheros o red local
Ejecución segura	Debe poder reproducirse sin malware real y sin afectar datos legítimos

Criterio	Descripción
Relación con ransomware/insiders	Se priorizan ejecución, persistencia, discovery, impacto, sabotaje, staging, archivado ZIP y exfiltración HTTP controlada en laboratorio
Reproducibilidad	Debe poder ejecutarse varias veces en la VM

Tabla 15: Criterios usados para seleccionar técnicas MITRE ATT&CK

3.2.2. Técnicas seleccionadas

La evaluación final comprende nueve casos experimentales, TEC-001 a TEC-009. Los casos TEC-007, TEC-008 y TEC-009 representan, respectivamente, cifrado controlado, destrucción de datos y preparación, archivado y transferencia HTTP. TEC-009 agrupa varias subtécnicas ATT&CK y se cuenta como un único caso experimental.

ID	MITRE	Técnica	Táctica
TEC-001	T1059.001	PowerShell	Execution
TEC-002	T1059.003	Windows Command Shell	Execution
TEC-003	T1053.005	Scheduled Task	Execution; Persistence; Privilege Escalation
TEC-004	T1547.001	Registry Run Keys / Startup Folder	Persistence; Privilege Escalation
TEC-005	T1569.002	Service Execution	Execution
TEC-006	T1518.001	Security Software Discovery	Discovery
TEC-007	T1486	Data Encrypted for Impact	Impact
TEC-008	T1485	Data Destruction	Impact
TEC-009	T1074.001/ T1560.001/ T1048.003	Local Data Staging / Archive Collected Data / Exfiltration Over Unencrypted Non-C2 Protocol	Collection; Exfiltration

Tabla 16: Técnicas MITRE ATT&CK seleccionadas para la fase experimental - Fuente: elaboración propia a partir de las técnicas ATT&CK [2], [20]-[30].

La descripción detallada y el mapeo MITRE ATT&CK de las técnicas evaluadas se recogen en el Anexo C: Técnicas MITRE seleccionadas.

3.2.3. Tácticas ATT&CK representadas por el subconjunto evaluado

El denominador de 14 tácticas se mantiene como criterio metodológico. La cobertura se presenta en tres niveles para no inflar el alcance real del estudio: cobertura inicial, cobertura ampliada y cobertura contextual con red.

Criterio	Tácticas consideradas	Cálculo	Cobertura
Cobertura principal estricta	Execution, Persistence, Discovery, Collection, Impact	5/14	35,71 %
Cobertura ampliada/secundaria	Las anteriores + Privilege Escalation contextual en TEC-003/TEC-004	6/14	42,86 %
Cobertura contextual con red	Las anteriores + Exfiltration mediante envío HTTP documentado en TEC-009	7/14	50,00 %

Tabla 17: Tácticas ATT&CK representadas por el subconjunto evaluado empleando denominador 14

3.3. Diseño del entorno

El laboratorio se diseñó como un entorno controlado sobre Windows 10, orientado a reproducir comportamientos host-based asociados a técnicas MITRE ATT&CK sin emplear malware real ni afectar a datos legítimos. La arquitectura se basa en el modelo cliente-servidor de Velociraptor, aunque tanto el servidor como el cliente corren en la misma VM. El servidor coordina la ejecución de artifacts y el endpoint ejecuta localmente las consultas VQL y devuelve resultados estructurados.

3.3.1. Configuración base de la máquina virtual

El trabajo se desarrolló sobre una máquina virtual proporcionada por el tutor. Como plataforma de virtualización se utilizó Oracle VirtualBox [31]. La configuración del laboratorio se documenta en la Tabla 18 y en el Anexo A.

Las Tablas 18 a 21 recogen la configuración de la máquina virtual, los componentes desplegados, la identificación del endpoint y los parámetros funcionales del servidor Velociraptor.

Parámetro	Valor
Hipervisor	Oracle VirtualBox
Sistema operativo	Windows 10 64 bits

Parámetro	Valor
Memoria RAM	6000 MB
Procesadores virtuales	4 vCPU
Memoria de vídeo	128 MB
Adaptador de red	NAT
Chipset	PIIX3
UEFI	Desactivado
Secure Boot	Desactivado
TPM	No configurado
Windows 10 Pro	22H2
Oracle VirtualBox	7.2.16
Velociraptor cliente/servidor	0.75.6
Sysmon	15.15
Wazuh Agent	4.14.5

Tabla 18: Parámetros iniciales de la máquina virtual

La elección de un laboratorio virtualizado permite repetir las pruebas, restaurar snapshots y aislar las simulaciones del sistema físico. No obstante, este diseño también introduce una limitación: los resultados de rendimiento deben interpretarse como mediciones relativas del laboratorio y no como una estimación directa de impacto en una infraestructura corporativa completa.

3.3.2. Componentes técnicos desplegados

Componentes	Versión / referencia	Uso metodológico
Velociraptor	0.75.6	Servidor, GUI y agente cliente instalados en el entorno
Sysmon	15.15 / schema 4.90	Generación de telemetría avanzada
PowerShell Script Block Logging	Event ID 4104	Visibilidad sobre bloques de código PowerShell ejecutados
Windows Event Logs	System / Security / PowerShell / Sysmon	Fuente base para Event ID 7045, PowerShell 4104 y eventos Sysmon

Componentes	Versión / referencia	Uso metodológico
Discord	app-1.0.9242	Validación de salida externa
Microsoft Excel	16.0	Tratamiento y representación de resultados
Python	3.14.5	Procesamiento auxiliar

Tabla 19: Componentes técnicos principales

3.3.3. Identificación del cliente y alcance del laboratorio

Elemento	Valor
ClientId	C.364907172793f8c7
Host Name	LT29.ait.us.es
Usuario	LT29\seguridad
Endpoint evaluado	Windows 10 Pro 22H2
Ámbito	Pruebas locales host-based sobre una única VM controlada

Tabla 20: Identificación del endpoint usado en el laboratorio

El alcance experimental queda limitado a un único endpoint Windows. Esta decisión permite controlar el entorno y documentar con precisión cada prueba, pero impide generalizar directamente los resultados a redes heterogéneas con múltiples sistemas operativos, usuarios y cargas de trabajo.

3.4. Instalación, instrumentación y configuración base

La preparación del entorno no se limitó al despliegue del agente de Velociraptor. Para que la evaluación tuviera valor, fue necesario instrumentar el endpoint con fuentes de telemetría suficientes y establecer mecanismos de validación de conectividad, evidencias y alertas. [10], [11]

El procedimiento detallado de instalación y configuración del entorno de laboratorio se recoge en el Anexo A: Instalación del entorno de laboratorio.

3.4.1. Servidor Velociraptor

Como se ha comentado en la introducción del capítulo, el servidor Velociraptor se desplegó en modo cliente-servidor, habilitando la GUI de administración y el frontend de comunicación con agentes. El archivo `server.config.yaml` concentró los parámetros de red, GUI, datastore y certificados. Durante la configuración inicial se creó un usuario administrador para acceder a la consola y gestionar el cliente.

Elemento	Valor	Función
Frontend	0.0.0.0:8443	Punto de escucha para agentes
GUI	127.0.0.1:8889	Interfaz local de administración
Configuración	server.config.yaml	Archivo principal del servidor
Datastore	Local del laboratorio	Almacenamiento de resultados y eventos
Autenticación GUI	Usuario administrador local	Acceso controlado a la consola de gestión

Tabla 21: Parámetros funcionales del servidor Velociraptor

3.4.2. Agente Velociraptor

El agente se generó desde la propia GUI mediante `Server.Utils.CreateMSI`. El instalador MSI incorporó la configuración del servidor y los parámetros de conexión necesarios para registrar el endpoint en la consola. Tras la instalación, se validó la conectividad mediante la aparición del cliente en la sección de endpoints gestionados y la ejecución de artifacts de inventario.

Las Figuras 4 y 5 muestran la instalación del agente Velociraptor y la posterior recepción de información desde el endpoint.

Tool VelociraptorWindowsMSI

Other Definitions	Server.Internal.ToolDependencies
Artifact Definition	Server.Internal.ToolDependencies
Tool Name	VelociraptorWindowsMSI
Tool Version	0.75.6
Upstream URL	https://github.com/Velocidex/velociraptor/releases/windows-amd64.msi
Endpoint Filename	velociraptor-v0.75.6-windows-amd64.msi
Serve Locally	
Serve URL	https://localhost:8080/public/27bd56abf77264cd761a
Admin Override	

Override Tool

As an admin you can manually upload a binary to be used as that tool. This will override the tool for your tool to all artifacts that need it. Alternative, set a URL for clients to fetch tools from.

Click to upload file

Set Serve URL

Served Locally	Placeholder Definition	Admin Override
Tool will be served from the Velociraptor server to clients if needed. The client will cache the tool on its own disk and compare the hash next time it is needed. Tools will only be downloaded if their hash has changed.	Tool hash is currently unknown. The first time the tool is needed, Velociraptor will download it from it's upstream URL and calculate its hash. <button>Materialize Hash</button>	Tool was manually uploaded by an admin - it will not be automatically upgraded on the next Velociraptor server update.

Figura 4: Instalación del agente Velociraptor

State	FlowId	Artifacts	Created	Last Active	Creator	MB	Rows
✖	F.D6MA323F7R970	Windows.Forensics.Pst	2026-03-07T22:07:04.182Z	2026-03-07T22:12:54.480Z	admin	0 b	0
✓	F.D6MA2RIQ9PQJU	Windows.Detection.Yara.Device Windows.Detection.Yara.Process	2026-03-07T22:06:38.465Z	2026-03-07T22:07:49.998Z	admin	0 b	0
✓	F.D6MA038S77U2K	Windows.System.Pslist	2026-03-07T22:00:45.539Z	2026-03-07T22:00:55.053Z	admin	0 b	160

Figura 5: Datos recibidos desde los agentes Velociraptor

Como comprobación básica, se ejecutaron artifacts como Windows.System.Pslist y Windows.System.Services. La devolución de procesos y servicios confirmó que el cliente podía ejecutar consultas VQL y enviar resultados al servidor.

3.4.3. Instrumentación con Sysmon, PowerShell 4104 y eventos Windows

Como se explicó en el capítulo anterior, la visibilidad de VR será tan buena como lo es la del propio endpoint, ya que es host-based, por lo que se hicieron varias pruebas para aumentarla. Para esto el endpoint se instrumentó con Sysmon y con PowerShell Script Block Logging. Como veremos más adelante, esta configuración resulta esencial para las técnicas asociadas a ejecución, persistencia, discovery, impacto, sabotaje y staging. [17], [18]

La Tabla 22 resume las fuentes de telemetría habilitadas y su relación con las técnicas evaluadas.

Fuente	Evento	Qué aporta	Técnicas relacionadas
Sysmon ID 1	Process Creation	Creación de procesos, línea de comandos, imagen y usuario	TEC-001, TEC-002, TEC-003, TEC-004, TEC-006
Sysmon ID 3	Network Connection	Conexión de red asociada a proceso	TEC-009 contextual
Sysmon ID 11	FileCreate	Creación de ficheros, extensiones y rutas	TEC-007, TEC-009
Sysmon ID 12/13/14	Registry	12 crea/elimina objetos, 13 modifica valores, 14 renombra claves/valores	TEC-004
Sysmon ID 22	DNS	Resoluciones DNS si están habilitadas	Apoyo contextual
Sysmon ID 26	FileDeleteDetected	Borrado de ficheros	Apoyo contextual

Tabla 22: Fuentes de telemetría habilitadas y técnicas relacionadas

3.4.4. Organización del entorno de trabajo

Para mantener la trazabilidad se separó la organización de la máquina física y la máquina virtual. En la máquina física se conservaron memoria, evidencias, planificación y documentación auxiliar. En la máquina virtual se mantuvieron los scripts de prueba, directorios de laboratorio, configuración de Velociraptor, Sysmon y ficheros de benchmark:

Las Figuras 6 y 7 muestran la organización de directorios utilizada respectivamente en la máquina física y en la máquina virtual.

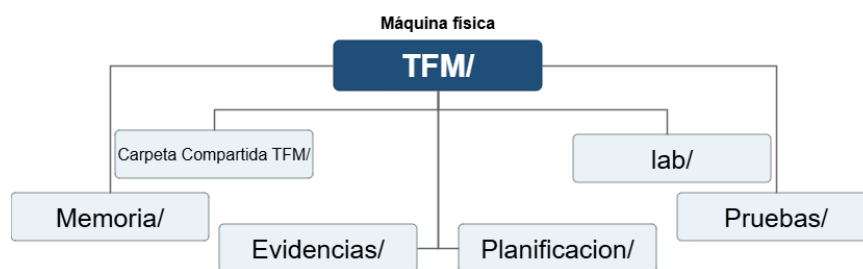


Figura 6: Estructura de directorios de la máquina física

Para la máquina virtual se usó la siguiente estructura de directorio:

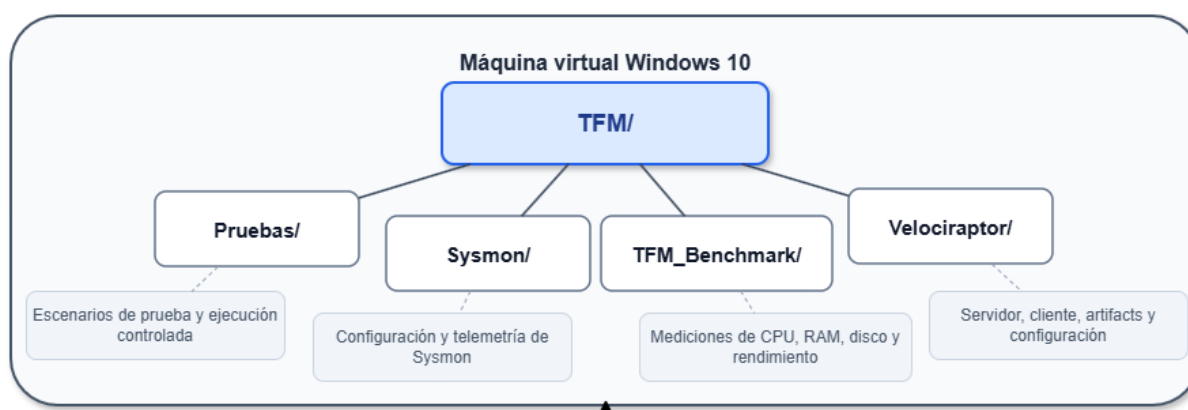


Figura 7: Directorio de la máquina virtual

3.4.5. Control de versiones y trazabilidad del desarrollo

El desarrollo aplicó control de versiones a scripts, artifacts, runners, configuraciones y documentos de apoyo. Cada elemento se vinculó a su versión, estado de validación, hash SHA-256 y evidencia asociada para conservar la trazabilidad entre código, ejecución y resultado.

La promoción de un componente a estado validado dependió de mi revisión y aceptación como definitivo, una vez comprobado que todo funcionaba correctamente. La denominación del archivo se utilizó como identificador auxiliar y nunca como prueba suficiente de validez experimental.

La máquina virtual se guardó en snapshots antes y después de hitos relevantes. Esta práctica permitió restaurar el entorno tras pruebas con impacto, repetir escenarios y reducir el riesgo de contaminar evidencias entre técnicas. La última snapshot fue la 08, ya que, a partir de ahí, no se crearon más pruebas con riesgo que pudieran salirse de control o tardaran en recuperarse. Además, para controlar el estado del proyecto y salvaguardarlo en caso de algún problema, se usó GitHub como copia de seguridad.

La Tabla 23 recoge las instantáneas empleadas para controlar y restaurar los distintos estados del laboratorio.

Snapshot	Finalidad
00_Windows_Inicial	Estado base del sistema operativo
01_TFM_Velociraptor_Base_Lab	Velociraptor instalado y entorno inicial disponible
02_VR_Con_Artifacts_Funcionando	Artifacts base validados
03_TFM_Montado_Configurado	Estructura de trabajo y laboratorio preparados
04_Antes_de_persistencia	Punto previo a pruebas con cambios de configuración
05_Despues_de_ransom	Estado posterior a simulación controlada de cifrado
06_Iniciando_pruebas	Estado de inicio de Runners
VR_7_Probando_REDES	Probando el ataque de red con TEC-009
Instantánea 8 19062026	Estado final donde se ejecutan todas las pruebas

Tabla 23: Snapshots utilizados para control del laboratorio

La estructura completa del repositorio y el contenido de sus diferentes directorios se describen en el Anexo B: Repositorio de GitHub.

3.4.6. Matriz principal de control experimental

El archivo Analisis_Tecnicas_TFM_Velociraptor.xlsx se empleó como matriz de control experimental. Su función fue centralizar la relación entre técnicas, artifacts, pruebas, evidencias, alertabilidad, brechas de cobertura y decisiones metodológicas.

Las Tablas 24 a 27 resumen la matriz de control experimental, las evidencias registradas, los controles de seguridad y los scripts utilizados durante las pruebas.

Hoja	Función metodológica
00_Guia_Indice	Guía de lectura, definiciones metodológicas e índice de navegación del libro.
01_Dashboard	Síntesis visual de los principales resultados, KPI y gráficos de cobertura, artifacts y telemetría.
02_Tecnicas	Técnicas TEC-001 a TEC-009, mapeo MITRE ATT&CK, procedimiento, evidencia esperada y observada y conclusión.

Hoja	Función metodológica
03_Visibilidad_VR	Matriz de visibilidad por fuente de telemetría y técnica, incluyendo Sysmon, PowerShell 4104, System 7045 y confirmación externa del receiver.
04_Matriz_Resultados	Matriz consolidada por técnica que separa visibilidad, detección y alerta, e integra Velociraptor público, Velociraptor custom y Wazuh.
05_Artifacts	Evaluación detallada de artifacts públicos y personalizados, incluyendo tipo, visibilidad, detección, alerta RT, filas TEC/FP, evidencias, resultado de campaña y limitaciones.
06_Comparativa_Wazuh	Comparación de cobertura entre Velociraptor custom, artifacts públicos/Hayabusa, Wazuh base y Wazuh custom.
07_Fuentes_Evidencias	Inventario de fuentes experimentales, scripts, rutas de evidencia, hashes SHA-256 y elementos de reproducibilidad.
08_Detalle_Alertas	Trazabilidad detallada de las filas CLIENT_EVENT.

Tabla 24: Hojas principales de la matriz de control experimental

Para el benchmark, como veremos en capítulos siguientes, se ha usado otro documento de control.

3.4.7. Registro de evidencias

Las evidencias se organizaron en carpetas por técnica siguiendo una nomenclatura basada en fecha, identificador interno y técnica MITRE ATT&CK: AAAAMMDD_TEC-XXX_TXXXX-XXX.

Tipo de evidencia	Uso
Capturas de Velociraptor	Resultados visibles en GUI, Client Events, Hunts o Collections
Consultas VQL	Consulta ejecutada o fragmento relevante del artifact
Scripts / comandos	Código ejecutado en la VM
Logs Windows / Sysmon	Event IDs asociados a la técnica
Exportaciones	CSV, JSONL o ZIP cuando es necesario para anexo o análisis
Benchmark	CSVs de control de rendimiento

Tabla 25: Tipos de evidencias documentadas por técnica

3.4.8. Diseño de pruebas controladas

Como se ha mencionado antes, las pruebas se diseñaron usando como núcleo principal un runner para reproducir comportamientos observables sin payload. Su estructura y metodología es la siguiente:

Criterio de seguridad	Aplicación
Datos dummy	Los ficheros pertenecen a directorios de laboratorio
Reversibilidad	Las pruebas incluyen limpieza, descifrado y restauración
Aislamiento	La ejecución se limita a la VM y a rutas controladas
No explotación real	No se realiza explotación de vulnerabilidades ni movimiento lateral

Tabla 26: Controles de seguridad aplicados a las pruebas

3.4.9. Scripts de laboratorio validados

Los scripts usados para el laboratorio son:

Script	Rol
crear_directorio_dummy.ps1	Preparación data set
enumeracion_v2.ps1	Enumeración previa TEC-007
Scriptransom_v2.ps1	TEC-007 cifrado controlado
descifrar.ps1	Restauración TEC-007
sabotaje.ps1	TEC-008 borrado controlado
exfiltracion_v3.ps1	TEC-009 staging / ZIP / upload controlado
TFM_Run_All_TEC_Tests_v6_PATCHED_4104.ps1	Runner global TEC-001 a TEC-009
TFM_Run_FP_Tests_v1.ps1	Runner falsos positivos

Tabla 27: Scripts de laboratorio y papel metodológico

La descripción funcional y el modo de uso de los runners y scripts desarrollados se amplían en el Anexo F: Runners, scripts y ground truth.

3.5. Estrategia de artifacts y monitorización

Para evaluar la facilidad de implementación de Velociraptor es clave, como se ha comentado previamente, analizar el catálogo de artifacts públicos o de fácil acceso que ofrece la herramienta. Luego, si no existe ningún artifact público que sea afín perfectamente, se modificarán y crearán artifacts que cumplan todos los requisitos previamente establecidos. El objetivo es comprobar en cada artifact la visibilidad, cuándo clasifica una actividad y cuánto esfuerzo y tiempo requiere para generar una alerta a un buen nivel de ruido. [3]-[6]

3.5.1. Artifacts públicos considerados

Antes de diseñar artifacts personalizados, se realizó una revisión de artifacts públicos y nativos de Velociraptor con posible utilidad para las técnicas seleccionadas. Esta revisión, como se ha explicado en la introducción del subcapítulo, sirve para establecer una línea base de comparación. Para clasificar la validez de estos artifacts públicos se considerará lo siguiente:

- Visibilidad: el artifact permite observar eventos o evidencias relacionadas con la técnica ejecutada.
- Detección: el artifact permite identificar un comportamiento relevante y asociarlo razonablemente a una técnica MITRE ATT&CK concreta.
- Alertabilidad: el artifact puede integrarse en un flujo de monitorización continua o alertado operativo.

Las Tablas 28 y 29 presentan los artifacts públicos considerados y las principales brechas identificadas durante su evaluación.

Artifact público	Tipo	Fuente de telemetría	Técnicas relacionadas
Windows.Events. ProcessCreation	CLIENT_EVENT	Creación de procesos y línea de comandos	TEC-001, TEC-002, TEC-003, TEC-004, TEC-005, TEC-006
Windows.EventLogs.PowerShell / eventos PowerShell 4104	Consulta sobre logs / monitorización según configuración	PowerShell Script Block Logging	TEC-001, TEC-006, TEC-007, TEC-008, TEC-009
Windows.Events. ServiceCreation	CLIENT_EVENT	Eventos de creación de servicios Windows	TEC-005

Artifact público	Tipo	Fuente de telemetría	Técnicas relacionadas
Windows.Sysmon.* / artifacts basados en Sysmon	Consulta sobre eventos Sysmon	Sysmon Event ID 1, 3, 11, 12, 13, 14, 26	TEC-001 a TEC-009, según evento
Generic.Events.TrackNetworkConnections	CLIENT_EVENT	Conexiones de red; sin atribución fiable de proceso en la campaña definitiva	TEC-009
Windows.System.Pslist	CLIENT / consulta puntual	Procesos activos	TEC-001, TEC-002, TEC-006
Windows.System.Services	CLIENT / consulta puntual	Servicios instalados y estado de servicios	TEC-005
Windows.Registry.* / artifacts de registro	CLIENT / consulta puntual	Claves y valores del registro de Windows	TEC-004
Windows.Hayabusa.Monitoring	Monitorización / correlación sobre eventos Windows	Logs de Windows procesados con lógica de detección	TEC-001 a TEC-009, según cobertura de reglas
Windows.ETW.Monitoring	CLIENT_EVENT	Eventos ETW	TEC-001 a TEC-006, según fuente habilitada

Tabla 28: Artifacts públicos considerados en la evaluación metodológica

Los artifacts públicos se emplean como base, pero en la mayoría de los casos aportan únicamente telemetría, no detecciones completas. Cuando generan eventos en tiempo casi real, se valora si dicha señal contiene suficientes datos como para asociarla a una técnica concreta.

3.5.2. Brechas detectadas y necesidad de artifacts personalizados

Mediante la revisión de los artifacts públicos como Hayabusa se pudo identificar que VR tiene capacidad de clasificación por severidad y tiene visibilidad sobre procesos, eventos de Windows, PowerShell, Sysmon, servicios, registro y conexiones de red.

Donde fallan es en la interpretación de estos datos, en varias técnicas, los artifacts públicos proporcionan eventos incluso clasificados ya con severidad, pero estos eventos requieren un tratamiento adicional, y sobre todo y lo más importante, se deben configurar para que alcancen toda la superficie que queremos monitorizar con las técnicas y para diferenciar actividad legítima de comportamiento potencialmente malicioso.

Por ejemplo, la ejecución de powershell.exe, cmd.exe, reg.exe, schtasks.exe o sc.exe puede ser legítima en un entorno administrativo, por lo que su mera aparición no debe tratarse automáticamente como detección. Para que el evento tenga valor como alerta, debe contextualizarse mediante parámetros, línea de comandos, fuente del evento, técnica asociada, severidad y relación con el escenario ejecutado.

A partir de esta limitación, se identificaron las siguientes brechas metodológicas:

Brecha	Consecuencia
PowerShell legítimo y sospechoso comparten proceso	Riesgo de FP
CMD, reg, sc y schtasks pueden ser administrativos	Requieren contexto
Borrado masivo genera muchos eventos	Necesita agregación
ZIP local y envío no genera evento	Requiere config
Sysmon aporta telemetría, no detección	Necesita lógica VQL

Tabla 29: Brechas y limitaciones de los artifacts públicos en la evaluación

Los artifacts públicos se emplean como base para generar los artifacts personalizados finales. No todos por igual, Hayabusa y ETW han sido los más útiles por estar orientados justo al objetivo de este TFM.

La definición y configuración de los artifacts finalmente validados se documentan en el Anexo D: Artifacts Velociraptor validados.

3.6. Benchmark y alertabilidad

Además de la capacidad de detección, se evaluó el impacto de las pruebas sobre el endpoint.

Esto se realizó en dos fases. La primera utilizó Performance Monitor mediante logman para recoger métricas del sistema, y posteriormente se sustituyó por la metodología centrada en el proceso cliente descrita a continuación.

Luego, para cada técnica se genera una medición normalmente cercana a un minuto, y siempre inferior a cinco minutos, suficiente para capturar el impacto inmediato de cada técnica sin introducir

ruido excesivo por actividad ajena al experimento en el que se ejecuta el ataque. Esta medición se realiza con y sin VR activo igualmente para ver cuánto penaliza VR en la máquina.

Las Tablas 30 y 31 describen, respectivamente, el planteamiento inicial y los escenarios definitivos utilizados para el benchmark de rendimiento.

Escenario	Objetivo
BASELINE_NO_VR	Medir el rendimiento basal de la máquina virtual sin el cliente Velociraptor activo.
VR_IDLE	Medir el impacto del endpoint con Velociraptor activo sin ejecutar la carga de técnicas.
VR_TEC_RUNNER	Medir el impacto del endpoint con Velociraptor durante la ejecución controlada del runner TEC.

Tabla 30: Planteamiento inicial del benchmark de rendimiento

Posteriormente, los CSV son procesados mediante scripts, los cuales extraen métricas globales como CPU, memoria e I/O, y métricas específicas del proceso cliente Velociraptor, evitando mezclar el coste del agente con el servidor/GUI del laboratorio.

Las métricas principales consideradas fueron:

- CPU total del sistema.
- RAM disponible.
- Lecturas de disco por segundo.
- Escrituras de disco por segundo
- CPU asociada al proceso de Velociraptor.
- Memoria privada del proceso de Velociraptor.
- I/O de lectura y escritura del proceso de Velociraptor.

Sin embargo, se detectó el problema del posible ruido generado por otros procesos y la limitada precisión de esta aproximación para aislar específicamente el consumo atribuible a Velociraptor. Por ello, esta primera metodología terminó por descartarse como método principal y se optó por una segunda aproximación más controlada, sencilla y precisa.

Escenario	Objetivo
BASELINE_NO_VR	Medir el rendimiento basal de la máquina virtual sin el cliente Velociraptor activo.

Escenario	Objetivo
VR_IDLE	Medir el impacto únicamente del proceso del agente Velociraptor activo sin ejecutar la carga de técnicas.
VR_TEC_RUNNER	Medir el impacto únicamente del proceso del agente Velociraptor durante la ejecución controlada del runner TEC.

Tabla 31: Escenarios definitivos del benchmark centrado en el cliente

Los escenarios BASELINE_NO_VR y VR_IDLE se configuraron con ventanas nominales de 180 s y VR_TEC_RUNNER con una ventana nominal de 300 s. Las duraciones efectivas se recogen en el Excel. En las tres repeticiones con carga, el runner seguía activo al cerrar la medición, por lo que los resultados describen la ventana observada y no el coste de completar toda la campaña.

Posteriormente, los resultados son procesados también mediante scripts, los cuales al igual que la anterior forma extraen tanto métricas globales del sistema, como CPU, memoria e I/O, como sobre todo y más importante métricas específicas asociadas al proceso cliente de Velociraptor. De esta forma se evita mezclar, en la medida de lo posible, el coste del agente instalado en el endpoint con el correspondiente al servidor o a la GUI utilizados en el laboratorio.

Lo que aporta esta medición es:

- CPU asociada al proceso cliente de Velociraptor.
- Memoria utilizada por el proceso cliente de Velociraptor.
- I/O de lectura y escritura del proceso cliente de Velociraptor.

Esta segunda metodología nos permite ver justo el consumo de VR en el endpoint, el cual es el objetivo real de este proyecto, por eso se terminó por seleccionar.

3.6.1. Validación operativa mediante alertas

Como se ha comentado varias veces, el objetivo es comprobar si una detección generada en el endpoint podía transformarse en una notificación externa accionable.

La Figura 8 representa la arquitectura y el flujo de alertas empleados durante la validación operativa.

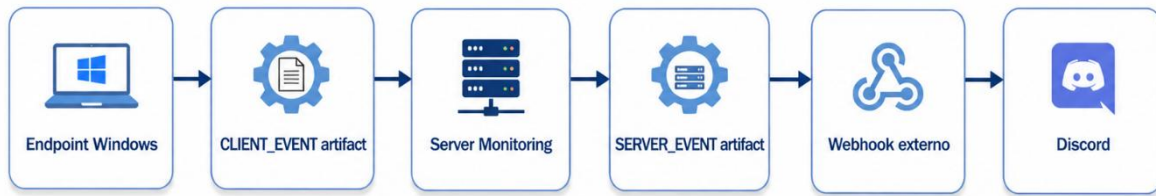


Figura 8: Arquitectura del flujo de alertas

En este modelo, el artifact de tipo CLIENT_EVENT observa eventos en el endpoint, el servidor recibe los resultados mediante Server Monitoring y un artifact de tipo SERVER_EVENT consume dichos eventos para enviarlos a un webhook externo. Para la validación, una alerta se consideró correctamente generada cuando se cumplían las siguientes condiciones:

1. El evento era observado por Velociraptor.
2. La lógica de detección lo asociaba a una técnica concreta.
3. El SERVER_EVENT procesaba el evento.
4. El webhook devolvía respuesta satisfactoria.
5. La notificación llegaba correctamente a Discord.

La prueba no se limitó a comprobar la existencia de telemetría, sino que verificó el flujo desde la observación del evento en el endpoint hasta su salida externa. Las capturas de Discord o JSONL se interpretan como notificación o persistencia posterior y final del flujo.

3.7. Ejecución de ataques

Todas las pruebas se realizaron mediante comandos o scripts controlados y reversibles. El runner global descrito en el apartado 3.4.9 automatiza los casos TEC-001 a TEC-009; su funcionamiento y uso se amplían en el Anexo F.

Las Figuras 9 a 17 documentan las principales evidencias de ejecución controlada de los escenarios TEC-001 a TEC-009.

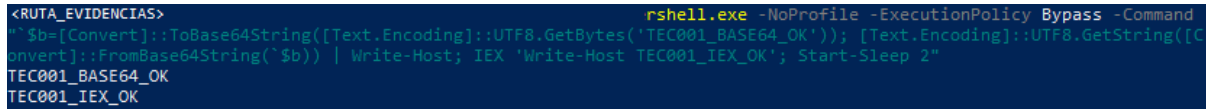
3.7.1. TEC-001 - Ejecución mediante PowerShell

La primera prueba se correspondió con T1059.001 - PowerShell. Se utilizó PowerShell porque es un intérprete legítimo de Windows ampliamente empleado en administración, pero también habitual en cadenas de ataque por su capacidad para ejecutar scripts, automatizar acciones y operar con parámetros avanzados.

- **Herramienta utilizada:** powershell.exe
- **Procedimiento ejecutado:**

```
$ powershell.exe -NoProfile -ExecutionPolicy Bypass -Command
"$b=[Convert]::ToBase64String([Text.Encoding]::UTF8.GetBytes('TEC001_BASE64_OK'));
[Text.Encoding]::UTF8.GetString([Convert]::FromBase64String('$b')) | Write-Host; IEX 'Write-
Host TEC001_IEX_OK'; Start-Sleep 2"
```

Código 5: Procedimiento de ejecución de TEC-001



```
<RUTA_EVIDENCIAS> powershell.exe -NoProfile -ExecutionPolicy Bypass -Command
"$b=[Convert]::ToBase64String([Text.Encoding]::UTF8.GetBytes('TEC001_BASE64_OK')); [Text.Encoding]::UTF8.GetString([C
onvert]::FromBase64String('$b')) | Write-Host; IEX 'Write-Host TEC001_IEX_OK'; Start-Sleep 2"
TEC001_BASE64_OK
TEC001_IEX_OK
```

Figura 9: Evidencia de ejecución de TEC-001

- **Comportamiento esperado en el host:**
 - creación de un proceso powershell.exe;
 - presencia de -NoProfile;
 - presencia de -ExecutionPolicy Bypass;
 - aparición de los marcadores de control TEC001_BASE64_OK y TEC001_IEX_OK en la ejecución del Código 5;
 - finalización automática tras unos segundos;
 - visibilidad en VR;
 - notificación/salida externa opcional mediante Discord.

Esta prueba sirvió como escenario inicial para validar ejecución de intérpretes de comandos y actividad PowerShell no destructiva. Además permite validar la recepción de este tipo de eventos por VR.

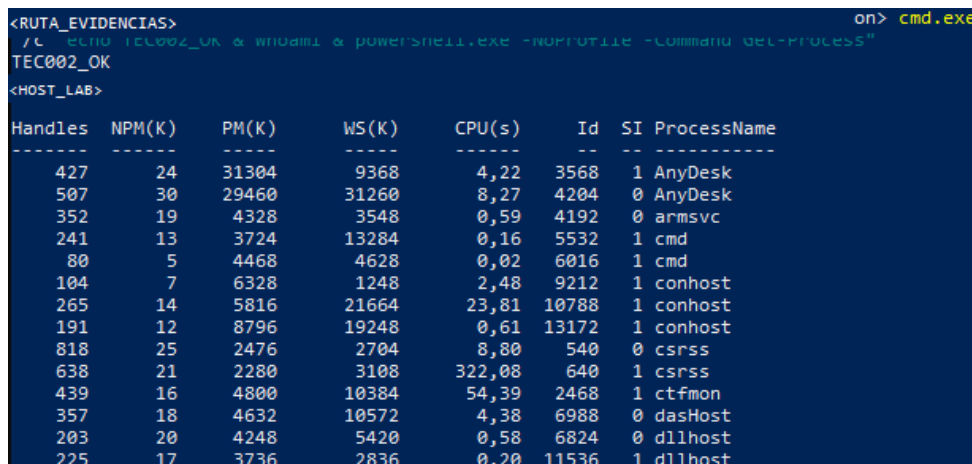
3.7.2. TEC-002 - Ejecución mediante Windows Command Shell

La segunda prueba se correspondió con T1059.003 - Windows Command Shell. TEC-002 genera actividad de cmd.exe y de su proceso hijo PowerShell, junto con salida de consola. Esta técnica permite representar ejecución de comandos del sistema mediante una utilidad nativa de Windows.

- **Herramienta utilizada:** cmd.exe
- **Procedimiento ejecutado:**

```
$ cmd.exe /c "echo TEC002_OK & whoami & powershell.exe -NoProfile -Command Get-Process"
```

Código 6: Procedimiento de ejecución de TEC-002



The screenshot shows a Windows command prompt window. The title bar reads '<RUTA_EVIDENCIAS> on> cmd.exe'. The command entered is `/c echo TEC002_OK & whoami & powershell.exe -NoProfile -Command Get-Process`. The output shows 'TEC002_OK' followed by the user 'HOST_LAB'. Below this, the command `Get-Process` is executed, displaying a list of running processes with columns: Handles, NPM(K), PM(K), WS(K), CPU(s), Id, SI, and ProcessName.

Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
427	24	31304	9368	4,22	3568	1	AnyDesk
507	30	29460	31260	8,27	4204	0	AnyDesk
352	19	4328	3548	0,59	4192	0	armsvc
241	13	3724	13284	0,16	5532	1	cmd
80	5	4468	4628	0,02	6016	1	cmd
104	7	6328	1248	2,48	9212	1	conhost
265	14	5816	21664	23,81	10788	1	conhost
191	12	8796	19248	0,61	13172	1	conhost
818	25	2476	2704	8,80	540	0	csrss
638	21	2280	3108	322,08	640	1	csrss
439	16	4800	10384	54,39	2468	1	ctfmon
357	18	4632	10572	4,38	6988	0	dasHost
203	20	4248	5420	0,58	6824	0	dllhost
225	17	3736	2836	0,20	11536	1	dllhost

Figura 10: Evidencia de ejecución de TEC-002

- **Comportamiento esperado en el host:**
 - creación de un proceso cmd.exe;
 - uso del parámetro /c;
 - ejecución de un comando simple;
 - presencia del marcador TEC002_OK;
 - visibilidad en VR;
 - notificación/salida externa opcional mediante Discord.

3.7.3. TEC-003 - Creación de tarea programada

La tercera prueba se correspondió con T1053.005 - Scheduled Task. Se utilizó `schtasks.exe` para crear una tarea programada, ejecutarla y eliminarla posteriormente. El procedimiento incluye creación, ejecución y limpieza de la tarea TFM-TEC-003.

- **Herramienta utilizada:** `schtasks.exe`
- **Procedimiento ejecutado:**

```

$ taskTime=(Get-Date).AddMinutes(5).ToString("HH:mm")

$ schtasks.exe /create /tn "TFM-TEC-003" /tr "cmd.exe /c echo TEC003_OK >
<HOST_LAB>\tec003_ok.txt" /sc once /st $taskTime /f

$ schtasks.exe /query /tn "TFM-TEC-003" /v /fo list

$ schtasks.exe /run /tn "TFM-TEC-003"

$ Start-Sleep 3

$ Get-Content "<HOST_LAB>\tec003_ok.txt"

$ schtasks.exe /delete /tn "TFM-TEC-003" /f

```

Código 7: Procedimiento de ejecución de TEC-003

```

<RUTA_EVIDENCIAS>
//
>> schtasks.exe /create /tn "TFM-TEC-003" /tr "cmd.exe /c echo TEC003_OK > C:\Users\Public\tec003_ok.txt" /sc once /s
t $taskTime /f>>
>> schtasks.exe /query /tn "TFM-TEC-003" /v /fo list
>>
>> schtasks.exe /run /tn "TFM-TEC-003"
>>
>> Start-Sleep 3
>>
>> Get-Content "C:\Users\Public\tec003_ok.txt"
>>
>> schtasks.exe /delete /tn "TFM-TEC-003" /f
Correcto: se creó correctamente la tarea programada "TFM-TEC-003".

Carpeta: \
Nombre de host: LT29
Nombre de tarea: \TFM-TEC-003
Hora próxima ejecución: 24/06/2026 9:59:00
Estado: Listo
Modo de inicio de sesión: Solo interactivo
Último tiempo de ejecución: 24/06/2026 9:54:08
Último resultado: 0
Autor: <HOST_LAB>
Tarea que se ejecutará: cmd.exe /c echo TEC003_OK > C:\Users\Public\tec003_ok.txt
Iniciar en: N/A
Comentario: N/A
Estado de tarea programada: Habilitado
Tiempo de inactividad: Deshabilitado
Administración de energía: Detener en modo Batería, No iniciar en Batería
Ejecutar como usuario: <HOST_LAB>
Eliminar tarea si no se vuelve a programar: Deshabilitado
Eliminar tarea si ejecuta durante X horas y X minutos: 72:00:00
Programación: Datos de programación no disponibles en este formato.
Tipo de programación: Solo una vez
Hora de inicio: 9:59:00
Fecha de inicio: 24/06/2026
Fecha final: N/A
Días: N/A
Meses: N/A
Repetir: cada: Deshabilitado
Repetir: hasta: hora: Deshabilitado
Repetir: hasta: duración: Deshabilitado
Repetir: detener si aún se ejecuta: Deshabilitado
CORRECTO: se ha intentado ejecutar la tarea programada "TFM-TEC-003".
TEC003_OK
Correcto: se eliminó correctamente la tarea programada "TFM-TEC-003".

```

Figura 11: Evidencia de la tarea programada TEC-003

- **Comportamiento esperado en el host:**
 - creación de una tarea programada temporal;
 - ejecución de la tarea;
 - lanzamiento de cmd.exe como acción asociada;

- creación del fichero <HOST_LAB>\tec003_ok.txt;
- eliminación posterior de la tarea;
- visibilidad en VR;
- notificación/salida externa opcional mediante Discord

Esta prueba reproduce de forma controlada un mecanismo típico de ejecución diferida o persistencia mediante tareas programadas.

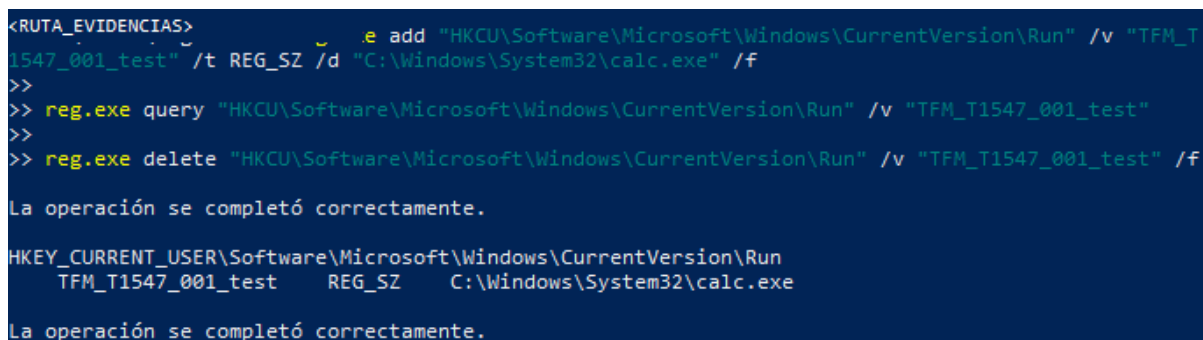
3.7.4. TEC-004 - Persistencia mediante claves Run del registro

La cuarta prueba se correspondió con T1547.001 - Registry Run Keys / Startup Folder. Se utilizó reg.exe para crear una entrada temporal en la clave Run del usuario actual y eliminarla posteriormente.

- **Herramienta utilizada:** reg.exe
- **Procedimiento ejecutado:**

```
$ reg add "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" /v "TFM_T1547_001_test" /t REG_SZ /d "C:\Windows\System32\calc.exe" /f
$ reg delete "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" /v "TFM_T1547_001_test" /f
```

Código 8: Procedimiento de ejecución de TEC-004



```
<RUTA_EVIDENCIAS>
>> reg.exe add "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" /v "TFM_T1547_001_test" /t REG_SZ /d "C:\Windows\System32\calc.exe" /f
>>
>> reg.exe query "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" /v "TFM_T1547_001_test"
>>
>> reg.exe delete "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" /v "TFM_T1547_001_test" /f
La operación se completó correctamente.
HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run
    TFM_T1547_001_test    REG_SZ    C:\Windows\System32\calc.exe
La operación se completó correctamente.
```

Figura 12: Evidencia de persistencia mediante clave Run en TEC-004

- **Comportamiento esperado en el host:**
 - ejecución de reg.exe;
 - creación de un valor temporal en HKCU\...\Run;
 - referencia al binario benigno calc.exe;
 - eliminación del valor creado;
 - ausencia de modificación persistente tras la limpieza;
 - visibilidad en VR;
 - notificación/salida externa opcional mediante Discord

Esta prueba permitió representar una técnica clásica de persistencia sin instalar software adicional ni modificar claves críticas del sistema.

3.7.5. TEC-005 - Ejecución mediante servicio Windows

La quinta prueba se correspondió con T1569.002 - Service Execution. Se utilizó sc.exe desde una consola con privilegios de administrador para crear, arrancar y eliminar un servicio temporal.

- **Herramienta utilizada:** sc.exe
- **Procedimiento ejecutado:**

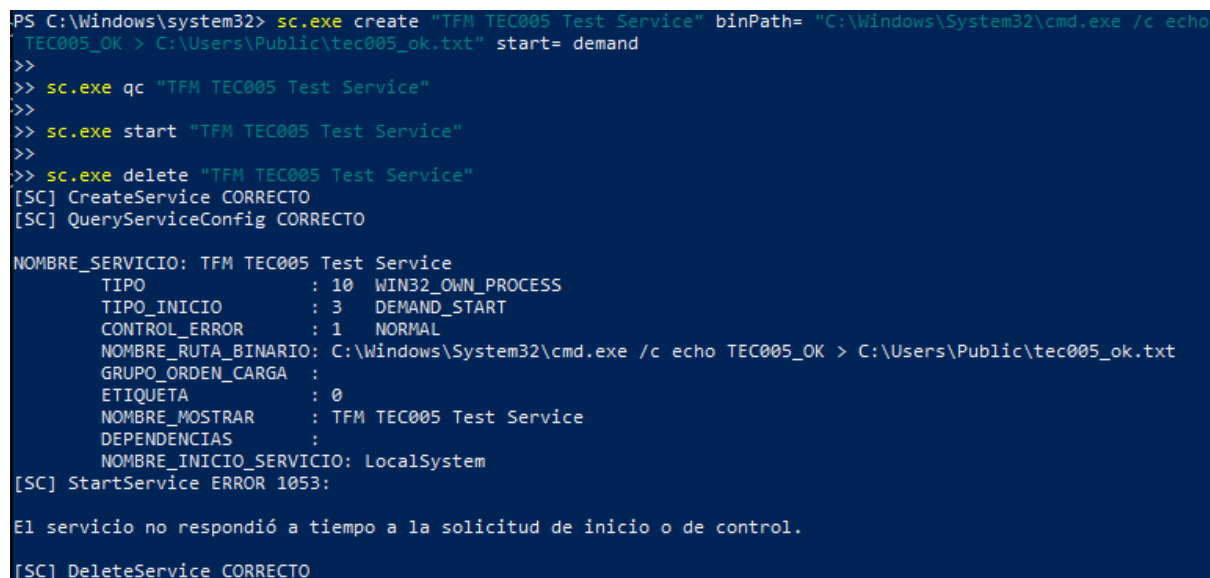
```
$ sc.exe create "TFM TEC005 Test Service" binPath=
"C:\Windows\System32\cmd.exe /c echo TEC005_OK >
<HOST_LAB>\tec005_ok.txt" start= demand

$ sc.exe qc "TFM TEC005 Test Service"

$ sc.exe start "TFM TEC005 Test Service"

$ sc.exe delete "TFM TEC005 Test Service"
```

Código 9: Procedimiento de ejecución de TEC-005



```
PS C:\Windows\system32> sc.exe create "TFM TEC005 Test Service" binPath= "C:\Windows\System32\cmd.exe /c echo
TEC005_OK > C:\Users\Public\tec005_ok.txt" start= demand
>>
>> sc.exe qc "TFM TEC005 Test Service"
>>
>> sc.exe start "TFM TEC005 Test Service"
>>
>> sc.exe delete "TFM TEC005 Test Service"
[SC] CreateService CORRECTO
[SC] QueryServiceConfig CORRECTO

NOMBRE_SERVICIO: TFM TEC005 Test Service
TIPO           : 10  WIN32_OWN_PROCESS
TIPO_INICIO    : 3   DEMAND_START
CONTROL_ERROR  : 1   NORMAL
NOMBRE_RUTA_BINARIO: C:\Windows\System32\cmd.exe /c echo TEC005_OK > C:\Users\Public\tec005_ok.txt
GRUPO_ORDEN_CARGA :
ETIQUETA       : 0
NOMBRE_MOSTRAR  : TFM TEC005 Test Service
DEPENDENCIAS   :
NOMBRE_INICIO_SERVICIO: LocalSystem
[SC] StartService ERROR 1053:

El servicio no respondió a tiempo a la solicitud de inicio o de control.
[SC] DeleteService CORRECTO
```

Figura 13: Evidencia de ejecución del servicio TEC-005

- **Comportamiento esperado en el host:**
 - creación de un servicio Windows temporal;
 - arranque del servicio;
 - ejecución de cmd.exe como binario asociado;
 - creación del fichero <HOST_LAB>\tec005_ok.txt;
 - eliminación posterior del servicio;

- visibilidad en VR;
- notificación/salida externa opcional mediante Discord

Esta prueba reproduce abuso de servicios Windows como mecanismo de ejecución, manteniendo un flujo controlado y reversible.

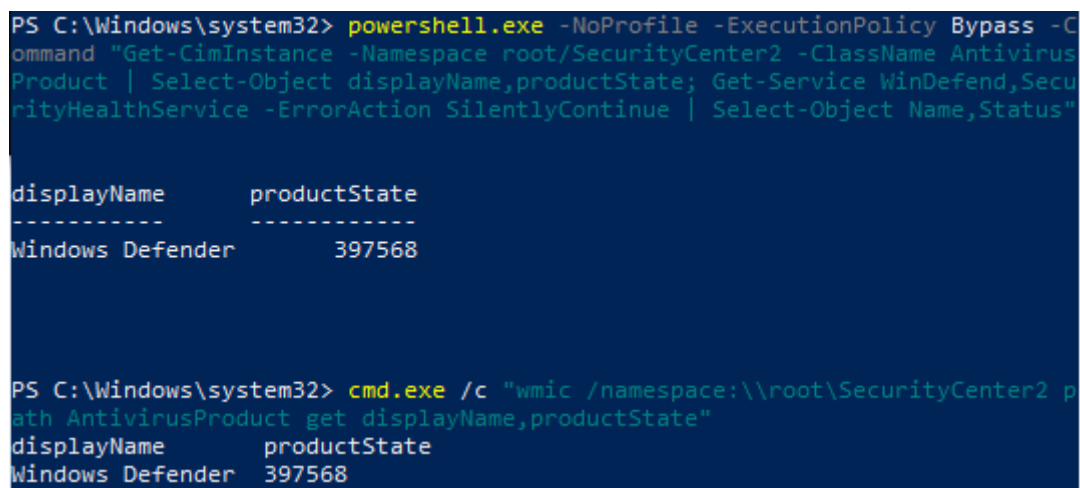
3.7.6. TEC-006 - Descubrimiento de software de seguridad

La sexta prueba se correspondió con T1518.001 - Security Software Discovery. Se ejecutaron consultas benignas para enumerar productos de seguridad registrados en root\SecurityCenter2, usando PowerShell y WMIC.

- **Herramienta utilizada:** powershell.exe, cmd.exe, wmic.exe
- **Procedimiento ejecutado mediante PowerShell:**

```
$ powershell.exe -NoProfile -ExecutionPolicy Bypass -Command "Get-CimInstance -Namespace root/SecurityCenter2 -ClassName AntivirusProduct | Select-Object displayName,productState; Get-Service WinDefend,SecurityHealthService -ErrorAction SilentlyContinue | Select-Object Name,Status"
```

Código 10: Procedimiento de ejecución de TEC-006 mediante PowerShell



```
PS C:\Windows\system32> powershell.exe -NoProfile -ExecutionPolicy Bypass -Command "Get-CimInstance -Namespace root/SecurityCenter2 -ClassName AntivirusProduct | Select-Object displayName,productState; Get-Service WinDefend,SecurityHealthService -ErrorAction SilentlyContinue | Select-Object Name,Status"

displayName      productState
-----
Windows Defender 397568

PS C:\Windows\system32> cmd.exe /c "wmic /namespace:\\root\SecurityCenter2 path AntivirusProduct get displayName,productState"

displayName      productState
Windows Defender 397568
```

Figura 14: Evidencia de descubrimiento de software de seguridad en TEC-006

- **Procedimiento ejecutado mediante WMIC:**

```
$ cmd.exe /c wmic /namespace:\\root\SecurityCenter2 path AntivirusProduct get displayName,productState
```

Código 11: Procedimiento de ejecución de TEC-006 mediante WMIC

- **Comportamiento esperado en el host:**
 - ejecución de PowerShell o CMD;

- consulta a root\SecurityCenter2;
- enumeración de productos AntivirusProduct;
- posible aparición de WMIC.exe como proceso hijo;
- ausencia de modificación sobre los productos de seguridad;
- visibilidad en VR;
- notificación/salida externa opcional mediante Discord

Esta prueba representa una fase de discovery en la que un atacante podría identificar soluciones defensivas antes de ejecutar acciones posteriores.

3.7.7. TEC-007 - Cifrado de datos

La séptima prueba se correspondió con T1486 - Data Encrypted for Impact, técnica principal del TFM. Se ejecutó un simulador benigno desarrollado en PowerShell, denominado Scriptransom.ps1, sobre una carpeta de laboratorio con datos dummy. El flujo documentado en el repositorio complementario [32] incluye lectura de lista_archivos.csv y password.txt, creación de backups, generación de ficheros temporales cifrados y salida final con extensión .aes.

- **Herramienta utilizada:** powershell.exe
- **Procedimiento ejecutado**

```
$ powershell.exe -ExecutionPolicy Bypass -File "<HOST_LAB>\
Desktop\TFM\Pruebas\Ransomware\Scriptransom.ps1"
```

Código 12: Procedimiento de ejecución de TEC-007

- **Comportamiento esperado en el host:**
 - ejecución de PowerShell con ExecutionPolicy Bypass;
 - lanzamiento del script Scriptransom.ps1;
 - lectura de ficheros auxiliares del laboratorio;
 - creación de carpeta backups;
 - generación de archivos cifrados con extensión .aes;
 - operación limitada exclusivamente a datos dummy;
 - visibilidad en VR;
 - notificación/salida externa opcional mediante Discord

TFM > Pruebas > TEC-007_LAB > Projects				Buscar en Projects
Nombre	Fecha de modificación	Tipo	Tamaño	
2025	10/04/2026 12:50	Carpeta de archivos		
2026	10/04/2026 12:50	Carpeta de archivos		
Acta_Reunion_355.aes	10/04/2026 12:50	Archivo AES	1 KB	
Acta_Reunion_364.aes	10/04/2026 12:50	Archivo AES	1 KB	
Acta_Reunion_551.aes	10/04/2026 12:50	Archivo AES	1 KB	
Acta_Reunion_786.aes	10/04/2026 12:50	Archivo AES	1 KB	

Figura 15: Resultado del cifrado controlado: directorio con ficheros cifrados

Los archivos originales se guardan en una carpeta de backups:

io > TFM > Pruebas > Ransomware > backups				Buscar en backups
Nombre	Fecha de modificación	Tipo	Tamaño	
Acta_Reunion_27.pptx_original.txt	09/04/2026 15:20	Documento de te...	1 KB	
Acta_Reunion_31.csv_original.txt	09/04/2026 15:20	Documento de te...	1 KB	
Acta_Reunion_34.jpg_original.txt	09/04/2026 15:20	Documento de te...	1 KB	
Acta_Reunion_37.xlsx_original.txt	09/04/2026 15:20	Documento de te...	1 KB	
Acta_Reunion_46.csv_original.txt	09/04/2026 15:20	Documento de te...	1 KB	
Acta_Reunion_61.pptx_original.txt	09/04/2026 15:20	Documento de te...	1 KB	
Acta_Reunion_90.docx_original.txt	09/04/2026 15:20	Documento de te...	1 KB	
Acta_Reunion_95.txt_original.txt	09/04/2026 15:20	Documento de te...	1 KB	
Acta_Reunion_120.pptx_original.txt	09/04/2026 15:20	Documento de te...	1 KB	

Figura 16: Copia de respaldo de los ficheros tras el cifrado controlado

3.7.8. TEC-008 – Destrucción de datos

La octava prueba se correspondió con T1485 - Data Destruction. Esta técnica representa acciones orientadas a destruir, borrar o inutilizar datos del sistema. En el laboratorio se simuló mediante un script benigno de PowerShell, denominado sabotaje.ps1, limitado exclusivamente al directorio de pruebas.

- **Herramienta utilizada:** powershell.exe

```
$ powershell.exe -ExecutionPolicy Bypass -File "<HOST_LAB>\Desktop
\TFM\01_ACTIVE_TESTS\Pruebas\TEC-008_Data_Destruction\sabotaje.ps1"
```

Código 13: Procedimiento de ejecución de TEC-008

- **Comportamiento esperado en el host:**
 - ejecución de powershell.exe;
 - uso de ExecutionPolicy Bypass;

- lanzamiento del script sabotaje.ps1;
- enumeración de ficheros dentro del directorio de laboratorio;
- eliminación de archivos dummy mediante Remove-Item;
- generación de eventos asociados a borrado de ficheros;
- ausencia de afectación sobre datos reales del sistema;
- posibilidad de restauración mediante recreación del dataset dummy;
- visibilidad en logs de PowerShell y Sysmon;
- posible generación de alerta si la lógica de detección identifica borrado masivo.

La telemetría esperada para esta técnica se basa principalmente en PowerShell Script Block Logging, especialmente Event ID 4104, y en eventos de Sysmon asociados a borrado de ficheros.

3.7.9. TEC-009 - Staging, archivado ZIP y exfiltración HTTP controlada

La novena prueba combina T1074.001 - Local Data Staging, T1560.001 - Archive Collected Data y T1048.003 - Exfiltration Over Unencrypted Non-C2 Protocol. Representa una exfiltración HTTP: preparación de datos dummy, compresión en ZIP y transferencia HTTP. La ejecución prepara datos dummy, genera un archivo comprimido del dummy, calcula un SHA-256 local y realiza un envío HTTP Invoke-WebRequest.

En el equipo receptor se ejecutó un script receiver_tfm_v4 en Python que quedaba a la escucha para poder recibir el zip correctamente.

```
$ python .\receiver_tfm_v4.py

--host 0.0.0.0

--port 8088

--out-dir .\received

--log-file .\received\receiver_log.jsonl

--max-seconds 600

--max-uploads 5

--max-bytes 104857600
```

Código 14: Ejecución del receptor HTTP controlado

```
PS C:\Users\julio> cd "C:\Users\julio\Desktop\TFM\02_SCRIPTS\candidate"
PS C:\Users\julio\Desktop\TFM\02_SCRIPTS\candidate>
PS C:\Users\julio\Desktop\TFM\02_SCRIPTS\candidate> python .\receiver_tfm_v4.py `
>> --host 0.0.0.0 `
>> --port 8088 `
>> --out-dir .\received `
>> --log-file .\received\receiver_log.jsonl `
>> --max-seconds 600 `
>> --max-uploads 5 `
>> --max-bytes 104857600
TFM receiver v4 listening on http://0.0.0.0:8088/upload
Health URL: http://0.0.0.0:8088/health
Output directory: C:\Users\julio\Desktop\TFM\02_SCRIPTS\candidate\received
Log file: C:\Users\julio\Desktop\TFM\02_SCRIPTS\candidate\received\receiver_log.jsonl
Limits: max_seconds=600, max_uploads=5, max_bytes=104857600
[2026-08-12T13:35:10.092899+00:00] 192.168.1.143 - "POST /upload HTTP/1.1" 200 -
```

Figura 17: Receptor HTTP controlado activo para la prueba TEC-009

- **Herramientas utilizadas: PowerShell y receptor HTTP controlado.**
- **Comportamiento esperado en el host:**
 - ejecución de powershell.exe;
 - preparación de datos dummy en staging controlado;
 - lectura de ficheros dummy del dataset de laboratorio;
 - creación de un archivo comprimido mediante Compress-Archive;
 - generación de un fichero .zip;
 - posible copia del archivo a una ruta de salida controlada;
 - envío HTTP hacia un receptor de laboratorio;
 - visibilidad mediante PowerShell 4104;
 - visibilidad mediante Sysmon Event ID 1 para creación de proceso;
 - visibilidad mediante Sysmon Event ID 11 para creación de ficheros;
 - visibilidad mediante Sysmon Event ID 3 si se genera conexión de red.

La técnica permite analizar los límites de la detección de red, ampliando los presuntos límites host-based de la herramienta.

3.8. Ejecución de pruebas legítimas

Además del runner de verdaderos positivos, se desarrolló un runner específico de falsos positivos denominado TFM_Run_FP_Tests_v1.ps1. Su objetivo fue ejecutar actividad benigna y controlada que pudiera parecer sospechosa desde el punto de vista de la telemetría, pero que no debía clasificarse automáticamente como ataque. Esto permite evaluar la precisión de los artifacts y reglas desarrolladas, así como el riesgo de generar alertas innecesarias ante operaciones normales del sistema.

El runner se ejecutó desde la carpeta de runners del TFM:

```
$ cd "<HOST>\Desktop\TFM\03_RUNNERS"  
$ Set-ExecutionPolicy -Scope Process Bypass -Force  
$ .\TFM_Run_FP_Tests_v1.ps1
```

Código 15: Ejecución del runner de falsos positivos

Las Figuras 18 y 19 muestran la ejecución y el resultado de la campaña de pruebas benignas, mientras que la Tabla 32 recoge los diez casos utilizados para evaluar falsos positivos.

```
==== TFM FP RUN SUMMARY ====  
RunId: TFM_FP_20260624_152518  
GlobalStatus: WARN  
OK/WARN/FAIL/SKIPPED: 9/0/0/1  
Total VR hits: 0
```

Figura 18: Resumen de la campaña de pruebas benignas

```

>> hniqes OK/WARN/FAIL: 9/0/0
>> .\TFM_Run_FP_Tests_v1.ps1p\TFM\03_RUNNERS>
[2026-08-12 15:45:44.813] [INFO] RunId: TFM_FP_20260812_154544
[2026-08-12 15:45:44.826] [INFO] Workspace: C:\Users\seguridad\Desktop\TFM\01_ACTIVE_TESTS\Pruebas\FP_WORKSPACE\TFM_FP_20260812_154544
[2026-08-12 15:45:44.826] [INFO] OutputDir: C:\Users\seguridad\Desktop\TFM\01_ACTIVE_TESTS\Pruebas\Logs_Pruebas_TFM\FPs
[2026-08-12 15:45:44.826] [INFO] JsonlPath: \\VBOXSVR\Carpeta_Compartida_TFM\inbox\soc_alerts.jsonl
[2026-08-12 15:45:44.840] [INFO] START FP-001 - PowerShell administrativo legitimo
[2026-08-12 15:45:51.740] [INFO] END FP-001 => OK (6,9s)
[2026-08-12 15:45:51.748] [INFO] START FP-002 - CMD legitimo
[2026-08-12 15:45:52.920] [INFO] END FP-002 => OK (1,17s)
[2026-08-12 15:45:52.945] [INFO] START FP-003 - Scheduled Task benigno temporal
[2026-08-12 15:45:57.726] [INFO] END FP-003 => WARN (4,78s)
[2026-08-12 15:45:57.745] [INFO] START FP-004 - Registro benigno no persistente
[2026-08-12 15:45:57.755] [INFO] END FP-004 => OK (0,01s)
[2026-08-12 15:45:57.769] [INFO] START FP-005 - Consulta de servicios
[2026-08-12 15:45:58.740] [INFO] END FP-005 => OK (0,97s)
[2026-08-12 15:45:58.746] [INFO] START FP-006 - Discovery legitimo de seguridad
[2026-08-12 15:46:01.795] [INFO] END FP-006 => OK (3,05s)
[2026-08-12 15:46:01.795] [INFO] START FP-007 - ZIP benigno local
[2026-08-12 15:46:03.838] [INFO] END FP-007 => OK (2,04s)
[2026-08-12 15:46:03.838] [INFO] START FP-008 - Borrado benigno controlado
[2026-08-12 15:46:04.678] [INFO] END FP-008 => OK (0,84s)
[2026-08-12 15:46:04.686] [INFO] START FP-009 - HTTP benigno local
[2026-08-12 15:46:10.538] [INFO] END FP-009 => SKIPPED (5,84s)
[2026-08-12 15:46:10.575] [INFO] START FP-010 - Operaciones normales masivas
[2026-08-12 15:46:10.870] [INFO] END FP-010 => OK (0,3s)
[2026-08-12 15:46:10.887] [INFO] Waiting 15s before reading SOC JSONL
[2026-08-12 15:46:29.031] [INFO] GlobalStatus: WARN
[2026-08-12 15:46:29.119] [INFO] Tests OK/WARN/FAIL/SKIPPED: 8/1/0/1
[2026-08-12 15:46:29.155] [INFO] Total VR hits: 0

===== TFM FP RUN SUMMARY =====
RunId: TFM_FP_20260812_154544
GlobalStatus: WARN
OK/WARN/FAIL/SKIPPED: 8/1/0/1
Total VR hits: 0

Hits by Profile:
Hits by FP_ID:
Generated files:
- Log: C:\Users\seguridad\Desktop\TFM\01_ACTIVE_TESTS\Pruebas\Logs_Pruebas_TFM\FPs\TFM_FP_20260812_154544.log
- Summary TXT: C:\Users\seguridad\Desktop\TFM\01_ACTIVE_TESTS\Pruebas\Logs_Pruebas_TFM\FPs\TFM_FP_20260812_154544_summary.txt
- Summary JSON: C:\Users\seguridad\Desktop\TFM\01_ACTIVE_TESTS\Pruebas\Logs_Pruebas_TFM\FPs\TFM_FP_20260812_154544_summary.json
- Summary CSV: C:\Users\seguridad\Desktop\TFM\01_ACTIVE_TESTS\Pruebas\Logs_Pruebas_TFM\FPs\TFM_FP_20260812_154544_summary.csv
- VR hits CSV: C:\Users\seguridad\Desktop\TFM\01_ACTIVE_TESTS\Pruebas\Logs_Pruebas_TFM\FPs\TFM_FP_20260812_154544_vr_hits.csv
- VR hits TXT: C:\Users\seguridad\Desktop\TFM\01_ACTIVE_TESTS\Pruebas\Logs_Pruebas_TFM\FPs\TFM_FP_20260812_154544_vr_hits.txt

```

Figura 19: Resultado detallado de la campaña de pruebas benignas

El script genera un identificador único de ejecución, crea un workspace temporal y almacena los resultados en la carpeta de falsos positivos del laboratorio. Además, puede consultar el fichero JSONL de alertas generado por el flujo de Velociraptor para comprobar si las operaciones benignas han producido alertas P1, P2, P3 o P4.

La ruta de salida utilizada por defecto es <HOST_LAB>\Desktop\TFM\01_ACTIVE_TESTS\Pruebas\Logs_Pruebas_TFM\FPs.

El runner genera los siguientes ficheros principales:

- log detallado de ejecución;
- resumen TXT;
- resumen JSON;
- resumen CSV;
- fichero TXT con posibles hits detectados por Velociraptor;
- fichero CSV con posibles hits detectados por Velociraptor.

Las pruebas benignas incluidas fueron:

ID	Descripción	Finalidad
FP-001	PowerShell administrativo legítimo	Verificar si comandos administrativos básicos generan alertas indebidas.
FP-002	CMD legítimo	Comprobar ruido asociado a ejecución normal de cmd.exe.
FP-003	Scheduled Task benigno temporal	Evaluar si una tarea programada legítima se clasifica como persistencia maliciosa.
FP-004	Registro benigno no persistente	Comprobar ruido en operaciones de registro sin finalidad maliciosa.
FP-005	Consulta de servicios	Validar si consultas normales sobre servicios generan detecciones.
FP-006	Discovery legítimo de seguridad	Contrastar actividad administrativa similar a Security Software Discovery.
FP-007	ZIP benigno local	Evaluar si una compresión local legítima se confunde con staging.
FP-008	Borrado benigno controlado	Medir ruido ante eliminación controlada de ficheros temporales.
FP-009	HTTP benigno local	Evaluar si una conexión HTTP controlada genera alerta de exfiltración.
FP-010	Operaciones normales masivas	Medir sensibilidad ante actividad de ficheros de volumen elevado.

Tabla 32: Casos benignos incluidos en el runner de falsos positivos

Cada caso registra estado, duración, artefactos generados y detalles de ejecución. Posteriormente, el runner revisa la ventana temporal correspondiente en el fichero soc_alerts.jsonl para identificar si alguna operación benigna ha generado una alerta. Cuando se encuentra una coincidencia, el script clasifica el impacto potencial del falso positivo según el perfil de severidad observado.

El resultado de estas pruebas permite valorar:

- si las reglas son demasiado genéricas;
- si PowerShell o CMD generan ruido excesivo;
- si las reglas P4 aportan visibilidad útil o demasiadas alertas;
- si las técnicas de staging, ZIP o HTTP requieren mayor contexto;
- si las reglas custom necesitan exclusiones, umbrales o correlación adicional.

Por tanto, el runner de falsos positivos completa la validación principal. Mientras el runner global evalúa la capacidad de detectar técnicas MITRE ATT&CK controladas, el runner de falsos positivos evalúa la capacidad de no alertar indebidamente ante actividad benigna.

3.9. Recopilación de telemetría

La recopilación de telemetría se diseñó para mantener la trazabilidad entre la técnica ejecutada, la fuente de datos observada y la evidencia conservada. En esta fase no se valoran todavía los resultados de detección, sino las fuentes desde las que se espera obtener información útil para el análisis posterior.

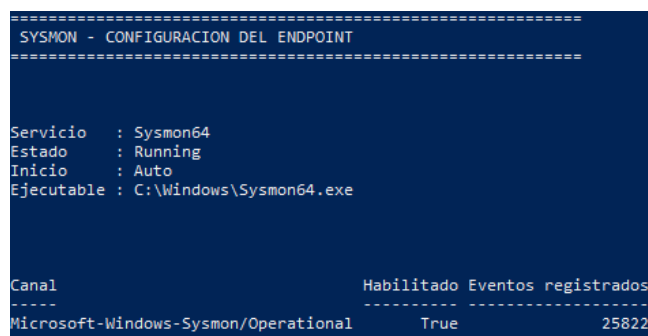
La telemetría se agrupó en tres niveles:

- telemetría generada por el propio sistema Windows;
- telemetría observada y procesada por Velociraptor;
- salida externa generada por el flujo de alertado hacia Discord.

La Tabla 33 y las Figuras 20 a 25 documentan la telemetría esperada y las principales evidencias obtenidas en Windows, Velociraptor y el canal de salida externa.

3.9.1. Telemetría captada en el sistema

La primera fuente de información procede del propio endpoint Windows. Para ello se utilizaron principalmente Sysmon, PowerShell Script Block Logging y eventos nativos de Windows.



```

=====
SYSMON - CONFIGURACION DEL ENDPOINT
=====

Servicio   : Sysmon64
Estado     : Running
Inicio     : Auto
Ejecutable : C:\Windows\Sysmon64.exe

Canal
-----
Microsoft-Windows-Sysmon/Operational      True      25822
  
```

Figura 20: Estado de Sysmon y del canal de telemetría del endpoint

Las fuentes principales fueron:

- Sysmon Event ID 1: creación de procesos, línea de comandos, usuario, imagen ejecutada y proceso padre.
- Sysmon Event ID 3: conexiones de red asociadas a procesos.
- Sysmon Event ID 11: creación de ficheros.
- Sysmon Event ID 12/13/14: creación, modificación y borrado de claves o valores de registro.
- Sysmon Event ID 26: evidencia de borrado de ficheros.

```
=====
SYSMON - VERIFICACION DE TELEMETRIA DEL TFM
=====
```

EventID	Telemetria	Evidencia	UltimoRegistro
1	Creacion de procesos	SI	14/08/2026 15:06:23
3	Conexiones de red	SI	14/08/2026 15:06:05
11	Creacion de ficheros	SI	14/08/2026 15:06:25
12	Creacion de claves de Registro	SI	14/08/2026 15:06:22
13	Modificacion de valores de Registro	SI	14/08/2026 15:06:22
14	Renombrado de claves de Registro	SI	14/08/2026 15:06:23
26	Borrado de ficheros	SI	14/08/2026 15:06:27

Figura 21: Verificación de las fuentes de telemetría Sysmon habilitadas y registradas en el endpoint

- PowerShell Event ID 4104: bloques de código PowerShell ejecutados.

```
=====
POWERSHELL - SCRIPT BLOCK LOGGING
=====
```

Configuracion	Habilitado	EventID	Evidencia	UltimoRegistro
Script Block Logging	SI	4104	SI	14/08/2026 14:49:10

Figura 22: Verificación de PowerShell Script Block Logging y Event ID 4104

- Windows System Event ID 7045: instalación o creación de servicios Windows.
- Logs del receptor HTTP: evidencia auxiliar para TEC-009 cuando el upload HTTP está habilitado.

La siguiente tabla resume la telemetría esperada por técnica:

Técnica	Telemetría esperada
TEC-001	Sysmon ID 1 por ejecución de powershell.exe; PowerShell 4104 con parámetros como NoProfile, ExecutionPolicy Bypass, Base64 o IEX.
TEC-002	Sysmon ID 1 por ejecución de cmd.exe; línea de comandos; posible proceso hijo de PowerShell; fichero de evidencia temporal.
TEC-003	Sysmon ID 1 por schtasks.exe y cmd.exe; eventos asociados a creación, ejecución y eliminación de tarea programada.
TEC-004	Sysmon ID 1 por reg.exe; Sysmon ID 12/13/14 por modificación temporal de clave Run del registro.
TEC-005	Sysmon ID 1 por sc.exe y cmd.exe; Windows System Event ID 7045 por creación del servicio temporal.
TEC-006	Sysmon ID 1 por powershell.exe, cmd.exe o wmic.exe; PowerShell 4104; consultas a root\SecurityCenter2.

Técnica	Telemetría esperada
TEC-007	Sysmon ID 1 por powershell.exe; PowerShell 4104; Sysmon ID 11 por creación de ficheros .aes y backups.
TEC-008	Sysmon ID 1 por powershell.exe; PowerShell 4104; Sysmon ID 26 por borrado de ficheros dummy.
TEC-009	Sysmon ID 1 por powershell.exe; PowerShell 4104; Sysmon ID 11 por creación del ZIP; Sysmon ID 3 si existe conexión HTTP; logs del receiver si hay upload.

Tabla 33: Telemetría esperada por técnica experimental

Por eficiencia, en las técnicas TEC-007, TEC-008 y TEC-009 no se usa cada evento como alerta directa, ya que un borrado/cifrado/envío masivo puede generar un volumen masivo de eventos individuales.

TEC	Estado	Evidencia VR
TEC-001	OK	TEC001_PowerShell_4104, 1 fila, Confidence=HIGH, FromBase64String, IEX
TEC-002	OK	TEC002_CMD_Sysmon, 27 filas, cmd.exe /c, algo ruidoso
TEC-003	OK	TEC003_ScheduledTask_Sysmon, 8 filas, schtasks.exe create/delete
TEC-004	OK	TEC004_RunKey_Sysmon, 6 filas, reg.exe, Run key
TEC-005	OK	TEC005_Service_System7045, 1 fila, System 7045
TEC-006	OK	TEC006_SecuritySoftwareDiscovery_4104, 2 filas; Sysmon fallback 5 filas
TEC-007	OK	TEC007_DataEncryptedForImpact_4104, 4 filas, crypto/AES
TEC-008	OK	TEC008_DataDestruction_4104, 1 fila, Get-ChildItem, Remove-Item, -Force
TEC-009	OK parcial	P1 4104 fuerte real + P3 Sysmon process context; P3 4104 fuerte tiene ruido

Figura 23: Evidencia de telemetría registrada en Windows durante las pruebas

3.9.2. Telemetría captada en Velociraptor

Velociraptor actúa como capa de recogida, filtrado y estructuración de eventos. La telemetría generada por Windows, Sysmon y PowerShell puede ser consultada mediante los distintos artifacts.

Como se ha declarado en el capítulo de metodología, se emplearon dos tipos de observación:

- puntual, mediante collections o consultas sobre logs;
- continua, mediante artifacts de tipo CLIENT_EVENT.

La monitorización continua es la más relevante para la aproximación HIDS, ya que permite emitir eventos durante la ejecución de la técnica. Aun así, la existencia de eventos en Velociraptor no equivale automáticamente a detección. Para que exista detección, el artifact debe aplicar una lógica capaz de clasificar la actividad como relevante y asociarla a una técnica o perfil concreto.

La telemetría observada por Velociraptor se utilizó para:

- registrar procesos y líneas de comando;
- consultar eventos PowerShell 4104;
- consultar eventos Sysmon;
- observar modificaciones del registro;
- detectar creación de servicios;
- identificar creación o eliminación de ficheros;
- observar conexiones de red asociadas al endpoint;
- normalizar eventos mediante artifacts personalizados;
- generar salidas estructuradas para análisis posterior.

Las evidencias obtenidas desde Velociraptor se conservaron como capturas de GUI, resultados de Client Events, exportaciones CSV/JSONL y registros de ejecución asociados a cada técnica.

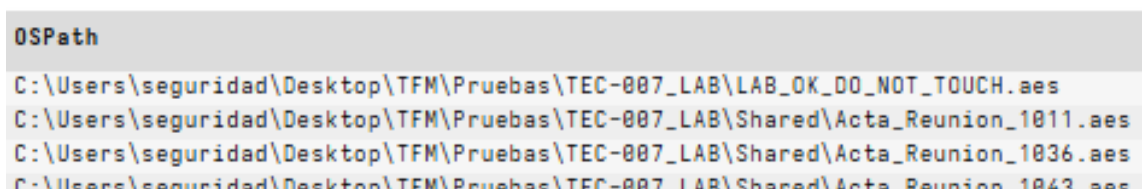


Figura 24: Evidencia de telemetría observada en Velociraptor durante las pruebas

3.9.3. Salida externa de alertado (JSONL/Discord)

Se ha usado Discord como canal externo de notificación por su facilidad de uso como webhook, al crear un canal, en la parte de configuración se puede obtener un token para usar ese canal para mostrar las alertas. En general su función es hacer el papel de un dashboard o una herramienta final como un SIEM al comprobar que una detección puede procesarse mediante SERVER_EVENT.

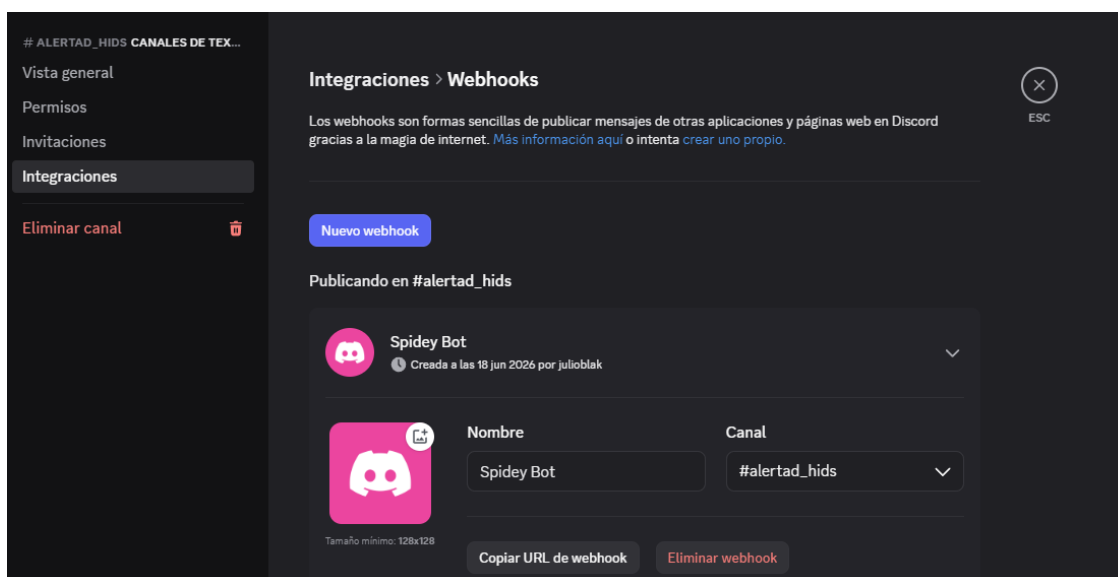


Figura 25: Configuración Discord-Webhook

En las capturas, el campo denominado «IOC» se interpreta como contexto o IOA capturado y generado por el laboratorio cuando contiene rutas, nombres de scripts o etiquetas. Esos valores no son IOC reales ni condiciones de detección y no se utilizan para acreditar cobertura, simplemente se usa como etiqueta metodológica.

El artifact CLIENT_EVENT observa el comportamiento en el endpoint. Posteriormente, un artifact SERVER_EVENT procesa los eventos recibidos por el servidor, genera la salida y la envía a un webhook (Discord).

Para considerar válida la salida externa se comprobaron los siguientes elementos:

- existencia del evento en Velociraptor;
- procesamiento por el artifact SERVER_EVENT;
- generación de una entrada estructurada en formato JSONL;
- envío mediante webhook;
- respuesta satisfactoria del servicio externo;
- recepción visible de la notificación en Discord.
- campos principales del evento procesado.

3.10. Configuración y uso de artifacts

Primero se revisaron artifacts de fábrica y públicos de Velociraptor para establecer una base. Después se desarrollaron artifacts personalizados para mejorar la clasificación, priorización y salida externa de eventos.

Los artifacts públicos utilizados son los descritos en el apartado 3.5.1, «Artifacts públicos considerados». A partir de las limitaciones observadas se desarrollaron los artifacts personalizados.

3.10.1. Artifacts personalizados

Por conveniencia para el proyecto, se desarrolló una arquitectura personalizada basada en perfiles de severidad P1, P2, P3 y P4. Su objetivo fue transformar eventos host-based en señales más estructuradas, priorizadas y trazables, es decir, alertas.

La Tabla 34 resume los artifacts personalizados desarrollados, indicando su perfil, tipo, volumen de filas y función operativa.

Perfil	Artifact	Tipo	Filas	Función
P1	Custom.TFM.HIDS.P1.Critical.Priority.Event_v1	CLIENT_EVENT	19	Señales críticas selectivas
P2	Custom.TFM.HIDS.P2.High.Forensic.Event_v1	CLIENT_EVENT	118	Detección, contexto y evidencia forense; no constituye una unidad homogénea

Perfil	Artifact	Tipo	Filas	Función
P3	Custom.TFM.HIDS.P3.Medium.Behavioral.Event_v1	CLIENT_EVENT	109	Detección de comportamiento
P4	Custom.TFM.HIDS.P4.Low.Basic_v2	CLIENT_EVENT	133	Visibilidad básica
Router	Custom.TFM.HIDS.Router.JSONL.Discord.SOC_v3	SERVER_EVENT	N/A	Enruta, persiste o notifica; no detecta

Tabla 34: Artifacts custom por perfil y función operativa

Los artifacts personalizados se diseñaron para:

- asociar eventos a técnicas TEC-001 a TEC-009;
- normalizar campos relevantes;
- asignar perfil de severidad;
- reducir dependencia de indicadores de laboratorio;
- evitar reglas basadas únicamente en nombres de scripts o rutas TFM;
- facilitar la salida estructurada hacia JSONL y Discord;
- diferenciar eventos de baja señal frente a comportamientos de impacto.

3.10.2. Limitaciones encontradas durante la configuración

En primer lugar, muchas actividades administrativas legítimas generan la misma telemetría base que una técnica ofensiva. La ejecución de PowerShell, CMD, reg.exe, sc.exe o schtasks.exe no es maliciosa por sí misma. Por ello, la detección requiere contexto adicional, como parámetros usados, ventana temporal, técnica ejecutada, fuente del evento y relación con otros eventos.

En TEC-008 no se considera cada evento de eliminación como una alerta independiente. Los resultados finales incluyen cuatro filas CLIENT_EVENT asociadas al caso y no contienen filas Sysmon ID 26. La agregación temporal de eventos de borrado se plantea como trabajo futuro; no se presenta como una capacidad validada en esta campaña.

En TEC-009 exige separar dos evidencias. Los artifacts custom CLIENT_EVENT aportan detección host-based sobre staging, creación del ZIP, PowerShell y actividad de proceso. Por separado, el summary del runner independiente documenta ReceiverReachable=True, HTTPStatus=200, UploadSucceeded=True, SHA-256 local y bytes transferidos. El HTTP 200 es ground truth de ejecución, no una alerta CLIENT_EVENT ni una detección del artifact TrackNetwork; este último observó la conexión sin atribución fiable de proceso.

En cuarto lugar, la salida a Discord valida el flujo externo de alertado, pero no demuestra por sí sola la calidad de la detección. La calidad de la detección debe evaluarse en función de la lógica del artifact, la técnica asociada, la evidencia disponible y el ruido generado.

Por último, la configuración personalizada requiere ajuste iterativo. Velociraptor permite hacerlo de forma sencilla y distribuida, pero esa misma flexibilidad obliga a diseñar, probar y refinar artefactos para reducir falsos positivos y evitar sobreclasificación.

Capítulo IV: Resultados de detección

Este capítulo expone los resultados finales obtenidos con Velociraptor en las campañas finales, campaña real del 24/06 y en la campaña de técnicas benignas del 26/06. Todas las salidas JSONL y Discord se conservan como salidas trazables derivadas del flujo de monitorización para ampliación en caso de necesitarlo.

Los artifacts personalizados alcanzaron cobertura sobre los nueve casos de prueba, mientras que la cobertura específica de los artifacts públicos fue más limitada y dependiente de las reglas disponibles.

La Figura 26 resume el resultado global obtenido en cada campaña experimental.

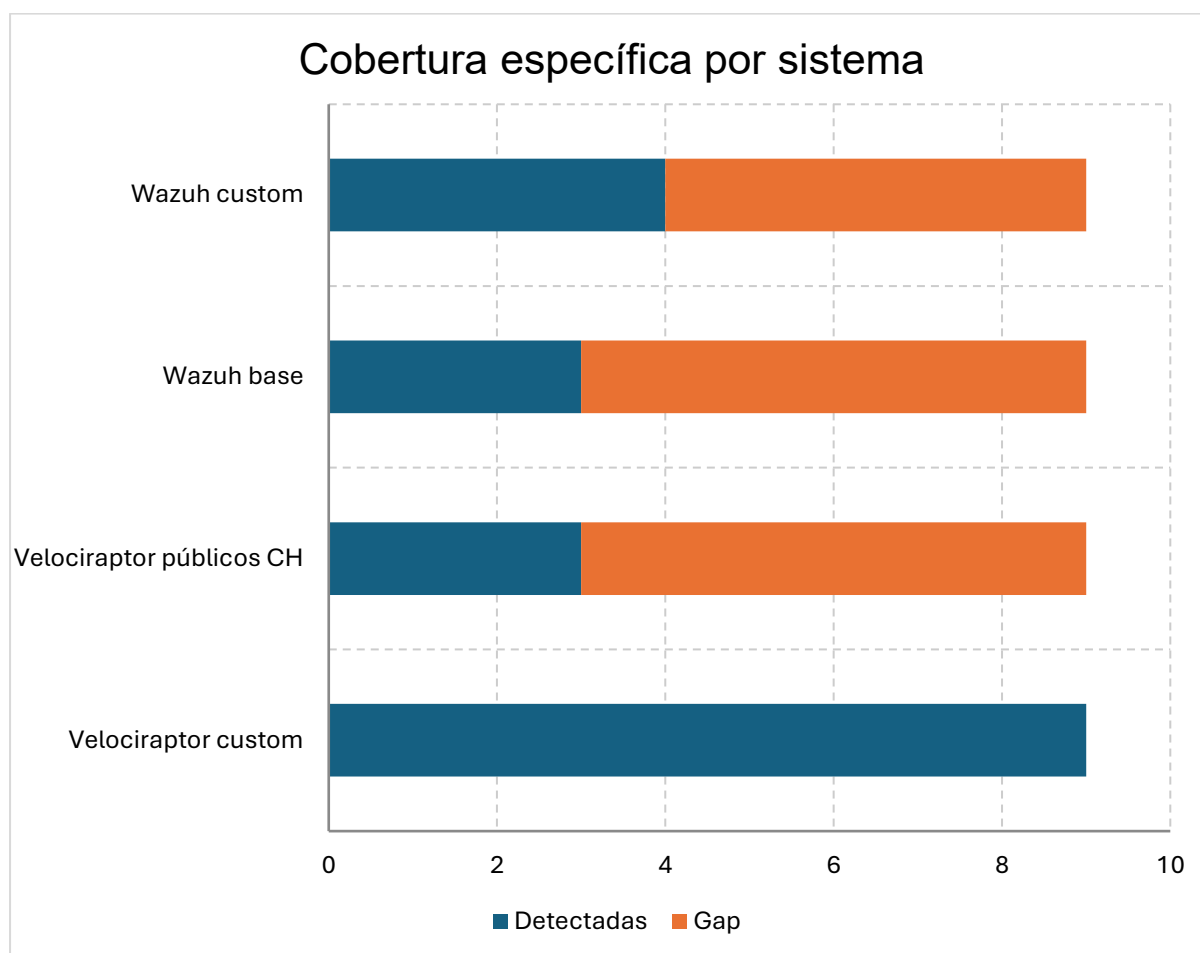


Figura 26: Resultado general de cobertura por campaña de ataques

Velociraptor custom alcanza 9/9, frente a 3/9 de los artifacts públicos CH, 3/9 de Wazuh base y 4/9 de Wazuh custom. La comparación es experimental y limitada a las reglas/configuraciones evaluadas.

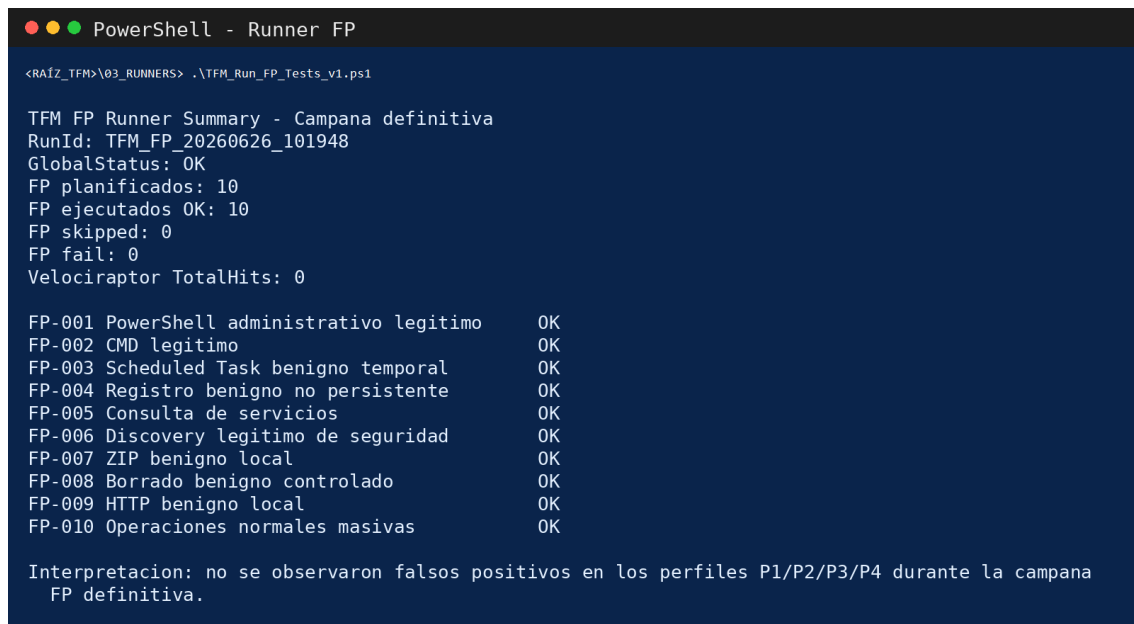
4.1. Detecciones ante actividad legítima

La campaña de actividad legítima se diseñó para comprobar si los artifacts personalizados generaban alertas ante acciones benignas que podían resultar similares a comportamientos sospechosos. Se ejecutaron diez casos controlados relacionados con PowerShell, CMD, tareas programadas, registro, servicios, discovery, compresión ZIP, borrado, tráfico HTTP y operaciones masivas sobre ficheros. La campaña finalizó con 10 ejecuciones correctas y ningún hit definitivo de Velociraptor, por lo que no se observaron falsos positivos en el conjunto de pruebas benignas evaluado.

4.1.1. Resultados de la campaña de técnicas benignas con artifacts personalizados

La campaña de técnicas benignas del 26/06 ejecutó diez acciones benignas y obtuvo 10 OK, 0 WARN, 0 FAIL y 0 SKIPPED. El resumen final indica VelociraptorHits.Total=0 y el fichero vr_hits.csv no contiene hits definitivos.

La Figura 27 y la Tabla 35 resumen los estados de ejecución y los hits obtenidos durante la campaña definitiva de técnicas benignas.



```

PowerShell - Runner FP

<RAIZ_TFM>\03_RUNNERS> .\TFM_Run_FP_Tests_v1.ps1

TFM FP Runner Summary - Campana definitiva
RunId: TFM_FP_20260626_101948
GlobalStatus: OK
FP planificados: 10
FP ejecutados OK: 10
FP skipped: 0
FP fail: 0
Velociraptor TotalHits: 0

FP-001 PowerShell administrativo legitimo      OK
FP-002 CMD legitimo                           OK
FP-003 Scheduled Task benigno temporal         OK
FP-004 Registro benigno no persistente         OK
FP-005 Consulta de servicios                   OK
FP-006 Discovery legitimo de seguridad         OK
FP-007 ZIP benigno local                       OK
FP-008 Borrado benigno controlado              OK
FP-009 HTTP benigno local                     OK
FP-010 Operaciones normales masivas            OK

Interpretacion: no se observaron falsos positivos en los perfiles P1/P2/P3/P4 durante la campana FP definitiva.

```

Figura 27: Campaña definitiva de falsos positivos

Prueba	Acción benigna	Estado	Hits custom
FP-001	PowerShell administrativo legítimo	OK	0
FP-002	CMD legítimo	OK	0
FP-003	Scheduled Task benigna temporal	OK	0
FP-004	Registro benigno no persistente	OK	0
FP-005	Consulta de servicios	OK	0
FP-006	Discovery legítimo de seguridad	OK	0

Prueba	Acción benigna	Estado	Hits custom
FP-007	ZIP benigno local	OK	0
FP-008	Borrado benigno controlado	OK	0
FP-009	HTTP benigno local	OK	0
FP-010	Operaciones normales masivas	OK	0

Tabla 35: Resultados definitivos del runner de técnicas benignas

4.2. Detecciones ante ataques

La evaluación frente a actividad maliciosa se realizó sobre las nueve técnicas MITRE ATT&CK definidas en el entorno experimental, ejecutadas de forma controlada mediante el runner global. Los resultados se analizan diferenciando dos enfoques: los artifacts personalizados, diseñados específicamente para las técnicas evaluadas, y los artifacts públicos, utilizados como referencia para comprobar qué capacidad de detección ofrecen sin adaptación específica. Esta separación permite comparar cobertura, volumen de señales, calidad de la detección y limitaciones de cada aproximación.

4.2.1. Resultados por ataque en campaña personalizada

Las nueve técnicas planificadas finalizaron en estado OK y generaron visibilidad de sistema y detección Velociraptor mediante CLIENT_EVENT. La cobertura funcional es, por tanto, 9/9 sobre el subconjunto experimental TEC-001 a TEC-009.

La Tabla 36 y las Figuras 28 a 32 sintetizan los resultados obtenidos por técnica, táctica y perfil en la campaña custom. En este análisis, cada detección corresponde a una fila CLIENT_EVENT emitida tras cumplirse la lógica de clasificación del artifact.

```

<RAÍZ_TFM>\03_RUNNERS> .\TFM_Run_All_TEC_Tests_v6_PATCHED_4104.ps1

<HOST_LAB>
-KeepExfilArtifacts $true -KeepArtifacts $true

<RUTA_EVIDENCIAS>
Start: 2026-06-24T14:50:22+02:00
End: 2026-06-24T14:55:53+02:00
DurationSeconds: 331.66

<HOST_LAB>
GlobalStatus: OK
Techniques OK/WARN/FAIL: 9/0/0

TEC-001 | PowerShell / T1059.001 | OK | Sysmon=4 | PS4104=3
TEC-002 | Windows Command Shell / T1059.003 | OK | Sysmon=4 | PS4104=1
TEC-003 | Scheduled Task / T1053.005 | OK | Sysmon=6 | PS4104=0
TEC-004 | Registry Run Key / T1547.001 | OK | Sysmon=5 | PS4104=0
TEC-005 | Service Execution / T1569.002 | OK | Sysmon=6 | PS4104=0
TEC-006 | Security Software Discovery / T1518.001 | OK | Sysmon=2 | PS4104=2
TEC-007 | Data Encrypted for Impact / T1486 | OK | Sysmon=9 | PS4104=4
TEC-008 | Data Destruction / T1485 | OK | Sysmon=4 | PS4104=3
TEC-009 | Staging / ZIP / Controlled HTTP Upload | OK | Sysmon=7 | PS4104=1

```

Figura 28: Resumen del runner global de técnicas

TEC	MITRE	Técnica evaluada	Sysmon relevantes	Estado
TEC-001	T1059.001	PowerShell	4	OK
TEC-002	T1059.003	Windows Command Shell	4	OK
TEC-003	T1053.005	Scheduled Task	6	OK
TEC-004	T1547.001	Registry Run Key	5	OK
TEC-005	T1569.002	Service Execution	6	OK
TEC-006	T1518.001	Security Software Discovery	2	OK
TEC-007	T1486	Data Encrypted for Impact	9	OK
TEC-008	T1485	Data Destruction	4	OK
TEC-009	T1074.001 / T1560.001 / T1048.003	Staging, ZIP y upload HTTP controlado	7	OK

Tabla 36: Resultado de ejecución y telemetría comprobada por el runner

Velociraptor ha mostrado una disparidad alta en la captación de técnicas. TEC-002 domina el volumen (214 filas) y se concentra sobre todo en P2/P3/P4. Esto demuestra que el número de alertas no equivale a mayor cobertura: una sola técnica puede producir mucho más ruido que otras.

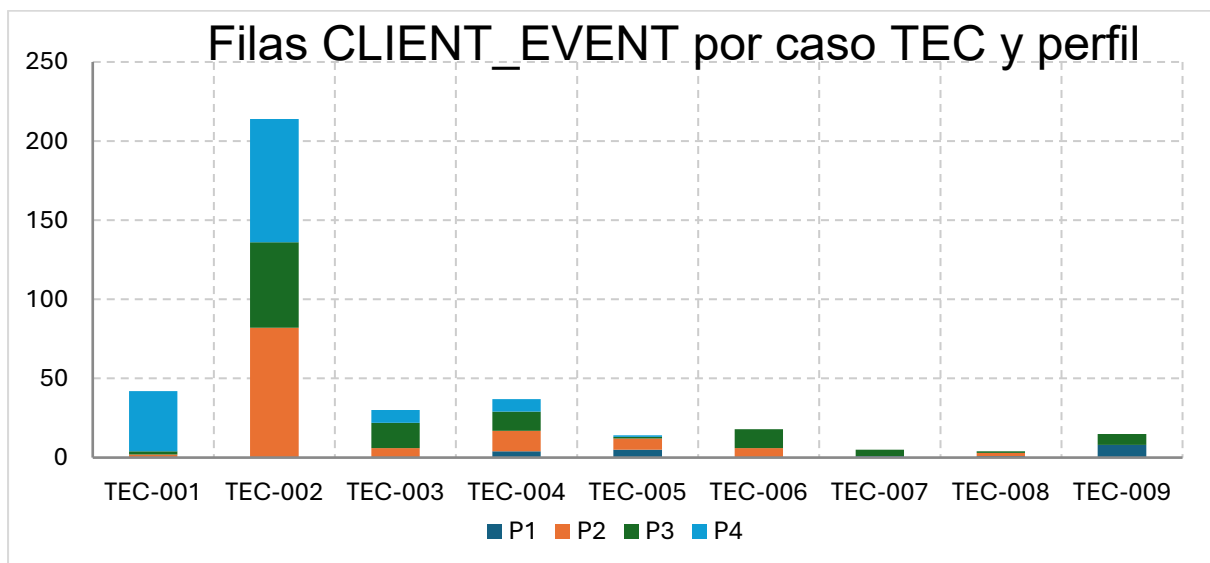


Figura 29: Filas CLIENT_EVENT por caso TEC y perfil

Si extrapolamos esto a MITRE tenemos que en el entorno experimental usado, las técnicas asociadas a Execution generaron la mayor densidad de señales:

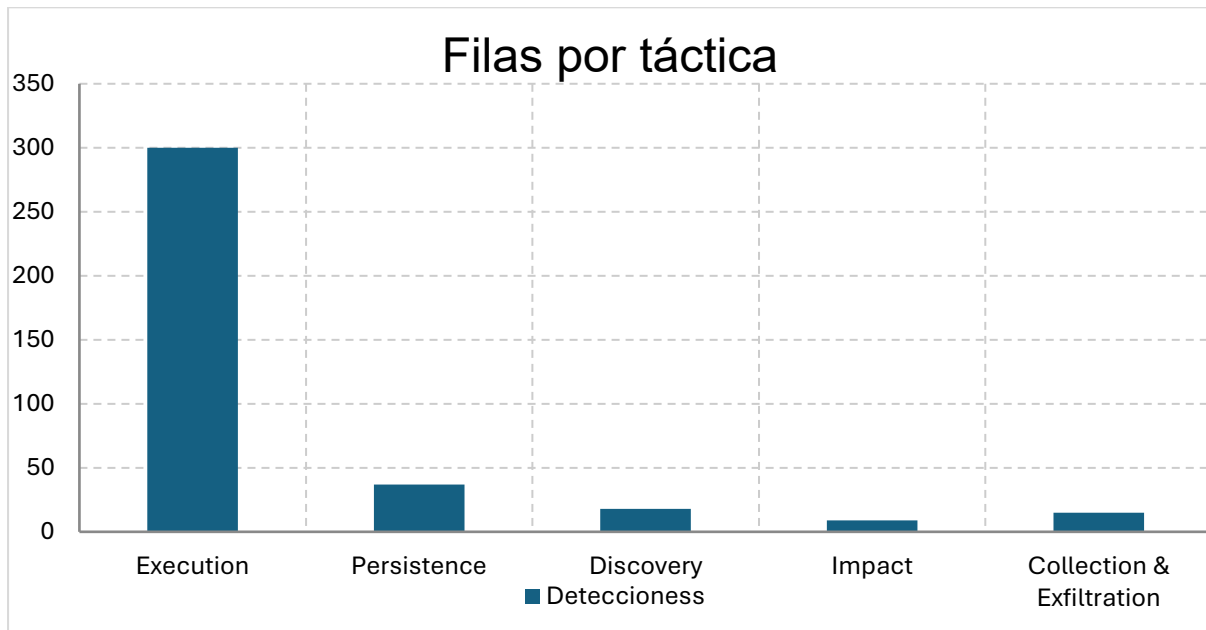


Figura 30: Filas CLIENT_EVENT por categoría táctica

Execution concentra 300/379 (79,2 %) de las señales. Parte del efecto procede de que contiene cuatro técnicas, por lo que conviene complementar este gráfico con la media por técnica. Esto podría ser predecible, pues son las que más huellas suelen dejar.

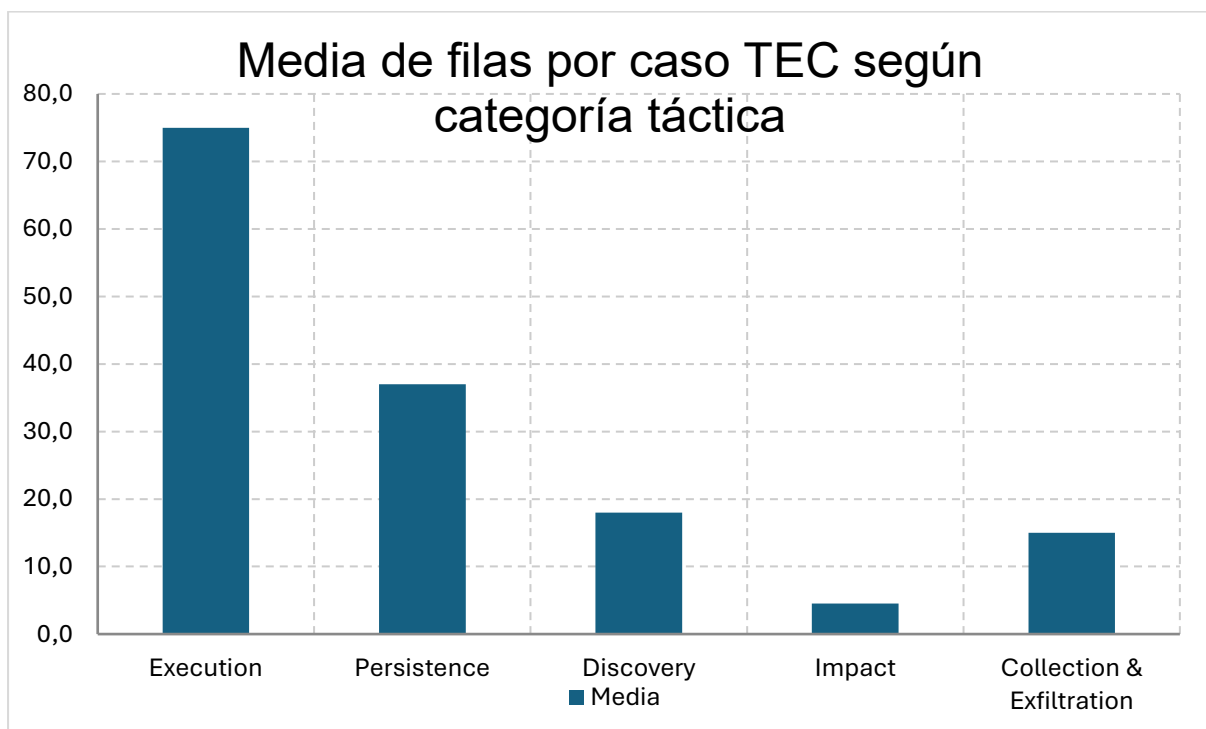


Figura 31: Media de filas CLIENT_EVENT por caso TEC según categoría táctica

Execution continúa siendo la categoría más intensa incluso tras normalizar (75 filas por técnica). Impact presenta la menor densidad ($\sim 4,5$), aunque ambas técnicas de impacto fueron detectadas.

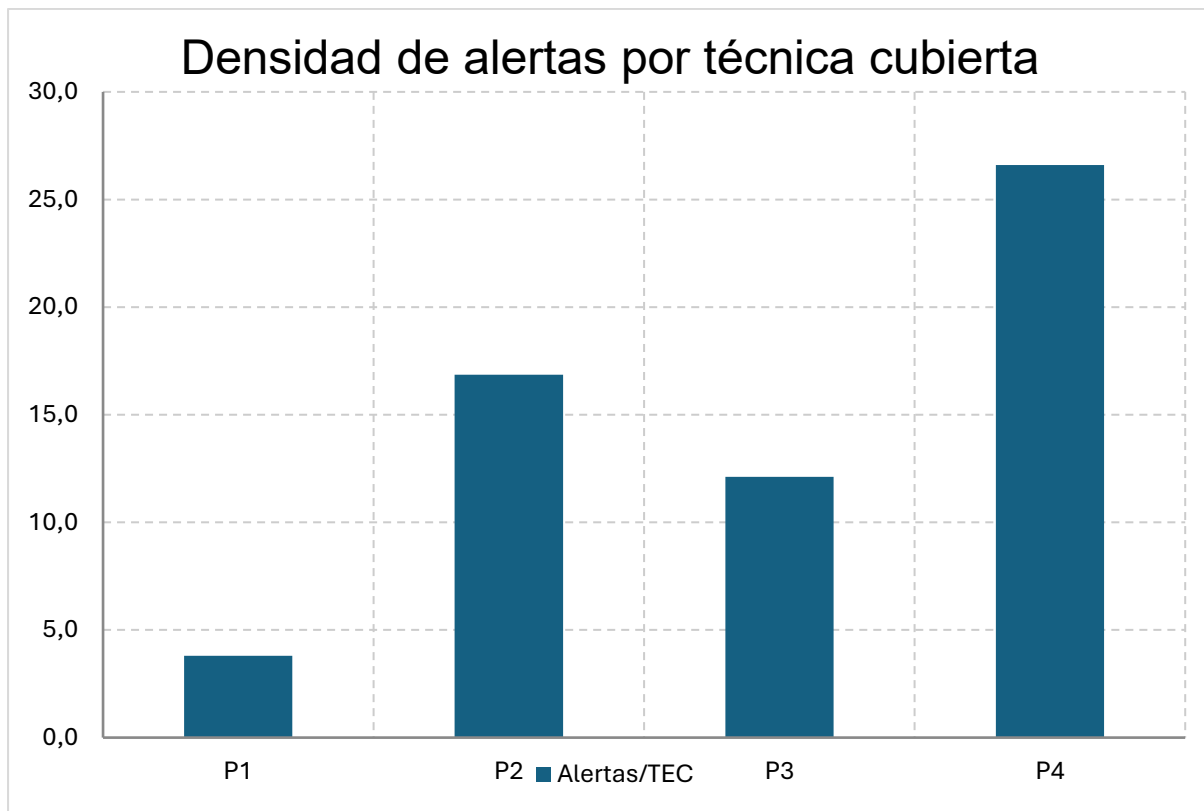


Figura 32: Distribución de alertas por técnica cubierta

Como es evidente y positivo, P1 es el perfil más selectivo ($\sim 3,8$ alertas por técnica cubierta), mientras P4 alcanza $\sim 26,6$. P3 destaca ya que presenta una cobertura completa con una densidad intermedia ($\sim 12,1$).

4.2.1.1. Cobertura de detección

La campaña final incluyó 379 filas CLIENT_EVENT reales dentro de la ventana de tiempo de ejecución del script e ingesta de estos datos. La distribución por prioridad/criticidad de artifacts fue P1=19, P2=118, P3=109 y P4=133.

La Figura 33 y las Tablas 37 y 38 relacionan la arquitectura custom con la distribución de filas CLIENT_EVENT por perfil y por técnica.

```

Arquitectura de artifacts Velociraptor
-----
CLIENT_EVENT P1_CRITICAL: comportamientos criticos; TEC-004, TEC-005, TEC-007, TEC-008, TEC-009
CLIENT_EVENT P2_HIGH: evidencia forense; TEC-001 a TEC-009
CLIENT_EVENT P3_MEDIUM: comportamiento relevante; TEC-001 a TEC-009
CLIENT_EVENT P4_LOW: visibilidad basica; TEC-001 a TEC-005
SERVER_EVENT Router SOC_v3: normalizacion y salida JSONL/Discord; no detecta por si mismo

Flujo: CLIENT_EVENT -> SERVER_EVENT -> JSONL / Webhook -> Discord

```

Figura 33: Arquitectura de artifacts custom y router

Perfil	Artifact	Filas exportadas	Lectura
P1	Custom.TFM.HIDS.P1.Critical.Priority.Event_v1	19	Señales críticas selectivas
P2	Custom.TFM.HIDS.P2.High.Forensic.Event_v1	118	Detección, contexto y evidencia forense; interpretar según la source
P3	Custom.TFM.HIDS.P3.Medium.Behavioral.Event_v1	109	Comportamiento relevante
P4	Custom.TFM.HIDS.P4.Low.Basic_v2	133	Visibilidad básica
Total	P1 + P2 + P3 + P4	379	379 filas emitidas/exportadas, incluidas emisiones repetidas; no equivalen a alertas únicas ni a cobertura

Tabla 37: Distribución de filas CLIENT_EVENT emitidas/exportadas por perfil custom

La distribución de detecciones por técnica ha sido la siguiente:

Técnica	Filas CLIENT_EVENT incluidas	Lectura
TEC-001	42	PowerShell observable por proceso y 4104.
TEC-002	214	CMD observable por proceso y línea de comandos.
TEC-003	30	Scheduled task observable por schtasks.exe.
TEC-004	37	Run Key observable por actividad de registro/proceso.
TEC-005	14	Servicio observable, con 7045 como evidencia principal.
TEC-006	18	Discovery observable por PowerShell/WMI/WMIC.

Técnica	Filas CLIENT_EVENT incluidas	Lectura
TEC-007	5	Cifrado controlado observable por 4104 y .aes.
TEC-008	4	Borrado controlado observable.
TEC-009	15	Staging y archivado observables; el HTTP 200 procede del runner independiente.

Tabla 38: Distribución de filas CLIENT_EVENT por técnica en la campaña real

Como se ha visto al principio, la táctica Execution concentra el mayor volumen de filas CLIENT_EVENT, con 300 de 379 (79,2 %), debido principalmente a TEC-002, que genera 214 filas. Por el contrario, Impact presenta una densidad considerablemente menor, aunque ambas técnicas evaluadas fueron cubiertas. Estas diferencias representan intensidad de señal en el escenario experimental y no implican que una táctica sea intrínsecamente más difícil de detectar o más peligrosa que otra.

4.2.1.2. Calidad de la evidencia y detección parcial

En la campaña final no se identificó ninguna técnica del subconjunto TEC-001 a TEC-009 sin detección. Todas las técnicas generaron algún tipo de visibilidad en Velociraptor, aunque no todas ofrecieron el mismo nivel de evidencia ni la misma facilidad de interpretación.

Las limitaciones detectadas no correspondieron tanto a una ausencia completa de detección, sino a problemas de visibilidad, correlación y calidad de la evidencia. En particular, los eventos generados por perfiles de baja criticidad, como P4, no deben interpretarse siempre como alertas reales, sino como señales básicas de observación. La clasificación operativa depende de los perfiles superiores y de la correlación con la telemetría del sistema.

Durante las iteraciones se reforzó la recogida de PowerShell 4104 y Sysmon para mejorar la trazabilidad de staging, creación del ZIP y procesos. La transferencia se validó mediante un receptor HTTP controlado y el summary del runner independiente. Esa validación externa se mantiene separada de la detección CLIENT_EVENT y de TrackNetwork, que en la campaña definitiva presentó PID=0 y ProcInfo vacío.

En TEC-009, los artifacts personalizados aportaron señales host de preparación, archivado y ejecución. En la captura de TrackNetwork se observó la conexión con PID=0 y ProcInfo vacío, por lo que no pudo atribuirse de forma fiable a un proceso. La causa concreta de esa ausencia de atribución no quedó acreditada. La confirmación HTTP procede del receptor y del runner y no constituye una detección adicional de Velociraptor.

```
PowerShell - TEC-009 evidence

TEC-009 EVIDENCE CHECK
-----
ReceiverPrecheck: Reachable=True | Status=OK
Command: powershell.exe -NoProfile -ExecutionPolicy Bypass -File exfiltracion_v3.ps1
<HOST_LABEL>
ExitCode: 0
HTTPStatus: 200
UploadSucceeded: True
ZIP: tec009_controlled_archive.zip
ZIP copy: tec009_controlled_archive_copy.zip
LocalSHA256: 31e202c49811f354b51162baecb4ed174e4b0f0805c50c9f91c9f7ec5f642a59
ReceiverResponse: {"status":"ok","bytes_received":6245,"sha256":"31e202c49811f354b51162baecb4ed174e4b0f0805c50c9f91c9f7ec5f642a59"}
PowerShell4104 upload evidence: Count=1; QueryStatus=OK

Interpretacion: staging + archivado ZIP + upload HTTP controlado. Limitacion: no describir como exfiltracion completa si no se conserva receiver_log.jsonl o ZIP recibido.
```

Figura 34: Captura textual de la evidencia de upload HTTP controlado en TEC-009

La Figura 34 documenta la confirmación del upload por el runner; no constituye una detección adicional de Velociraptor.

4.2.1.3. Resultados recibidos en Discord

Las alertas generadas por los artifacts personalizados fueron enviadas al canal de Discord configurado como mecanismo de notificación. Las Figuras 35 a 51 muestran las notificaciones obtenidas durante la ejecución de las técnicas TEC-001 a TEC-009, permitiendo comprobar tanto la generación de la detección como su correcta transmisión al canal de alertas.

4.2.1.3.1 Resultado final recibido en Discord - TEC-001

Hay tres alertas adjudicadas a la TEC-001 según la lógica de los artifacts. La ejecución de TEC-001 produjo tres alertas diferenciadas según la lógica implementada en los artifacts asociados a la técnica. Las Figuras 35 a 37 muestran las tres notificaciones recibidas en Discord, confirmando que las detecciones generadas fueron correctamente procesadas y enviadas al canal configurado.

[MEDIUM / HIGH] CU-001 TEC-001 - TEC-001 PowerShell suspicious execution

PowerShell script block contains execution bypass, decoder or IEX-style behavior.
| PowerShell 4104 | EventID=4104

Timestamp	Host / Client	CU / TEC
2026-06-21T13:31:34Z	LT29.ait.us.es / C.364907172793f8c7	CU-001 / TEC-001

MITRE	Profile	Source
T1059.001	P3	Microsoft-Windows- PowerShell/Operational / EventID 4104

Artifact
Custom.TFM.HIDS.P3.Medium.Behavioral.Event_v1/TEC001_PowerShell_4104

IOA
powershell -ExecutionPolicy Bypass -Command "Invoke-WebRequest -Uri
<HOST_LAB>

IOC
powershell -ExecutionPolicy Bypass -Command "Invoke-WebRequest -Uri
<HOST_LAB>

Action
Review parent process and whether this was expected administrative automation.

Velociraptor • 21/06/2026 15:31

Figura 35: Notificación recibida en Discord para TEC-001 (1 de 3)

[Medium / Medium] CU-001 TEC-001 - PowerShell suspicious execution - basic profile v2

powershell.exe/pwsh.exe with suspicious execution flags or command patterns |
Sysmon ID 1 + PowerShell image + suspicious flags; excludes router JSONL self-
events | EventID=1

Timestamp	Host / Client	CU / TEC
2026-06-21T13:30:24Z	LT29.ait.us.es / C.364907172793f8c7	CU-001 / TEC-001

MITRE	Profile	Source
T1059.001	P4	Microsoft-Windows- Sysmon/Operational / EventID 1

Artifact
Custom.TFM.HIDS.P4.Low.Basic_v2/CU001_Source_TEC001_PowerShell_Basic

IOA
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -
ExecutionPolicy Bypass -Command "Invoke-WebRequest -Uri
<HOST_LAB>

IOC
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -
ExecutionPolicy Bypass -Command "Invoke-WebRequest -Uri
<HOST_LAB>

Action
Review command line, parent process and whether this is expected
administration.

Velociraptor • 21/06/2026 15:30

Figura 36: Notificación recibida en Discord para TEC-001 (2 de 3)



Figura 37: Notificación recibida en Discord para TEC-001 (3 de 3)

4.2.1.3.2 Resultado final recibido en Discord - TEC-002

La ejecución de TEC-002 generó una notificación asociada a la utilización del intérprete de comandos de Windows. La Figura 38 muestra la alerta recibida en Discord como resultado final del flujo de detección configurado para esta técnica.



Figura 38: Notificación recibida en Discord para TEC-002

4.2.1.3.3 Resultado final recibido en Discord - TEC-003

Para TEC-003 se obtuvo correctamente una alerta tras la ejecución de la técnica relacionada con la creación o utilización de una tarea programada. La Figura 39 recoge la notificación recibida en Discord y confirma el funcionamiento del mecanismo de alertado establecido.

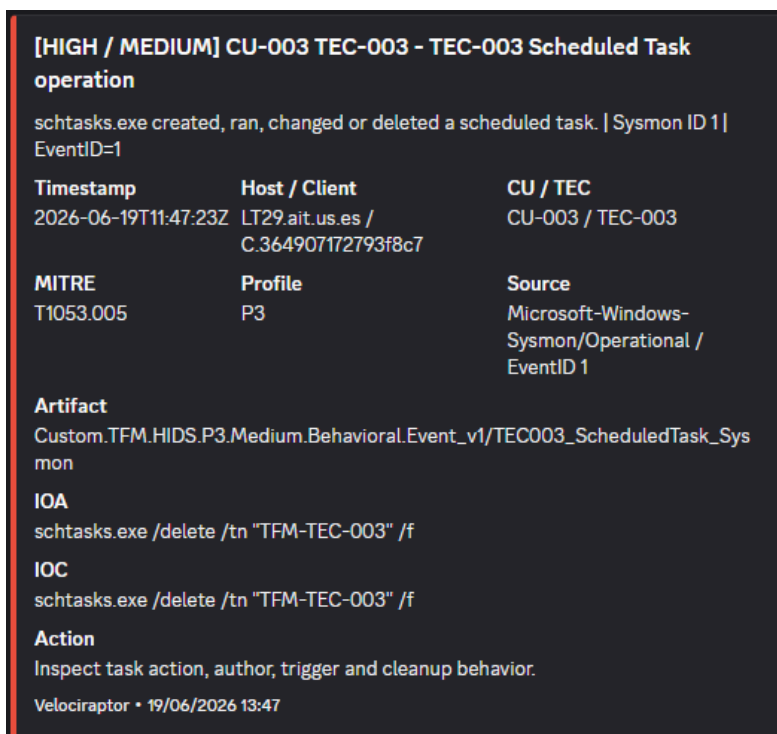


Figura 39: Notificación recibida en Discord para TEC-003

4.2.1.3.4 Resultado final recibido en Discord - TEC-004

La ejecución de TEC-004 fue identificada por el artifact correspondiente y generó la alerta esperada. La Figura 40 muestra la notificación enviada a Discord asociada a la detección de la técnica de persistencia mediante claves de ejecución del Registro.

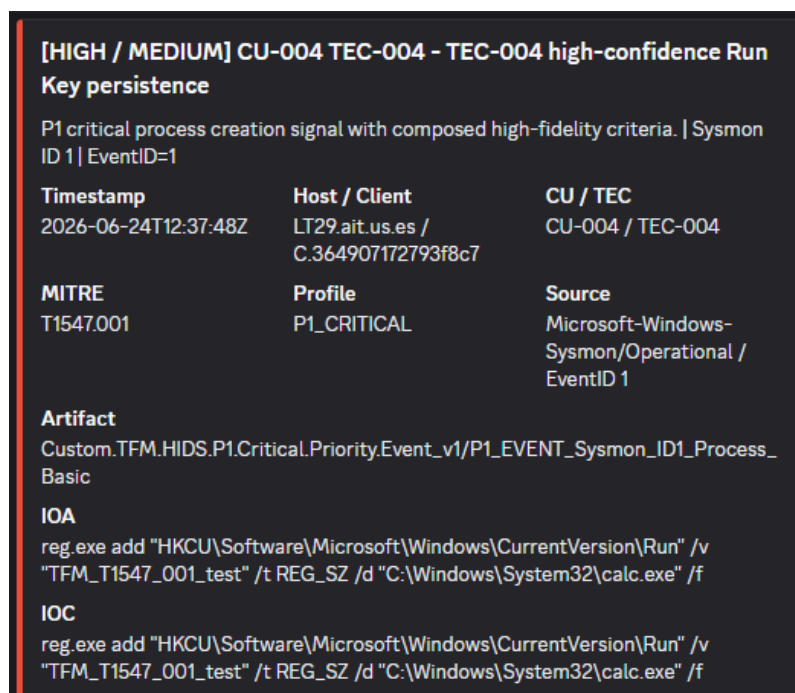


Figura 40: Notificación recibida en Discord para TEC-004

4.2.1.3.5 Resultado final recibido en Discord - TEC-005

En TEC-005 se detectó la actividad relacionada con la ejecución mediante servicios de Windows. La Figura 41 presenta la alerta recibida en Discord, verificando la correcta transmisión de la detección obtenida durante la prueba.

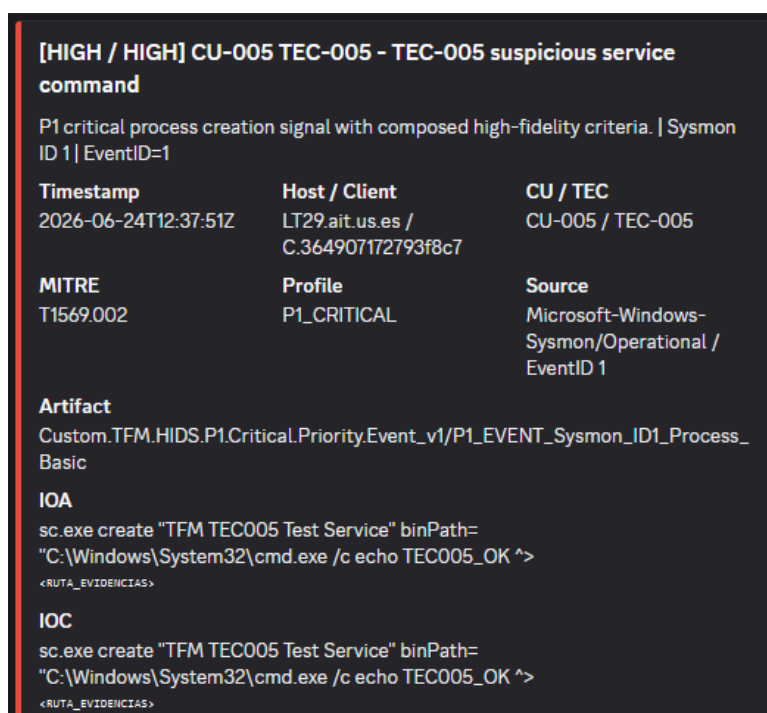


Figura 41: Notificación recibida en Discord para TEC-005

4.2.1.3.6 Resultado final recibido en Discord - TEC-006

La ejecución de TEC-006 produjo dos notificaciones asociadas a la lógica de detección empleada. Las Figuras 42 y 43 muestran ambas alertas recibidas en Discord durante la identificación de actividad relacionada con el descubrimiento de software de seguridad.

[MEDIUM / MEDIUM] CU-006 TEC-006 - TEC-006 Security Software
Discovery process context

Process creation command line queries security product or Defender inventory. | Sysmon ID 1 | EventID=1

Timestamp	Host / Client	CU / TEC
2026-06-21T13:15:59Z	LT29.ait.us.es / C.364907172793f8c7	CU-006 / TEC-006

MITRE	Profile	Source
T1518.001 context	P3	Microsoft-Windows-Sysmon/Operational / EventID 1

Artifact
 Custom.TFM.HIDS.P3.Medium.Behavioral.Event_v1/TEC006_SecuritySoftwareDiscovery_Sysmon

IOA
 "C:\ProgramData\Microsoft\Windows Defender\Platform\4.18.26050.15-0\MpCmdRun.exe" -IdleTask -TaskName WdCacheMaintenance

IOC
 "C:\ProgramData\Microsoft\Windows Defender\Platform\4.18.26050.15-0\MpCmdRun.exe" -IdleTask -TaskName WdCacheMaintenance

Action
 Correlate with PowerShell 4104 if available.

Velociraptor • 21/06/2026 15:16

Figura 42: Notificación recibida en Discord para TEC-006 (1 de 2)

[MEDIUM / MEDIUM] CU-006 TEC-006 - TEC-006 forensic Sysmon ID 1

Sysmon ID 1 command-line behavior | EvidenceRole=Detection | EventID=1

Timestamp	Host / Client	CU / TEC
2026-06-21T13:15:17Z	LT29.ait.us.es / C.364907172793f8c7	CU-006 / TEC-006

MITRE	Profile	Source
T1518.001	P2_EVENT	watch_evt Sysmon Operational / EventID 1

Artifact
 Custom.TFM.HIDS.P2.High.Forensic.Event_v1/Forensic_EVENT_Sysmon_ID1_Process_By_Technique

IOA
 "C:\ProgramData\Microsoft\Windows Defender\Platform\4.18.26050.15-0\MpCmdRun.exe" -IdleTask -TaskName WdCleanup

IOC
 "C:\ProgramData\Microsoft\Windows Defender\Platform\4.18.26050.15-0\MpCmdRun.exe" -IdleTask -TaskName WdCleanup

Figura 43: Notificación recibida en Discord para TEC-006 (2 de 2)

4.2.1.3.7 Resultado final recibido en Discord - TEC-007

Para TEC-007 se recibieron dos alertas durante la ejecución de la técnica de cifrado de datos. Las Figuras 44 y 45 muestran estas notificaciones y permiten comprobar que la actividad observada fue identificada y trasladada correctamente al canal de Discord.

[HIGH / HIGH] CU-007 TEC-007 - TEC-007 Data Encrypted for Impact

PowerShell script block contains AES/CryptoStream/CreateEncryptor or encrypted-extension behavior. | PowerShell 4104 | EventID=4104

Timestamp	Host / Client	CU / TEC
2026-06-19T11:47:25Z	LT29.ait.us.es / C.364907172793f8c7	CU-007 / TEC-007

MITRE	Profile	Source
T1486	P3	Microsoft-Windows-PowerShell/Operational / EventID 4104

Artifact
Custom.TFM.HIDS.P3.Medium.Behavioral.Event_v1/TEC007_DataEncryptedForImpact_4104

IOA

```
{
$.FullName -and
($$.FullName -like "$WorkDir") -and
(Test-Path -LiteralPath $.FullName) -and
($$.FullName -notlike ".aes")
}
```

IOC

```
{
$.FullName -and
($$.FullName -like "$WorkDir") -and
(Test-Path -LiteralPath $.FullName) -and
($$.FullName -notlike ".aes")
}
```

Action
Confirm scope is limited to dummy/lab files and review generated encrypted artifacts.

Velociraptor • 19/06/2026 13:47

Figura 44: Notificación recibida en Discord para TEC-007 (1 de 2)

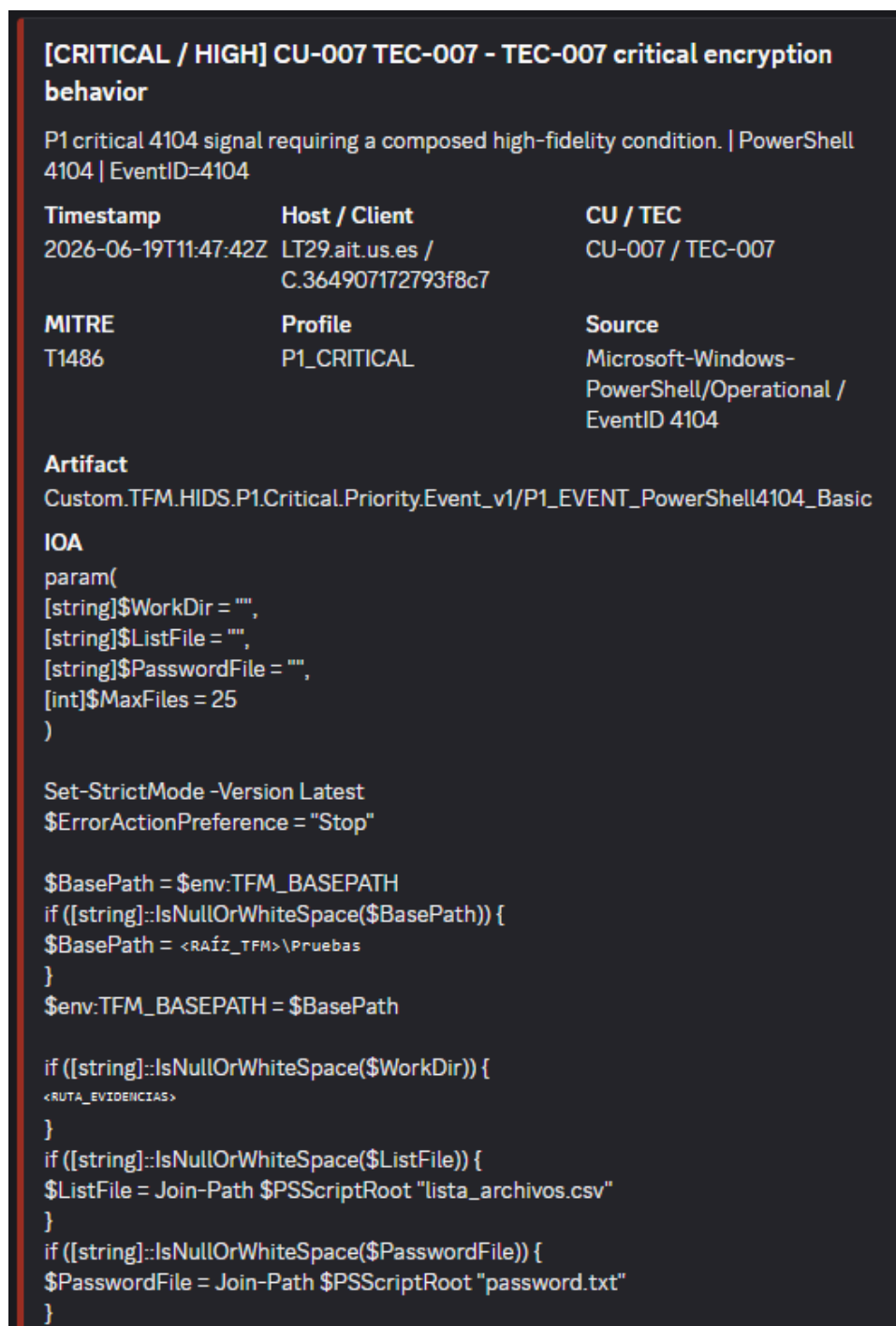


Figura 45: Notificación recibida en Discord para TEC-007 (2 de 2)

4.2.1.3.8 Resultado final recibido en Discord - TEC-008

La ejecución de TEC-008 generó tres alertas correspondientes a los eventos detectados por los artifacts utilizados. Las Figuras 46 a 48 recogen las notificaciones recibidas en Discord y evidencian la detección de la actividad asociada a destrucción de datos.

[MEDIUM / MEDIUM] CU-008 TEC-008 - TEC-008 forensic Sysmon ID 1

Sysmon ID 1 command-line behavior | EvidenceRole=Context | EventID=1

Timestamp	Host / Client	CU / TEC
2026-06-19T11:49:03Z	LT29.ait.us.es / C.364907172793f8c7	CU-008 / TEC-008

MITRE	Profile	Source
T1485 context	P2_EVENT	watch_evtx Sysmon Operational / EventID 1

Artifact
Custom.TFM.HIDS.P2.High.Forensic.Event_v1/Forensic_EVENT_Sysmon_ID1_Process_By_Technique

IOA
powershell.exe -NoProfile -ExecutionPolicy Bypass -Command "\$target = <RUTA_EVIDENCIAS>
Write-Host 'TEC008OK'; Get-ChildItem \$target -Recurse -File | ForEach-Object {
Remove-Item \$_.FullName -Force }; Start-Sleep 3"

IOC
powershell.exe -NoProfile -ExecutionPolicy Bypass -Command "\$target = <RUTA_EVIDENCIAS>
Write-Host 'TEC008OK'; Get-ChildItem \$target -Recurse -File | ForEach-Object {
Remove-Item \$_.FullName -Force }; Start-Sleep 3"

Action
Review process tree, parent command line and related 4104 evidence.

Velociraptor • 19/06/2026 13:49

Figura 46: Notificación recibida en Discord para TEC-008 (1 de 3)

[HIGH / HIGH] CU-008 TEC-008 - TEC-008 Data Destruction

PowerShell script block combines enumeration and forced or recursive deletion behavior. | PowerShell 4104 | EventID=4104

Timestamp	Host / Client	CU / TEC
2026-06-19T11:48:01Z	LT29.ait.us.es / C.364907172793f8c7	CU-008 / TEC-008

MITRE	Profile	Source
T1485	P3	Microsoft-Windows- PowerShell/Operational / EventID 4104

Artifact
Custom.TFM.HIDS.P3.Medium.Behavioral.Event_v1/TEC008_DataDestruction_4104

IOA
\$target =
<RUTA_EVIDENCIAS>
Write-Host 'TEC008OK'; Get-ChildItem \$target -Recurse -File | ForEach-Object {
Remove-Item \$_.FullName -Force }; Start-Sleep 3

IOC
\$target =
<RUTA_EVIDENCIAS>
Write-Host 'TEC008OK'; Get-ChildItem \$target -Recurse -File | ForEach-Object {
Remove-Item \$_.FullName -Force }; Start-Sleep 3

Action
Review deletion scope and validate whether affected files are controlled test data.

Velociraptor • 19/06/2026 13:48

Figura 47: Notificación recibida en Discord para TEC-008 (2 de 3)

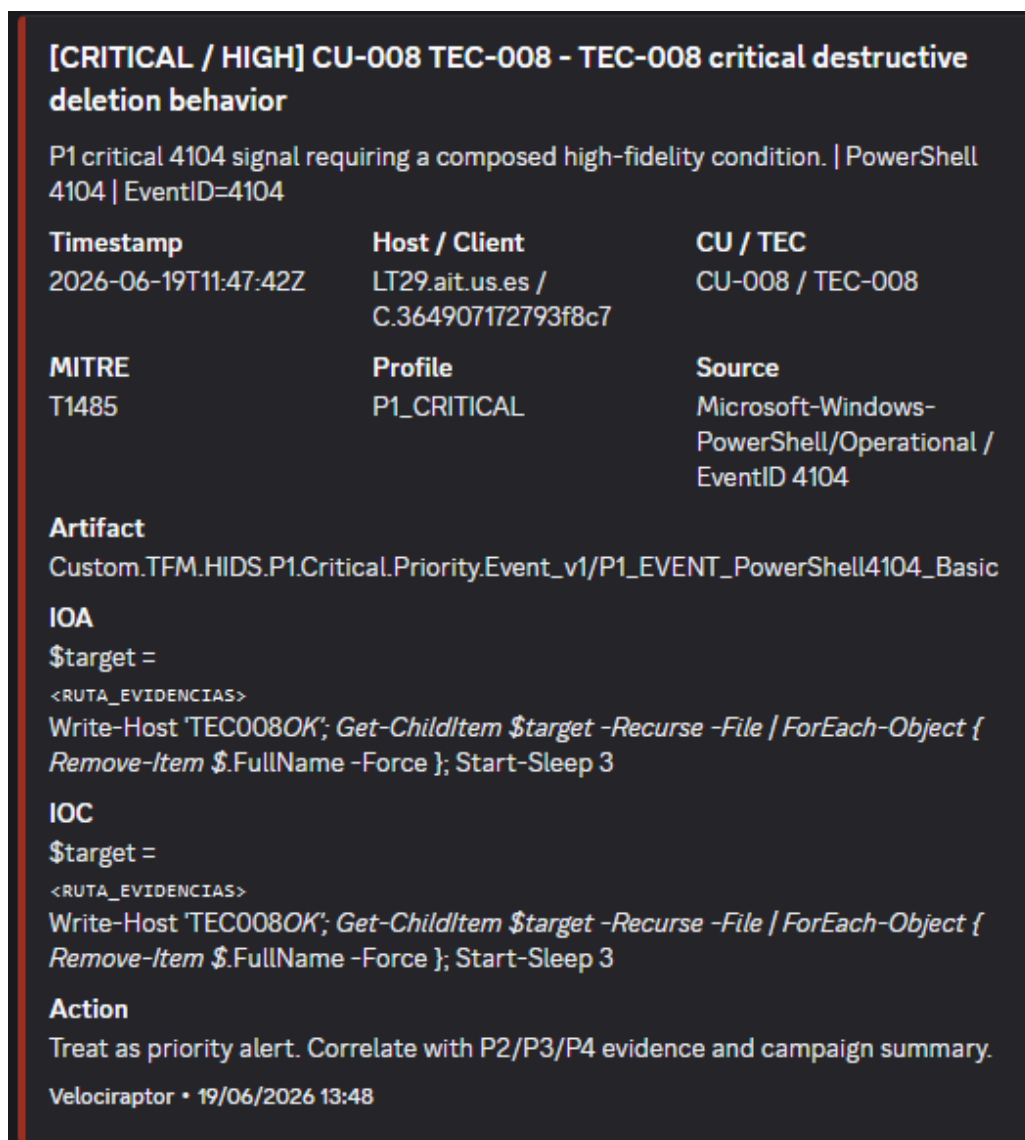


Figura 48: Notificación recibida en Discord para TEC-008 (3 de 3)

4.2.1.3.9 Resultado final recibido en Discord - TEC-009

En TEC-009 se generaron tres alertas relacionadas con las distintas acciones monitorizadas durante el escenario de preparación y exfiltración de datos. Las Figuras 49 a 51 muestran las notificaciones obtenidas, confirmando que las diferentes fases observadas fueron transmitidas correctamente al canal de Discord.

[HIGH / MEDIUM] CU-009 TEC-009 - TEC-009 HTTP upload process context

PowerShell process command line contains file execution/upload parameters or HTTP upload indicators. | Sysmon ID 1 | EventID=1

Timestamp	Host / Client	CU / TEC
2026-06-21T13:31:36Z	LT29.ait.us.es / C.364907172793f8c7	CU-009 / TEC-009

MITRE	Profile	Source
T1074.001 / T1560.001 / T1048.003 context	P3	Microsoft-Windows-Sysmon/Operational / EventID 1

Artifact
Custom.TFM.HIDS.P3.Medium.Behavioral.Event_v1/TEC009_HTTP_Upload_Sysmon_Process_Context

IOA
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" - ExecutionPolicy Bypass -Command "Invoke-WebRequest -Uri <HOST_LAB>

IOC
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" - ExecutionPolicy Bypass -Command "Invoke-WebRequest -Uri <HOST_LAB>

Action
Use as context; the primary alert is TEC009_HTTP_ZIP_Upload_4104.

Velociraptor • 21/06/2026 15:31

Figura 49: Notificación recibida en Discord para TEC-009 (1 de 3)

[HIGH / HIGH] CU-009 TEC-009 - TEC-009 controlled upload process invocation

P1 critical process creation signal with composed high-fidelity criteria. | Sysmon ID 1 | EventID=1

Timestamp	Host / Client	CU / TEC
2026-06-24T12:37:45Z	LT29.ait.us.es / C.364907172793f8c7	CU-009 / TEC-009

MITRE	Profile	Source
T1074.001 / T1560.001 / T1048.003	P1_CRITICAL	Microsoft-Windows-Sysmon/Operational / EventID 1

Artifact
Custom.TFM.HIDS.P1.CriticalPriority.Event_v1/P1_EVENT_Sysmon_ID1_Process_Basic

IOA
powershell.exe -NoProfile -ExecutionPolicy Bypass -File <RUTA_ARTIFACTS>\exfiltracion_v3.ps1 <HOST_LAB>

IOC
powershell.exe -NoProfile -ExecutionPolicy Bypass -File <RUTA_ARTIFACTS>\exfiltracion_v3.ps1 <HOST_LAB>

Action
Review command line, parent process and corresponding forensic P2/P3 evidence.

Velociraptor • hoy a las 14:37

Figura 50: Notificación recibida en Discord para TEC-009 (2 de 3)

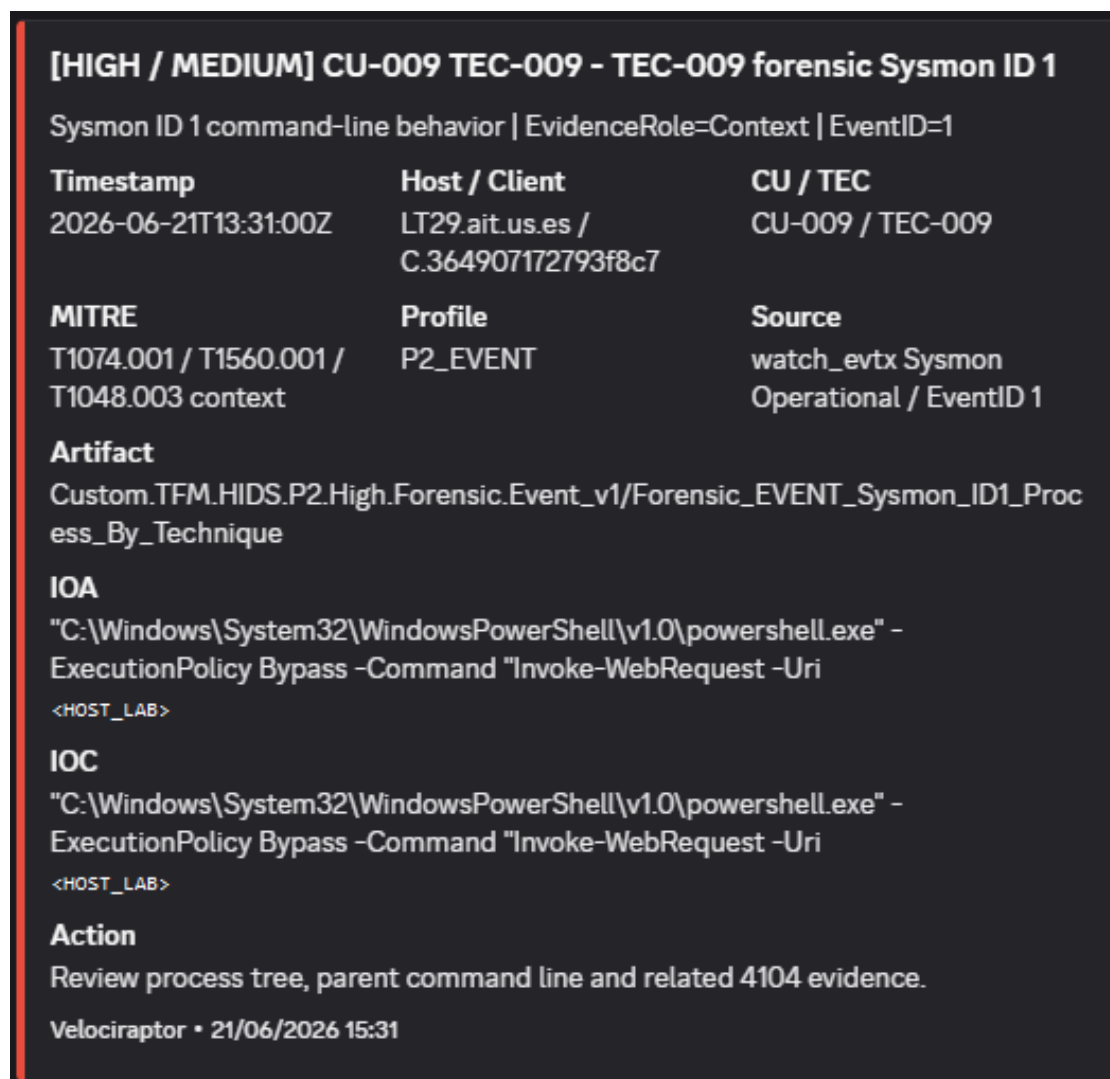


Figura 51: Notificación recibida en Discord para TEC-009 (3 de 3)

4.2.2. Campaña de ataque con Artifacts públicos

La evaluación pública se mantiene separada de la arquitectura custom. La siguiente tabla resume las campañas realizadas para los distintos artifacts públicos, y la siguiente figura distingue su resultado. Los resultados de las campañas siguen almacenados en caso de considerar oportuno ampliar la información sobre estos.

La campaña solo se decidió realizar con estos seis artifacts por ser los más cercanos a artifacts reales pensados para el uso que queremos darle a VR. Las Tablas 42 a 44 y las Figuras 60 y 61 resumen el impacto observado, los escenarios, la duración, la estructura del libro de resultados y el coste operativo por escenario.

Artifact público	Visibilidad	Detección acreditada	Resultado	Limitación
Windows.Hayabusa.Monitoring	Sí	3/9 casos (TEC-001, TEC-003 y TEC-005)	Positivo	CH high: 4 coincidencias en FP; CHM no ejecutada
Windows.Events.ServiceCreation	Sí: System 7045	No por sí solo	Negativo	El 7045 acredita creación de servicio, no malicia
Windows.Events.ProcessCreation	Sí	No por sí solo	Negativo	Proceso observado, no equivale a técnica detectada
Windows.Sysinternals.SysmonLogForward	Sí: Sysmon	No por sí solo	Negativo	Telemetría forense; ID 26 no es alerta individual
Windows.ETW.Monitoring	Sí	No por sí solo	Negativo	Proceso observado, no equivale a técnica detectada
Generic.Events.TrackNetworkConnections	Sí: conexión	No por sí solo	Negativo	PID=0 y ProcInfo vacío; HTTP 200 del runner independiente

Tabla 39: Resultados definitivos de las campañas con artifacts públicos

Hayabusa alcanzó 3/9 técnicas específicas. Por ello, es el artifact del que se partió para la creación de los demás artifacts personalizados.

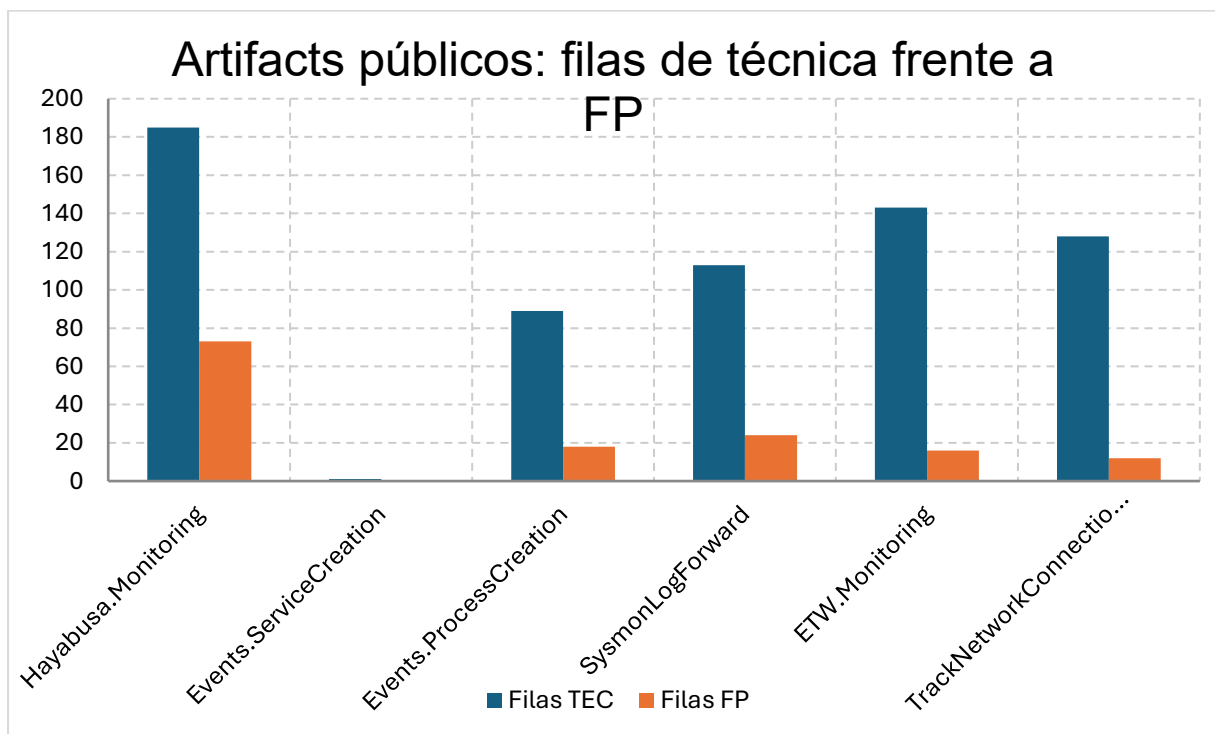


Figura 52: Artifacts públicos: filas en la campaña TEC y en la campaña benigna

Hayabusa es el artifact público con mayor capacidad de detección específica, pero también registra el mayor volumen FP, se observaron cuatro filas high/critical asociadas a tres eventos subyacentes, incluyendo un caso benigno FP-003. El resto aporta sobre todo visibilidad y contexto.

4.3. Análisis comparativo integrado de los resultados

Este apartado integra los resultados de detección obtenidos tanto en la campaña de actividad legítima como en las campañas de ataque, comparando el comportamiento de los artifacts públicos y personalizados. El análisis considera la cobertura por técnica, el volumen de señales CLIENT_EVENT, la distribución por perfiles P1-P4 y el comportamiento ante actividad benigna. El benchmark de rendimiento y la comparación experimental con Wazuh quedan fuera de este apartado y se analizan, respectivamente, en los capítulos V y VI.

La cobertura se mide por técnicas únicas detectadas; el volumen se mide por filas CLIENT_EVENT; las filas observadas en ventanas benignas de artifacts de visibilidad no se consideran automáticamente falsos positivos; y la salida externa (Discord/HTTP) se mantiene separada de la detección host.

4.3.1. Artifacts públicos frente a personalizados

La comparación más directa se obtiene manteniendo el mismo denominador de nueve técnicas. La capa pública con detección específica CH alcanzó 3/9 técnicas, mientras que los artifacts personalizados alcanzaron 9/9. Esto supone pasar de una cobertura experimental del 33,3 % al 100 %, es decir, una diferencia de 66,7 puntos porcentuales dentro del subconjunto evaluado. La Figura 53 compara la cobertura específica agregada.

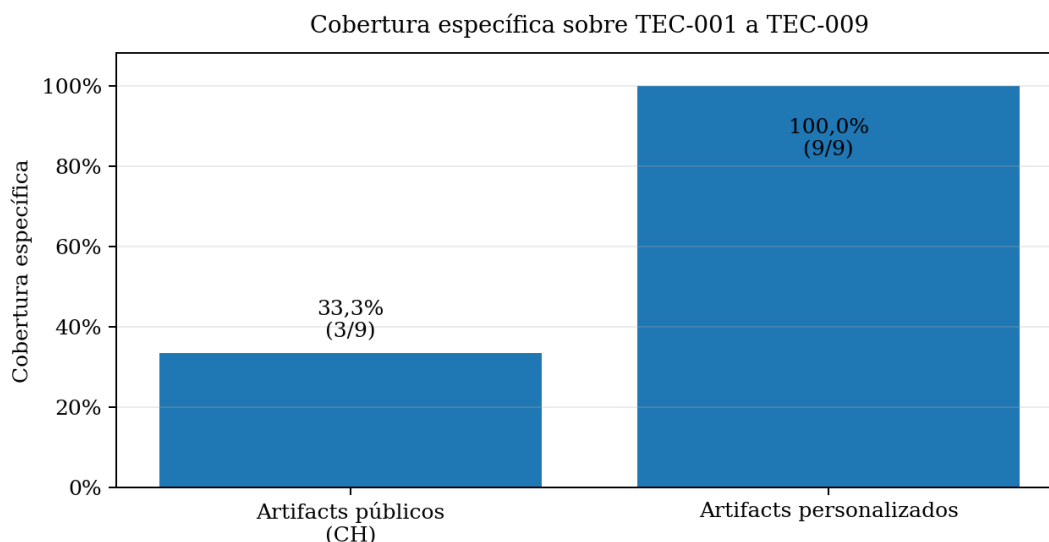


Figura 53: Cobertura específica agregada de artifacts públicos y personalizados

La mejora no puede atribuirse simplemente a un aumento del número de eventos recogidos. Los artifacts públicos ya proporcionaban telemetría de procesos, Sysmon, ETW, servicios y conexiones. La diferencia principal está en la incorporación de lógica de clasificación específica capaz de convertir esa telemetría en señales asociadas a las técnicas evaluadas

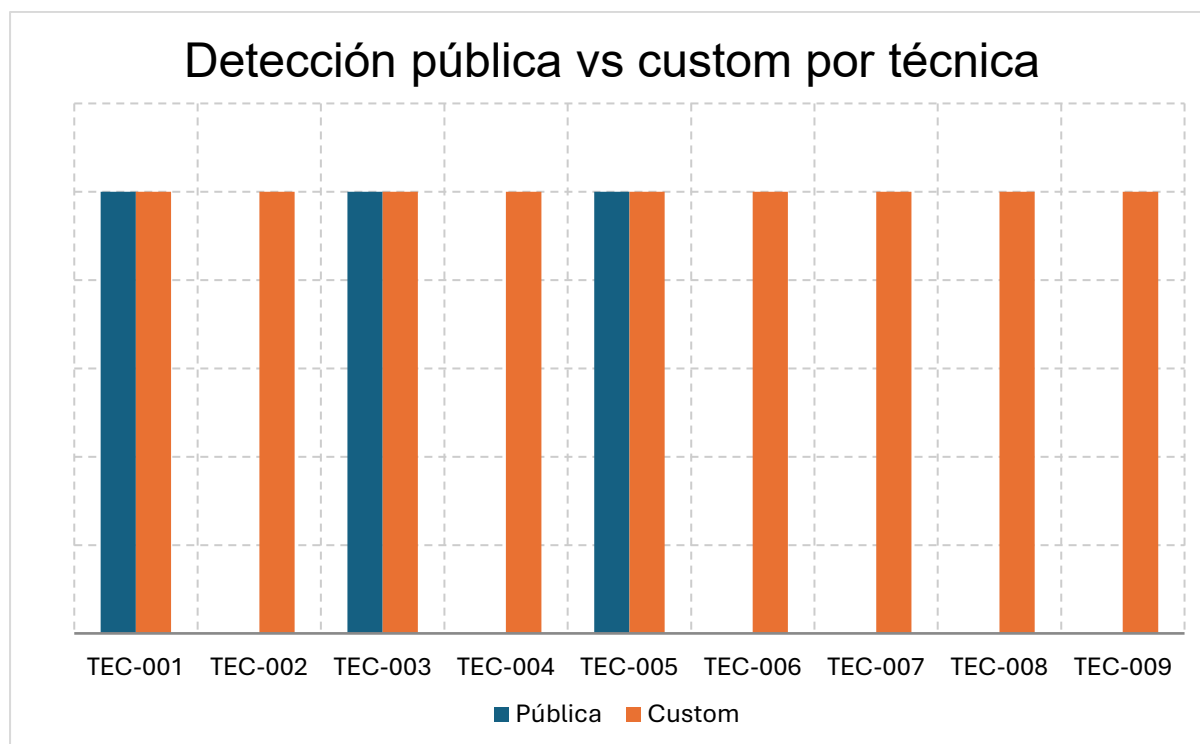


Figura 54: Comparación de detección por técnica entre artifacts públicos y custom

La Figura 54 permite localizar exactamente dónde aparece la ganancia de cobertura. La capa pública detectó TEC-001, TEC-003 y TEC-005. La arquitectura custom añadió TEC-002, TEC-004, TEC-006, TEC-007, TEC-008 y TEC-009. Estas seis incorporaciones no se concentran en una única

táctica: incluyen Execution, Persistence, Discovery, Impact y Collection/Exfiltration. Por ello, la mejora no solo incrementa el número total de técnicas detectadas, sino que amplía la diversidad táctica cubierta.

Este resultado es especialmente relevante para el objetivo del trabajo. Velociraptor no se comporta como un HIDS cerrado cuya cobertura viene determinada únicamente por reglas preinstaladas; su utilidad aumenta cuando la telemetría se combina con artifacts adaptados al entorno y a los comportamientos que se desea identificar.

4.3.2. Intensidad de señal por técnica

Una vez establecida la cobertura, el siguiente análisis estudia la intensidad de señal. Las nueve técnicas generaron 379 filas `CLIENT_EVENT` en total, con una media de 42,1 filas por técnica y una mediana de 18. La diferencia entre ambas medidas anticipa una distribución fuertemente asimétrica. La Figura 55 muestra la concentración de filas por etiqueta TEC y la Figura 56 su distribución por fuente host.

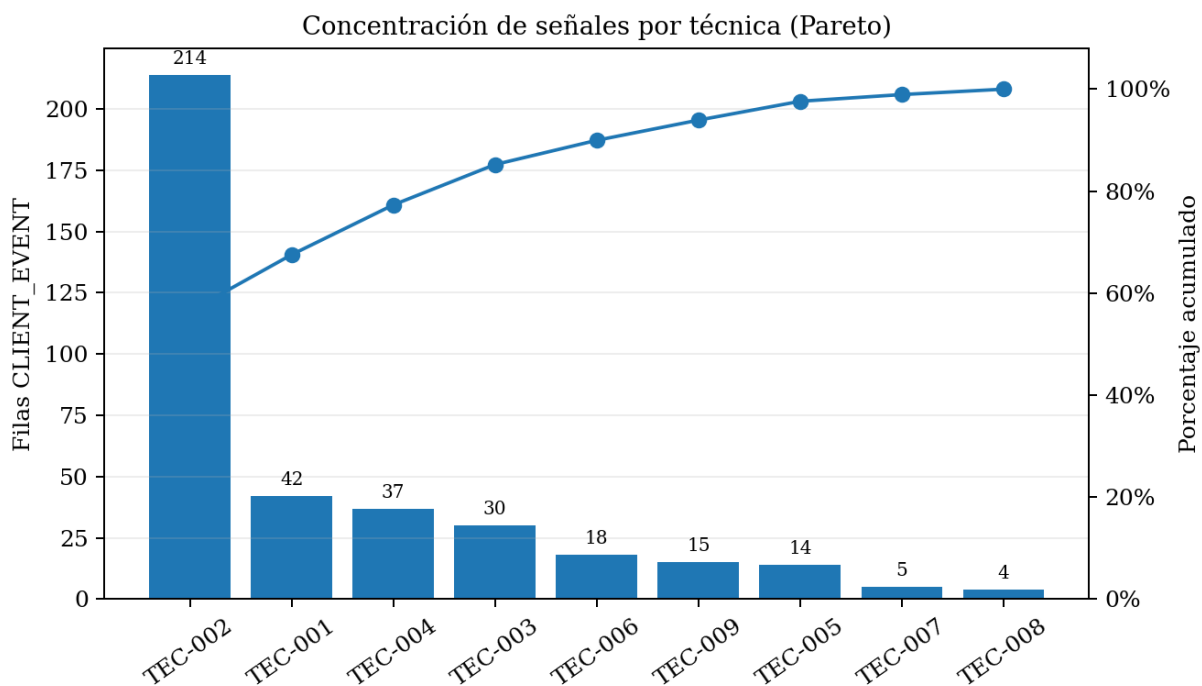


Figura 55: Concentración de señales `CLIENT_EVENT` por técnica

Las filas etiquetadas como TEC-002 representan 214 de las 379 filas (56,5 %). La agrupación utiliza la etiqueta asignada por la source, no una atribución causal exclusiva a la ventana de ejecución de cada caso. El conjunto conserva emisiones repetidas y puede incluir actividad auxiliar del laboratorio; por ello describe el volumen clasificado, no el número de ataques ni de alertas únicas.

Esta dispersión demuestra que el recuento bruto de filas no puede utilizarse como métrica de eficacia. Un mayor volumen puede reflejar más eventos asociados al procedimiento, mayor verbosidad

de una fuente o varias sources activadas simultáneamente. La métrica de cobertura debe permanecer separada de la métrica de densidad de señales.

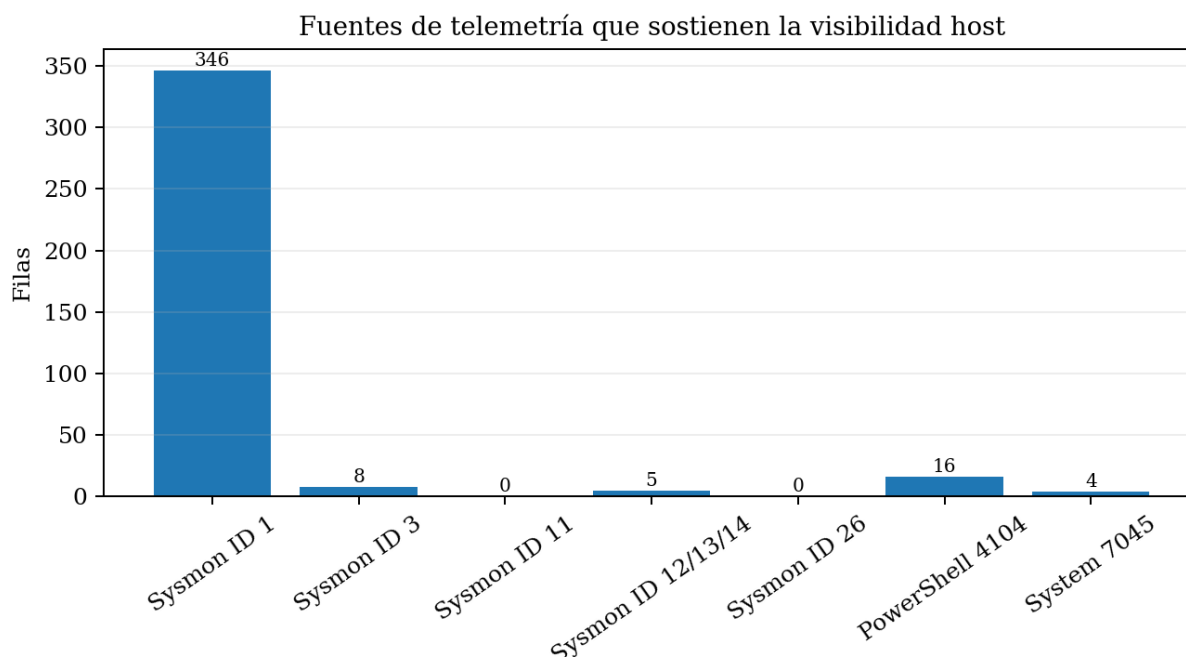


Figura 56: Fuentes de telemetría que sostienen la visibilidad host

La dependencia de fuentes también es desigual. Sysmon Event ID 1 aporta 346 de las 379 filas host consideradas, un 91,3 %. PowerShell 4104, System 7045 y los eventos de registro aportan un volumen mucho menor, pero contienen semántica esencial para distinguir comportamientos concretos. El resultado revela una dependencia fuerte de la telemetría de creación de procesos y, por tanto, una sensibilidad del sistema a la configuración de Sysmon y a la calidad de la línea de comandos registrada.

Desde un punto de vista operativo, esto implica que una reducción o pérdida de ProcessCreation tendría un impacto notable en la visibilidad general. Sin embargo, las fuentes menos voluminosas siguen siendo necesarias porque aportan contexto que no puede deducirse únicamente de la existencia de un proceso.

4.3.3. Cobertura y selectividad de los perfiles P1-P4

Los cuatro perfiles no tienen la misma finalidad. P1 prioriza señales críticas y selectivas; P2 aporta contexto y evidencia forense; P3 se orienta a comportamiento; y P4 proporciona visibilidad básica. Por ello, su comparación debe considerar conjuntamente cobertura y volumen, evitando interpretar el perfil con más filas como el más eficaz. La Tabla 40 resume los casos con señal y la densidad por perfil; la Figura 57 muestra su volumen.

Perfil	Técnicas	Cobertura de TEC (%)	Filas	Filas/TEC cubierta	Lectura
--------	----------	----------------------	-------	--------------------	---------

P1	5/9	55,6 %	19	3,8	Mayor selectividad; señales de alta prioridad.
P2	7/9	77,8 %	118	16,9	Cobertura alta con mayor contexto forense.
P3	9/9	100 %	109	12,1	Único perfil con cobertura completa; densidad intermedia.
P4	5/9	55,6 %	133	26,6	Mayor densidad; visibilidad básica, no cobertura adicional.

Tabla 40: Cobertura y densidad de los perfiles personalizados P1-P4

La cobertura por perfil indica casos TEC con alguna fila emitida. No equivale por sí sola a detección específica: P4 aporta visibilidad y P2 puede aportar contexto/evidencia. La densidad es el cociente filas/casos con señal; «selectividad» se utiliza aquí únicamente en ese sentido descriptivo, no como estimación de precisión.

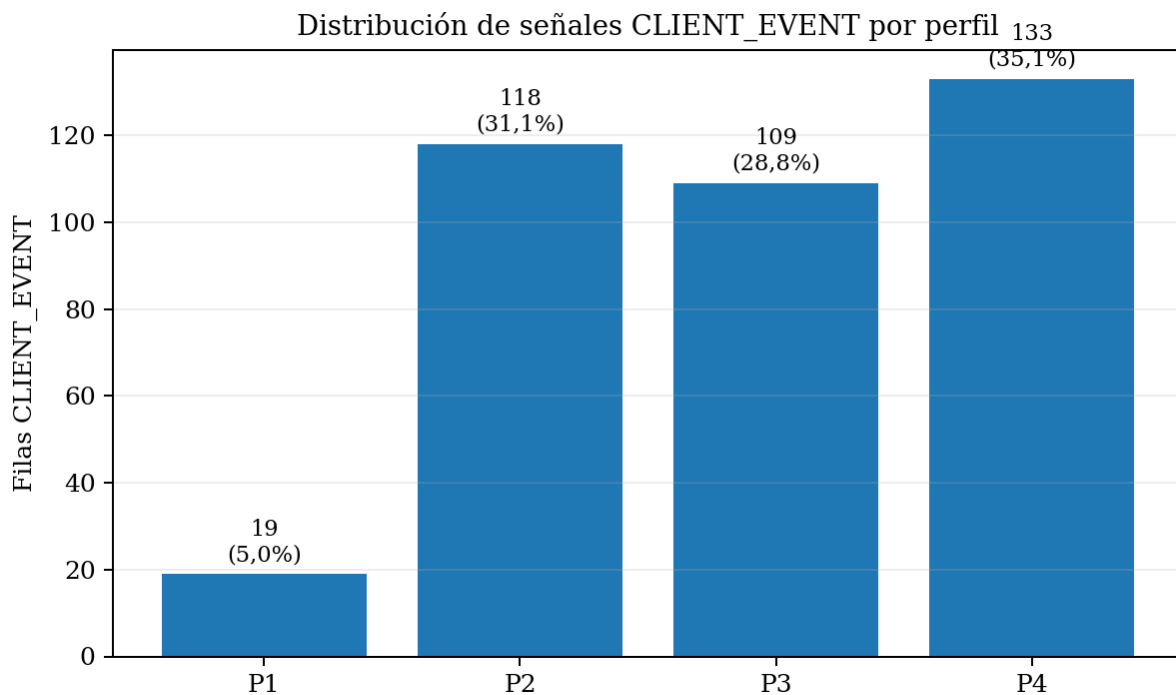


Figura 57: Distribución de las 379 filas CLIENT_EVENT por perfil

P4 genera 133 filas, el 35,1 % del total, seguido de P2 con 118 (31,1 %), P3 con 109 (28,8 %) y P1 con 19 (5,0 %). Si se observase únicamente este gráfico, podría parecer que P4 es el perfil más relevante. Sin embargo, su cobertura es de 5/9, idéntica a P1 y muy inferior a P3.

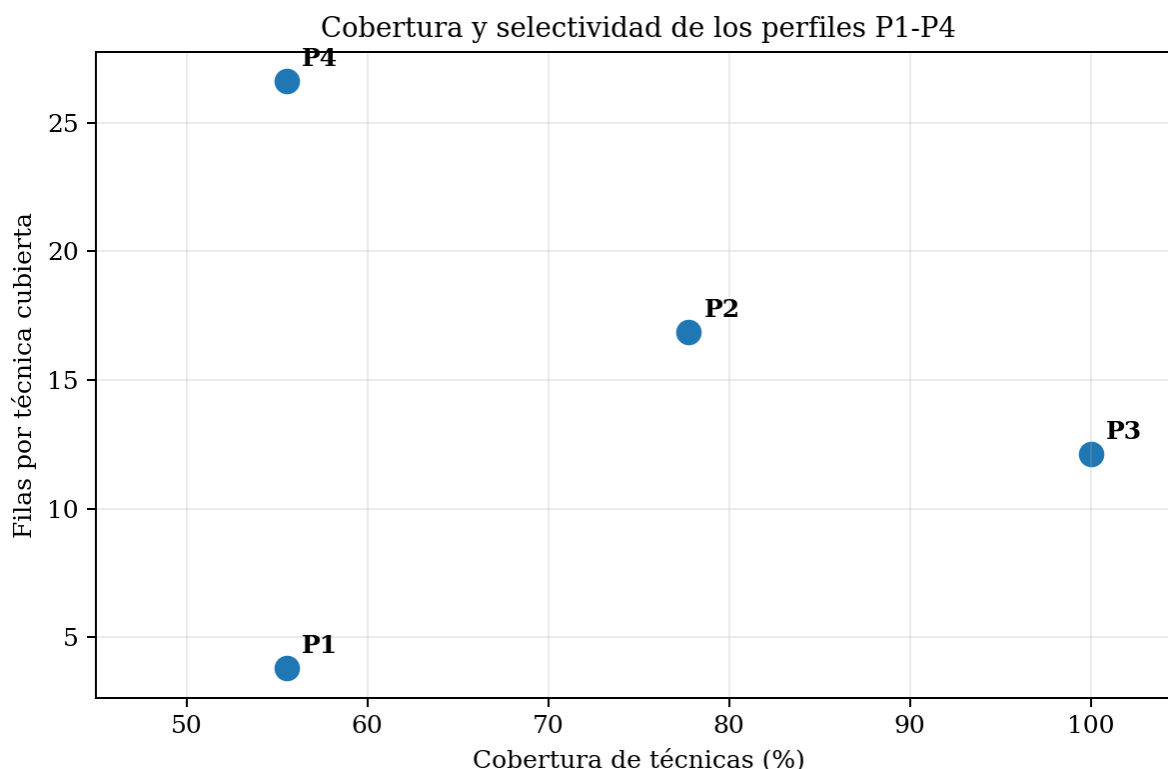


Figura 58: Relación entre cobertura y densidad de señales de los perfiles P1-P4

La Figura 58 sintetiza mejor la función de cada nivel. P1 es el perfil más selectivo, con 3,8 filas por técnica cubierta, aunque solo cubre cinco técnicas. P4 se sitúa en el extremo opuesto, con 26,6 filas por técnica cubierta y la misma cobertura 5/9. P2 amplía la cobertura hasta 7/9 con una densidad de 16,9. P3 constituye el punto más equilibrado del conjunto: es el único perfil que cubre las nueve técnicas y lo hace con 12,1 filas por técnica cubierta, por debajo de P2 y muy por debajo de P4.

Este patrón confirma el valor de una arquitectura multicapa. P1 no debe sustituir a P3 ni P4 debe interpretarse como una capa de detección final. La combinación permite separar señales de distinta prioridad, conservar contexto y mantener una capa comportamental capaz de cubrir todo el subconjunto experimental.

4.3.4. Relación entre detección y falsos positivos

El análisis de calidad debe considerar simultáneamente la campaña de ataque y la campaña benigna. A nivel de escenario experimental, los nueve casos de ataque produjeron detección custom y los diez escenarios benignos no produjeron hits definitivos. La Figura 59 resume los veredictos de los casos controlados.

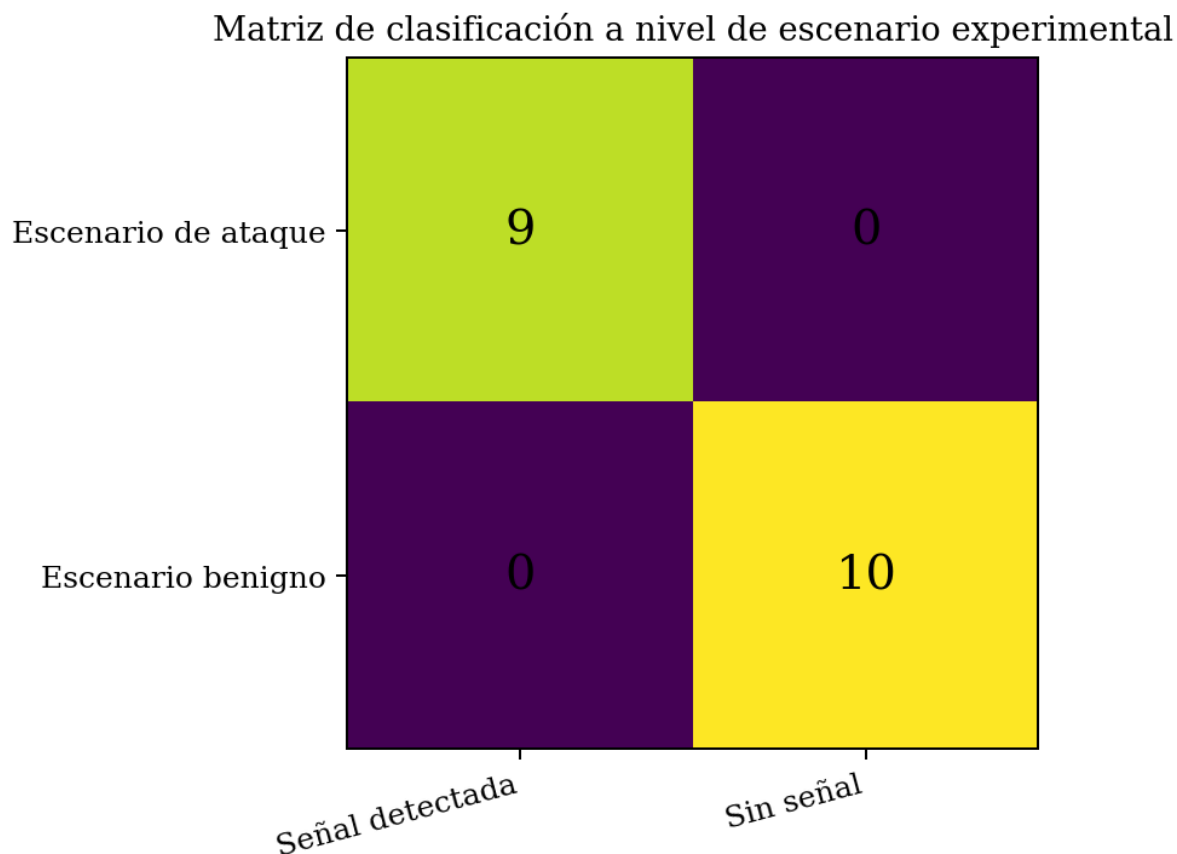


Figura 59: Matriz de clasificación a nivel de escenario experimental

Dentro de estas 19 ejecuciones controladas se observan 9 verdaderos positivos, 0 falsos negativos, 0 falsos positivos y 10 verdaderos negativos. Aunque estos valores equivalen a una clasificación perfecta en la muestra ensayada, no deben interpretarse como estimaciones de sensibilidad, especificidad o precisión en producción. La muestra es pequeña, deliberadamente diseñada y no representa la diversidad de comportamiento legítimo de un endpoint real. Además, gracias a que estos artifacts son modificables es sencillo excepcionar esos falsos positivos.

4.3.5. Síntesis de resultados

Tras analizar conjuntamente cobertura, volumen, tácticas, perfiles y actividad benigna, pueden extraerse las siguientes conclusiones principales:

- La capa personalizada amplía la cobertura específica de 3/9 a 9/9 técnicas dentro del subconjunto evaluado. La mejora añade seis técnicas y extiende la detección más allá de Execution hacia Persistence, Discovery, Impact y Collection/Exfiltration.
- La cantidad de filas no equivale a eficacia. TEC-002 concentra el 56,5 % del volumen total, mientras TEC-007 y TEC-008 se detectan con cinco y cuatro filas. La cobertura y la densidad de señal deben tratarse como métricas distintas.

- Execution es la táctica con mayor intensidad de señal: 79,2 % del volumen total y 75 filas por técnica tras normalizar. La cobertura pública se concentra también en Execution, mientras la arquitectura custom cubre todas las tácticas representadas.
- P3 presenta el equilibrio más favorable entre cobertura y volumen: 9/9 técnicas con 109 filas y 12,1 filas por técnica cubierta. P1 es el perfil más selectivo y P4 el más voluminoso, lo que confirma que los niveles cumplen funciones complementarias.
- La visibilidad depende fuertemente de Sysmon Event ID 1, que aporta 346/379 filas. Esta dependencia convierte la configuración de telemetría en un requisito técnico crítico para reproducir los resultados.
- En los diez escenarios benignos custom no se observaron hits definitivos. Este resultado es positivo dentro del laboratorio, pero no permite extrapolar una tasa general de falsos positivos nula.
- TEC-009 muestra la importancia de separar detección host, evidencia contextual y ground truth externo: staging y ZIP fueron observables en Velociraptor, mientras la confirmación HTTP procede del runner controlado y no se contabiliza como una detección adicional. La Tabla 41 sintetiza los resultados y sus límites.

Dimensión	Resultado cuantitativo	Conclusión
Cobertura específica	Públicos: 3/9 · Custom: 9/9	La personalización añade seis casos TEC y extiende la representación desde Execution a Persistence, Discovery, Impact y Collection/Exfiltration.
Volumen	379 filas CLIENT_EVENT	TEC-002 concentra 214 filas (56,5 %); volumen ≠ cobertura.
Tácticas	Execution: 300/379	79,2 % del volumen; 75 filas/técnica tras normalización.
Perfiles	P3: 9/9 con 109 filas	P3 combina cobertura completa con densidad intermedia.
Actividad benigna	10 escenarios · 0 hits custom	Resultado favorable pero limitado al conjunto benigno diseñado.
Telemetría	Sysmon ID 1: 346/379	Alta dependencia de ProcessCreation y de la configuración del endpoint.

Tabla 41: Síntesis cuantitativa de los resultados de detección

En conjunto, los resultados muestran que Velociraptor puede utilizarse como base para construir capacidades HIDS/DFIR host-based en Windows cuando se combina una telemetría adecuada con lógica de detección personalizada. La principal aportación observada no es simplemente recopilar más eventos, sino transformar señales de bajo nivel en detecciones específicas y contextualizadas. Al mismo tiempo, el análisis evidencia que esa capacidad depende del diseño de los artifacts, de la configuración

de Sysmon y Windows Event Logs y de una validación experimental que separe de forma estricta ejecución, visibilidad, detección, evidencia y salida externa.

Por ello, la conclusión del capítulo no debe formularse como una superioridad universal de Velociraptor, sino como la demostración de que, dentro del laboratorio y del subconjunto TEC-001 a TEC-009, una arquitectura custom correctamente ajustada puede alcanzar cobertura completa con una relación favorable entre detección y actividad benigna observada. La evaluación comparativa con Wazuh se desarrolla de forma independiente en el Capítulo VI para evitar mezclar unidades de análisis distintas.

Capítulo V: Evaluación de Velociraptor y benchmark de rendimiento

Este capítulo evalúa el coste operativo observado del cliente Velociraptor en el endpoint del laboratorio mediante el benchmark recogido en TFM_BENCHMARK_RENDIMIENTO.xlsx. Para ello, se utilizan tres escenarios controlados: BASELINE_NO_VR, con el cliente Velociraptor detenido; VR_IDLE, con el cliente activo sin ejecutar carga experimental; y VR_TEC_RUNNER, con el cliente activo durante la ejecución controlada del runner TEC-001 a TEC-009. Cada escenario se repite tres veces y se analizan principalmente el consumo de CPU, memoria y la actividad de disco observada.

La evaluación de rendimiento se mantiene separada de los resultados de detección, falsos positivos, alerta en tiempo real, evidencia forense y salida externa.

El libro TFM_BENCHMARK_RENDIMIENTO.xlsx acredita nueve ejecuciones válidas, tres repeticiones por escenario, 422 muestras brutas de sistema y cliente, y 770 registros de procesos. Las cifras agregadas de CPU y RAM se toman de las hojas de resumen; los estadísticos adicionales de distribución y disco se recalculan sobre RAW_SAMPLES sin modificar el libro original. El cliente presentó una CPU media del 0 % en VR_IDLE y del 0,95 % en VR_TEC_RUNNER, con un pico puntual del 54 %; una huella media de memoria próxima a 57 MB y estable entre reposo y carga; y contadores de E/S del proceso cliente iguales a 0 B/s en las muestras disponibles. La actividad global de disco sí aumentó durante el runner, pero no puede atribuirse al cliente porque incluye la propia carga de prueba, Windows y efectos de caché.

Las Tablas 42 a 44 y las Figuras 60 y 61 resumen los escenarios utilizados, la estructura del benchmark y los principales resultados de rendimiento obtenidos.

Dimensión	VR_IDLE	VR_TEC_RUNNER	Lectura defendible
CPU del cliente	Media 0 %; máximo 0 %	Media 0,95 %; máximo 54 %	Carga media baja y comportamiento en ráfagas.
Memoria del cliente	Media 57,48 MB; máximo 57,92 MB	Media 57,42 MB; máximo 57,97 MB	Huella próxima a 57 MB, sin aumento asociado al runner.
E/S del proceso cliente	0 B/s en las muestras	0 B/s en las muestras	Resultado no prueba acceso a disco.
Disco del sistema	Media 0,12 MB/s	Media 0,67 MB/s	Métrica contextual contaminada por runner, SO y

			cachés; no atribuible al cliente.
--	--	--	-----------------------------------

Tabla 42: Síntesis del impacto observado

5.1. Metodología de benchmark

El benchmark utiliza la metodología definitiva del apartado 3.6 y la campaña BENCH_20260619_FINAL01. Se comparan BASELINE_NO_VR, VR_IDLE y VR_TEC_RUNNER, con tres repeticiones por escenario. VR_TEC_RUNNER utiliza la carga controlada de las técnicas TEC-001 a TEC-009; la campaña benigna no forma parte de esta carga. SERVER_GUI se registra separadamente y se excluye de las métricas del cliente.

La carga local del benchmark se lanza mediante TFM_Benchmark_TEC_Load_v1.ps1, que invoca TFM_Run_All_TEC_Tests_v6_PATCHED_4104.ps1 con la transferencia HTTP deshabilitada. Por tanto, no debe confundirse con la campaña canónica de TEC-009 que confirma HTTP 200.

El estudio de rendimiento se realizó mediante tres escenarios diferenciados: BASELINE_NO_VR, con el cliente Velociraptor detenido; VR_IDLE, con el cliente activo pero sin ejecutar pruebas; y VR_TEC_RUNNER, con el cliente activo durante la ejecución controlada del runner de ataques TEC-001 a TEC-009. Por tanto, el benchmark no utiliza la campaña de actividad benigna como carga de prueba, sino la campaña de técnicas de ataques reales definida previamente. Y como se ha dicho en el apartado anterior, cada escenario se ejecutó tres veces para facilitar la comparación entre reposo, ausencia del agente y carga experimental.

Escenario	Cliente	Runner	Runs válidos	Duración media
BASELINE_NO_VR	Servicio detenido	No observado	3/3	182,69 s
VR_IDLE	Servicio activo	No observado	3/3	181,57 s
VR_TEC_RUNNER	Servicio activo	139 muestras observadas	3/3	315,60 s

Tabla 43: Escenarios y ventanas de medición

La mayor duración de VR_TEC_RUNNER se debe a que tenía que tener una duración acorde a la ejecución del runner.

El detalle de las ejecuciones, métricas y resultados del benchmark se recoge en el Anexo E: Benchmark y rendimiento.

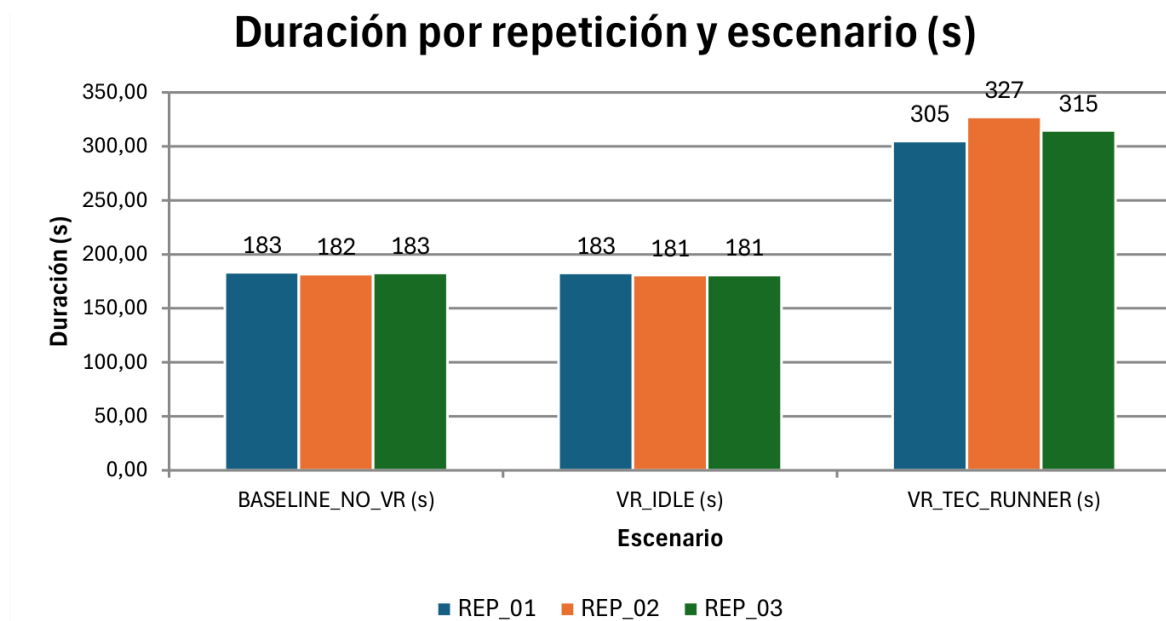


Figura 60: Duración por repetición y escenario

5.2. Métricas, unidad de análisis y exclusiones

Para CPU se emplea porcentaje de uso; para memoria, megabytes de memoria del cliente; para E/S, bytes por segundo del proceso y del sistema; y para duración, segundos por run. SERVER_GUI se registra por trazabilidad, pero se excluye del cálculo principal mediante IncludeServerGuiInTotal=False y ServerGuiExcludedFromClientMetrics=True. Esto se debe a que estamos midiendo el consumo del cliente de VR en el endpoint, ya que SERVER_GUI se excluye porque el objeto del benchmark es el cliente en el endpoint. El estudio no evalúa el coste del servidor ni la escalabilidad del despliegue. Del mismo modo, el runner no se contabiliza como proceso cliente: su actividad únicamente define la carga de trabajo del escenario.

Hoja	Contenido	Uso en el capítulo
00_README	Alcance, campaña experimental, criterios metodológicos y estado final del benchmark.	Contextualización y navegación del conjunto de datos.
01_RESUMEN_EJECUTIVO	KPI consolidados y resultado global del benchmark.	Cifras principales de CPU, RAM, duración y validez.
02_RUNS_VALIDOS	Nueve ejecuciones válidas y métricas por repetición.	Comparación y variabilidad entre REP_01, REP_02 y REP_03.

03_RESUMEN_ESCENARIO	Agregados de BASELINE_NO_VR, VR_IDLE y VR_TEC_RUNNER.	Comparación principal entre baseline, reposo y carga.
04_RAW_SAMPLES	422 muestras temporales de sistema y cliente.	Cálculo de distribuciones, CPU, RAM y actividad global de disco.
05_PROCESS_SAMPLES	770 registros de procesos.	Validación de roles, proceso cliente, SERVER_GUI y contadores de E/S.
06_CALIDAD_DATOS	Controles de integridad, exclusiones y criterios de validación.	Acreditar la validez de los 9 runs y las decisiones metodológicas.
07_GRAFICAS	Seis gráficos de rendimiento.	Representación visual de CPU, RAM y duración por escenario y repetición.
08_TRAZABILIDAD	Fuentes experimentales, rutas y hashes SHA-256.	Trazabilidad y reproducibilidad de las métricas.
09_LIMITACIONES	Limitaciones del entorno y de la metodología de medida.	Delimitar el alcance de las conclusiones del benchmark.
10_DISCREPANCIAS	Advertencias y decisiones metodológicas relevantes.	Documentar RunnerStillRunningAtEnd y la exclusión de SERVER_GUI.

Tabla 44: Estructura del Excel definitivo de benchmark

RunnerStillRunningAtEnd=True no invalida las muestras ya recogidas del cliente, solo indica que el runner continuaba corriendo en el momento en el que la prueba benchmark terminó.

El impacto de las pruebas, en resumen, se especifica en la siguiente imagen:

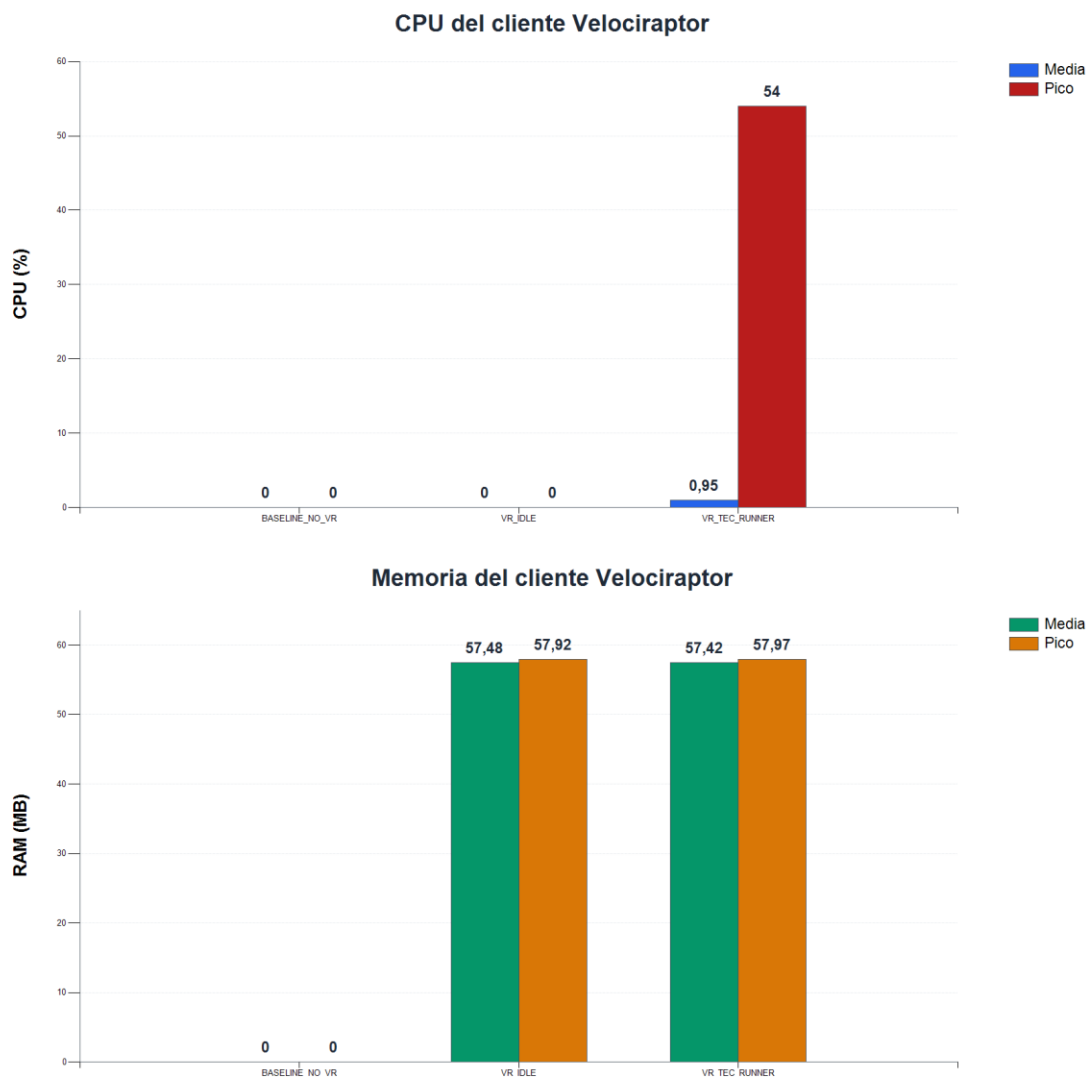


Figura 61: Coste operativo observado del cliente Velociraptor por escenario

5.3. Impacto en CPU

La Tabla 45 y la Figura 62 detallan la CPU media, el percentil 95 y el máximo del cliente durante VR_TEC_RUNNER.

El consumo del cliente se registró con una media del 0 % tanto en BASELINE_NO_VR como en VR_IDLE. En la baseline, el cero es esperado porque el servicio estaba detenido. En VR_IDLE, el proceso sí estaba presente en todas las muestras y no se registraron valores nulos. Por tanto, se entiende que la carga del cliente en estado de reposo es prácticamente nula.

Durante VR_TEC_RUNNER, la media agregada fue 0,95 %, la mediana 0 %, el percentil 95 5 % y el máximo 54 %. Solo 15 de 139 muestras (10,79 %) fueron mayores que cero. La combinación de mediana nula, P95 de 5 % y un máximo aislado del 54 % caracteriza una carga en ráfagas: la mayor parte de la ventana no presenta CPU visible del cliente, mientras algunos intervalos concentran trabajo.

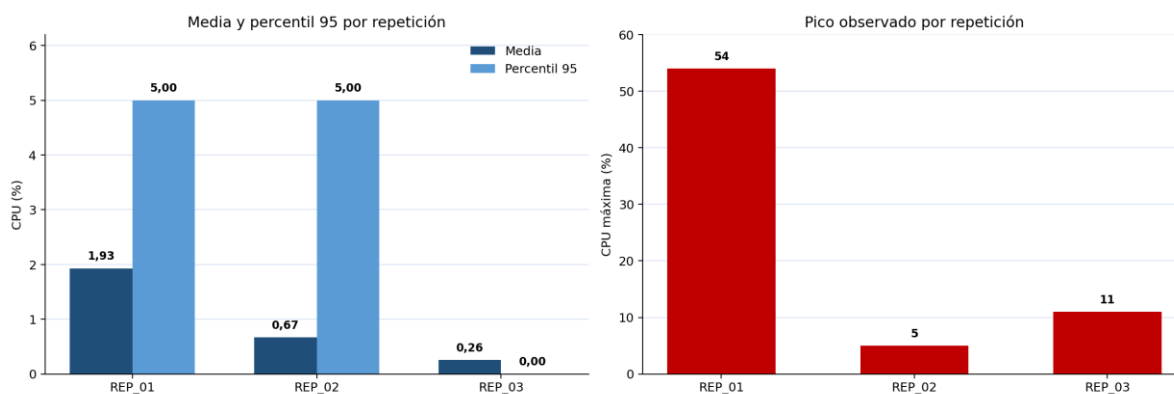
El cliente Velociraptor introduce una carga en el endpoint, especialmente visible en picos puntuales de CPU. En el laboratorio evaluado, la CPU media fue 0 % en reposo y 0,95 % durante VR_TEC_RUNNER, con un pico observado del 54 %.

Run	n	Media	Mediana	P95	Máximo	Muestras > 0
REP_01	45	1,93 %	0 %	5 %	54 %	15,56 %
REP_02	51	0,67 %	0 %	5 %	5 %	13,73 %
REP_03	43	0,26 %	0 %	0 %	11 %	2,33 %
Agregado	139	0,95 %	0 %	5 %	54 %	10,79 %

Tabla 45: CPU del cliente en VR_TEC_RUNNER por repetición

Las medias por repetición descendieron de 1,93 % a 0,67 % y 0,26 %. Los máximos, en cambio, fueron 54 %, 5 % y 11 %. Esta dispersión hace que el 54 % se confirme como comportamiento atípico: se trata del mayor valor observado en REP_01. El percentil 95 de 5 % en REP_01 y REP_02 es una referencia más representativa de la cola alta, mientras REP_03 concentró su actividad en una única muestra no nula.

CPU del cliente durante VR_TEC_RUNNER



El 54 % es un máximo puntual de REP_01; no representa la carga sostenida del cliente.

Figura 62: CPU media, percentil 95 y máximo por repetición en VR_TEC_RUNNER

La CPU se analiza a partir de 139 muestras del cliente reunidas en tres repeticiones VR_TEC_RUNNER. Los valores registrados son enteros; 15 muestras son no nulas. La media agrupada es 0,95 %, el P95 es 5 % y el máximo es 54 %. La cadencia efectiva mediana entre muestras es 4,59 s y presenta intervalos de hasta 24,51 s. Estas magnitudes describen el muestreo disponible y no excluyen picos no capturados. El script utiliza el contador Win32_PerfFormattedData_PerfProc_Process.PercentProcessorTime y una espera nominal de 2 s, a la que se suma el tiempo de consulta. El valor se interpreta como contador del proceso y no como porcentaje normalizado sobre el total de CPU de la máquina virtual.

La conclusión es que el cliente supuso una carga media observada inferior al 1 % durante el runner, acompañada de ráfagas puntuales. Esto induce a la misma conclusión anterior de que VR no supone una carga elevada para el sistema.

5.4. Impacto en memoria

La memoria media del cliente fue 57,48 MB en VR_IDLE y 57,42 MB en VR_TEC_RUNNER. La diferencia es de -0,06 MB, aproximadamente un -0,10 % respecto a reposo, menor que la variación entre repeticiones. Los máximos fueron 57,92 MB y 57,97 MB, respectivamente. En consecuencia, el runner no produjo un incremento apreciable en el consumo de memoria del cliente.

El análisis de las 145 muestras de VR_IDLE sitúa la mediana en 57,41 MB, el P95 en 57,79 MB y la desviación estándar muestral en 0,20 MB. En las 139 muestras de VR_TEC_RUNNER, la mediana fue 57,43 MB, el P95 57,87 MB y la desviación estándar 0,36 MB. Estos valores describen una banda estrecha alrededor de 57 MB.

La Tabla 46 y la Figura 63 resumen la memoria media, el percentil 95 y el máximo del cliente por escenario y repetición.

Escenario	Run	Media	P95	Máximo	Rango muestral
VR_IDLE	REP_01	57,40 MB	57,54 MB	57,57 MB	57,34-57,57 MB
VR_IDLE	REP_02	57,75 MB	57,88 MB	57,92 MB	57,71-57,92 MB
VR_IDLE	REP_03	57,30 MB	57,43 MB	57,43 MB	57,18-57,43 MB
VR_TEC_RUNNER	REP_01	57,01 MB	57,45 MB	57,53 MB	56,77-57,53 MB
VR_TEC_RUNNER	REP_02	57,85 MB	57,93 MB	57,97 MB	57,79-57,97 MB
VR_TEC_RUNNER	REP_03	57,40 MB	57,51 MB	57,57 MB	57,32-57,57 MB

Tabla 46: Memoria del cliente por escenario y repetición

La repetición con mayor media fue REP_02 en ambos escenarios: 57,75 MB en VR_IDLE y 57,85 MB en VR_TEC_RUNNER. La menor media fue 57,30 MB en VR_IDLE REP_03 y 57,01 MB en VR_TEC_RUNNER REP_01. El intervalo entre la menor y la mayor media de los seis runs es 0,84 MB, por lo que se observa que el cliente VR no usa demasiada memoria a la hora de realizar ingesta y envío de datos. Las medias globales de RAM del resumen se calculan promediando las medias de las tres repeticiones: 57,48 MB en VR_IDLE y 57,42 MB en VR_TEC_RUNNER. No corresponden a la media ponderada de todas las muestras, que sería aproximadamente 57,49 MB y 57,44 MB, respectivamente.

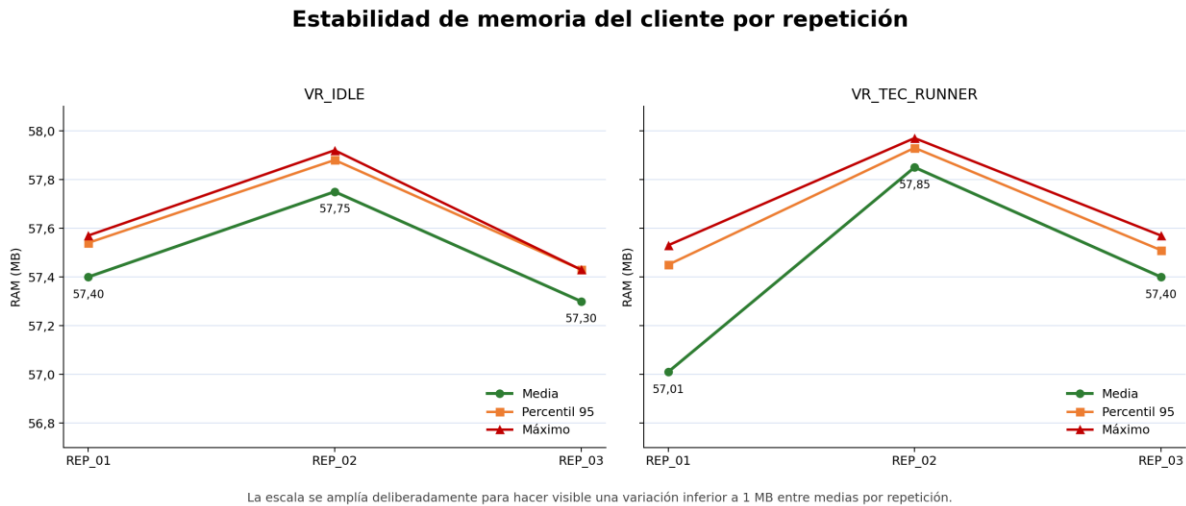


Figura 63: Memoria media, percentil 95 y máxima por repetición

Las ventanas medias fueron de aproximadamente 182 s para VR_IDLE y 316 s para VR_TEC_RUNNER, y el laboratorio solo incluye una VM y una versión/configuración del cliente. Para evaluar crecimiento acumulativo serían necesarias pruebas de horas o días, reinicios controlados y repetición sobre varios endpoints. Por ahora y con la evidencia que hemos conseguido, podemos afirmar una huella media próxima a 57 MB y estable durante las ventanas ensayadas. No permite afirmar estabilidad indefinida, consumo absoluto ni ausencia de presión de memoria en equipos con distinta configuración.

5.5. Impacto en disco

Hay dos niveles de telemetría de disco, el primero son los contadores de E/S del proceso cliente: lectura, escritura y total en bytes por segundo. El segundo es System_Disk_BytesSec, es decir, la actividad global del sistema. Solo el primer nivel sería directamente atribuible al cliente; el segundo mezcla Velociraptor, runner, servicios de Windows, paginación, antivirus, sistema de archivos y cachés.

El contador instantáneo de E/S del proceso cliente registró cero en las muestras conservadas. Esto no demuestra ausencia de operaciones de disco; la granularidad del muestreo o la caché son explicaciones posibles, no causas verificadas en este experimento.

Las Tablas 47 y 48 y la Figura 64 presentan la actividad global de disco por escenario y repetición, sin atribuirla exclusivamente al cliente Velociraptor.

Escenario	n	Media	Mediana	P95	Máximo	Muestras > 0
BASELINE_NO_VR	138	0,09 MB/s	0,00 MB/s	0,57 MB/s	2,16 MB/s	21,01 %

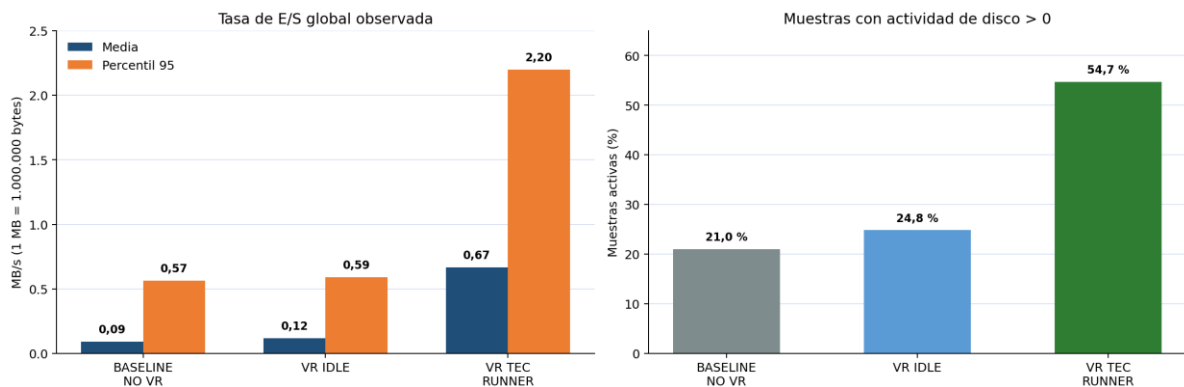
Escenario	n	Media	Mediana	P95	Máximo	Muestras > 0
VR_IDLE	145	0,12 MB/s	0,00 MB/s	0,59 MB/s	2,17 MB/s	24,83 %
VR_TEC_RUNNER	139	0,67 MB/s	0,01 MB/s	2,20 MB/s	25,36 MB/s	54,68 %

Tabla 47: Actividad global de disco por escenario

La actividad media del sistema fue 0,09 MB/s en BASELINE_NO_VR, 0,12 MB/s en VR_IDLE y 0,67 MB/s en VR_TEC_RUNNER. El P95 pasó de 0,59 MB/s en reposo a 2,20 MB/s con el runner, y la proporción de muestras activas aumentó del 24,83 % al 54,68 %. La mediana de VR_TEC_RUNNER, sin embargo, fue solo 0,0148 MB/s frente a un máximo de 25,36 MB/s. Esta asimetría muestra que la media está influida por ráfagas intensas.

El aumento es coherente con un escenario que ejecuta operaciones de fichero, cifrado controlado, borrado, staging y archivado. Precisamente por ello no puede usarse para estimar el coste de disco de Velociraptor: la carga que se pretende observar también genera E/S. La comparación VR_IDLE frente a BASELINE_NO_VR es más cercana a un contraste del agente en reposo, pero la diferencia de medias (0,12 frente a 0,09 MB/s) es pequeña respecto a la dispersión y al ruido residual del sistema.

Actividad de disco del sistema durante el benchmark



Métrica contextual de sistema: incluye runner, Windows y cachés. Los contadores de E/S del cliente fueron 0 B/s en 284/284 muestras.

Figura 64: Actividad de disco del sistema por escenario

Run	n	Media	P95	Máximo	Muestras > 0
REP_01	45	0,18 MB/s	0,86 MB/s	2,78 MB/s	53,33 %
REP_02	51	1,12 MB/s	3,31 MB/s	25,36 MB/s	56,86 %
REP_03	43	0,65 MB/s	1,44 MB/s	10,50 MB/s	53,49 %

Tabla 48: Disco global en VR_TEC_RUNNER por repetición

La variabilidad entre repeticiones refuerza la cautela: las medias fueron 0,18, 1,12 y 0,65 MB/s; los máximos 2,78, 25,36 y 10,50 MB/s. REP_02 concentra el mayor pico y eleva el agregado. Sin una separación de procesos del runner y una métrica acumulativa fiable del cliente, no es posible convertir estos valores en bytes escritos por Velociraptor ni en sobrecarga porcentual de disco atribuible.

Conclusión de disco: el dataset no acredita una sobrecarga de disco cuantificable y atribuible al cliente. Acredita que el contador por proceso no registró E/S no nula y que la actividad global del sistema aumentó bajo la carga del runner. Ambas observaciones son compatibles y deben presentarse juntas.

Una campaña futura orientada específicamente a disco debería registrar bytes acumulados de lectura/escritura por PID, separar los procesos del runner, aumentar la frecuencia o duración del muestreo y contrastar con ETW/WPA o una herramienta equivalente. Ese diseño permitiría diferenciar E/S directa, caché del sistema de archivos y ruido de fondo sin depender únicamente de tasas instantáneas.

5.6. Relación detección / coste

La relación detección/coste se construye a partir de dos resultados independientes, el benchmark sobre el laboratorio y el cliente evaluado, y los artifacts custom y la cobertura documentada previamente. Por su parte, el cliente presentó la huella de recursos descrita en este capítulo. No se calcula un cociente general de 'detecciones por CPU' porque mezclaría unidades sin relación causal y ocultaría la naturaleza distinta de ambas pruebas.

La Tabla 49 sintetiza las evidencias utilizadas, las afirmaciones permitidas y sus límites, mientras que la Tabla 50 recoge los controles de calidad del benchmark.

Evidencia	Conclusión
CPU del cliente	Media 0,95 %, P95 5 % y máximo 54 % durante VR_TEC_RUNNER.
Memoria del cliente	Huella media próxima a 57 MB en las ventanas ensayadas.
E/S del proceso	0 B/s registrados en 284/284 muestras CLIENT_SERVICE.
Disco del sistema	Mayor actividad global durante el runner.
CLIENT_EVENT	Cobertura custom documentada en el experimento de detección.

Tabla 49: Síntesis de evidencias y conclusiones

Los resultados se obtuvieron a partir de un script el cual tenía flags de seguridad, en esta tabla se detallan los campos más relevantes del control de calidad.

Control de calidad	Estado	Decisión
SchemaVersion 1.1	OK	Todos los runs válidos usan el esquema esperado.
9/9 runs válidos	OK	No faltan repeticiones.
SERVER_GUI excluido	OK	No suma en métricas de cliente.
notepad.exe runner = 0	OK	Evita contaminación del runner.
RunnerStillRunningAtEnd=True	WARN	No bloqueante en VR_TEC_RUNNER
Benchmark separado de detección	OK	No se usa para inferir FP ni cobertura.

Tabla 50: Controles de calidad del benchmark final

5.7. Conclusiones

La conclusión es clara: el cliente de Velociraptor mostró un coste medio de CPU bajo en reposo y durante la ejecución controlada, pero con picos puntuales; una memoria estable alrededor de 57 MB; y ausencia de E/S no nula en el contador instantáneo del proceso. Esto concuerda con la hipótesis presentada en el capítulo de metodología, en la que se detalla el porqué VR destaca como herramienta y era precisamente por su eficiencia y sencillez a la hora de hacerla distribuida (el poco consumo del cliente y la comunicación con el servidor). Velociraptor custom presentó el mayor nivel de detección específica acreditado entre las configuraciones contrastadas -9/9 frente a 3/9 de Hayabusa CH, 3/9 de Wazuh base, 4/9 de Wazuh custom y 7/9 para la unión complementaria de los dos grupos Wazuh- y lo hizo con un consumo de recursos reducido en el entorno experimental evaluado.

Capítulo VI: Validación comparativa con Wazuh como HIDS

Este capítulo documenta el despliegue de Wazuh como plataforma HIDS/SIEM de contraste dentro del entorno experimental del TFM. Su finalidad no es sustituir la evaluación principal de Velociraptor, sino aportar una línea externa para comparar visibilidad host-based, generación de alertas, trazabilidad de evidencias, severidad, ruido operativo y facilidad de configuración.

La comparación se plantea de forma separada para evitar sesgos metodológicos y no trata Wazuh y Velociraptor como unidades equivalentes directas. Wazuh trabaja con logs, reglas, archives y alertas; Velociraptor trabaja con artifacts, VQL, CLIENT_EVENT y evidencia host-based. La coexistencia simultánea Velociraptor + Wazuh se reserva únicamente para analizar ruido operativo.

- Arquitectura, instalación, agente y recogida de logs: [34]-[37]
- Reglas locales/custom: [33].

La configuración completa del entorno Wazuh y los resultados detallados de las campañas se recogen en el Anexo G: Configuración y resultados de Wazuh.

6.1. Objetivo de la comparación

El objetivo del despliegue fue disponer de una plataforma independiente con la que contrastar modelos de visibilidad y alerta. Wazuh se utilizó como herramienta HIDS/SIEM de referencia operativa, no como sustituto de Velociraptor, por su capacidad para recoger eventos de Windows, procesar telemetría de Sysmon, aplicar reglas de correlación y mostrar alertas en un dashboard centralizado.

La comparación se orienta a los siguientes criterios: visibilidad sobre eventos crudos, capacidad de detección, evidencias disponibles, trazabilidad, severidad, volumen de ruido y esfuerzo de despliegue/configuración.

Criterio	Interpretación en el capítulo
Visibilidad	Existencia de eventos crudos en archives.json o en la GUI de Wazuh.
Detección	Generación de alertas en alerts.json o en el dashboard a partir de reglas base o reglas custom.
Trazabilidad	Capacidad de vincular alerta, agente, fuente de telemetría, Event ID, regla y técnica MITRE.

Criterio	Interpretación en el capítulo
Severidad	Nivel de alerta de Wazuh y mapeo aproximado con perfiles P1/P2/P3/P4 de Velociraptor.
Ruido	Eventos o alertas no atribuibles a la técnica evaluada, incluyendo actividad operacional legítima.
Velocidad de implementación	Velocidad en la que se ha implementado y desplegado la herramienta.

Tabla 51: Criterios empleados para evaluar Wazuh como solución de comparación

6.2. Arquitectura del entorno Wazuh

La arquitectura se desplegó en un entorno Ubuntu virtualizado sobre Oracle VirtualBox. Se utilizó una máquina virtual Ubuntu Server dedicada para los componentes centrales de Wazuh y la misma máquina Windows de laboratorio utilizada en las pruebas HIDS como endpoint monitorizado. Esta decisión permitió mantener constante el sistema observado y reutilizar las mismas técnicas TEC empleadas en el resto del TFM.

El despliegue de los componentes de Wazuh se realizó tomando como referencia el procedimiento de instalación asistida descrito en la documentación oficial. [22]

Las Tablas 51 a 55 definen los criterios de comparación, el entorno, las fuentes de telemetría, las reglas personalizadas y las campañas de Wazuh, mientras que las Figuras 66 y 67 verifican el agente y la carga de reglas custom.

Elemento	Valor / descripción
Servidor Wazuh	Ubuntu Server 24.04.3 LTS en VirtualBox.
Modo de instalación	All-in-one: Wazuh Manager, Wazuh Indexer y Wazuh Dashboard en la misma VM.
IP servidor	<HOST_LAB>.
Recursos VM Ubuntu	4 vCPU, 8 GB RAM y disco VDI dinámico de 60 GB.
Red	Adaptador puente para comunicación directa con el endpoint y el host.
Endpoint monitorizado	Windows LT29 / LT29.ait.us.es.
IP endpoint	<HOST_LAB>.
Wazuh Agent ID	001.
Estado agente	Active.

Tabla 52: Parámetros principales del entorno Wazuh desplegado

```
admin@admin:~$ sudo /var/ossec/bin/agent_control -l

Wazuh agent_control. List of available agents:
  ID: 000, Name: admin (server), IP: 127.0.0.1, Active/Local
  ID: 001, Name: LT29, IP: any, Active

List of agentless devices:
```

Figura 65: Validación del agente LT29 como agente activo desde agent_control

6.3. Fuentes de telemetría configuradas

Para que Wazuh pudiera observar las mismas fuentes relevantes que Velociraptor, el agente Windows se configuró para remitir eventos de PowerShell Operational, Sysmon, Security y System. Estas fuentes resultan necesarias para observar ejecución de scripts, creación de procesos, creación de ficheros, instalación de servicios y eventos generales del sistema.

```

<localfile>

  <location>Microsoft-Windows-PowerShell/Operational</location>

  <log_format>eventchannel</log_format>

</localfile>

<localfile>

  <location>Microsoft-Windows-Sysmon/Operational</location>

  <log_format>eventchannel</log_format>

</localfile>

<localfile>

  <location>Security</location>

  <log_format>eventchannel</log_format>

</localfile>

<localfile>

  <location>System</location>

  <log_format>eventchannel</log_format>

</localfile>

```

Código 16: Fuentes eventchannel configuradas en el agente Windows

Tras modificar la configuración del agente, se reinició el servicio para aplicar los cambios.

```

$ Restart-Service WazuhSvc

$ Get-Service WazuhSvc

```

Código 17: Reinicio y comprobación del agente Wazuh en Windows

Fuente	Evento / uso principal	Utilidad para el TFM
PowerShell Operational	Event ID 4104	Visibilidad de Script Block Logging y comandos PowerShell.
Sysmon	Event ID 1	Creación de procesos, imagen, usuario y línea de comandos.

Fuente	Evento / uso principal	Utilidad para el TFM
Sysmon	Event ID 11	Creación de ficheros y evidencias de staging/cifrado/borrado.
System	Event ID 7045	Instalación de servicios Windows.
Security	Eventos de seguridad Windows	Apoyo contextual y auditoría del endpoint.

Tabla 53: Fuentes de telemetría utilizadas por Wazuh en la campaña

6.4. Reglas personalizadas TFM en Wazuh

Durante las pruebas se deshabilitó temporalmente una de las versiones de las reglas personalizadas para aislar la configuración evaluada. La versión exacta afectada por esta operación no puede verificarse de forma inequívoca con la evidencia conservada.

```
$ sudo mv /var/ossec/etc/rules/tfm_wazuh_custom_rules_v2.xml
$ <HOST_LAB>/tfm_wazuh_custom_rules_v1.DISABLED.xml
$ sudo systemctl restart wazuh-manager
$ sudo systemctl status wazuh-manager --no-pager
```

Código 18: Deshabilitación de la primera versión de reglas tras problemas de estabilidad

Rule ID	Perfil equivalente	Descripción funcional	Level
110201	P1	Posible exfiltración HTTP/ZIP mediante PowerShell 4104.	15
110202	P1	Borrado destructivo mediante PowerShell.	14
110203	P1	Comportamiento de cifrado/ransomware mediante PowerShell.	15
110301	P2	PowerShell sospechoso detectado en ScriptBlock 4104.	10
110401	P3	Posible persistencia Run Key por Sysmon Event ID 1.	8
110402	P3	Instalación de servicio Windows Event ID 7045.	8
110501	P4	Ejecución de PowerShell observada por Sysmon Event ID 1.	5

Tabla 54: Reglas personalizadas Wazuh utilizadas para aproximar los perfiles P1-P4 de Velociraptor

```
$ sudo /var/ossec/bin/wazuh-analysisd -t
$ sudo systemctl restart wazuh-manager
$ sudo systemctl status wazuh-manager --no-pager
```

Código 19: Validación sintáctica y reinicio tras cargar reglas custom TFM

```
sudo /var/ossec/bin/wazuh-analysisd -t
sudo systemctl restart wazuh-manager
sudo systemctl status wazuh-manager --no-pager
● wazuh-manager.service - Wazuh manager
   Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-06-22 11:25:14 UTC; 81ms ago
     Process: 8108 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
    Tasks: 200 (limit: 9261)
  Memory: 487.8M (peak: 488.1M)
    CPU: 27.446s
   CGroup: /system.slice/wazuh-manager.service
           └─8174 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             └─8175 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
               └─8176 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                 └─8179 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                   └─8182 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                     └─8223 /var/ossec/bin/wazuh-authd
                       └─8238 /var/ossec/bin/wazuh-db
                         └─8273 /var/ossec/bin/wazuh-execd
                           └─8287 /var/ossec/bin/wazuh-analysisd
                             └─8297 /var/ossec/bin/wazuh-syscheckd
                               └─8315 /var/ossec/bin/wazuh-remoted
                                 └─8397 /var/ossec/bin/wazuh-logcollector
                                   └─8414 /var/ossec/bin/wazuh-monitord
                                     └─8430 /var/ossec/bin/wazuh-modulesd
                                       └─9117 dpkg-query -s vsftpd

Jun 22 11:25:09 admin env[8108]: Started wazuh-syscheckd...
Jun 22 11:25:10 admin env[8108]: Started wazuh-remoted...
Jun 22 11:25:10 admin env[8108]: Started wazuh-logcollector...
Jun 22 11:25:11 admin env[8108]: Started wazuh-monitord...
Jun 22 11:25:11 admin env[8427]: 2026/06/22 11:25:11 wazuh-modulesd:router: INFO: Loaded router module.
Jun 22 11:25:11 admin env[8427]: 2026/06/22 11:25:11 wazuh-modulesd:content_manager: INFO: Loaded module.
Jun 22 11:25:11 admin env[8427]: 2026/06/22 11:25:11 wazuh-modulesd:inventory-harvester: INFO: Loaded module.
Jun 22 11:25:12 admin env[8108]: Started wazuh-modulesd...
Jun 22 11:25:14 admin env[8108]: Completed.
Jun 22 11:25:14 admin systemd[1]: Started wazuh-manager.service - Wazuh manager.
```

Figura 66: Reactivación de reglas personalizadas TFM y servicio wazuh-manager activo

6.5. Metodología de campañas separadas

La validación se estructuró en campañas separadas para diferenciar visibilidad, detección y ruido de coexistencia. Esta separación es crítica porque Wazuh puede registrar actividad legítima generada por Velociraptor, especialmente si el router SERVER_EVENT de Velociraptor ejecuta PowerShell, escribe JSONL o envía alertas mediante webhook.

Campaña	Condición	Objetivo	Uso en la comparación
A - Velociraptor custom	Artifacts P1/P2/P3/P4 + router	Medir detección/alertabilidad de Velociraptor.	Línea base del TFM.

Campaña	Condición	Objetivo	Uso en la comparación
B - Wazuh base	Reglas TFM desactivadas	Medir visibilidad y detección out-of-the-box.	Línea base Wazuh.
C - Wazuh custom	Reglas TFM activadas	Medir mejora aportada por reglas equivalentes.	Comparación funcional frente a VR custom.

Tabla 55: Campañas definidas para la validación comparativa

6.6. Campaña B: Wazuh base

En la campaña base se desactivaron las reglas custom TFM y se ejecutaron los mismos runners de pruebas desde el endpoint Windows. El objetivo fue medir qué telemetría y alertas genera Wazuh sin lógica personalizada diseñada específicamente para el TFM.

```
$ sudo mv /var/ossec/etc/rules/tfm_wazuh_custom_rules_v2.xml
<HOST_LAB>/tfm_wazuh_custom_rules_v2.DISABLED.xml
$ sudo systemctl restart wazuh-manager
$ sudo systemctl status wazuh-manager --no-pager
```

Código 20: Desactivación de reglas custom para la campaña Wazuh base

```
$ cd "<HOST_LAB>\Desktop\TFM\03_RUNNERS"
$ Set-ExecutionPolicy -Scope Process Bypass -Force
$ .\TFM_Run_All_TEC_Tests_v6_PATCHED_4104.ps1
$ .\TFM_Run_FP_Tests_v1.ps1
```

Código 21: Ejecución de runners TEC y falsos positivos desde Windows

```

$ BASE=~ /TFM_WAZUH_EVIDENCE/base_OK_$(date -u +"%Y%m%d_%H%M%S")
$ mkdir -p "$BASE"
$ date -u +"%Y-%m-%dT%H:%M:%SZ" | tee "$BASE/start_utc.txt"
$ sudo bash -c 'wc -l < /var/ossec/logs/archives/archives.json' | tee
"$BASE/archives_start_line.txt"
$ sudo bash -c 'wc -l < /var/ossec/logs/alerts/alerts.json' | tee
"$BASE/alerts_start_line.txt"

```

Código 22: Marcado de inicio y líneas iniciales de logs en campaña base

```

$ date -u +"%Y-%m-%dT%H:%M:%SZ" | tee "$BASE/end_utc.txt"
$ sudo bash -c 'wc -l < /var/ossec/logs/archives/archives.json' | tee
"$BASE/archives_end_line.txt"
$ sudo bash -c 'wc -l < /var/ossec/logs/alerts/alerts.json' | tee
"$BASE/alerts_end_line.txt"
$ ARCH0=$(cat "$BASE/archives_start_line.txt")
$ ARCH1=$(cat "$BASE/archives_end_line.txt")
$ ALERT0=$(cat "$BASE/alerts_start_line.txt")
$ ALERT1=$(cat "$BASE/alerts_end_line.txt")
$ sudo awk -v s="$ARCH0" -v e="$ARCH1" 'NR>s && NR<=e'
/var/ossec/logs/archives/archives.json > "$BASE/wazuh_base_archives_delta
.jsonl"
$ sudo awk -v s="$ALERT0" -v e="$ALERT1" 'NR>s && NR<=e'
/var/ossec/logs/alerts/alerts.json > "$BASE/wazuh_base_alerts_delta
.jsonl"
$ sudo chown -R admin:admin "$BASE"

```

Código 23: Extracción de delta de archives.json y alerts.json en campaña base

Las Tablas 56 a 58 y las Figuras 68 a 70 resumen la visibilidad, las reglas activadas y la cobertura específica obtenida en las campañas Wazuh base y custom.

Métrica	Resultado
Eventos crudos en archives.json	904

Métrica	Resultado
Alertas en alerts.json	58
PowerShell Event ID 4104	34
Sysmon Event ID 1	457
Sysmon Event ID 11	413
System Event ID 7045	0
Reglas custom 110xxx	0

Tabla 56: Resultados de visibilidad y alerta de la campaña Wazuh base

```

admin@admin:~$ cat "$BASE/wazuh_base_summary.txt"
=== WAZUH BASE VISIBILITY SUMMARY ===
Folder: <RUTA_EVIDENCIAS>

Total archives delta:
90 <RUTA_EVIDENCIAS>

Total alerts delta:
58 <RUTA_EVIDENCIAS>

PowerShell 4104:
34

Sysmon Event ID 1:
457

Sysmon Event ID 11:
413

System 7045:
0

TFM custom rule IDs should be 0 in base:

```

Figura 67: Resumen de la campaña Wazuh base, con ausencia de reglas TFM 110xxx

El resultado confirma que Wazuh base recibe eventos del endpoint y genera alertas propias sin depender de reglas TFM. En la GUI se observaron alertas base asociadas a Sysmon, por ejemplo eventos de creación de ficheros en ubicaciones temporales clasificados como posible Ingress Tool Transfer mediante reglas internas de Wazuh.

6.7. Campaña C: Wazuh custom

En la campaña custom se reactivaron las reglas TFM y se repitió el flujo de ejecución con los mismos runners. El objetivo fue comprobar si Wazuh podía transformar la telemetría del endpoint en alertas metodológicamente comparables a los perfiles de severidad definidos en Velociraptor, sin asumir equivalencia directa entre sus unidades.

```
$ sudo mv <HOST_LAB>/tfm_wazuh_custom_rules_v2.DISABLED.xml  
/var/ossec/etc/rules/tfm_wazuh_custom_rules_v2.xml  
  
$ sudo /var/ossec/bin/wazuh-analysisd -t  
  
$ sudo systemctl restart wazuh-manager  
  
$ sudo systemctl status wazuh-manager --no-pager
```

Código 24: Reactivación de reglas custom TFM para la campaña Wazuh custom

```
$ CUSTOM=~/TFM_WAZUH_EVIDENCE/custom_OK_$(date -u +"%Y%m%d_%H%M%S")  
  
$ mkdir -p "$CUSTOM"  
  
$ date -u +"%Y-%m-%dT%H:%M:%SZ" | tee "$CUSTOM/start_utc.txt"  
  
$ sudo bash -c 'wc -l < /var/ossec/logs/archives/archives.json' | tee  
"$CUSTOM/archives_start_line.txt"  
  
$ sudo bash -c 'wc -l < /var/ossec/logs/alerts/alerts.json' | tee  
"$CUSTOM/alerts_start_line.txt"
```

Código 25: Inicio de campaña Wazuh custom

```

$ date -u +"%Y-%m-%dT%H:%M:%SZ" | tee "$CUSTOM/end_utc.txt"

$ sudo bash -c 'wc -l < /var/ossec/logs/archives/archives.json' | tee
"$CUSTOM/archives_end_line.txt"

$ sudo bash -c 'wc -l < /var/ossec/logs/alerts/alerts.json' | tee
"$CUSTOM/alerts_end_line.txt"


$ ARCH0=$(cat "$CUSTOM/archives_start_line.txt")
$ ARCH1=$(cat "$CUSTOM/archives_end_line.txt")
$ ALERT0=$(cat "$CUSTOM/alerts_start_line.txt")
$ ALERT1=$(cat "$CUSTOM/alerts_end_line.txt")

$ sudo awk -v s="$ARCH0" -v e="$ARCH1" 'NR>s && NR<=e'
/var/ossec/logs/archives/archives.json > "$CUSTOM/wazuh_custom_archives
_delta.jsonl"

$ sudo awk -v s="$ALERT0" -v e="$ALERT1" 'NR>s && NR<=e'
/var/ossec/logs/alerts/alerts.json > "$CUSTOM/wazuh_custom_alerts
_delta.jsonl"

$ sudo chown -R admin:admin "$CUSTOM"

```

Código 26: Extracción de delta en campaña Wazuh custom

Métrica	Resultado
Eventos crudos en archives.json	18.285
Alertas en alerts.json	110
PowerShell Event ID 4104	99
Sysmon Event ID 1	127
Sysmon Event ID 11	24
System Event ID 7045	1
Alertas TFM 110xxx	56

Tabla 57: Resultados agregados de la campaña Wazuh custom

Rule ID	Número de alertas	Interpretación
110201	0	TEC-009 / exfiltración HTTP-ZIP no activada en esta campaña.
110202	5	Borrado destructivo mediante PowerShell.
110203	4	Comportamiento de cifrado/ransomware mediante PowerShell.
110301	2	PowerShell sospechoso en ScriptBlock 4104.
110402	1	Instalación de servicio Windows Event ID 7045.
110501	44	Ejecución de PowerShell observada por Sysmon Event ID 1.

Tabla 58: Distribución de reglas TFM activadas durante la campaña custom

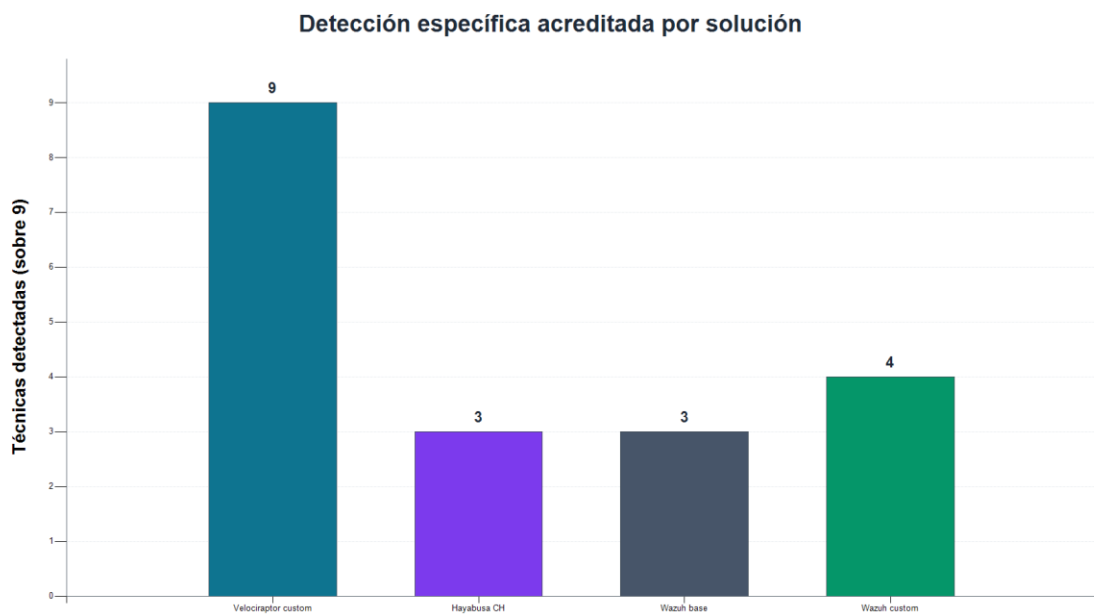


Figura 68: Cobertura específica comparada sobre nueve técnicas experimentales

```

=== WAZUH CUSTOM VISIBILITY SUMMARY ===
Folder: <RUTA_EVIDENCIAS>

Total archives delta:
1{<RUTA_EVIDENCIAS>                                jsonl

Total alerts delta:
1{<RUTA_EVIDENCIAS>                                jsonl

PowerShell 4104:
99

Sysmon Event ID 1:
127

Sysmon Event ID 11:
24

System 7045:
1

TFM custom rule IDs:
  5 "id":"110202"
  4 "id":"110203"
  2 "id":"110301"
  1 "id":"110402"
  44 "id":"110501"
  
```

Figura 69: Resumen de campaña Wazuh custom con reglas TFM activadas

6.8. Evidencias gráficas en el dashboard de Wazuh

El dashboard permitió validar visualmente la recepción de eventos del agente LT29, la visibilidad de fuentes concretas y la aparición de reglas personalizadas. Las capturas se obtuvieron aplicando filtros por agente, Event ID o rule.id, dentro de la ventana temporal de cada campaña.

Las Figuras 71 a 78 muestran las principales evidencias del dashboard filtradas por agente, Event ID, severidad y rule.id.

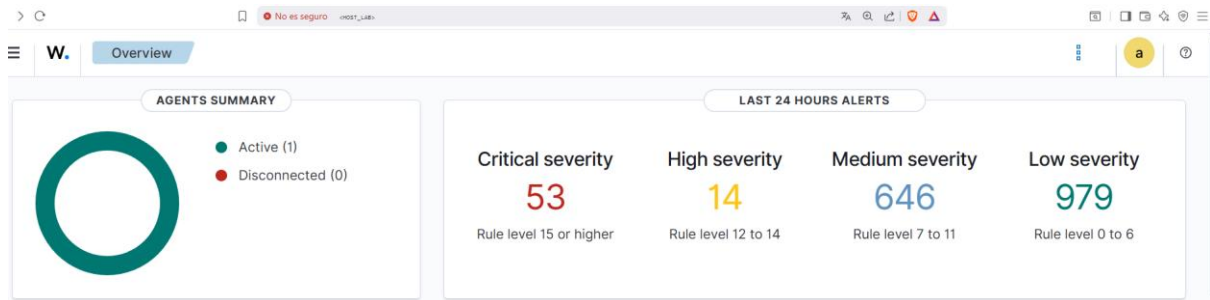


Figura 70: Resumen de severidades en el dashboard tras la campaña

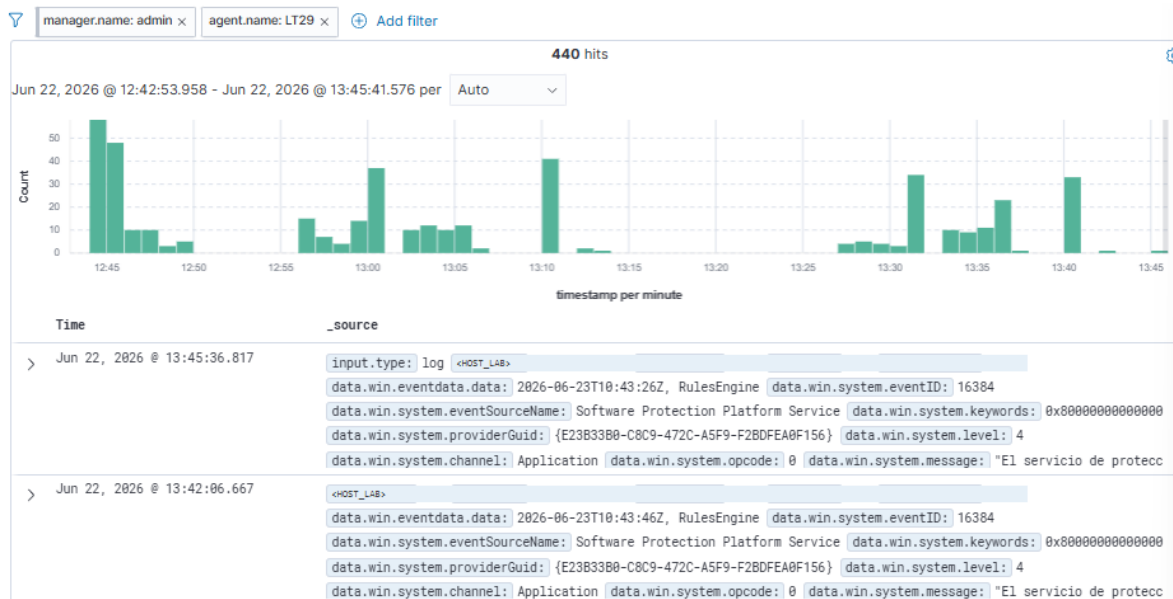


Figura 71: Dashboard de Wazuh filtrado por agent.name LT29

Evaluación de Velociraptor como HIDS para la detección de técnicas MITRE ATT&CK Enterprise

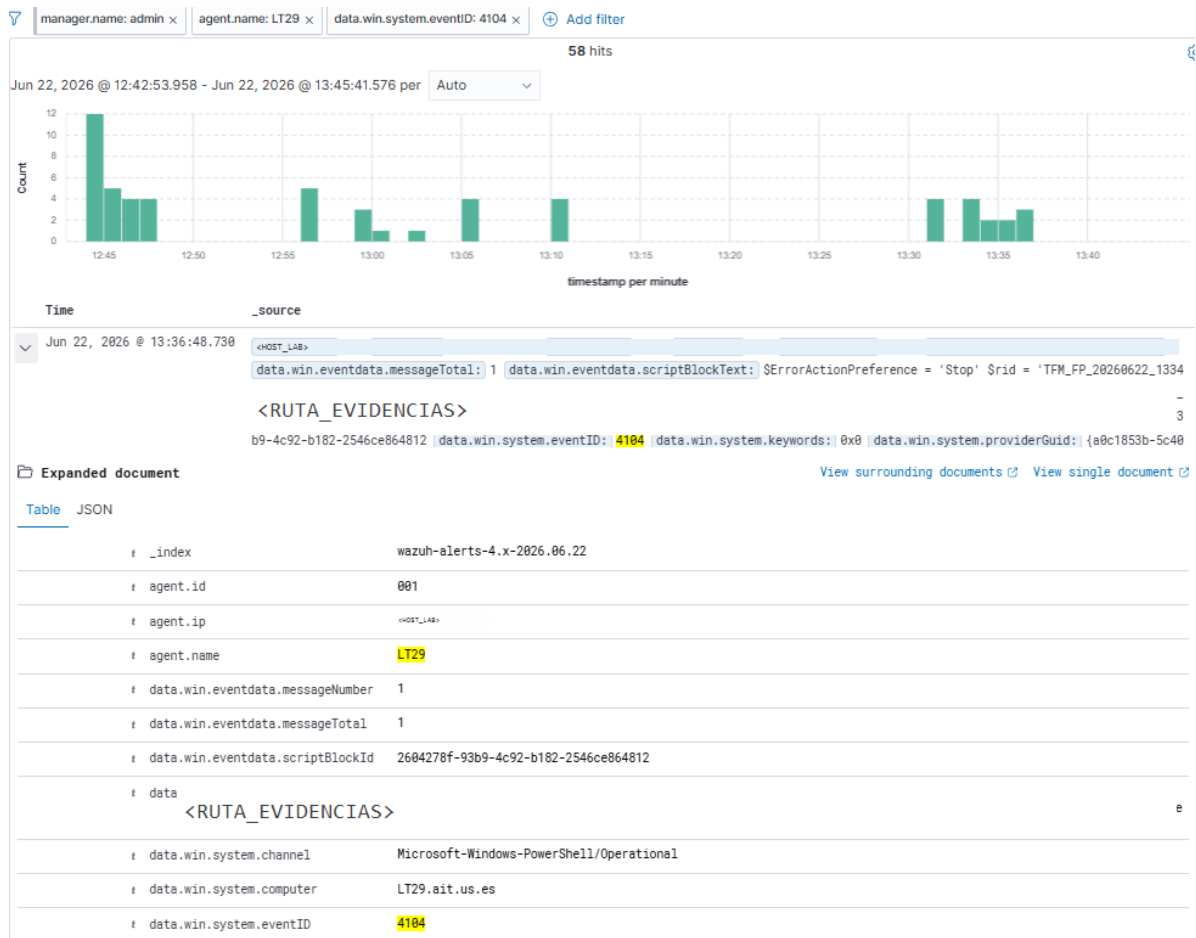


Figura 72: Visibilidad de PowerShell 4104 en Wazuh

# data.win.eventdata.terminalSessionId	1
# data.win.eventdata.user	<HOST_LAB>
# data.win.eventdata.utcTime	2026-06-22 11:29:19.394
# data.win.system.channel	Microsoft-Windows-Sysmon/Operational
# data.win.system.computer	LT29.ait.us.es
# data.win.system.eventID	1

Figura 73: Visibilidad de Sysmon Event ID 1, asociado a creación de procesos

# data.win.eventdata.user	<HOST_LAB>
# data.win.eventdata.utcTime	2026-06-22 11:09:47.214
# data.win.system.channel	Microsoft-Windows-Sysmon/Operational
# data.win.system.computer	LT29.ait.us.es
# data.win.system.eventID	11

Figura 74: Visibilidad de Sysmon Event ID 11, asociado a creación de ficheros



Figura 75: Ejemplo de detección custom mediante la regla 110202

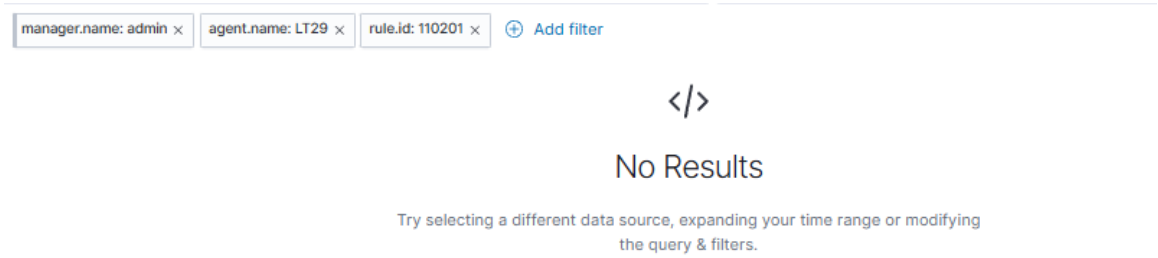


Figura 76: Ausencia de resultados para rule.id 110201 en la campaña custom

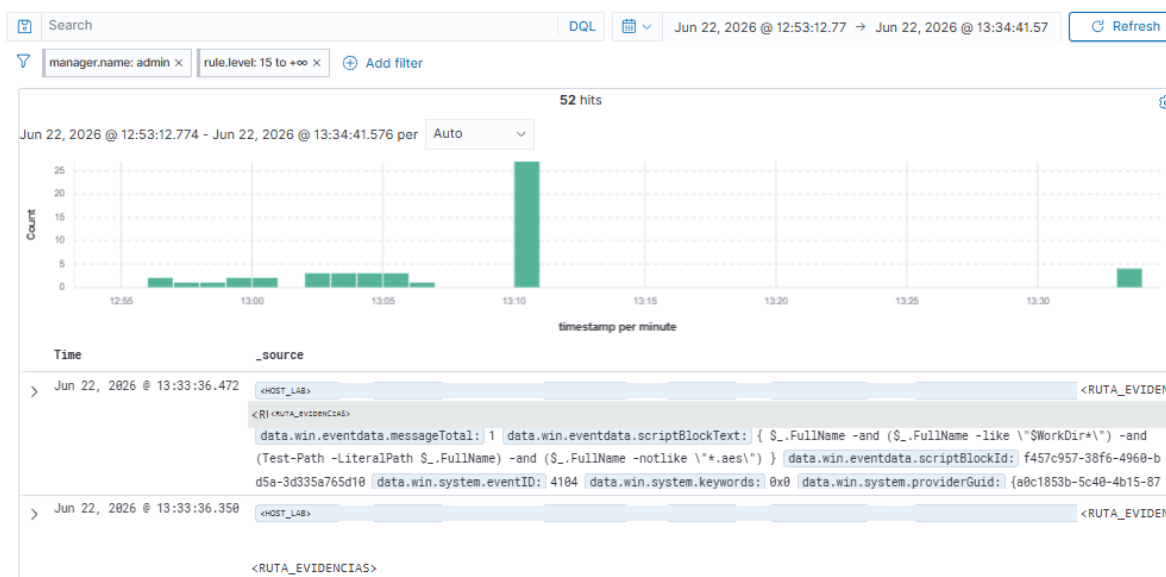


Figura 77: Vista de alertas de severidad alta/crítica generadas en Wazuh

6.9. Comparación Wazuh – Velociraptor

Los resultados obtenidos muestran una diferencia significativa en el esfuerzo de configuración y puesta en marcha de ambas soluciones. Velociraptor destaca por su arquitectura distribuida, flexibilidad y elevada capacidad de personalización, permitiendo alcanzar buenos niveles de visibilidad y detección mediante el desarrollo y ajuste de artifacts específicos. Sin embargo, esta flexibilidad implica una mayor carga de configuración, validación y mantenimiento, especialmente cuando se pretende implementar una lógica de detección adaptada a técnicas concretas de MITRE ATT&CK.

En comparación, Wazuh permitió obtener un nivel de detección aceptable con un esfuerzo de configuración considerablemente inferior. En el entorno experimental empleado, el tiempo necesario para alcanzar una configuración funcional fue mucho menor que el requerido para desarrollar y ajustar la arquitectura equivalente en Velociraptor. Este resultado no implica una menor capacidad técnica de Velociraptor, sino que evidencia un compromiso entre flexibilidad y personalización frente a facilidad de despliegue y configuración: Velociraptor ofrece un mayor control sobre la telemetría y la lógica de detección, mientras que Wazuh facilita una implantación más rápida mediante mecanismos de detección y reglas más directamente orientados a un entorno SIEM/HIDS.

Las Tablas 59 y 60 comparan la cobertura acreditada y el mapeo de severidad entre Velociraptor y Wazuh. Y las figuras 78 y 79 comparan la visibilidad y cobertura alcanzada entre wazuh y velociraptor.

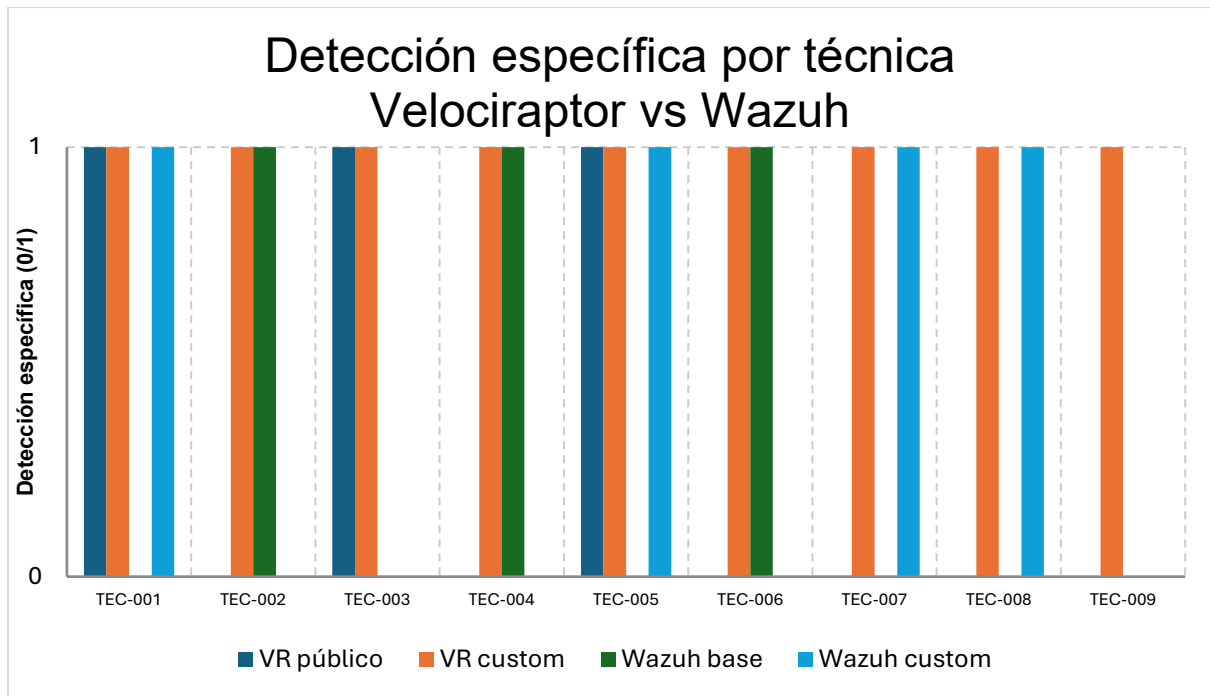


Figura 78: Detección por técnica Velociraptor vs Wazuh

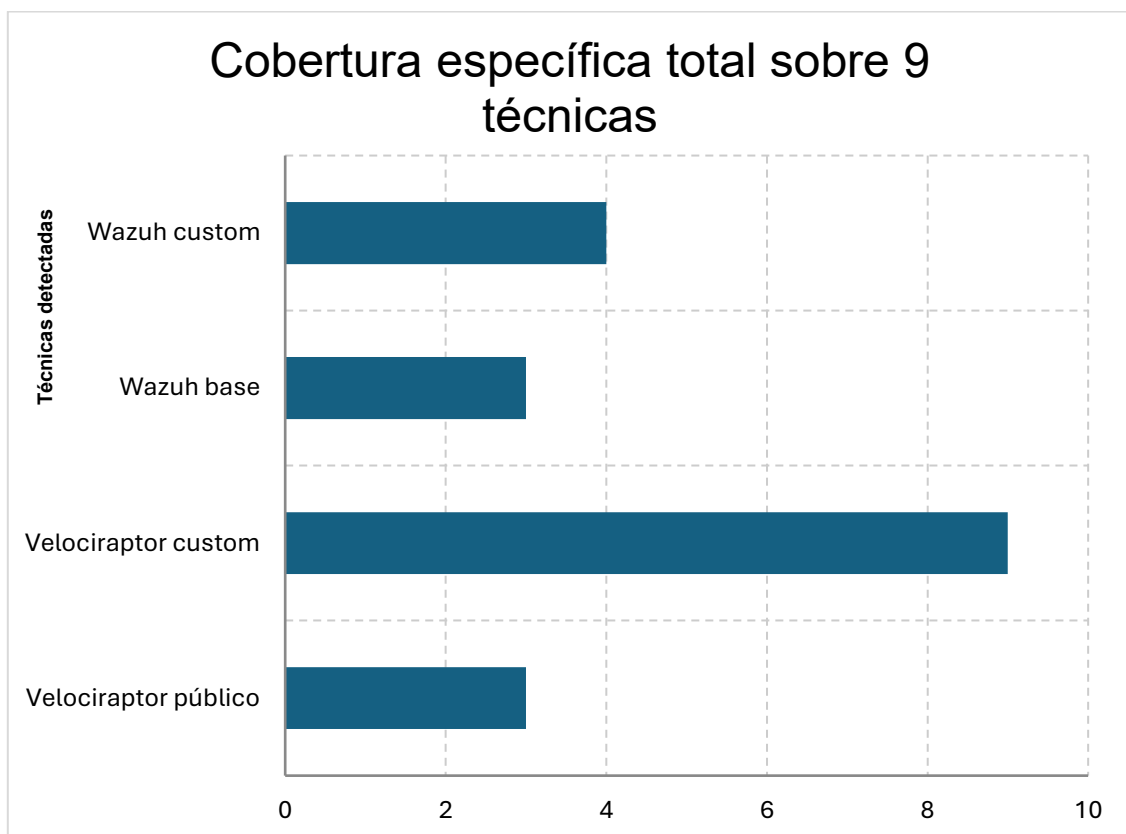


Figura 79: Cobertura por técnica wazuh vs VR

Sistema/campaña	Cobertura	Técnicas	Limitación principal
Velociraptor custom	9/9	TEC-001 a TEC-009	CLIENT_EVENT detecta; Router/Discord no.
Velociraptor públicos - Hayabusa CH	3/9	TEC-001, TEC-003, TEC-005	CHM no ejecutada; otros artifacts aportan visibilidad.
Wazuh base	3/9	TEC-002, TEC-004, TEC-006	Ruleset nativo; alertas genéricas no suman técnica.
Wazuh custom	4/9	TEC-001, TEC-005, TEC-007, TEC-008	La regla 110201 no produjo alertas en esta campaña. La unión de los casos detectados por reglas base y custom alcanza 7/9; es una combinación de resultados, no otra configuración ensayada.

Tabla 59: Comparación final entre Wazuh base, Wazuh custom y Velociraptor custom

La cobertura se refiere a nueve casos controlados sobre los que se desarrollaron e iteraron las configuraciones. En TEC-009 se acredita evidencia host de preparación, archivado y ejecución; la confirmación de transferencia HTTP procede del runner/receptor y no suma una detección de Velociraptor. La cobertura por subfase no equivale a atribución de red completa.

Perfil VR	Mapeo Wazuh aproximado	Criterio
P1_CRITICAL	Level 13-15	Eventos críticos o de impacto alto.
P2_HIGH	Level 10-12	Alta evidencia forense o PowerShell sospechoso.
P3_MEDIUM	Level 7-9	Comportamientos relevantes de persistencia o servicio.
P4_LOW	Level 3-6	Visibilidad básica o ejecución observada.

Tabla 60: Mapeo de severidad usado entre perfiles Velociraptor y niveles Wazuh

La comparación entre Velociraptor y Wazuh se limita a la cobertura de detección obtenida durante la campaña de ataques. No se realizó una comparación equivalente sobre actividad legítima ni falsos positivos, al no haberse ejecutado una campaña benigna homogénea sobre ambas herramientas.

6.10. Limitaciones observadas

En primer lugar, Wazuh base genera alertas útiles sin reglas custom, pero algunas pueden tener severidad elevada ante actividad legítima o contextual del sistema. Esto obliga a revisar manualmente la semántica de la alerta antes de asociarla a una técnica MITRE concreta.

En segundo lugar, la regla 110501 generó un volumen notable de alertas P4 al detectar ejecución de PowerShell mediante Sysmon Event ID 1. Esta regla aporta visibilidad, pero puede generar ruido si no se contextualiza con la técnica ejecutada, el comando concreto y la ventana temporal de prueba.

En tercer lugar, la regla 110201, asociada a TEC-009, no se activó durante la campaña custom. Este resultado se conserva como gap de Wazuh y evidencia que la cobertura depende de la lógica de reglas y de la fuente de eventos disponible.

Por último, al coexistir con Velociraptor, Wazuh puede registrar actividad operacional del propio sistema de alertado de VR, como ConvertFrom-Json, FromBase64String, Add-Content, Invoke-RestMethod, escritura en soc_alerts.jsonl o envío a Discord. Por ello, dicha coexistencia debe tratarse como experimento de ruido operativo, no como campaña principal de detección.

Capítulo VII: Conclusiones

Este capítulo recoge las conclusiones principales del TFM, relacionando los resultados obtenidos con los objetivos planteados al inicio del trabajo. La evaluación se centra en determinar hasta qué punto Velociraptor puede aproximarse a un comportamiento HIDS/DFIR en Windows mediante monitorización continua, artifacts personalizados y una metodología reproducible basada en técnicas MITRE ATT&CK Enterprise.

7.1. Conclusiones principales

El resultado del TFM muestra que Velociraptor puede utilizarse como una aproximación HIDS/DFIR en Windows cuando se combina monitorización continua mediante CLIENT_EVENT, artifacts custom orientados a comportamiento y una metodología estricta de validación. En la campaña final, el subconjunto TEC-001 a TEC-009 obtuvo detección mediante CLIENT_EVENT para las nueve técnicas evaluadas.

La aportación técnica principal no reside únicamente en la ejecución de técnicas MITRE, sino en la construcción de una cadena completa de detection engineering. Esta cadena incluye la selección de técnicas, la ejecución controlada mediante runners, la generación de telemetría Windows, el desarrollo de artifacts VQL, la clasificación por perfiles de artifacts P1/P2/P3/P4, la separación entre detección y salida externa, el análisis de técnicas benignas, el benchmark de rendimiento y la validación comparativa con Wazuh.

La evaluación definitiva de artifacts públicos comprende seis campañas CLIENT_EVENT: Windows.Hayabusa.Monitoring aportó detección específica CH en 3/9 técnicas -TEC-001, TEC-003 y TEC-005-, mientras que ServiceCreation, ProcessCreation, ETW, TrackNetwork y SysmonLogForward aportaron visibilidad o evidencia sin equivaler por sí solos a detección MITRE.

También se concluye que Velociraptor ofrece una arquitectura muy flexible para construir detecciones propias, pero exige un esfuerzo significativo de ajuste. La calidad de los resultados depende de la telemetría disponible en el endpoint, de la configuración de Sysmon y Windows Event Logs, de la precisión de los artifacts y de la validación experimental frente a ground truth. Por tanto, Velociraptor no debe entenderse como un HIDS cerrado y listo para producción, sino como una plataforma DFIR extensible sobre la que es posible construir capacidades de detección host-based.

La comparativa con Wazuh contextualiza los resultados sin afirmar superioridad universal. Sobre el mismo denominador de nueve técnicas, Wazuh base acreditó detección específica en 3/9 -TEC-002, TEC-004 y TEC-006- y las reglas Wazuh custom en 4/9 -TEC-001, TEC-005, TEC-007 y TEC-008-. La unión complementaria alcanza 7/9, pero no se trata como una quinta solución.

7.2. Respuesta a objetivos

La Tabla 61 relaciona cada pregunta de investigación con el resultado obtenido, la evidencia principal y las limitaciones aplicables.

Pregunta	Resultado	Evidencia principal
RQ1 - Cobertura custom	9/9 en TEC-001 a TEC-009	Filas CLIENT_EVENT de P1–P4 y matriz por técnica – Excel de visibilidad
RQ2 - Artifacts públicos	Seis campañas positivas de visibilidad; Hayabusa CH 3/9	Campañas públicas y matriz de resultados
RQ3 - Rendimiento	CPU media: 0 % VR_IDLE y 0,95 % VR_TEC_RUNNER; RAM media: 57,48 y 57,42 MB; duración media: 181,57 y 315,60 s; 9/9 runs válidos.	BASELINE_NO_VR, VR_IDLE y VR_TEC_RUNNER; CPU, RAM y duración – Excel de benchmark
RQ4 - Arquitectura personalizada	P1–P4 cubren las brechas evaluadas y Router no detecta	Cuatro detectores CLIENT_EVENT y un Router SERVER_EVENT
RQ5 - Falsos positivos	10/10 pruebas benignas; 0 hits definitivos	summary.json y vr_hits.csv reconciliados – Excel de visibilidad
RQ6 - Comparación con Wazuh	Wazuh base 3/9; custom 4/9; unión complementaria 7/9	Reglas nativas y tfm_wazuh_custom_rules_v2.xml – Excel de visibilidad

Tabla 61: Correspondencia entre preguntas de investigación, resultados y evidencias

En conjunto, los objetivos se consideran cumplidos dentro del alcance experimental.

7.3. Limitaciones identificadas

Las conclusiones se limitan a un laboratorio controlado, con Windows 10, telemetría configurada, scripts dummy y un subconjunto reducido de técnicas. No se evalúa malware real, no se comprometen sistemas de terceros y no se afirma cobertura completa de MITRE ATT&CK Enterprise. Los resultados deben interpretarse como una validación experimental acotada, no como una certificación general de capacidad defensiva.

Una limitación relevante fue la dependencia de la visibilidad del endpoint. Algunas técnicas solo pudieron analizarse correctamente tras ajustar la telemetría disponible, especialmente mediante Sysmon y Windows Event Logs. Esto demuestra que la detección no depende únicamente del artifact de Velociraptor, sino también de la calidad de las fuentes host-based configuradas en el sistema.

En Wazuh se conserva la limitación asociada a la regla 110201, que no llegó a activarse para TEC-009. Esto no invalida la comparativa, pero sí refleja que la cobertura de una técnica puede variar notablemente según la lógica de reglas disponible y la fuente de eventos utilizada.

El benchmark de rendimiento también presenta limitaciones. Mide el coste operativo agregado del cliente Velociraptor en el laboratorio, no la calidad de detección, el consumo individual de P1–P4, la escalabilidad en flotas ni el comportamiento bajo cargas prolongadas. Los valores observados -0,95 % de CPU media y 54 % de pico en VR_TEC_RUNNER; 57,42 MB de RAM media y 57,97 MB de pico- no se extrapolan a otros equipos o despliegues.

Los artifacts se desarrollaron e iteraron observando los comportamientos del mismo subconjunto experimental empleado en la evaluación final. Por tanto, el resultado 9/9 acredita cobertura de los casos controlados con la configuración ajustada, no capacidad de detección generalizable a variantes no vistas. No se utilizó un conjunto independiente de validación ciega. Una evaluación posterior debería congelar las versiones antes de ejecutar nuevas variantes y actividad legítima no usada durante el ajuste.

7.4. Valor del enfoque

El valor principal del enfoque reside en hacer auditable la cadena completa de razonamiento. Para cada técnica evaluada se puede identificar qué acción se ejecutó, qué fuente generó telemetría, qué artifact la clasificó, qué perfil se asignó, qué evidencia se conservó y qué salida externa se produjo. Esta trazabilidad permite defender los resultados frente a un tribunal técnico y evita conclusiones basadas únicamente en capturas aisladas o en afirmaciones no verificables.

La separación entre visibilidad, detección, alerta y evidencia aporta rigor metodológico. Un evento visible en Windows o Sysmon no se ha considerado automáticamente como detección; solo se ha tratado como detección cuando un artifact CLIENT_EVENT lo ha clasificado de forma reproducible y asociada a una técnica concreta. Esta decisión fortalece la validez del trabajo y reduce el riesgo de sobreinterpretar los resultados.

El uso de perfiles de artifacts P1/P2/P3/P4 también aporta valor operativo. Permite diferenciar entre señales críticas, evidencia forense relevante, comportamientos sospechosos y visibilidad básica. Esta clasificación evita que todos los eventos se traten con la misma severidad y aproxima el sistema a una lógica operativa de SOC sin confundir estos perfiles con prioridades documentales de planificación.

Finalmente, el TFM demuestra que Velociraptor puede ser una herramienta útil para construir capacidades HIDS/DFIR personalizadas, siempre que se acompañe de una metodología rigurosa, telemetría suficiente, validación experimental y documentación de limitaciones. La solución desarrollada no sustituye a un EDR comercial ni a un SIEM completo, pero sí muestra que Velociraptor

puede aportar visibilidad, detección reproducible y evidencia forense trazable en escenarios host-based controlados.

Capítulo VIII: Discusión y trabajo futuro

Este capítulo presenta las principales líneas de trabajo futuro derivadas del desarrollo y validación experimental realizados. Aunque los resultados obtenidos permiten confirmar la viabilidad de Velociraptor como herramienta host-based para detección y respuesta forense en un entorno controlado, el alcance del proyecto queda limitado a un laboratorio Windows, a nueve técnicas MITRE ATT&CK y a una arquitectura basada en artifacts personalizados.

8.1. Discusión integrada de resultados

Los resultados solo son comparables cuando se conserva la unidad de análisis. Velociraptor custom acredita técnicas mediante filas CLIENT_EVENT clasificadas; los artifacts públicos aportan visibilidad o detección específica según su lógica; Wazuh acredita reglas y alertas; el benchmark mide recursos. Un evento, una alerta, una técnica, una evidencia forense y una salida externa no son magnitudes intercambiables.

Dentro del subconjunto TEC-001 a TEC-009, Velociraptor custom alcanzó 9/9, Hayabusa CH 3/9, Wazuh base 3/9 y Wazuh custom 4/9. La unión base+custom de Wazuh es complementaria y alcanza 7/9, pero no constituye una solución adicional. Estos resultados describen la configuración ensayada y no demuestran superioridad universal de Velociraptor ni equivalencia funcional con un EDR completo.

Las 379 filas CLIENT_EVENT emitidas/exportadas, incluidas emisiones repetidas describen volumen: P1=19, P2=118, P3=109 y P4=133. No miden por sí mismas cobertura ni calidad. De forma análoga, los volúmenes de alerts y archives de Wazuh se mantienen como contexto y no se utilizan para sumar técnicas.

El benchmark acredita 9/9 runs válidos y un coste agregado del cliente. No mide cada artifact P1-P4 de manera individual, ni detección, falsos positivos o escalabilidad. La evaluación conserva TrackNetwork como visible sin atribución fiable de proceso en la captura analizada. En TEC-009, el HTTP 200 es ground truth del runner independiente, separado de la detección CLIENT_EVENT.

Además, como línea de mejora de integración de datos, se propone realizar una evaluación comparativa homogénea entre Velociraptor y Wazuh, aplicando las mismas campañas de ataque y actividad benigna y recogiendo métricas equivalentes de rendimiento. Esto permitiría comparar conjuntamente cobertura, falsos positivos y consumo de recursos.

8.2. Ampliación de técnicas MITRE

El trabajo futuro más directo consiste en ampliar el subconjunto TEC-001 a TEC-009 hacia más tácticas de MITRE ATT&CK Enterprise, especialmente Defense Evasion, Credential Access, Lateral

Movement y Command and Control. La ampliación debería mantener el mismo criterio: técnica observable en host, runner reproducible, datos dummy y separación entre ejecución, visibilidad, detección, alerta y evidencia.

Por proponer algunas, primero se deberían priorizar comportamientos frecuentes en campañas actuales. Por ejemplo, la Tabla 62 recoge las técnicas MITRE ATT&CK propuestas para ampliar el alcance del trabajo en futuras iteraciones.

MITRE	Técnica	Por qué interesa	Emulación segura
T1003	OS Credential Dumping	Muy relevante en Credential Access y movimiento lateral posterior	Simulación con eventos y accesos controlados, sin extraer credenciales reales
T1078	Valid Accounts	Muy usada en intrusiones reales y ransomware	Cuentas dummy, logon controlado, eventos 4624/4625
T1021	Remote Services	Clave para lateral movement	RDP/WinRM/SMB controlado entre VMs
T1562.001	Disable or Modify Tools	Muy relevante en evasión de defensas	Cambios simulados o sobre servicios dummy
T1055	Process Injection	Técnica muy usada para evasión	Solo PoC inocuo o detección teórica; cuidado con complejidad

Tabla 62: Técnicas MITRE ATT&CK propuestas para trabajo futuro

8.3. Desarrollo de nuevos artifacts

La arquitectura P1/P2/P3/P4 puede evolucionar con reglas más agregadas, correlación temporal y nuevas fuentes de telemetría. El objetivo sería reducir ruido en los perfiles P3/P4 sin perder evidencia forense, y reservar P1/P2 para señales de mayor criticidad.

Una mejora clara sería aplicar agregación temporal en técnicas como TEC-008. En lugar de generar una alerta por cada fichero eliminado, el artifact podría agrupar múltiples eventos Sysmon Event ID 26 dentro de una misma ventana temporal. De esta forma, el sistema detectaría el patrón de borrado masivo sin producir un volumen excesivo de alertas.

Otra línea de mejora sería correlacionar, dentro de una ventana temporal, la ejecución de PowerShell, la creación de un archivo comprimido y una conexión de salida, utilizando exclusivamente telemetría host y relaciones entre procesos. El summary del runner se conservaría como ground truth externo para evaluar la regla, nunca como condición del detector.

8.4. Automatización de detecciones

La automatización debería evolucionar hacia un modelo más cercano a detection engineering. Actualmente, los artifacts CLIENT_EVENT generan eventos y el router SOC_v3 actúa como canal de salida hacia JSONL y Discord. Como trabajo futuro, este flujo podría ampliarse con validaciones automáticas de calidad, etiquetado de severidad y generación de evidencias normalizadas.

Una mejora útil sería generar automáticamente resúmenes por campaña, incluyendo número de técnicas ejecutadas, detecciones por perfil, alertas enviadas, eventos descartados y posibles falsos positivos. Esto permitiría comparar campañas de forma más rápida y reducir el trabajo manual de revisión. También podría incorporarse un sistema de validación continua de artifacts. Cada vez que se modifique un artifact P1/P2/P3/P4, el runner podría ejecutar una campaña mínima de regresión para comprobar que no se rompe la detección de técnicas ya validadas. Esta aproximación permitiría tratar las reglas como código, con control de versiones, hashes, changelog y pruebas reproducibles.

8.5. Integración SIEM/SOC y despliegue en entornos reales

Un despliegue real requeriría probar retención de eventos, control de ruido, seguridad de webhooks, gestión de credenciales, tuning por baseline de usuario y exportación hacia SIEM. En este trabajo se ha validado la viabilidad técnica en laboratorio, pero un entorno corporativo exige controles adicionales. La integración con SIEM permitiría centralizar los eventos generados por Velociraptor, correlacionarlos con otras fuentes y construir casos de uso SOC más completos. Por ejemplo, una alerta de PowerShell sospechoso podría enriquecerse con eventos de autenticación, conexiones de red, inventario de endpoint o actividad previa del usuario.

Otro punto importante sería la seguridad operativa del canal de salida. Los webhooks, tokens y credenciales no deberían quedar en texto plano dentro de artifacts o scripts. En un despliegue real deberían protegerse mediante mecanismos de gestión de secretos, permisos mínimos y rotación periódica.

Por último, sería necesario repetir el benchmark en hardware distinto, con más endpoints, mayor duración y cargas más realistas. Las métricas obtenidas en este TFM son válidas para el laboratorio evaluado, pero no deben extrapolarse directamente a producción sin pruebas adicionales.

La declaración responsable relativa al uso de herramientas de inteligencia artificial durante la elaboración del trabajo se incluye en el Anexo I: Declaración responsable sobre el uso de inteligencia artificial.

Anexo A: Instalación del entorno de laboratorio

A.1. Máquina Virtual Windows 10

Para la realización de las pruebas experimentales se utilizó una máquina virtual proporcionada y preconfigurada. Esta fue desplegada mediante Oracle VirtualBox, empleando Windows 10 de 64 bits como sistema operativo invitado.

En primer lugar, se instaló Oracle VirtualBox 7.2.16, una vez finalizada la instalación, se inició la aplicación para incorporar la máquina virtual utilizada durante las pruebas.

Como buena práctica, antes de empezar cualquier instalación, pasar el ejecutable por alguna herramienta de escaneo.

Las Figuras 80 a 84 documentan la validación del software, el despliegue de la máquina virtual y la instalación y ejecución del agente Wazuh.

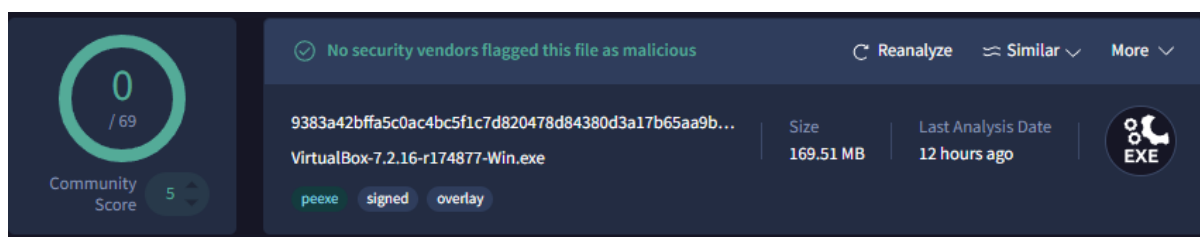


Figura 80: Resultado del análisis de un componente Oracle en VirusTotal

Luego, se importó la máquina virtual desde el apartado de añadir/importar una máquina virtual existente:

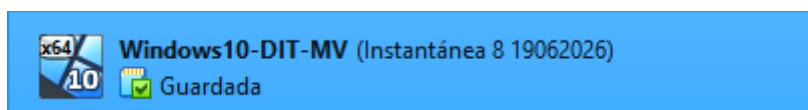


Figura 81: Máquina virtual del laboratorio desplegada y en ejecución

Una vez incorporada la máquina virtual, se inició Windows y se verificó que el sistema operativo arrancaba correctamente antes de proceder con la instalación de las herramientas de monitorización y detección.

A.2. Velociraptor

Para el desarrollo del trabajo se utilizó Velociraptor 0.75.6 para Windows x64. La herramienta se desplegó siguiendo una arquitectura cliente-servidor, utilizando una instancia como servidor de administración y un agente instalado en el endpoint Windows monitorizado.

A.2.1. Velociraptor Servidor

Para facilitar su utilización desde línea de comandos, el ejecutable fue renombrado como:

```
$ velociraptor.exe
```

Código 27: Nombre VR

En PowerShell admin se genera la configuración con:

```
$ .\velociraptor.exe config generate -i
```

Código 28: Generación interactiva de la configuración de Velociraptor

Esto genera el fichero server.config.yaml, que contiene la configuración para el funcionamiento del servidor, incluyendo los parámetros de comunicación, acceso a la interfaz gráfica, almacenamiento y certificados empleados para establecer la comunicación con los clientes.

Toda la configuración y su restablecimiento se ha gestionado mediante un txt con los comandos necesarios:

```
# Para cargar la configuración
velociraptor.exe --config server.config.yaml gui

# SI SE REINICIA LA CONTRASEÑA O SE PIERDE LA CUENTA:
velociraptor.exe --config server.config.yaml user add admin --role
administrator
```

Código 29: Documento_config_VR

Una vez levantado, debemos ingresar desde nuestro navegador en <https://127.0.0.1:8889/>.

A.2.2. Velociraptor Agente

Una vez desplegado el servidor, se procedió a instalar el agente de Velociraptor sobre el endpoint Windows utilizado durante las pruebas. El agente necesita disponer de una configuración de cliente que contenga la información necesaria para establecer comunicación con el servidor. Para ello se realizó la instalación mediante GUI con Server.Utils.CreateMSI.

De esta forma es posible comprobar que el endpoint establece correctamente comunicación con el servidor antes de instalar el agente como servicio. Una vez verificado su funcionamiento, el cliente se instaló como servicio de Windows ejecutando una consola con privilegios de administrador y utilizando:

```
$ .\velociraptor.exe service install --config client.config.yaml -v
```

Código 30: Agente instalación II

Este procedimiento instala Velociraptor como un servicio de Windows, permitiendo que el agente se inicie automáticamente junto con el sistema operativo y permanezca disponible para recibir consultas desde el servidor. El estado del servicio puede comprobarse usando `sc.exe query velociraptor`.

Tras la instalación se verificó igualmente desde la interfaz web del servidor que el endpoint aparecía registrado y conectado correctamente.

A.3. Sysmon

Como herramienta complementaria de monitorización del sistema se utilizó Sysmon v15.15, perteneciente a la suite Microsoft Sysinternals. Su función dentro del laboratorio fue proporcionar una mayor visibilidad sobre la actividad generada en el endpoint Windows, registrando eventos relacionados con procesos, conexiones de red, modificaciones de archivos y otras acciones relevantes para el análisis de seguridad. En primer lugar, se descargó Sysmon desde la página oficial de Microsoft Sysinternals. Tras descomprimir el paquete descargado, se utilizó el ejecutable correspondiente a sistemas Windows de 64 bits:

```
$ Sysmon64.exe
```

Código 31: Sysmon

Para realizar la instalación se abrió una consola de PowerShell con privilegios de administrador y se realizó:

```
$ .\Sysmon64.exe -accepteula -i sysmonconfig.xml
```

Código 32: Instalación de Sysmon

Una vez completada la instalación, se verificó que el servicio se encontraba correctamente desplegado mediante:

```
$ sc.exe query Sysmon64
```

Código 33: Sysmon comprobación

Los eventos generados se verificaron en el canal Microsoft-Windows-Sysmon del Visor de eventos. En este registro pueden consultarse los eventos generados por el endpoint de acuerdo con las reglas definidas en el fichero de configuración. De este modo, Sysmon quedó instalado como servicio permanente en el endpoint Windows utilizado para las pruebas y preparado para registrar la telemetría necesaria durante la ejecución de las técnicas analizadas.

A.4. PowerShell Script Block Logging

Durante las pruebas se habilitó PowerShell Script Block Logging con el objetivo de disponer de eventos Event ID 4104 en el canal Microsoft-Windows-PowerShell/Operational. Esta fuente de telemetría fue utilizada posteriormente para analizar distintas técnicas ejecutadas mediante PowerShell [18].

```
$ New-Item -Path  
"HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging  
" -Force | Out-Null  
  
$ New-ItemProperty -Path  
"HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging  
" -Name "EnableScriptBlockLogging" -Value 1 -PropertyType DWord -Force
```

Código 34: habilitar PowerShell Script Block Logging

Para comprobar que haya funcionado podemos usar el siguiente código:

```
$ Get-WinEvent -LogName "Microsoft-Windows-PowerShell/Operational" -  
FilterXPath "[System[(EventID=4104)]]" -MaxEvents 10
```

Código 35: Comprobar_Script_block

Una vez habilitado, se verificó la generación de eventos desde el Visor de eventos de Windows:

Applications and Services Logs -> Microsoft -> Windows -> PowerShell -> Operational

A.5. Wazuh

La instalación se realizó sobre Ubuntu Server 24.04.3 LTS. Tras instalar el sistema operativo, se habilitó OpenSSH para administrar la máquina desde el host Windows mediante PowerShell. Posteriormente se descargó el script oficial de instalación de Wazuh y se ejecutó el modo all-in-one.

```
$ sudo apt update && sudo apt upgrade -y  
$ sudo apt install openssh-server -y  
$ sudo systemctl enable --now ssh  
$ sudo apt install curl nano net-tools -y  
$ curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh  
$ sudo bash ./wazuh-install.sh -a  
$ sudo systemctl status wazuh-manager --no-pager  
# Active  
$ sudo systemctl status wazuh-indexer --no-pager  
# Active  
$ sudo systemctl status wazuh-dashboard --no-pager  
# Active
```

Código 36: Instalación base de dependencias y despliegue all-in-one de Wazuh

El dashboard quedó disponible desde el host físico en la URL https://<HOST_LAB>. Se utilizó el usuario admin generado durante la instalación. La contraseña generada automáticamente por el instalador no se incorpora en la memoria ni en las capturas por criterio de seguridad.

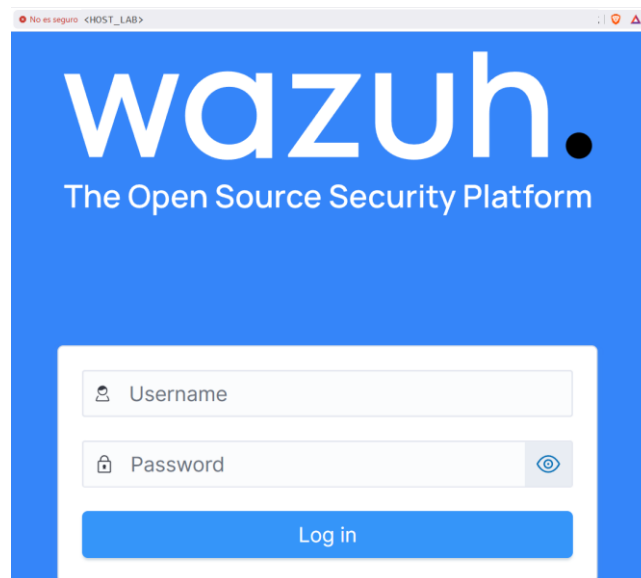


Figura 82: Panel de acceso al dashboard de Wazuh

Antes de instalar el agente se validó la conectividad desde Windows hacia el servidor Wazuh. Se comprobaron los puertos 443, 1514 y 1515, asociados respectivamente al acceso web, la comunicación del agente y el enrolamiento/registro.

```
$ Test-NetConnection <HOST_LAB> -Port 443  
$ Test-NetConnection <HOST_LAB> -Port 1514  
$ Test-NetConnection <HOST_LAB> -Port 1515
```

Código 37: Validación de conectividad desde Windows hacia Wazuh

El agente se instaló mediante el paquete MSI oficial de Wazuh correspondiente a la versión 4.14.5. En la instalación se indicó la IP del manager y posteriormente se inició el servicio WazuhSvc.

```

$ $WAZUH_MANAGER="<HOST_LAB>"

$ Invoke-WebRequest

  -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.5-1.msi -
OutFile "$env:TEMP\wazuh-agent.msi"

$ Start-Process msixec.exe -Wait -ArgumentList "/I "$env:TEMP\wazuh-
agent.msi" /qn WAZUH_MANAGER="$WAZUH_MANAGER`""

$ Start-Service WazuhSvc

$ Get-Service WazuhSvc

```

Código 38: Instalación del agente Wazuh en Windows

```

PS <HOST_LAB> -Port 443
>> <HOST_LAB> -Port 1514
>> <HOST_LAB> -Port 1515

ComputerName      : <HOST_LAB>
RemoteAddress     : <HOST_LAB>
RemotePort        : 443
InterfaceAlias    : Ethernet 9
SourceAddress     : <HOST_LAB>
TcpTestSucceeded : True

ComputerName      : <HOST_LAB>
RemoteAddress     : <HOST_LAB>
RemotePort        : 1514
InterfaceAlias    : Ethernet 9
SourceAddress     : <HOST_LAB>
TcpTestSucceeded : True

ComputerName      : <HOST_LAB>
RemoteAddress     : <HOST_LAB>
RemotePort        : 1515
InterfaceAlias    : Ethernet 9
SourceAddress     : <HOST_LAB>
TcpTestSucceeded : True

PS C:\Windows\system32> $WAZUH_MANAGER="<HOST_LAB>"
>>
>> Invoke-WebRequest `
>>   -Uri "https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.5-1.msi" `
>>   -OutFile "$env:TEMP\wazuh-agent.msi"
>>
>> msixec.exe /i "$env:TEMP\wazuh-agent.msi" /q WAZUH_MANAGER="$WAZUH_MANAGER"
>>
>> Start-Service wazuhsvc

```

Figura 83: Validación de conectividad e instalación del agente Wazuh desde PowerShell

```

PS C:\>
<RUTA_EVIDENCIAS>
Status  Name      DisplayName
-----
Running WazuhSvc  Wazuh

```

Figura 84: Comprobación del servicio WazuhSvc en ejecución en el endpoint Windows

Anexo B: Repositorio de GitHub

En la referencia [38] se encuentra el repositorio de GitHub asociado al presente Trabajo Fin de Máster. Su objetivo es conservar de forma estructurada los principales elementos utilizados durante el desarrollo experimental y facilitar la reproducibilidad, trazabilidad y validación de los resultados obtenidos. El repositorio se encuentra disponible en https://github.com/Juliog-a/VR_HIDS-EDR

B.1. Contenido del repositorio

El repositorio se organiza por bloques funcionales para separar los componentes de detección, scripts, automatización, evidencias, documentación y resultados. Esta estructura permite relacionar cada elemento utilizado durante las pruebas con las evidencias y resultados obtenidos.

La Tabla 62 describe las carpetas principales del repositorio.

Ruta	Contenido y finalidad
01_ARTIFACTS/	Artifacts de Velociraptor utilizados durante el proyecto. Incluye los artifacts personalizados validados, los perfiles P1-P4 y los componentes de procesamiento y enrutamiento empleados en el flujo de alertas.
02_SCRIPTS/	Scripts desarrollados para reproducir de forma controlada las técnicas evaluadas y tareas auxiliares de preparación, restauración y validación del laboratorio.
03_RUNNERS/	Runners utilizados para automatizar las campañas experimentales. Incluye el runner global TEC-001 a TEC-009 y el runner destinado a las pruebas de falsos positivos.
04_EVIDENCE/	Evidencias y resultados derivados de las campañas experimentales. Incluye resultados estructurados, matrices de validación, ficheros de análisis y evidencias utilizadas para elaborar los resultados de la memoria.
06_CONTROLLED_RERUN/	Material utilizado para repetir de forma controlada determinadas campañas y verificar la reproducibilidad de los resultados obtenidos.
07_DOCS/	Documentación técnica y de trazabilidad del proyecto, incluyendo índices de artifacts, matrices de pruebas, relación de evidencias y documentación auxiliar.

08_MEMORIA/	Documentación asociada a la elaboración del TFM, incluyendo la memoria, planificación y ficheros de análisis empleados para generar tablas y gráficas.
10_WAZUH/	Elementos específicos de la validación comparativa con Wazuh: configuración, reglas, evidencias y resultados de las campañas base y personalizada.
README.md	Documento principal de orientación del repositorio. Describe su finalidad, estructura y los componentes necesarios para reproducir el trabajo.
.gitignore	Define los archivos y elementos locales que no deben incorporarse al repositorio, especialmente resultados temporales o información sensible.

Tabla 63: Repositorio de GitHub

Esta separación permite distinguir claramente entre código y configuración (01_ARTIFACTS, 02_SCRIPTS y 03_RUNNERS), evidencias experimentales (04_EVIDENCE y 06_CONTROLLED_RERUN), documentación y entregables (07_DOCS y 08_MEMORIA) y la validación comparativa con Wazuh (10_WAZUH). Las versiones consideradas definitivas se encuentran separadas de los elementos históricos o de desarrollo para evitar utilizar accidentalmente componentes no validados.

Los componentes definitivos se localizan en los subdirectorios de versiones validadas indicados en los índices del repositorio. Los logs locales de 05_LOGS no forman parte del repositorio público; la evidencia distribuida se identifica por su ruta publicada y por la revisión de entrega.

B.2. Control de versiones e integridad

Durante el desarrollo se utilizaron diferentes versiones de scripts y artifacts. Las versiones consideradas válidas para los resultados finales se conservaron de forma separada de las versiones de desarrollo. Cuando resultó necesario, se utilizaron además valores SHA-256 para comprobar la integridad de determinados artifacts y archivos empleados durante la validación. Por motivos de seguridad y privacidad, el repositorio público no incluye información que pueda comprometer el entorno utilizado durante las pruebas. Por seguridad no se publican contraseñas, credenciales de acceso, tokens, webhooks activos, claves privadas, información sensible del entorno, máquinas virtuales completas, configuraciones que contengan secretos reutilizables.

Anexo C. Técnicas MITRE seleccionadas

Las técnicas incluidas en el entorno experimental fueron seleccionadas atendiendo principalmente a su observabilidad en sistemas Windows, posibilidad de ejecución controlada, reproducibilidad mediante scripts o runners y disponibilidad de telemetría host-based suficiente para evaluar su detección. El conjunto seleccionado no pretende proporcionar una cobertura completa de la matriz MITRE ATT&CK Enterprise, sino establecer un subconjunto heterogéneo y reproducible de técnicas pertenecientes a diferentes tácticas. La matriz Enterprise utilizada como referencia se encuentra disponible en [2].

Las Tablas 64 y 65 resumen las técnicas evaluadas, su ejecución, la evidencia observada y la conclusión técnica.

C.1. TEC-001 - T1059.001 PowerShell

La sub-técnica T1059.001 PowerShell, perteneciente a la táctica Execution, representa el uso de PowerShell como intérprete de comandos y scripts. Durante las pruebas se ejecutaron comandos PowerShell de forma controlada con el objetivo de generar actividad identificable en el endpoint. La evidencia esperada incluye la ejecución de powershell.exe, la línea de comandos asociada, eventos de creación de procesos y, cuando se encuentra habilitado PowerShell Script Block Logging, eventos 4104.

Esta técnica permite evaluar la capacidad de Velociraptor para identificar actividad basada en PowerShell a partir de la telemetría generada por Windows y Sysmon.

MITRE ATT&CK: [20] <https://attack.mitre.org/techniques/T1059/001/>

C.2. TEC-002 - T1059.003 Windows Command Shell

La sub-técnica T1059.003 Windows Command Shell, perteneciente a la táctica Execution, representa la utilización del intérprete de comandos de Windows, principalmente cmd.exe. Durante el laboratorio se realizaron ejecuciones controladas mediante cmd.exe y comandos encadenados, incluyendo la generación de procesos secundarios.

La evidencia esperada incluye eventos de creación de proceso, principalmente Sysmon Event ID 1, la línea de comandos ejecutada y la relación entre procesos padre e hijo.

MITRE ATT&CK: [21] <https://attack.mitre.org/techniques/T1059/003/>

C.3. TEC-003 - T1053.005 Scheduled Task

La sub-técnica T1053.005 Scheduled Task se encuentra asociada a las tácticas Execution, Persistence y Privilege Escalation. En el laboratorio se realizó la creación, consulta, ejecución y posterior eliminación de una tarea programada controlada utilizando mecanismos nativos de Windows.

La evidencia esperada incluye la ejecución de `schtasks.exe`, eventos asociados a la creación y ejecución de tareas programadas y eventos de creación de procesos registrados durante la prueba.

MITRE ATT&CK: [22] <https://attack.mitre.org/techniques/T1053/005/>

C.4. TEC-004 - T1547.001 Registry Run Keys / Startup Folder

La sub-técnica T1547.001 Registry Run Keys / Startup Folder está asociada principalmente a las tácticas Persistence y Privilege Escalation. La prueba experimental consistió en crear de forma controlada una entrada dentro de las claves Run de Windows, consultar posteriormente su contenido y eliminar finalmente la entrada generada.

La evidencia esperada incluye la ejecución de `reg.exe` y eventos relacionados con modificaciones del Registro, especialmente cuando Sysmon tiene habilitada la monitorización correspondiente.

MITRE ATT&CK: [23] <https://attack.mitre.org/techniques/T1547/001/>

C.5. TEC-005 - T1569.002 Service Execution

La sub-técnica T1569.002 Service Execution, perteneciente a la táctica Execution, representa la utilización de servicios de Windows como mecanismo de ejecución. Durante la prueba se creó un servicio controlado, se consultó su configuración, se realizó un intento de ejecución y posteriormente se eliminó.

La evidencia esperada incluye la ejecución de `sc.exe`, eventos de creación de proceso y eventos asociados a la creación de servicios, como System Event ID 7045 cuando corresponde.

MITRE ATT&CK: [24] <https://attack.mitre.org/techniques/T1569/002/>

C.6. TEC-006 - T1518.001 Security Software Discovery

La sub-técnica T1518.001 Security Software Discovery, perteneciente a la táctica Discovery, representa la identificación de productos y servicios de seguridad presentes en un sistema. Durante la prueba se realizaron consultas controladas destinadas a identificar soluciones antivirus y servicios de seguridad instalados en Windows mediante PowerShell y herramientas nativas.

La evidencia esperada incluye consultas a SecurityCenter2, enumeración de servicios de seguridad, eventos de creación de proceso y eventos PowerShell 4104.

MITRE ATT&CK: [25] <https://attack.mitre.org/techniques/T1518/001/>

C.7. TEC-007 - T1486 Data Encrypted for Impact

La técnica T1486 Data Encrypted for Impact, perteneciente a la táctica Impact, representa el cifrado de datos con el objetivo de impedir o dificultar su disponibilidad. La prueba se realizó

exclusivamente sobre un conjunto de archivos dummy creado específicamente para el laboratorio. El script empleado realiza un cifrado controlado mediante AES y genera archivos con extensión .aes.

La evidencia esperada incluye ejecución de PowerShell, utilización de componentes criptográficos, operaciones sobre archivos y generación de los ficheros cifrados.

MITRE ATT&CK: [26] <https://attack.mitre.org/techniques/T1486/>

C.8. TEC-008 - T1485 Data Destruction

La técnica T1485 Data Destruction, perteneciente a la táctica Impact, representa la destrucción deliberada de datos para afectar a su disponibilidad. La prueba se realizó exclusivamente sobre archivos dummy del entorno de laboratorio. El script ejecuta una eliminación controlada de los archivos existentes y permite comparar el número de ficheros antes y después de la operación.

La evidencia esperada incluye ejecución de PowerShell, comandos de eliminación, eventos 4104 y, cuando se encuentra habilitada la telemetría correspondiente, eventos de eliminación de archivos como Sysmon Event ID 26.

MITRE ATT&CK: [27] <https://attack.mitre.org/techniques/T1485/>

C.9. TEC-009 - T1074.001, T1560.001 y T1048.003

TEC-009 combina tres comportamientos consecutivos relacionados con la preparación, archivado y transferencia de información. T1074.001 Local Data Staging, perteneciente a la táctica Collection, representa la preparación y agrupación local de información antes de su transferencia [28].

- <https://attack.mitre.org/techniques/T1074/001/>

T1560.001 Archive via Utility, también asociada a Collection, representa el archivado de los datos mediante utilidades capaces de generar archivos comprimidos [29].

- <https://attack.mitre.org/techniques/T1560/001/>

Finalmente, T1048.003 Exfiltration Over Unencrypted Non-C2 Protocol, perteneciente a la táctica Exfiltration, representa la transferencia de datos mediante un protocolo no cifrado diferente de un canal de mando y control [30].

- <https://attack.mitre.org/techniques/T1048/003/>

C.10. Resultado global de las técnicas seleccionadas

Las nueve pruebas, identificadas como TEC-001 a TEC-009, fueron ejecutadas de forma controlada dentro del entorno experimental. Las técnicas seleccionadas abarcan diferentes fases de la matriz MITRE ATT&CK Enterprise, incluyendo Execution, Persistence, Privilege Escalation,

Discovery, Collection, Exfiltration e Impact, permitiendo evaluar la capacidad de Velociraptor frente a comportamientos heterogéneos observables desde el endpoint.

La matriz MITRE ATT&CK Enterprise completa utilizada como referencia puede consultarse en [2]: <https://attack.mitre.org/matrices/enterprise/>

La correspondencia entre las técnicas ejecutadas, la telemetría obtenida y los resultados de detección fue registrada en el fichero Excel de análisis de visibilidad empleado durante el trabajo.

TEC	MITRE	Técnica	Táctica	Procedimiento	Evidencia esperada
TEC-001	T1059.001	PowerShell	Execution	PowerShell con -NoProfile / -ExecutionPolicy Bypass	Sysmon ID 1; ProcessCreation; 4104 si aplica
TEC-002	T1059.003	Windows Command Shell	Execution	cmd.exe /c con comandos dual-use	Sysmon ID 1; ProcessCreation
TEC-003	T1053.005	Scheduled Task	Execution	schtasks /create /run /delete	ProcessCreation; TaskScheduler; logs de tarea
TEC-004	T1547.001	Registry Run Keys / Startup Folder	Persistence	reg add/delete sobre Run/RunOnce	Sysmon 12/13/14; StartupItems; Autoruns
TEC-005	T1569.002	Service Execution	Execution	sc create/start/delete servicio temporal	Event ID 7045; ServiceCreation; ProcessCreation
TEC-006	T1518.001	Security Software Discovery	Discovery	Get-CimInstance root\SecurityCenter2 / AntivirusProduct	ProcessCreation; 4104; CommandLine
TEC-007	T1486	Data Encrypted for Impact	Impact	cifrado controlado de ficheros	4104 crypto; Sysmon ID 11; extensiones .aes
TEC-008	T1485	Data Destruction	Impact	sabotaje.ps1: borrado masivo controlado dentro de DUMB_LAB	PowerShell 4104: Get-ChildItem + Remove-Item + ForEach-Object; Sysmon ID 26 como evidencia forense
TEC-009	T1074.001/ T1560.001/ T1048.003	Local Data Staging / Archive Collected Data / Exfiltration Over Unencrypted Non-C2 Protocol	Collection & Exfiltration	exfiltracion_v3.ps1: staging + Compress-Archive + upload HTTP	PowerShell 4104; creación de ZIP mediante Sysmon ID 11; Sysmon ID 3/TrackNetworkConnections como contexto de red; ground truth del runner HTTP

Tabla 64: Técnicas MITRE evaluadas, procedimientos y telemetría esperada (I)

Evidencia host observada	Cobertura pública	VR custom	Conclusión técnica
ProcessCreation visible + Hayabusa CH: 2 high	Detección CH específica	DETECTADA	Visible por proceso y eventos PowerShell 4104; detección básica, requiere contexto sobre parámetros y script blocks.
ProcessCreation/Sysmon visibles; 0 high/critical CH	N/A	DETECTADA	Visible por ejecución de cmd.exe y línea de comandos; detección básica, con riesgo de confusión con administración legítima.
ProcessCreation visible + Hayabusa CH: 1 high	Detección CH específica	DETECTADA	Visible por schtasks.exe y ciclo de creación, ejecución y eliminación; requiere diferenciar tarea temporal benigna de persistencia real.
Sysmon 12/13 visible; reglas medium fuera de CH	N/A	DETECTADA	Visible parcialmente por actividad de registro; requiere correlación entre clave Run, valor configurado y proceso asociado.
ServiceCreation 7045 + Hayabusa: 4 high (3 semánticos)	Detección CH específica	DETECTADA	Evidencia fuerte por Windows Event ID 7045; detección sólida al combinar ServiceName, ImagePath y contexto temporal.
WMIC/proceso visible; 0 high/critical CH	N/A	DETECTADA	Visible por PowerShell/WMI/WMIC y eventos 4104; requiere contexto para evitar clasificar consultas administrativas como maliciosas.
Proceso/ID11 y reglas genéricas; sin regla CH específica de cifrado	N/A	DETECTADA	Visible por PowerShell 4104, creación de archivos .aes y escritura en disco; emulación controlada, no ransomware real.
Proceso visible; sin ID26 y sin regla CH específica de destrucción	N/A	DETECTADA	Visible por PowerShell 4104; Sysmon Event ID 26 no fue observado en la campaña. Requiere agregación para evitar ruido por borrado masivo de archivos.
TrackNetwork: 3 filas destino sin PID; runner HTTP 200/UploadSucceeded=True	N/A	DETECTADA	Staging y archivado. TrackNetworkConnections aportó visibilidad de red sin atribución fiable de proceso.

Tabla 65: Evidencia observada y conclusión por técnica MITRE evaluada (II)

Anexo D. Artifacts Velociraptor validados

Durante el desarrollo del trabajo se diseñaron y ajustaron distintos artifacts personalizados de Velociraptor con el objetivo de transformar la telemetría disponible en el endpoint en eventos estructurados y utilizables para la detección de las técnicas MITRE ATT&CK seleccionadas.

Los artifacts fueron desarrollados mediante VQL (Velociraptor Query Language) y desplegados principalmente como artifacts de tipo CLIENT_EVENT, permitiendo ejecutar consultas de monitorización continua sobre los endpoints. Velociraptor permite definir este tipo de artifacts y utilizar colas de eventos para procesar posteriormente la información generada [3]-[5].

La arquitectura final se organizó en cuatro perfiles de prioridad, denominados P1, P2, P3 y P4, complementados por un artifact SERVER_EVENT encargado del procesamiento y enrutamiento posterior de los eventos.

Las Tablas 66 a 69 describen la definición, los resultados y las fuentes de evidencia de los artifacts personalizados y públicos.

D.1. Control de versiones

Durante el desarrollo se generaron distintas versiones de trabajo de los artifacts. Una vez que una versión superaba las pruebas experimentales definidas, se conservaba como versión validada y no se modificaba posteriormente. Cualquier cambio sobre la lógica de detección se realizaba sobre una nueva versión, que debía volver a superar el proceso de validación antes de sustituir a la anterior.

Este procedimiento permite mantener la correspondencia entre la versión del artifact, la campaña ejecutada, los eventos obtenidos y los resultados registrados. Además, para determinados artifacts recalibrados se generaron valores SHA-256, utilizados como mecanismo adicional de control de integridad.

D.2. P1 - Critical Priority

El artifact Custom.TFM.HIDS.P1.Critical.Priority.Event_v1.yaml corresponde al perfil de mayor prioridad definido dentro de la arquitectura.

Su finalidad es representar eventos cuya combinación de comportamiento y contexto presenta una relevancia elevada dentro del entorno experimental. Este perfil se orientó principalmente hacia comportamientos relacionados con acciones de impacto o actividades que, dentro de las condiciones del laboratorio, requieren una atención prioritaria.

Entre los escenarios evaluados se encuentran comportamientos asociados al cifrado de información, destrucción de datos y determinadas fases de exfiltración controlada.

D.3. P2 – High / Forensic

El artifact Custom.TFM.HIDS.P2.High.Forensic.Event_v1.yaml corresponde al perfil High / Forensic.

Su objetivo es recoger evidencias con una elevada utilidad forense que permitan reforzar o contextualizar las detecciones realizadas por otros perfiles. P2 prioriza la disponibilidad de información útil para reconstruir posteriormente una actividad, incluyendo atributos como procesos ejecutados, líneas de comandos, eventos de Windows y otros elementos disponibles en la telemetría utilizada durante las pruebas. Por tanto, su finalidad no es únicamente generar una alerta, sino proporcionar suficiente información para mantener la trazabilidad técnica de la actividad detectada.

D.4. P3 – Medium / Behavioral

El artifact Custom.TFM.HIDS.P3.Medium.Behavioral.Event_v1.yaml corresponde al perfil Medium / Behavioral.

Este perfil está orientado a comportamientos potencialmente sospechosos que, considerados de forma aislada, no necesariamente representan una actividad maliciosa. Su utilidad reside en aportar contexto adicional y permitir relacionar varias señales dentro de una misma ventana temporal o ejecución experimental. Por este motivo, los eventos generados por P3 deben interpretarse junto con el resto de la telemetría disponible y no como evidencia suficiente de compromiso por sí solos.

D.5. P4 – Low / Basic

El artifact Custom.TFM.HIDS.P4.Low.Basic_v2.yaml corresponde al nivel de menor prioridad de la arquitectura desarrollada.

P4 tiene como finalidad proporcionar visibilidad básica sobre la actividad del endpoint, registrando comportamientos que pueden resultar relevantes para el análisis pero que presentan una menor especificidad. Las señales generadas por este perfil no deben considerarse alertas de seguridad ni evidencias suficientes de actividad maliciosa.

Su función principal dentro del trabajo es complementar los perfiles superiores y permitir evaluar hasta qué nivel de visibilidad puede llegar Velociraptor utilizando la telemetría disponible.

D.6. Router SERVER_EVENT

Para centralizar el tratamiento de los eventos generados por los perfiles anteriores se desarrolló el artifact Custom.TFM.HIDS.Router.JSONL.Discord.SOC_v3.yaml.

A diferencia de P1-P4, este componente funciona como un artifact de tipo SERVER_EVENT. Su función principal es consumir los eventos generados por los CLIENT_EVENT y realizar tareas posteriores de procesamiento, entre ellas:

- normalización de los eventos;
- persistencia en formato estructurado;
- clasificación de la información;
- preparación de la salida para su análisis;
- generación de notificaciones externas cuando corresponde.

El router no constituye un mecanismo de detección independiente. La detección se produce previamente en los artifacts ejecutados sobre el cliente; el router únicamente procesa la salida generada por estos.

Este diseño sigue el modelo de automatización y monitorización de eventos proporcionado por Velociraptor [5], [6].

D.7. Validación experimental de los artifacts

Los artifacts fueron sometidos a ejecuciones controladas de las técnicas TEC-001 a TEC-009 descritas en el Anexo C.

Para cada técnica se compararon tres elementos principales: ground truth de ejecución, telemetría disponible y eventos generados por Velociraptor.

La existencia de una ejecución correcta de una técnica no se consideró por sí sola una detección. Para considerar una técnica visible o detectada fue necesario identificar evidencia generada por los artifacts de Velociraptor dentro de la ventana temporal correspondiente.

Los resultados fueron registrados en el fichero Excel de análisis de visibilidad utilizado durante el trabajo.

D.7.1. Resultados de los artifacts personalizados

Tras la ejecución de las campañas experimentales, los artifacts personalizados permitieron obtener evidencia para las técnicas evaluadas dentro de las condiciones definidas en el laboratorio.

Los resultados mostrados deben interpretarse únicamente respecto a las pruebas concretas ejecutadas y no como una afirmación de cobertura general frente a cualquier implementación posible de cada técnica MITRE ATT&CK.

Artifact	Perfil	Tipo	Sources internas	Técnicas declaradas
Custom.TFM.HIDS.P1.Critical.Priority.Event_v1	P1	CLIENT_EVENT	P1_EVENT_PowerShell4104_Basic; P1_EVENT_Sysmon_ID1_Process_Basic; P1_EVENT_System7045_ServiceCreation; P1_EVENT_TEC009_Sysmon_Context	TEC-004/005/007/008/009
Custom.TFM.HIDS.P2.High.Forensic.Event_v1	P2	CLIENT_EVENT	5 declaradas; 4 con alertas. Incluye PowerShell, Process, 7045, Registry/File/Network y ScheduledTask opcional	TEC-001 a TEC-009 declaradas
Custom.TFM.HIDS.P3.Medium.Behavioral.Event_v1	P3	CLIENT_EVENT	13 declaradas; 12 con alertas. Sources específicas TEC001-TEC009	TEC-001 a TEC-009
Custom.TFM.HIDS.P4.Low.Basic_v2	P4	CLIENT_EVENT	5: CU001 PowerShell; CU002 CMD; CU003 ScheduledTask; CU004 RunKeys; CU005 ServiceCreation	TEC-001/002/003/004/005

Tabla 66: Definición de artifacts personalizados: perfil, tipo, sources y técnicas declaradas (I)

Técnicas detectadas	Alertas	FP hits	Función metodológica	Evidencia principal	Limitaciones
5/9	19	0	Correlación/prioridad crítica	normalized/alertas_vr.csv; ALERTAS_VR	Señales de alta prioridad; el volumen no equivale a cobertura.
7/9	118	0	Evidencia forense y contexto	normalized/alertas_vr.csv; ALERTAS_VR	Source Security ScheduledTask Optional sin filas en la ventana final.
9/9	109	0	Detección de comportamiento	normalized/alertas_vr.csv; ALERTAS_VR	TEC009_ZIP_FileCreate_Sysmon_Context no generó filas en ventana; otras sources cubren TEC-009.
5/9	133	0	Señales básicas de baja complejidad	normalized/alertas_vr.csv; ALERTAS_VR	Detección CH específica

Tabla 67: Resultados y función metodológica de los artifacts personalizados (II)

D.7.2. Resultados de los artifacts públicos

Los artifacts públicos fueron evaluados utilizando el mismo conjunto de técnicas y evidencias de referencia, permitiendo comparar su capacidad de visibilidad con la obtenida mediante los artifacts personalizados desarrollados durante el trabajo.

Artifact	Tipo	Detección	Alerta RT	Discord	Salida externa HTTP	Técnicas principales	Veredicto
Windows.Hayabusa.Monitoring	CLIENT_EVENT	Sí (Sigma CH)	Sí	No	No	TEC-001/003/005 en CH high	Positivo con FP; no extrapolar niveles inferiores
Windows.Events.ServiceCreation	CLIENT_EVENT	No: 7045 aporta visibilidad, no atribuye malicia	No	No	No	TEC-005	Creación de servicio observada
Windows.Events.ProcessCreation	CLIENT_EVENT	No por sí solo	No por sí solo	No	No	TEC-001 a TEC-009	Telemetría de procesos
Windows.Sysinternals.SysmonLogForward	CLIENT_EVENT	No por sí solo	No por sí solo	No	No	Contexto TEC-001 a TEC-009	Reenvío/forense
Windows.ETW.Monitoring	CLIENT_EVENT	No por sí solo	No	No	No	No atribuible	Telemetría básica
Generic.Events.TrackNetworkConnections	CLIENT_EVENT	No por sí solo	No por sí solo	No	No	TEC-009 contextual	Conexión visible; salida HTTP acreditada aparte

Tabla 68: Capacidad de detección, alerta y salida externa de los artifacts públicos (I)

Ventana FP UTC	Filas TEC	Filas FP	FP CH high/critical	Fuentes exactas
2026-07-12T13:41:00.378Z– 13:41:29.197Z	185	73	4	04_EVIDENCE\PRUEBAS_CON_ARTIFACTS_PUBLICOS\ 04_HayabusaMonitoring_CH\ Windows.Hayabusa.Monitoring_REAL.json ..._FP.json ventanas
2026-07-12T12:37:57.100Z– 12:38:39.571Z	1	0	0	04_EVIDENCE\PRUEBAS_CON_ARTIFACTS_PUBLICOS\ 02_ServiceCreation\ Windows.Events.ServiceCreation_REAL.json ..._FP.json summary TEC
2026-07-12T12:01:47.092Z– 12:02:15.729Z	89	18	0	04_EVIDENCE\PRUEBAS_CON_ARTIFACTS_PUBLICOS\ 01_ProcessCreation\ Windows.Events.ProcessCreation_REAL.json ..._FP.json ventanas
2026-07-12T12:55:41.452Z– 12:56:21.064Z	113	24	0	04_EVIDENCE\PRUEBAS_CON_ARTIFACTS_PUBLICOS\ 03_SysmonLogForward\ Windows.Sysinternals.SysmonLogForward_REAL.json ..._FP.json ventanas
2026-07-12T14:32:20.307Z– 14:32:57.519Z	143	16	0	04_EVIDENCE\PRUEBAS_CON_ARTIFACTS_PUBLICOS\ 05_ETWMonitoring\ Windows.ETW.Monitoring_REAL.json ..._FP.json ventanas
2026-07-12T14:52:39.396Z– 14:53:04.206Z	128	12	0	04_EVIDENCE\PRUEBAS_CON_ARTIFACTS_PUBLICOS\ 06_TrackNetworkConnections\ Generic.Events.TrackNetworkConnections_REAL.json summary TEC

Tabla 69: Resultados y fuentes de evidencia de las campañas con artifacts públicos (II)

Anexo E. Benchmark y rendimiento

Este anexo recoge las tablas y evidencias complementarias correspondientes al benchmark de rendimiento presentado y analizado en el capítulo correspondiente de la memoria. Los resultados completos se encuentran disponibles en el fichero TFM_BENCHMARK_RENDIMIENTO.xlsx. La campaña final consta de nueve ejecuciones válidas, distribuidas en tres escenarios y tres repeticiones por escenario.

Las Tablas 70 y 71 y las Figuras 85 a 90 presentan el resumen, los controles de calidad y los gráficos de CPU, memoria y duración del benchmark.

Escenario	Ejecuciones	AvgDuration	AvgCPUPercent	MaxCPUPercent	AvgRAMMB	MaxRAMMB	Warnings
BASELINE_NO_VR	3	182,69	0	0	0	0	N/A
VR_IDLE	3	181,57	0	0	57,48	57,92	N/A
VR_TEC_RUNNER	3	315,6	0,95	54	57,42	57,97	El runner sigue activo al finalizar la ventana de benchmark.

Tabla 70: Resumen de rendimiento por escenario de benchmark

KPI	Valor	Observaciones
runs totales	9	Esperado: 9
runs válidos	9	9/9
escenarios	3	BASELINE_NO_VR, VR_IDLE, VR_TEC_RUNNER
repeticiones por escenario	3/3	3 repeticiones válidas por escenario
CPU media BASELINE_NO_VR	0	%
CPU media VR_IDLE	0	%
CPU media VR_TEC_RUNNER	0,95	%
RAM media BASELINE_NO_VR	0	MB
RAM media VR_IDLE	57,48	MB
RAM media VR_TEC_RUNNER	57,42	MB
pico CPU cliente VR	54	%
pico RAM cliente VR	57,97	MB
duración media BASELINE_NO_VR	182,69	segundos
duración media VR_IDLE	181,57	segundos
duración media VR_TEC_RUNNER	315,6	segundos
estado global	APTO	Validación de calidad sin fallos críticos
Notas/Tags	RunnerStillRunningAtEnd=True en VR_TEC_RUNNER	WARN no bloqueante

Tabla 71: Indicadores globales y controles de calidad del benchmark

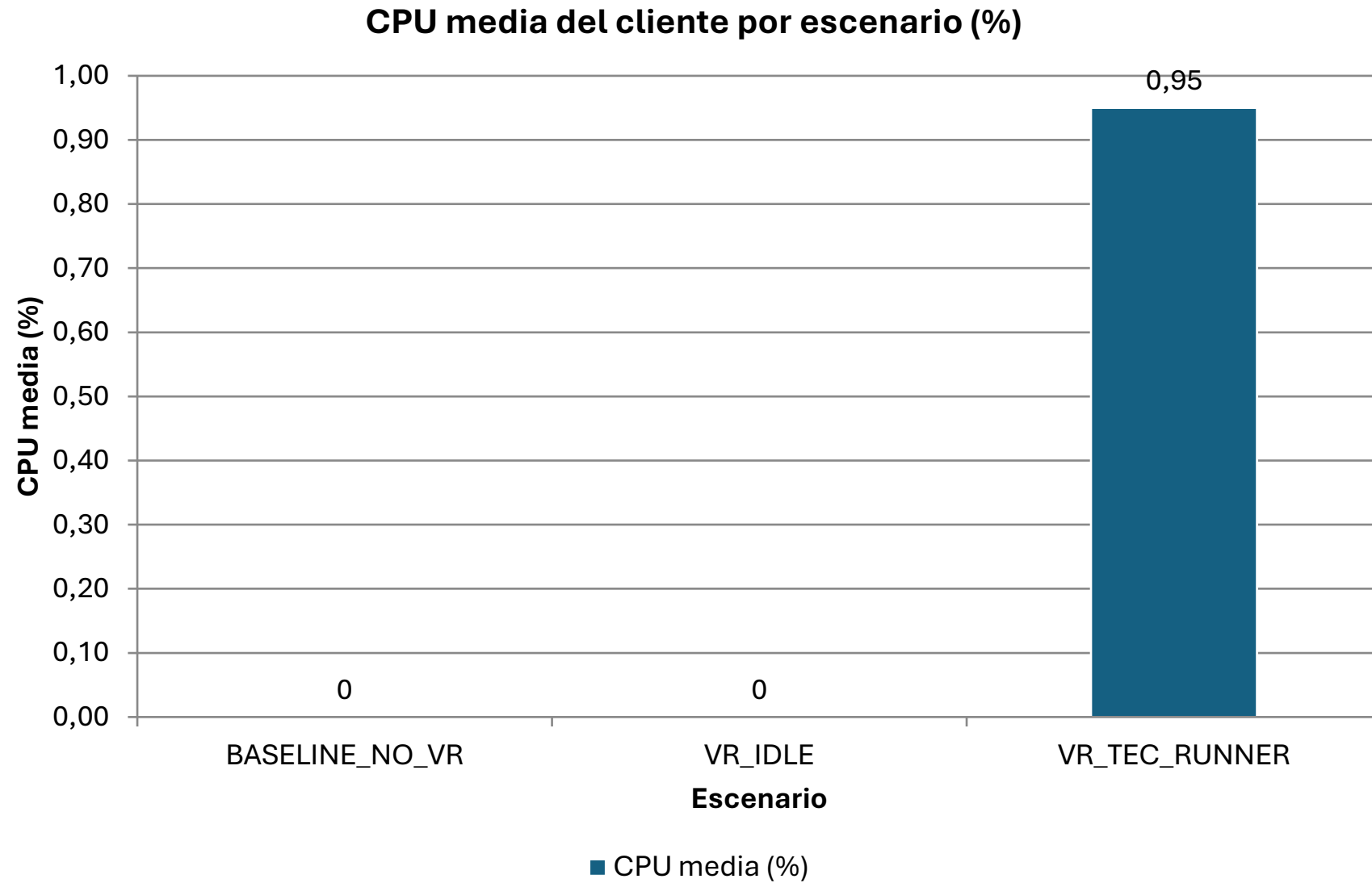


Figura 85: CPU media del cliente Velociraptor por escenario de benchmark

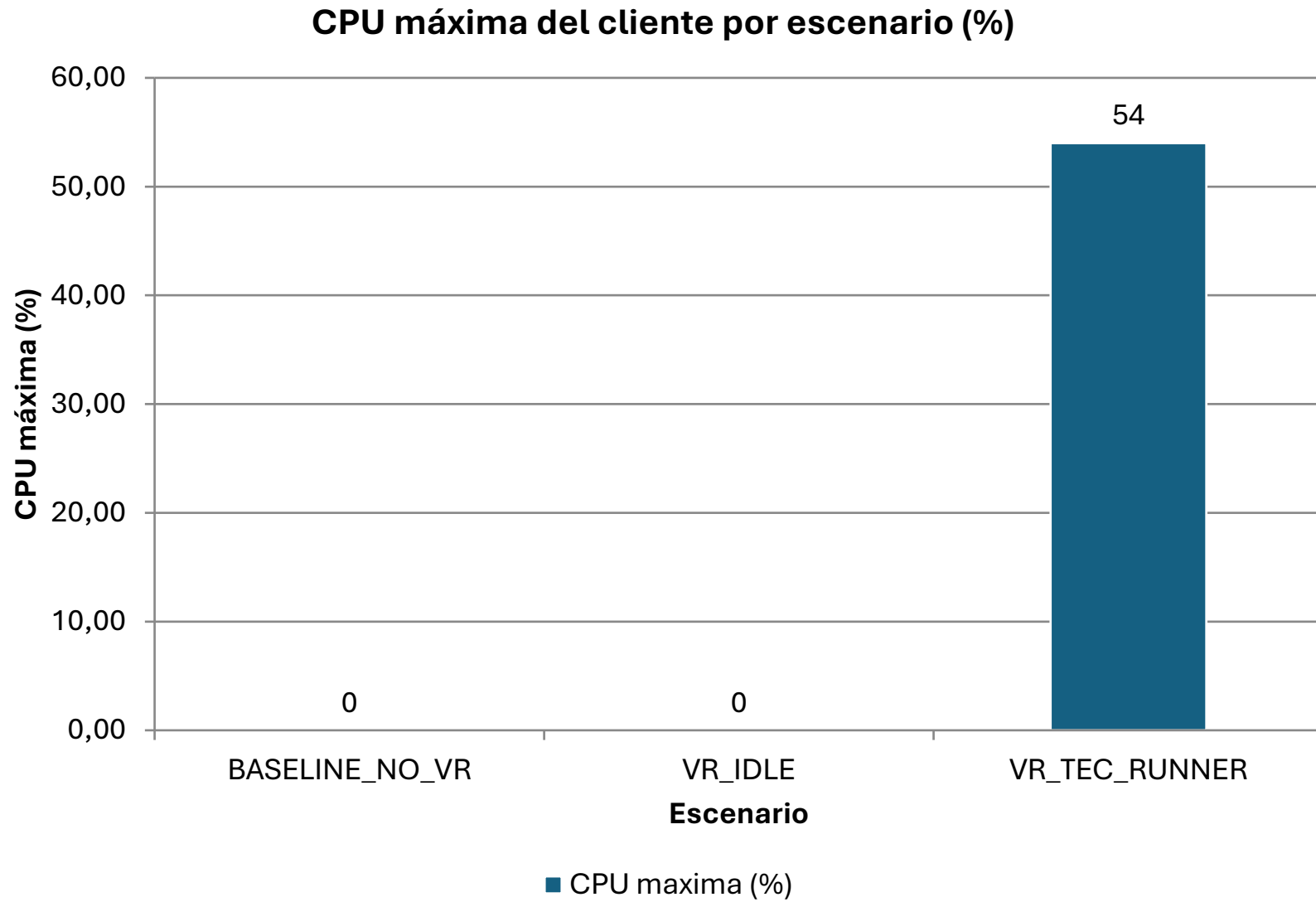


Figura 86: CPU máxima del cliente Velociraptor por escenario de benchmark

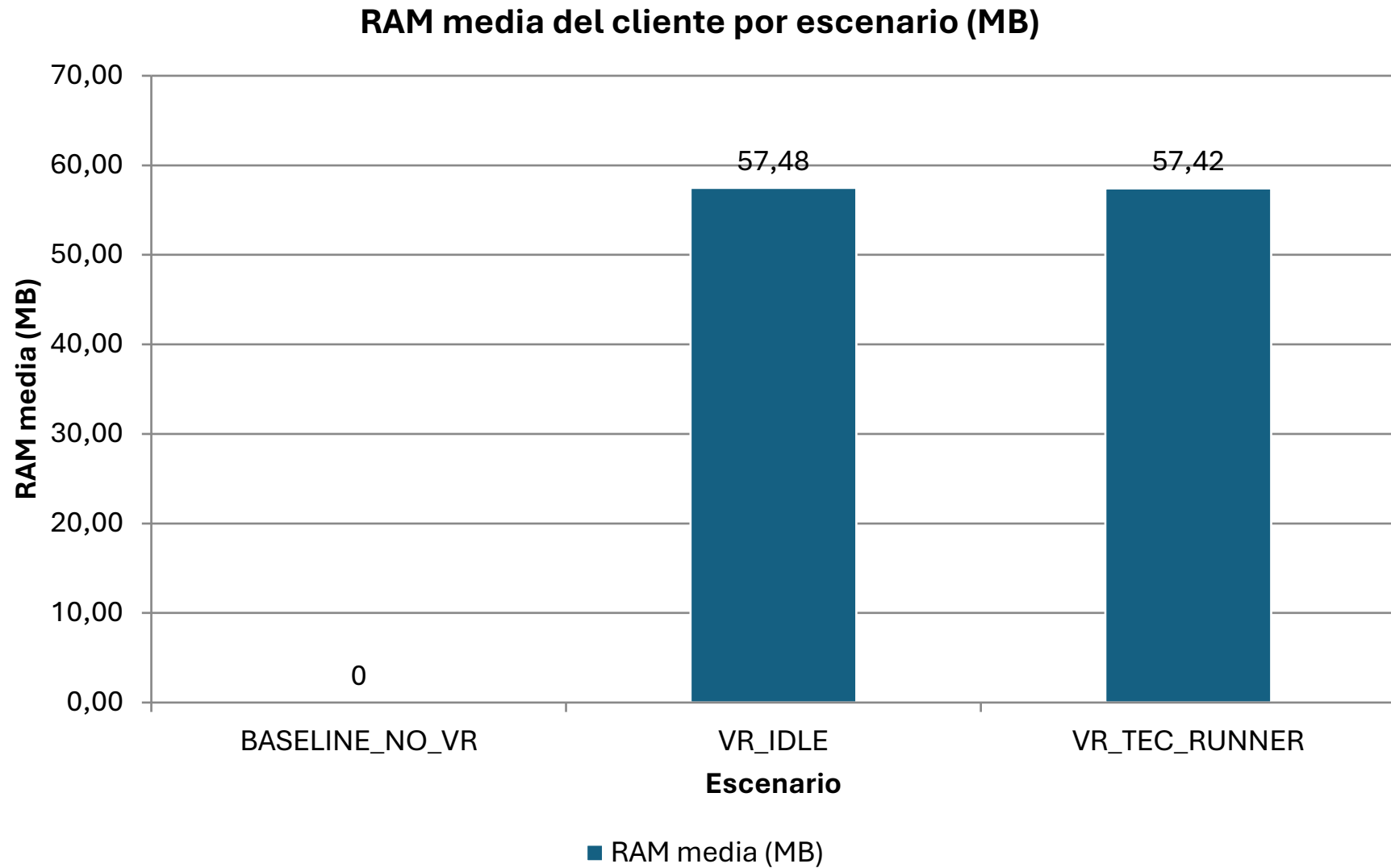


Figura 87: Memoria RAM media del cliente Velociraptor por escenario de benchmark

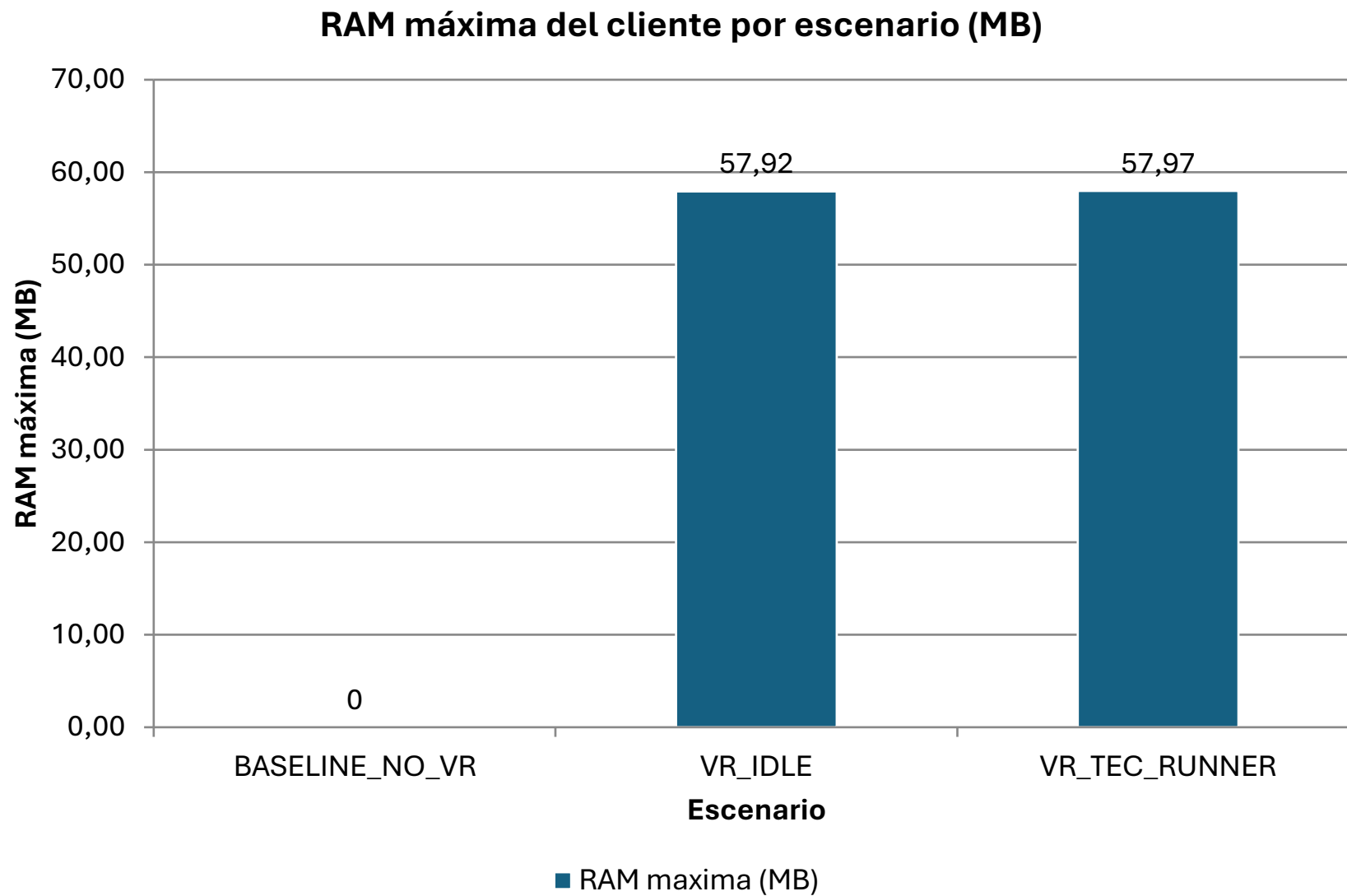


Figura 88: Memoria RAM máxima del cliente Velociraptor por escenario de benchmark

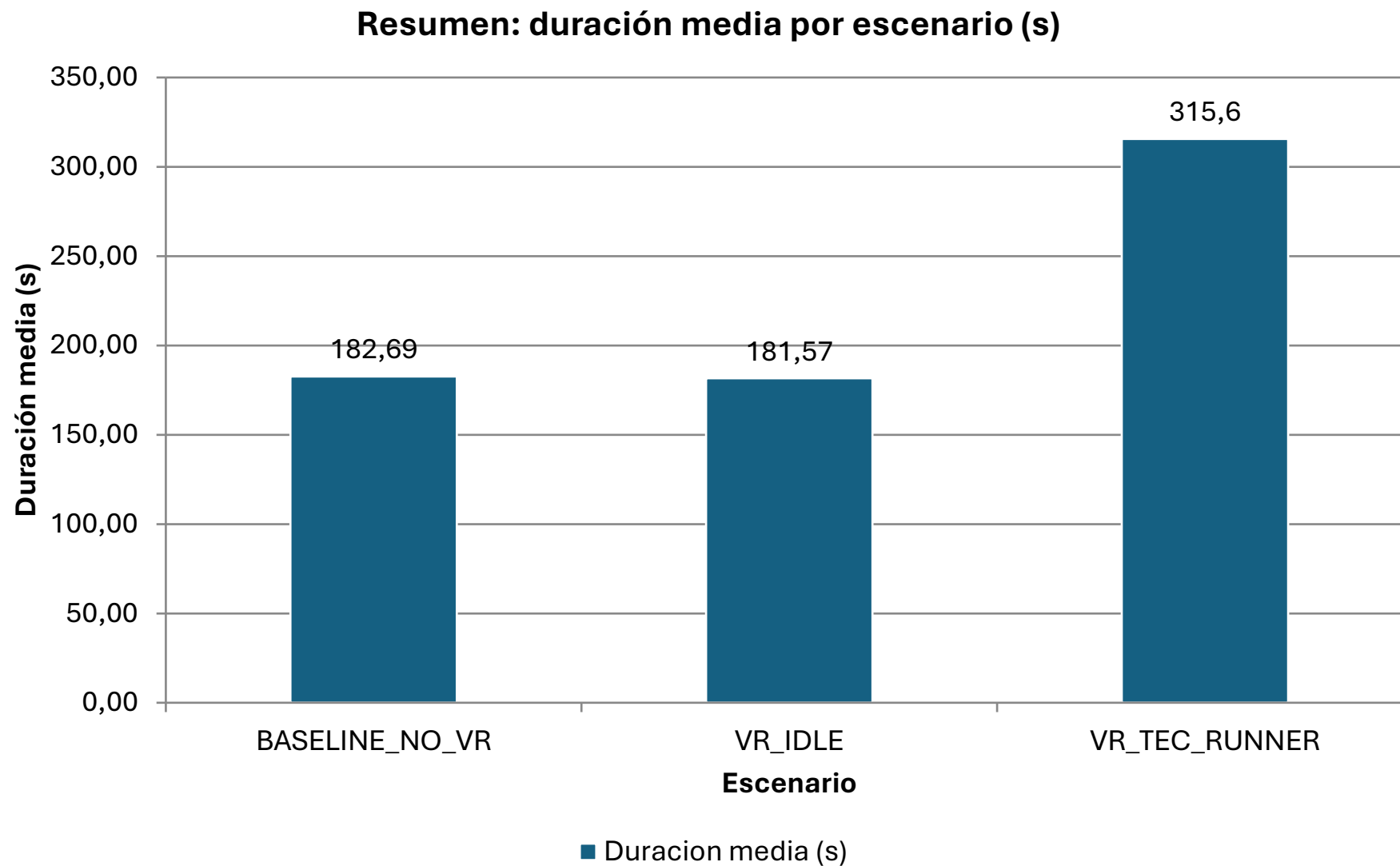


Figura 89: Duración media de las ejecuciones por escenario de benchmark

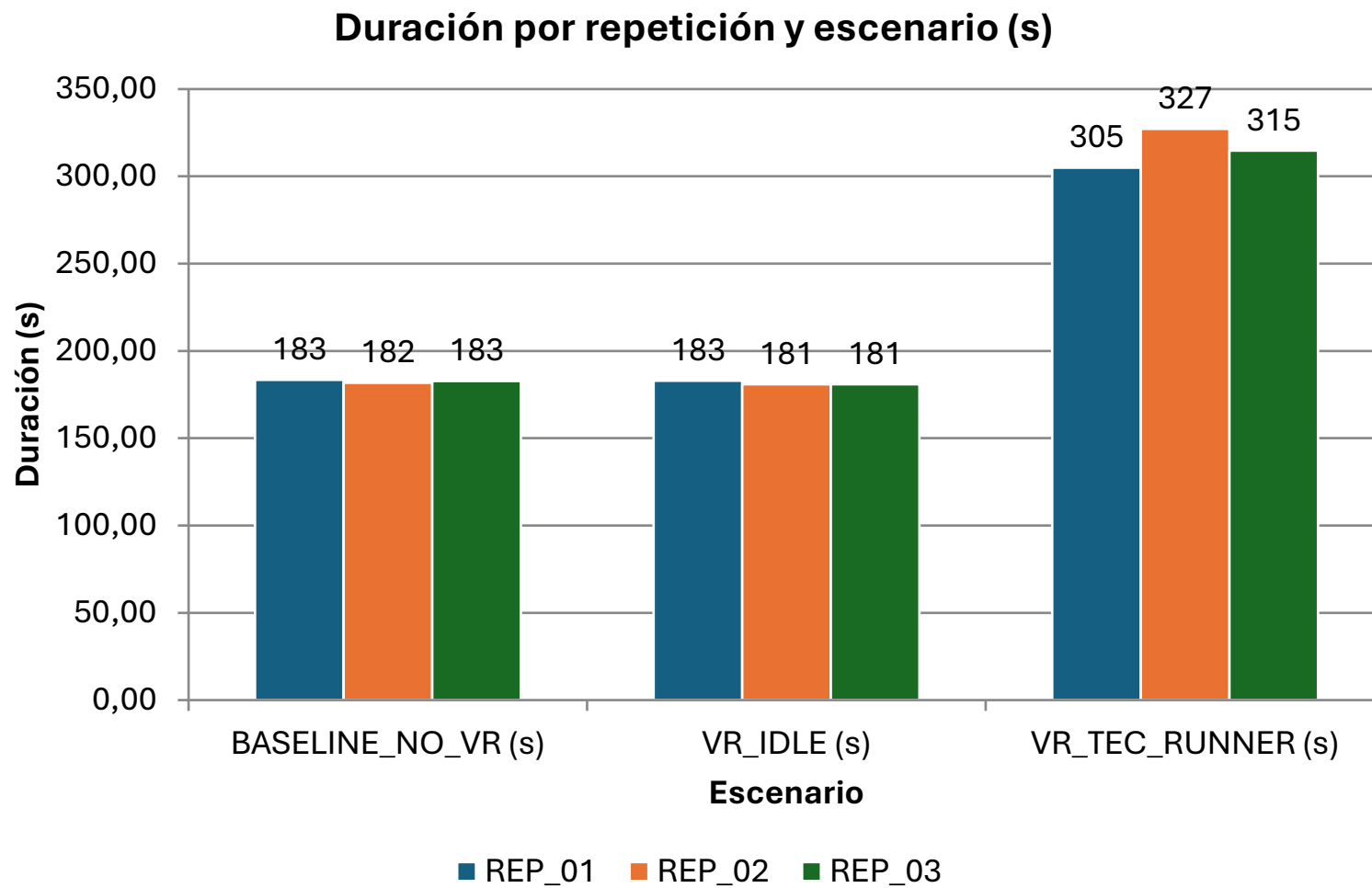


Figura 90: Duración de cada repetición por escenario de benchmark

Anexo F. Runners, scripts y ground truth

Este anexo describe los principales scripts desarrollados para automatizar y reproducir las pruebas experimentales. El código fuente completo no se incluye en la memoria, ya que se encuentra disponible en el repositorio asociado al proyecto [38]. En su lugar, se documenta la finalidad de cada componente, su relación con las técnicas evaluadas, su modo de utilización y las evidencias que genera.

Estos scripts tienen como objetivo generar de forma controlada el ground truth del experimento, es decir, proporcionar una referencia verificable de qué acción se ejecutó, cuándo se ejecutó y cuál fue su resultado. Por tanto, los runners y scripts no constituyen mecanismos de detección, sino herramientas para generar actividad conocida contra la que posteriormente se evalúan Velociraptor y Wazuh.

Anexo F.1. Runner de ejecución global de técnicas

Como se ha comentado varias veces, además de la ejecución individual de cada técnica, se desarrolló un runner global para automatizar la campaña completa. El runner utilizado fue TFM_Run_All_TEC_Tests_v6_PATCHED_4104.ps1, ya que se hicieron pruebas con múltiples versiones anteriores. Este script ejecuta de forma secuencial las técnicas definidas en el laboratorio, introduce pausas entre pruebas, registra tiempos de inicio y fin, valida evidencias generadas y almacena los resultados en ficheros de salida. Además, integra comprobaciones sobre eventos relevantes de Sysmon y PowerShell 4104, lo que permite relacionar cada técnica con la telemetría generada en el endpoint.

```
$ $env:TFM_BASEPATH = '<RUTA_ABSOLUTA_DEL_DATASET_DE_LABORATORIO>'
$ $parametrosRunner = @{
    ReceiverUrl = 'http://<HOST_LAB>:8088/upload'
    EnableExfilUpload = $true
    ExfilTimeoutSec = 60
    KeepExfilArtifacts = $true
    KeepArtifacts = $true}
$ & '<RUTA_ABSOLUTA_DEL_RUNNER_VALIDADO.ps1>' @parametrosRunner
```

Código 39: Ejecución del runner global de técnicas TEC-001 a TEC-009

Las Figuras 91 y 92 muestran la evidencia de finalización de la campaña global ejecutada mediante el runner.

```

CAMPAIGN FINISHED
Global result: OK
Full log: C:\Users\seguridad\Desktop\TFM\01_ACTIVE_TESTS\Pruebas\Logs_Pruebas_TFM\TFM_TEC_Run_CANDIDATE_v5_20260812_152954.log
Summary TXT: C:\Users\seguridad\Desktop\TFM\01_ACTIVE_TESTS\Pruebas\Logs_Pruebas_TFM\TFM_TEC_Run_CANDIDATE_v5_20260812_152954_summary.txt
Summary JSON: C:\Users\seguridad\Desktop\TFM\01_ACTIVE_TESTS\Pruebas\Logs_Pruebas_TFM\TFM_TEC_Run_CANDIDATE_v5_20260812_152954_summary.json
Summary CSV: C:\Users\seguridad\Desktop\TFM\01_ACTIVE_TESTS\Pruebas\Logs_Pruebas_TFM\TFM_TEC_Run_CANDIDATE_v5_20260812_152954_summary.csv
Techniques OK/WARN/FAIL: 9/0/0

```

Figura 91: Evidencia de finalización de la campaña global (1 de 2)

```

[2026-08-12 15:36:31] [INFO] STDOUT => OK. Dataset dummy creado en C:\Users\seguridad\Desktop\TFM\Pruebas\DUMB_LAB
[2026-08-12 15:36:31] [INFO] ExitCode => 0
[2026-08-12 15:36:31] [INFO] Restore dataset from sabotage folder
[2026-08-12 15:36:31] [INFO] CMD => powershell.exe -NoProfile -ExecutionPolicy Bypass -File "C:\Users\seguridad\Desktop\TFM\01_ACTIVE_TESTS\Pruebas\TEC-008_Sabotaje\crear_directorio_dummy.ps1"
[2026-08-12 15:36:58] [INFO] STDOUT => OK. Dataset dummy creado en C:\Users\seguridad\Desktop\TFM\Pruebas\DUMB_LAB
[2026-08-12 15:36:58] [INFO] ExitCode => 0
[2026-08-12 15:36:58] [INFO] Restore dataset from exfiltration folder
[2026-08-12 15:36:58] [INFO] CMD => powershell.exe -NoProfile -ExecutionPolicy Bypass -File "C:\Users\seguridad\Desktop\TFM\01_ACTIVE_TESTS\Pruebas\TEC-009_Exfiltracion\crear_directorio_dummy.ps1"
[2026-08-12 15:37:15] [INFO] STDOUT => OK. Dataset dummy creado en C:\Users\seguridad\Desktop\TFM\Pruebas\DUMB_LAB
[2026-08-12 15:37:15] [INFO] ExitCode => 0
[2026-08-12 15:37:15] [INFO] Candidate campaign v5 finished
[2026-08-12 15:37:15] [INFO] Final log: C:\Users\seguridad\Desktop\TFM\01_ACTIVE_TESTS\Pruebas\Logs_Pruebas_TFM\TFM_TEC_Run_CANDIDATE_v5_20260812_152954.log
[2026-08-12 15:37:15] [INFO] PRE-SUMMARY active technique => NONE
[2026-08-12 15:37:15] [INFO] PRE-SUMMARY technique records => 9
[2026-08-12 15:37:15] [INFO] PRE-SUMMARY OK/WARN/FAIL => 9/0/0
[2026-08-12 15:37:15] [INFO] PRE-SUMMARY TEC-009 4104 QueryStatus/Count => MISSING/0
[2026-08-12 15:37:15] [INFO] PRE-SUMMARY output JSON => C:\Users\seguridad\Desktop\TFM\01_ACTIVE_TESTS\Pruebas\Logs_Pruebas_TFM\TFM_TEC_Run_CANDIDATE_v5_20260812_152954_summary.json
[2026-08-12 15:37:15] [INFO] PRE-SUMMARY JSON write completed => C:\Users\seguridad\Desktop\TFM\01_ACTIVE_TESTS\Pruebas\Logs_Pruebas_TFM\TFM_TEC_Run_CANDIDATE_v5_20260812_152954_summary.json
[2026-08-12 15:37:15] [INFO] Summary TXT => C:\Users\seguridad\Desktop\TFM\01_ACTIVE_TESTS\Pruebas\Logs_Pruebas_TFM\TFM_TEC_Run_CANDIDATE_v5_20260812_152954_summary.txt
[2026-08-12 15:37:15] [INFO] Summary JSON => C:\Users\seguridad\Desktop\TFM\01_ACTIVE_TESTS\Pruebas\Logs_Pruebas_TFM\TFM_TEC_Run_CANDIDATE_v5_20260812_152954_summary.json
[2026-08-12 15:37:15] [INFO] Summary CSV => C:\Users\seguridad\Desktop\TFM\01_ACTIVE_TESTS\Pruebas\Logs_Pruebas_TFM\TFM_TEC_Run_CANDIDATE_v5_20260812_152954_summary.csv

```

Figura 92: Evidencia de finalización de la campaña global (2 de 2)

El runner no modifica artifacts, reglas ni configuración del sistema de alertado. Su función es exclusivamente ejecutar las técnicas de laboratorio y generar evidencias operativas. Las acciones se mantienen dentro del dataset dummy y de las rutas controladas del TFM.

Las funciones principales del runner son:

- ejecutar TEC-001 a TEC-009 de forma secuencial;
- restaurar el dataset de laboratorio cuando una técnica lo modifica;
- evitar que TEC-007, TEC-008 o TEC-009 contaminen pruebas posteriores;
- registrar hora de inicio, hora de fin y duración de cada técnica;
- recoger códigos de salida de los comandos ejecutados;
- comprobar evidencias esperadas por técnica;
- contar eventos relevantes en Sysmon y PowerShell 4104;
- generar logs y resúmenes de campaña;
- facilitar la repetición de pruebas en campañas posteriores.

En el caso de TEC-007, el runner ejecuta la enumeración del dataset y el script de cifrado controlado, validando posteriormente la aparición de archivos .aes. En TEC-008, ejecuta un borrado controlado sobre ficheros dummy y compara el número de archivos antes y después de la operación. En TEC-009, ejecuta la prueba de staging, ZIP y upload HTTP, registra el receptor configurado, el estado del upload, el hash SHA-256 del ZIP generado y el resumen de transferencia.

La ejecución genera evidencias en la carpeta de logs del laboratorio, incluyendo un log detallado de campaña y ficheros resumen en formato TXT, JSON y CSV. Estos ficheros permiten reconstruir la ejecución completa sin depender únicamente de capturas de pantalla.

Las salidas principales son:

- log completo de ejecución;
- resumen textual de la campaña;
- resumen JSON estructurado;
- resumen CSV para análisis tabular;
- evidencias por técnica;
- recuentos de eventos Sysmon y PowerShell 4104.

El uso del runner fue especialmente importante para asegurar que las campañas de Velociraptor, Wazuh base y Wazuh custom se ejecutaran sobre un patrón de actividad comparable. De este modo, las diferencias observadas posteriormente pueden atribuirse principalmente a la capacidad de visibilidad, detección y alertado de cada herramienta, y no a variaciones manuales en la ejecución de las pruebas.

Anexo F.2. Simulación controlada de técnicas y Scripts de ejecución de pruebas

Para cada iteración de laboratorio había que restaurarlo:

- `crear_directorio_dummy.ps1`: Genera o restaura el dataset de archivos dummy sobre el que se realizan las pruebas con modificación de datos.
- `enumeracion_v2.ps1`: Enumera los archivos disponibles y genera los inventarios TXT/CSV utilizados posteriormente por TEC-007.
- `descifrar.ps1`: Permite recuperar los archivos afectados por el cifrado controlado cuando es necesario restaurar el escenario.

También se exponen los scripts empleados para simular el cifrado de archivos, el borrado destructivo y la exfiltración de datos correspondientes a TEC-007, TEC-008 y TEC-009. Las pruebas se realizan sobre archivos de laboratorio con el objetivo de generar actividad observable y evaluar la capacidad de detección de Velociraptor.

Como preparación para TEC-007, el script de enumeración identifica los archivos del directorio de pruebas y genera un inventario en TXT y CSV. El simulador de cifrado utiliza el CSV para seleccionar los archivos sobre los que actuará. Una vez hecha la enumeración podemos ejecutar los ataques de ransomware, exfiltración o sabotaje.

Anexo F.2.1. Script Ransomware

Scriptransom_v2.ps1 reproduce de forma controlada el comportamiento asociado a T1486 – Data Encrypted for Impact. El script utiliza únicamente archivos dummy, conserva una copia de los originales y cifra un subconjunto limitado mediante AES, generando archivos con extensión .aes.

Su finalidad no es reproducir ransomware real, sino generar evidencias observables de cifrado, como uso de componentes criptográficos, operaciones sobre archivos y creación de extensiones .aes, que puedan ser posteriormente analizadas por los sistemas evaluados.

Anexo F.2.2. Script Sabotaje

sabotaje.ps1 reproduce de forma controlada T1485 – Data Destruction. El script enumera los archivos del dataset dummy y los elimina mediante operaciones PowerShell. La ejecución permanece limitada al directorio preparado específicamente para el laboratorio.

La comparación entre el número de archivos antes y después de la ejecución proporciona ground truth de que la operación se realizó correctamente, mientras que PowerShell 4104 y los eventos de Sysmon aportan la telemetría utilizada durante el análisis.

Anexo F.2.3. Script Exfiltración

exfiltracion_v3.ps1 reproduce las fases de preparación, archivado y transferencia controlada de TEC-009. El script comprime los datos dummy en un archivo ZIP, genera una copia local, calcula su SHA-256 y, cuando se habilita la transferencia, realiza un HTTP POST hacia el receptor del laboratorio. Finalmente genera un resumen JSON con los datos principales de la operación.

Como extremo receptor se utiliza receiver_tfm_v4.py, un servidor HTTP desarrollado específicamente para la prueba. Este recibe el archivo, lo almacena y registra información como marca temporal, origen, tamaño y hash SHA-256. De esta forma puede verificarse independientemente que la transferencia se completó correctamente.

Anexo F.3. Evidencia y reproducibilidad

Los runners generan evidencias estructuradas que permiten reconstruir posteriormente cada campaña sin depender únicamente de capturas de pantalla. Entre las salidas se encuentran logs de ejecución, tiempos de inicio y fin, códigos de salida, resúmenes TXT/JSON/CSV, recuentos de eventos Sysmon y PowerShell 4104 y evidencias específicas de las técnicas ejecutadas.

El código fuente completo y las versiones utilizadas durante las campañas permanecen disponibles en el repositorio del proyecto [38], permitiendo consultar la implementación exacta y reproducir las pruebas sin aumentar innecesariamente la extensión de la memoria.

Anexo G. Configuración y resultados de Wazuh

Este anexo recoge la configuración y las evidencias complementarias correspondientes a la evaluación de Wazuh realizada durante el trabajo. La instalación del servidor y del agente se describe en el Anexo A, mientras que este apartado se centra exclusivamente en las reglas utilizadas y los resultados obtenidos.

La evaluación se realizó sobre las mismas pruebas TEC-001 a TEC-009 empleadas para Velociraptor, diferenciando dos configuraciones:

- Wazuh base: ruleset nativo de Wazuh, sin reglas específicas desarrolladas para el TFM.
- Wazuh custom: ruleset base complementado con reglas personalizadas destinadas a detectar comportamientos concretos de las técnicas seleccionadas.

Las reglas personalizadas fueron implementadas siguiendo el mecanismo de reglas locales proporcionado por Wazuh [33]. La recogida de eventos de Windows utilizada durante las pruebas se realizó mediante el agente desplegado en el endpoint [34], [36].

G.1. Reglas del ruleset base observadas

Durante la campaña se identificaron diferentes reglas nativas de Wazuh relacionadas con las técnicas ejecutadas.

Las Tablas 72 a 74 recogen las reglas observadas y los resultados agregados y por técnica de la comparación con Wazuh.

rule_id	grupo	descripción	TEC	MITRE	alertas	veredicto para métrica
92032	BASE	Suspicious Windows cmd shell execution	TEC-002	T1059.003	14	DETECTADA TEC-002; otras coincidencias no suman TEC.
92302	BASE	Run key modified using reg.exe	TEC-004	T1547.001	1	DETECTADA
91835	BASE	PowerShell Antivirus Software discovery	TEC-006	T1518.001	1	DETECTADA
92077	BASE	WMI AV product discovery	TEC-006	T1518.001	3	DETECTADA
92307	BASE	New service creation in registry	TEC-005	T1543.003	1	Visibilidad de creación de servicio (T1543.003); no acredita la detección específica de TEC-005/T1569.002 y no suma a la cobertura base.
110201	CUSTOM	TFM TEC-009 posible exfiltracion HTTP ZIP	TEC-009	T1048.003 / T1560.001	0	SIN DETECCIÓN
110202	CUSTOM	TFM TEC-008 borrado destructivo PowerShell	TEC-008	T1485	5	DETECTADA; incluye ruido fuera de TEC-008.
110203	CUSTOM	TFM TEC-007 cifrado PowerShell	TEC-007	T1486	4	DETECTADA
110301	CUSTOM	TFM PowerShell sospechoso 4104	TEC-001	T1059.001	2	DETECTADA
110401	CUSTOM	TFM Run Key Sysmon	TEC-004	T1547.001	0	SIN DETECCIÓN
110402	CUSTOM	TFM instalacion de servicio Windows	TEC-005	T1569.002	1	DETECTADA
110501	CUSTOM	TFM PowerShell observada por Sysmon	TEC-001	T1059.001	44	SIN DETECCIÓN

Tabla 72: Reglas Wazuh observadas y veredicto por técnica

G.2. Resultado global

Wazuh base proporcionó detección específica para TEC-002, TEC-004 y TEC-006, mientras que las reglas personalizadas permitieron detectar específicamente TEC-001, TEC-005, TEC-007 y TEC-008.

La unión de ambas configuraciones proporcionó, por tanto, detección específica para 7 de las 9 pruebas evaluadas. Este valor representa la unión de las técnicas detectadas y no la suma del número de alertas generadas

Sistema	Técnicas probadas	Detectadas	Tipo de reglas	Observaciones
Velociraptor custom	9/9	9/9	Artifacts custom P1-P4 CLIENT_EVENT	Detección y alerta específica 9/9; transporte y salida externa se evalúan aparte.
Velociraptor públicos CH	9/9	3/9	Artifacts públicos + Hayabusa Sigma CH	Detección 3/9: TEC-001, TEC-003 y TEC-005; CHM no ejecutada.
Wazuh base	9/9	3/9	Ruleset nativo Wazuh	TEC-002, TEC-004 y TEC-006; rules 92032, 92302, 91835 y 92077. No se cuentan alertas genéricas.
Wazuh custom	9/9	4/9	Reglas TFM 110xxx	TEC-001, TEC-005, TEC-007 y TEC-008; rules 110301, 110402, 110203 y 110202. 110201=0.

Tabla 73: Comparación agregada de cobertura entre Velociraptor y Wazuh

G.3. Resultado por técnica global

La comparación técnica entre el ruleset base y las reglas personalizadas queda resumida en la siguiente matriz. Los resultados muestran un comportamiento complementario entre el ruleset nativo y las reglas desarrolladas para el trabajo.

TEC	MITRE	Técnica	Detección pública	Alerta RT pública	Artifact alertante público	VR custom	Wazuh base	Wazuh custom
TEC-001	T1059.001	PowerShell	DETECTADA	DETECTADA	Windows.Hayabusa.Monitoring	DETECTADA	SIN DETECCIÓN	DETECTADA
TEC-002	T1059.003	Windows Command Shell	SIN DETECCIÓN	SIN DETECCIÓN	N/A	DETECTADA	DETECTADA	SIN DETECCIÓN
TEC-003	T1053.005	Scheduled Task	DETECTADA	DETECTADA	Windows.Hayabusa.Monitoring	DETECTADA	SIN DETECCIÓN	SIN DETECCIÓN
TEC-004	T1547.001	Registry Run Keys / Startup Folder	SIN DETECCIÓN	SIN DETECCIÓN	N/A	DETECTADA	DETECTADA	SIN DETECCIÓN
TEC-005	T1569.002	Service Execution	DETECTADA	DETECTADA	Windows.Hayabusa.Monitoring	DETECTADA	SIN DETECCIÓN	DETECTADA
TEC-006	T1518.001	Security Software Discovery	SIN DETECCIÓN	SIN DETECCIÓN	N/A	DETECTADA	DETECTADA	SIN DETECCIÓN
TEC-007	T1486	Data Encrypted for Impact	SIN DETECCIÓN	SIN DETECCIÓN	N/A	DETECTADA	SIN DETECCIÓN	DETECTADA
TEC-008	T1485	Data Destruction	SIN DETECCIÓN	SIN DETECCIÓN	N/A	DETECTADA	SIN DETECCIÓN	DETECTADA
TEC-009	T1074.001 / T1560.001 / T1048.003	Staging / Archive Collected Data / HTTP Exfiltration	SIN DETECCIÓN	SIN DETECCIÓN	N/A	DETECTADA	SIN DETECCIÓN	SIN DETECCIÓN

Tabla 74: Resultados de detección por técnica en Velociraptor y Wazuh

Anexo H. Planificación del proyecto ampliada

La planificación inicial fijó una dedicación total de 150 h al tratarse de un TFM con 6 créditos asignados, con fecha de inicio el 05/03/2026 y fecha de finalización estimada el 05/06/2026. Como se ha comentado antes, el seguimiento de la dedicación se realizó mediante Toggl Track, lo que permitió registrar de forma sistemática el tiempo invertido.

La ejecución real alcanzó 290 h y finalizó el 20/09/2026. El incremento se debió a la ampliación del alcance, especialmente al desarrollo y la depuración de artifacts, la ejecución y repetición de pruebas, la campaña de técnicas benignas, el benchmark de rendimiento y la comparación con Wazuh.

El seguimiento temporal del proyecto mediante Toggl Track se muestra en la Figura 93:

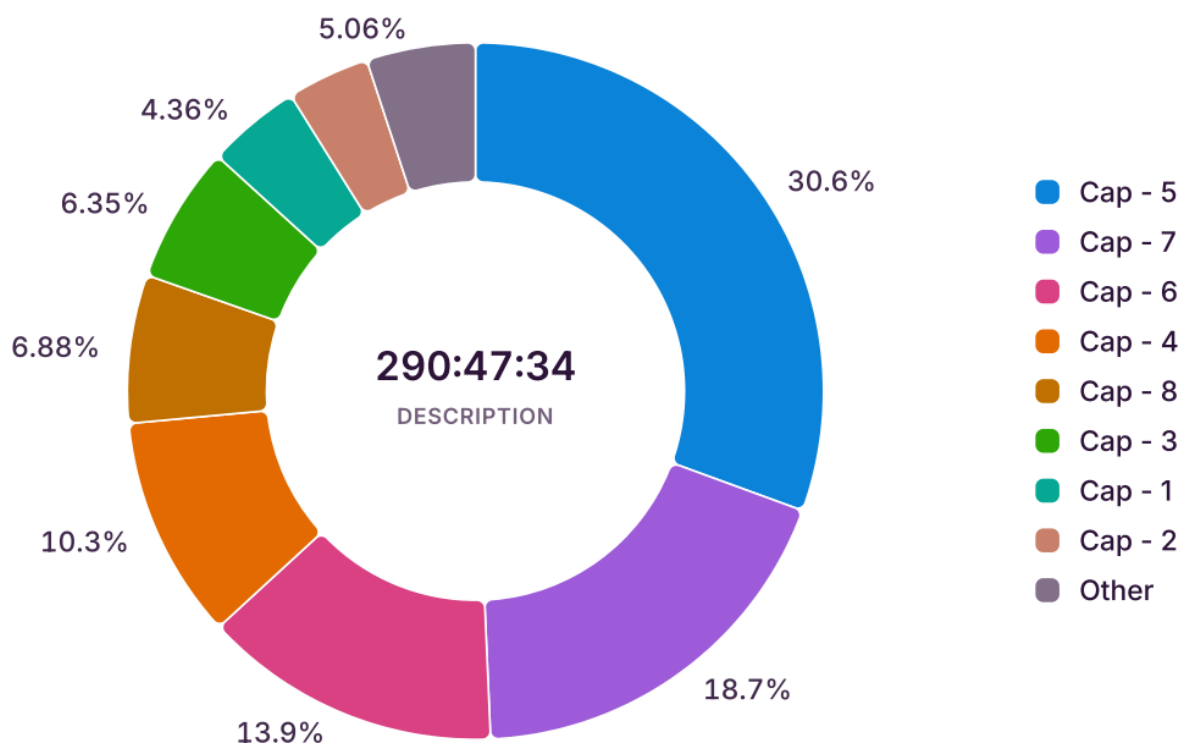


Figura 93: Monitorización con Toggl Track

- Planificación inicial: 150 h; fecha de finalización estimada: 05/06/2026.
- Ejecución real: 290 h; fecha de finalización real: 20/09/2026.

La planificación inicial se conserva como documentación histórica y no reproduce necesariamente la estructura final de la memoria. El resumen de dedicación se presenta en H.1; el desglose histórico y los diagramas se recogen en H.2 y H.3. Las fechas y horas efectivamente realizadas deben distinguirse de las previstas.

H.1. Tabla de planificación general

Capítulos	Duración estimada	Horas estimadas	Horas reales	Fecha de inicio	Fecha de finalización
Capítulo I: Introducción	3 semanas	20 h	22 h	05/03/2026	20/03/2026
Capítulo II: Fundamentos teóricos	1 semana	15 h	18 h	21/03/2026	28/03/2026
Capítulo III: Desarrollo experimental y ejecución de pruebas	8 semanas	60 h	120 h	29/03/2026	24/05/2026
Capítulo IV: Resultados de detección	3 semanas	20 h	40 h	25/05/2026	14/06/2026
Capítulo V: Evaluación de Velociraptor y benchmark de rendimiento	3 semanas	10 h	55 h	15/06/2026	05/07/2026
Capítulo VI: Validación comparativa con Wazuh como HIDS	2 semanas	10 h	20 h	06/07/2026	02/08/2026
Capítulo VII: Conclusiones	4 días	5 h	5 h	03/08/2026	16/08/2026
Capítulo VIII: Discusión y trabajo futuro	6 días	5 h	5 h	17/08/2026	06/09/2026
Referencias bibliográficas y anexos	2 días	5 h	5 h	07/09/2026	20/09/2026
Total	6 meses	150 h	290 h	05/03/2026	20/09/2026

Tabla 75: Planificación general

H.2. Tabla de planificación por capítulos

Este apartado presenta la planificación estructurada del trabajo desglosada por capítulos, con el fin de detallar con mayor profundidad el contenido y los objetivos de cada sección de la memoria.

Las tablas siguientes conservan el desglose histórico de planificación por capítulos. Sus rótulos no se utilizan para referenciar la estructura final; las horas previstas y registradas se contrastan con el resumen H.1.

Las Tablas 76 a 84 desglosan la planificación temporal y la dedicación estimada y real de los capítulos I a VIII y de las referencias bibliográficas y anexos.

H.2.1. Planificación del capítulo I

Subcapítulo	Duración estimada	Horas estimadas	Horas reales	Fecha de inicio	Fecha de finalización
1.1. Motivación y descripción del problema	2 días	3 h	3 h	05/03/2026	06/03/2026
1.2. Objetivos del proyecto	2 días	2 h	3 h	07/03/2026	08/03/2026
1.3. Estimación de tiempos y plazos	2 días	2 h	3 h	13/03/2026	14/03/2026
1.4. Presupuesto	1 día	2 h	2 h	15/03/2026	15/03/2026
1.5. Tabla de planificación general	2 días	3 h	3 h	16/03/2026	17/03/2026
1.6. Tabla de planificación por capítulos	1 día	4 h	4 h	18/03/2026	19/03/2026
1.7. Diagrama de Gantt	1 día	4 h	4 h	20/03/2026	20/03/2026
Total Capítulo I	3 semanas	20 h	22 h	05/03/2026	20/03/2026

Tabla 76: Planificación del capítulo I

H.2.2. Planificación del capítulo II

Subcapítulo	Duración estimada	Horas estimadas	Horas reales	Fecha de inicio	Fecha de finalización
2.1. Enfoques de detección y análisis en endpoint	2 días	3 h	4 h	21/03/2026	22/03/2026
2.2. Arquitectura interna Velociraptor	1 día	3 h	3 h	23/03/2026	23/03/2026
2.3. Artifacts y modelo de ejecución	1 día	3 h	4 h	24/03/2026	24/03/2026

Subcapítulo	Duración estimada	Horas estimadas	Horas reales	Fecha de inicio	Fecha de finalización
2.4. Velociraptor Query Language (VQL)	1 día	2 h	3 h	25/03/2026	26/03/2026
2.5. Telemetría disponible en el endpoint	1 día	2 h	2 h	27/03/2026	27/03/2026
2.6. Limitaciones de visibilidad en HIDS/DFIR	1 día	2 h	2 h	28/03/2026	28/03/2026
Total Capítulo II	1 semana	15 h	18 h	21/03/2026	28/03/2026

Tabla 77: Planificación del capítulo II

H.2.3. Planificación del capítulo III

Subcapítulo	Duración estimada	Horas estimadas	Horas reales	Fecha de inicio	Fecha de finalización
3.1. Diseño del entorno	2 días	4 h	5 h	29/03/2026	30/03/2026
3.2. Instalación, instrumentación y configuración base	2 días	4 h	5 h	31/03/2026	01/04/2026
3.3. Metodología experimental	3 días	5 h	6 h	02/04/2026	04/04/2026
3.4. Selección de técnicas MITRE ATT&CK	2 días	3 h	3 h	05/04/2026	06/04/2026
3.5. Estrategia de artifacts y monitorización	2 días	4 h	5 h	07/04/2026	08/04/2026
3.6. Benchmark y alertabilidad	2 días	3 h	4 h	09/04/2026	10/04/2026
3.7. Decisiones metodológicas cerradas	1 día	2 h	2 h	11/04/2026	11/04/2026
3.8. Escenarios de ataque	2 días	5 h	8 h	12/04/2026	16/04/2026
3.9. Desarrollo experimental por técnica	7 días	14 h	40 h	17/04/2026	07/05/2026

Subcapítulo	Duración estimada	Horas estimadas	Horas reales	Fecha de inicio	Fecha de finalización
3.10. Recopilación de telemetría	2 días	7 h	20 h	08/05/2026	17/05/2026
3.11. Configuración y uso de artifacts	3 días	9 h	22 h	18/05/2026	24/05/2026
Total Capítulo III	8 semanas	60 h	120 h	29/03/2026	24/05/2026

Tabla 78: Planificación del capítulo III

H.2.4. Planificación del capítulo IV

Subcapítulo	Duración estimada	Horas estimadas	Horas reales	Fecha de inicio	Fecha de finalización
4.1. Resultados por técnica MITRE	3 días	5 h	10 h	25/05/2026	29/05/2026
4.2. Cobertura de detección	2 días	4 h	7 h	30/05/2026	02/06/2026
4.3. Casos sin detección y detección parcial	2 días	3 h	5 h	03/06/2026	05/06/2026
4.4. Evidencias observadas y falsos positivos	2 días	3 h	6 h	06/06/2026	08/06/2026
4.5. Calidad de datos y limitaciones	2 días	2 h	5 h	09/06/2026	11/06/2026
4.6. Resultados definitivos de artifacts públicos	3 días	3 h	7 h	12/06/2026	14/06/2026
Total Capítulo IV	3 semanas	20 h	40 h	25/05/2026	14/06/2026

Tabla 79: Planificación del capítulo IV

H.2.5. Planificación del capítulo V

Subcapítulo	Duración estimada	Horas estimadas	Horas reales	Fecha de inicio	Fecha de finalización
5.1. Metodología de benchmark	2 días	2 h	10 h	15/06/2026	18/06/2026
5.2. Impacto en CPU	3 días	2 h	11 h	19/06/2026	22/06/2026
5.3. Impacto en memoria	3 días	2 h	10 h	23/06/2026	26/06/2026
5.4. Impacto en disco	3 días	2 h	9 h	27/06/2026	30/06/2026
5.5. Relación detección / coste	3 días	2 h	15 h	01/07/2026	05/07/2026
Total Capítulo V	3 semanas	10 h	55 h	15/06/2026	05/07/2026

Tabla 80: Planificación del capítulo V

H.2.6. Planificación del capítulo VI

Subcapítulo	Duración estimada	Horas estimadas	Horas reales	Fecha de inicio	Fecha de finalización
6.1. Objetivo de la comparación	1 día	0,5 h	1 h	06/07/2026	07/07/2026
6.2. Arquitectura del entorno Wazuh	1 día	0,5 h	1 h	08/07/2026	09/07/2026
6.3. Instalación del servidor Wazuh	1 día	1 h	2 h	10/07/2026	11/07/2026
6.4. Instalación y validación del agente Windows	1 día	0,5 h	1 h	12/07/2026	13/07/2026
6.5. Fuentes de telemetría configuradas	1 día	0,5 h	1 h	14/07/2026	15/07/2026
6.6. Reglas personalizadas TFM en Wazuh	2 días	1,5 h	3 h	16/07/2026	18/07/2026
6.7. Metodología de campañas separadas	1 día	0,5 h	1 h	19/07/2026	20/07/2026

Subcapítulo	Duración estimada	Horas estimadas	Horas reales	Fecha de inicio	Fecha de finalización
6.8. Campaña B: Wazuh base	1 día	1 h	2 h	21/07/2026	22/07/2026
6.9. Campaña C: Wazuh custom	2 días	1,5 h	3 h	23/07/2026	24/07/2026
6.10. Evidencias gráficas en el dashboard de Wazuh	1 día	0,5 h	1 h	25/07/2026	26/07/2026
6.11. Comparación Wazuh – Velociraptor	1 día	1 h	2 h	27/07/2026	28/07/2026
6.12. Limitaciones observadas	1 día	0,5 h	1 h	29/07/2026	30/07/2026
6.13. Registro y almacenamiento de evidencias	1 día	0,5 h	1 h	31/07/2026	02/08/2026
Total Capítulo VI	2 semanas	10 h	20 h	06/07/2026	02/08/2026

Tabla 81: Planificación del capítulo VI

H.2.7. Planificación del capítulo VII

Subcapítulo	Duración estimada	Horas estimadas	Horas reales	Fecha de inicio	Fecha de finalización
7.1. Conclusiones principales	1 día	2 h	2 h	03/08/2026	04/08/2026
7.2. Respuesta a objetivos	1 día	1 h	1 h	05/08/2026	07/08/2026
7.3. Limitaciones identificadas	1 día	1 h	1 h	09/08/2026	10/08/2026
7.4. Valor del enfoque	1 día	1 h	1 h	12/08/2026	16/08/2026
Total Capítulo VII	4 días	5 h	5 h	03/08/2026	16/08/2026

Tabla 82: Planificación del capítulo VII

H.2.8. Planificación del capítulo VIII

Subcapítulo	Duración estimada	Horas estimadas	Horas reales	Fecha de inicio	Fecha de finalización
8.1. Discusión integrada de resultados	1 día	1,5 h	1,5 h	17/08/2026	21/08/2026
8.2. Ampliación de técnicas MITRE	1 día	0,5 h	0,5 h	23/08/2026	24/08/2026
8.3. Desarrollo de nuevos artifacts	1 día	1 h	1 h	25/08/2026	26/08/2026
8.4. Ciclo de vida y trazabilidad de los artifacts	1 día	0,5 h	0,5 h	27/08/2026	28/08/2026
8.5. Automatización de detecciones	1 día	0,5 h	0,5 h	29/08/2026	29/08/2026
8.6. Integración SIEM/SOC y despliegue en entornos reales	1 día	1 h	1 h	30/08/2026	06/09/2026
Total Capítulo VIII	6 días	5 h	5 h	17/08/2026	06/09/2026

Tabla 83: Planificación del capítulo VIII

H.2.9. Planificación de referencias bibliográficas y anexos

Subcapítulo	Duración estimada	Horas estimadas	Horas reales	Fecha de inicio	Fecha de finalización
Revisión y normalización de referencias bibliográficas	2 días	5 h	5 h	07/09/2026	20/09/2026
Total de referencias bibliográficas y anexos	2 días	5 h	5 h	07/09/2026	20/09/2026

Tabla 84: Planificación de referencias bibliográficas y anexos

H.3. Diagrama de Gantt

La Figura 1 del capítulo I y las Figuras 94 a la 100 del Anexo H representan la planificación preliminar general y por capítulos elaborada al inicio del proyecto mediante GanttProject. Se conservan sin modificaciones porque documentan la previsión original y permiten compararla con la ejecución

real consolidada en la Tabla 74. Durante el desarrollo surgieron actividades no previstas inicialmente, especialmente la ampliación de las pruebas experimentales, la campaña de técnicas benignas, el benchmark de rendimiento, la evaluación de artifacts públicos y la validación comparativa con Wazuh. Por este motivo, las fechas y cargas de trabajo representadas en los diagramas no corresponden al calendario final ejecutado.

H.3.1. Diagrama de Gantt

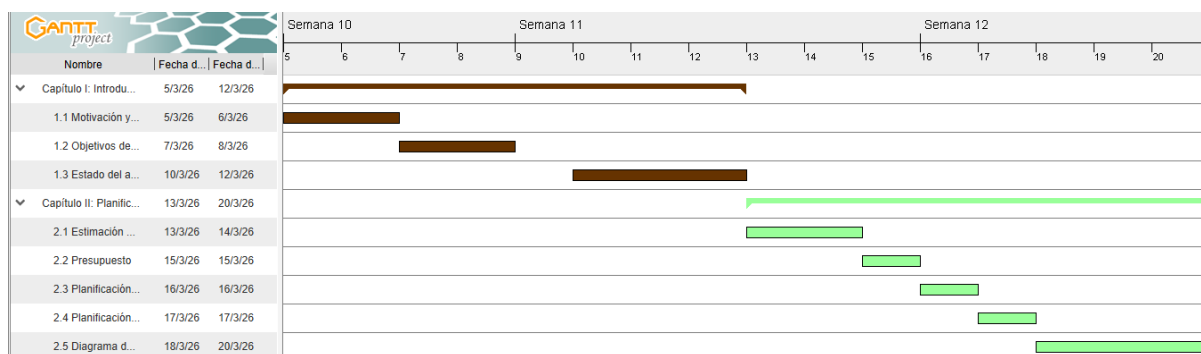


Figura 94: Planificación Gantt de los capítulos I y II

H.3.2. Diagrama de Gantt

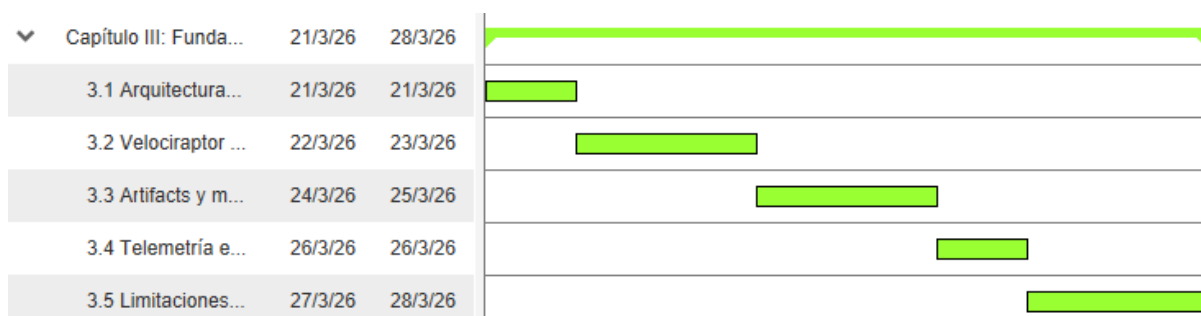


Figura 95: Planificación Gantt del capítulo III

H.3.3. Diagrama de Gantt



Figura 96: Planificación Gantt del capítulo IV

H.3.4. Diagrama de Gantt

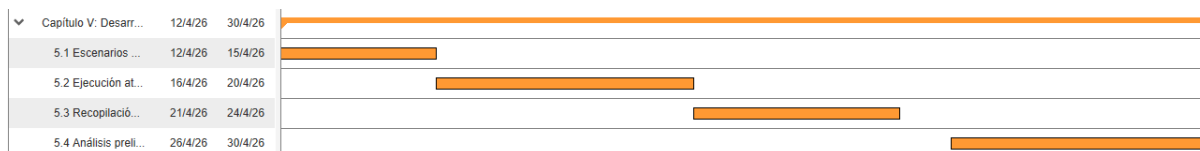


Figura 97: Planificación Gantt del capítulo V

H.3.5. Diagrama de Gantt



Figura 98: Planificación Gantt del capítulo VI

H.3.6. Diagrama de Gantt

El capítulo V se cerró finalmente como benchmark de rendimiento de Velociraptor. La comparación prevista con una solución EDR comercial se descartó por falta de licencia y se sustituyó por un capítulo específico de validación comparativa con Wazuh como plataforma HIDS/SIEM de contraste.

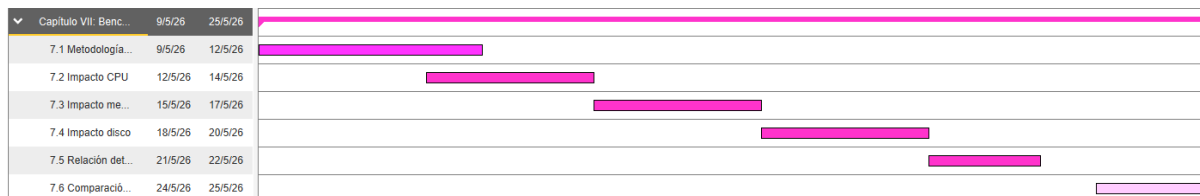


Figura 99: Planificación Gantt del capítulo VII

H.3.7. Diagrama de Gantt

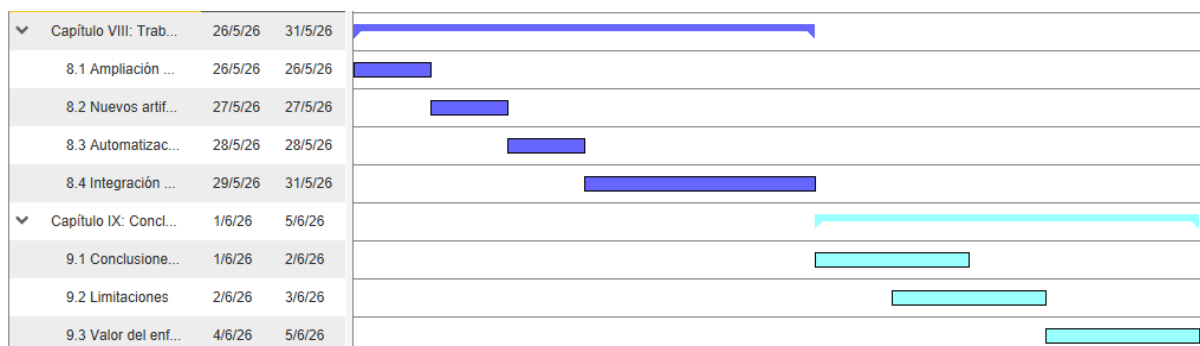


Figura 100: Planificación inicial del capítulo VIII y del bloque de referencias y anexos

Anexo I. Declaración responsable sobre el uso de inteligencia artificial

Yo, Julio García-Amorena Reina, autor del Trabajo Fin de Máster titulado «Evaluación de Velociraptor como HIDS para la detección de técnicas MITRE ATT&CK Enterprise», declaro bajo mi responsabilidad que durante su elaboración se han utilizado herramientas de inteligencia artificial generativa únicamente como apoyo auxiliar.

En concreto, estas herramientas se han empleado para:

- Revisar la redacción, ortografía, claridad y estructura de determinados apartados.
- Proponer mejoras de estilo y organización del contenido.
- Ayudar en la revisión de scripts, comandos y procedimientos técnicos.
- Facilitar la interpretación, síntesis y presentación de resultados obtenidos durante las pruebas.
- Apoyar la elaboración de tablas, gráficos y documentación complementaria.

El uso de estas herramientas no ha sustituido el trabajo técnico, experimental ni intelectual realizado por el autor. La definición de la metodología, el diseño y ejecución de las pruebas, la recopilación de evidencias, el análisis de los resultados y la formulación de las conclusiones han sido realizados y supervisados personalmente.

Asimismo, declaro que:

- Todas las respuestas generadas mediante inteligencia artificial han sido revisadas, verificadas y, cuando ha sido necesario, corregidas.
- No se han incorporado deliberadamente datos, referencias bibliográficas, resultados experimentales o evidencias sin comprobar su autenticidad.
- Las herramientas de inteligencia artificial no han sido consideradas fuentes académicas ni técnicas por sí mismas.
- Las decisiones finales sobre el contenido incluido en la memoria corresponden exclusivamente a mí, como autor.

Las principales herramientas utilizadas han sido ChatGPT y OpenAI Codex, empleadas como apoyo para la redacción, revisión documental, análisis técnico y asistencia en tareas de programación.

Referencias

-
- [1] Verizon, “2025 Data Breach Investigations Report,” *Verizon Business* [En línea], 2025. Último acceso: septiembre de 2026. Enlace: <https://www.verizon.com/business/resources/T49f/reports/2025-dbir-data-breach-investigations-report.pdf>
- [2] MITRE, “Enterprise Matrix,” *MITRE ATT&CK* [En línea], s. f. Último acceso: julio de 2026. Enlace: <https://attack.mitre.org/matrices/enterprise/>
- [3] Velociraptor, “Artifacts,” *Velociraptor Documentation* [En línea], s. f. Último acceso: julio de 2026. Enlace: <https://docs.velociraptor.app/docs/artifacts/>
- [4] Velociraptor, “VQL,” *Velociraptor Documentation* [En línea], s. f. Último acceso: julio de 2026. Enlace: <https://docs.velociraptor.app/docs/vql/>
- [5] Velociraptor, “Event Queues,” *Velociraptor Documentation* [En línea], s. f. Último acceso: julio de 2026. Enlace: https://docs.velociraptor.app/docs/artifacts/event_queues/
- [6] Velociraptor, “Server Monitoring,” *Velociraptor Documentation* [En línea], s. f. Último acceso: julio de 2026. Enlace: https://docs.velociraptor.app/docs/server_automation/server_monitoring/
- [7] National Institute of Standards and Technology, “Host-Based Intrusion Detection and Prevention System,” *NIST Computer Security Resource Center* [En línea], s. f. Último acceso: septiembre de 2026. Enlace: https://csrc.nist.gov/glossary/term/host_based_intrusion_detection_and_prevention_system
- [8] National Institute of Standards and Technology, “Guide to Integrating Forensic Techniques into Incident Response,” *NIST Special Publication 800-86* [En línea], 2006. Último acceso: septiembre de 2026. Enlace: <https://csrc.nist.gov/pubs/sp/800/86/final>
- [9] Microsoft, “What is EDR? Endpoint Detection and Response,” *Microsoft Security* [En línea], s. f. Último acceso: septiembre de 2026. Enlace: <https://www.microsoft.com/en-us/security/business/security-101/what-is-edr-endpoint-detection-response>
- [10] Velociraptor, “Quickstart Guide,” *Velociraptor Documentation* [En línea], s. f. Último acceso: septiembre de 2026. Enlace: <https://docs.velociraptor.app/docs/deployment/quickstart/>
- [11] Velociraptor, “Deploying Clients,” *Velociraptor Documentation* [En línea], s. f. Último acceso: septiembre de 2026. Enlace: <https://docs.velociraptor.app/docs/deployment/clients/>
- [12] Velociraptor, “Artifact Sources,” *Velociraptor Documentation* [En línea], s. f. Último acceso: septiembre de 2026. Enlace: <https://docs.velociraptor.app/docs/artifacts/sources/>
- [13] Velociraptor, “Basic Fields,” *Velociraptor Documentation* [En línea], s. f. Último acceso: septiembre de 2026. Enlace: https://docs.velociraptor.app/docs/artifacts/basic_fields/

- [14] Velociraptor, “VQL Fundamentals,” *Velociraptor Documentation* [En línea], s. f. Último acceso: septiembre de 2026. Enlace: <https://docs.velociraptor.app/docs/vql/fundamentals/>
- [15] Velociraptor, “Event Queries,” *Velociraptor Documentation* [En línea], s. f. Último acceso: septiembre de 2026. Enlace: <https://docs.velociraptor.app/docs/vql/events/>
- [16] Velociraptor, “Client Monitoring,” *Velociraptor Documentation* [En línea], s. f. Último acceso: septiembre de 2026. Enlace: <https://docs.velociraptor.app/docs/clients/monitoring/>
- [17] M. Russinovich y T. Garnier, “Sysmon,” *Microsoft Learn* [En línea]. Último acceso: agosto de 2026. Enlace: <https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon>
- [18] Microsoft, “about_Logging,” *Microsoft Learn — Windows PowerShell 5.1* [En línea], s. f. Último acceso: agosto de 2026. Enlace: https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_logging?view=powershell-5.1
- [19] A. Melendez Casado, *Análisis de la capacidad de detección de los IDS Snort, FortiGate y Palo Alto NGFW en entornos ICS bajo la matriz MITRE ATT&CK* [En línea], s. f. Universidad de Sevilla. Último acceso: septiembre de 2026. Enlace: <https://idus.us.es/bitstreams/6f899d44-e18b-44ce-8026-c78c45f59b42/download>
- [20] MITRE, “Command and Scripting Interpreter: PowerShell (T1059.001),” *MITRE ATT&CK* [En línea], s. f. Último acceso: julio de 2026. Enlace: <https://attack.mitre.org/techniques/T1059/001/>
- [21] MITRE, “Command and Scripting Interpreter: Windows Command Shell (T1059.003),” *MITRE ATT&CK* [En línea], s. f. Último acceso: julio de 2026. Enlace: <https://attack.mitre.org/techniques/T1059/003/>
- [22] MITRE, “Scheduled Task/Job: Scheduled Task (T1053.005),” *MITRE ATT&CK* [En línea], s. f. Último acceso: julio de 2026. Enlace: <https://attack.mitre.org/techniques/T1053/005/>
- [23] MITRE, “Boot or Logon Autostart Execution: Registry Run Keys / Startup Folder (T1547.001),” *MITRE ATT&CK* [En línea], s. f. Último acceso: julio de 2026. Enlace: <https://attack.mitre.org/techniques/T1547/001/>
- [24] MITRE, “System Services: Service Execution (T1569.002),” *MITRE ATT&CK* [En línea], s. f. Último acceso: julio de 2026. Enlace: <https://attack.mitre.org/techniques/T1569/002/>
- [25] MITRE, “Software Discovery: Security Software Discovery (T1518.001),” *MITRE ATT&CK* [En línea], s. f. Último acceso: julio de 2026. Enlace: <https://attack.mitre.org/techniques/T1518/001/>
- [26] MITRE, “Data Encrypted for Impact (T1486),” *MITRE ATT&CK* [En línea], s. f. Último acceso: julio de 2026. Enlace: <https://attack.mitre.org/techniques/T1486/>
- [27] MITRE, “Data Destruction (T1485),” *MITRE ATT&CK* [En línea], s. f. Último acceso: julio de 2026. Enlace: <https://attack.mitre.org/techniques/T1485/>

- [28] MITRE, “Data Staged: Local Data Staging (T1074.001),” *MITRE ATT&CK* [En línea], s. f. Último acceso: julio de 2026. Enlace: <https://attack.mitre.org/techniques/T1074/001/>
- [29] MITRE, “Archive Collected Data: Archive via Utility (T1560.001),” *MITRE ATT&CK* [En línea], s. f. Último acceso: julio de 2026. Enlace: <https://attack.mitre.org/techniques/T1560/001/>
- [30] MITRE, “Exfiltration Over Alternative Protocol: Exfiltration Over Unencrypted Non-C2 Protocol (T1048.003),” *MITRE ATT&CK* [En línea], s. f. Último acceso: julio de 2026. Enlace: <https://attack.mitre.org/techniques/T1048/003/>
- [31] Oracle, “Oracle VirtualBox,” *Oracle Documentation* [En línea], 2026. Último acceso: septiembre de 2026. Enlace: <https://www.virtualbox.org/>
- [32] J. García-Amorena Reina, “Controlled File Encryption,” *GitHub* [En línea], 2026. Último acceso: septiembre de 2026. Enlace: <https://github.com/Juliog-a/Controlled-File-Encryption>
- [33] Wazuh, “Custom Rules,” *Wazuh Documentation* [En línea], s. f. Último acceso: agosto de 2026. Enlace: <https://documentation.wazuh.com/current/user-manual/ruleset/rules/custom.html>
- [34] Wazuh, “Deploying Wazuh Agents on Windows Endpoints,” *Wazuh Documentation* [En línea], s. f. Último acceso: agosto de 2026. Enlace: <https://documentation.wazuh.com/current/installation-guide/wazuh-agent/wazuh-agent-package-windows.html>
- [35] Wazuh, “Installing the Wazuh Indexer Using the Assisted Installation Method,” *Wazuh Documentation* [En línea], s. f. Último acceso: agosto de 2026. Enlace: <https://documentation.wazuh.com/current/installation-guide/wazuh-indexer/installation-assistant.html>
- [36] Wazuh, “Log Data Collection and Windows Event Monitoring,” *Wazuh Documentation* [En línea], s. f. Último acceso: agosto de 2026. Enlace: <https://documentation.wazuh.com/current/user-manual/capabilities/log-data-collection/configuration.html>
- [37] Wazuh, “Quickstart,” *Wazuh Documentation* [En línea], s. f. Último acceso: septiembre de 2026. Enlace: <https://documentation.wazuh.com/current/quickstart.html>
- [38] J. García-Amorena Reina, “VR_HIDS-EDR,” *GitHub* [En línea], 2026. Último acceso: septiembre de 2026. Enlace: https://github.com/Juliog-a/VR_HIDS-EDR